

This PDF will Contains

1. Angular Framework
 - **125 theoretical questions**
2. Angular Framework
 - **20 coding and problem-solving questions**
3. Behavioral Questions
 - **10 questions**

Angular Framework (Questions with Answer)

1.What are the main features of Angular?

1. *Two-Way Data Binding*

- What it does: Automatically synchronizes data between the model (component logic) and the view (UI).
- Example: If a user updates an input field, the model updates instantly, and vice versa.
- Benefit: Reduces manual DOM manipulation, improving development speed.

2. *Component-Based Architecture*

- What it does: Applications are built using reusable components (each with its own template, styles, and logic).

```
@Component({  
  selector: 'app-user',  
  templateUrl: './user.component.html',  
  styleUrls: ['./user.component.css']  
})  
  
export class UserComponent { ... }
```

- Benefit: Promotes modularity, reusability, and easier maintenance.

3. Dependency Injection (DI)

- What it does: Provides dependencies (e.g., services) to components rather than hardcoding them.
- Example:

```
constructor(private userService: UserService) {}
```

- Benefit: Makes code more testable and modular.

4. Directives

- What they do: Extend HTML with custom behaviors.
- Types:
 - Structural Directives (Change DOM layout, e.g., `*ngIf`, `*ngFor`).
 - Attribute Directives (Modify element behavior/appearance, e.g., `ngStyle`, `ngClass`).

5. Services and RxJS for State Management

- What it does:
 - Services encapsulate business logic (e.g., HTTP requests).
 - RxJS handles asynchronous operations (e.g., API calls via `Observables`).
- Example:

```
this.http.get<User[]>('/api/users').subscribe(users => this.users = users);
```

6. Routing (Angular Router)

- What it does: Enables navigation between views without page reloads.
- Example:

```
const routes: Routes = [  
  
  { path: 'home', component: HomeComponent },  
  
  { path: 'users', component: UserListComponent }  
  
];
```

- Benefit: Creates a seamless SPA experience.

7. Forms (Reactive & Template-Driven)

- Reactive Forms: Model-driven approach with `FormControl`, `FormGroup`.
- Template-Driven Forms: HTML-centric (uses `ngModel`).
- Benefit: Simplifies complex form validation and handling.

8. Ahead-of-Time (AOT) Compilation

- What it does: Converts Angular HTML/templates into optimized JavaScript during build time.
- Benefit: Faster rendering, smaller bundle size, and better security.

9. Internationalization (i18n)

- What it does: Supports multi-language apps via built-in `i18n` tools.
- Benefit: Easier localization for global audiences.

10. Testing (Jasmine/Karma, Protractor)

- Unit Testing: Jasmine + Karma.
- E2E Testing: Protractor (or alternatives like Cypress).
- Benefit: Ensures reliability with test-first practices.

11. Angular CLI

- What it does: Command-line tool for scaffolding, building, and testing apps.
- Commands:

```
ng new my-app      # Create a new project
```

```
ng generate component user  # Generate a component
```

```
ng serve # Run dev server
```

- Benefit: Boosts productivity with automation.

12. PWA Support

- What it does: Turns apps into Progressive Web Apps (offline mode, push notifications).
- Setup: `ng add @angular/pwa`.

13. Ivy Renderer (Optimized Compilation)

- What it does: Angular's latest rendering engine (smaller bundles, faster runtime).

14. Cross-Platform Development

- Angular Universal: Server-side rendering (SSR) for SEO.
- Native Apps: Ionic + Angular for mobile.

2. Can you explain the Angular lifecycle hooks? When would you use `ngOnInit`?

Angular components go through a lifecycle from creation to destruction. Lifecycle hooks are methods that let you tap into key moments to run your code.

Key Lifecycle Hooks

1. `ngOnChanges()`
 - Runs when Angular sets/changes input properties.
 - Use it to react to input changes (e.g., `@Input()` values).
2. `ngOnInit()`
 - Runs once after Angular initializes the component and sets inputs.

- When to use it:
 - Fetch initial data (e.g., API calls).
 - Initialize component state (default values).

Example:

```
ngOnInit() {

    this.loadUserData();}
```

3. `ngDoCheck()`
 - Runs during every change detection cycle.
 - Use it for custom change detection (rarely needed).
4. `ngAfterViewInit()`
 - Runs after the component's view/DOM is fully rendered.
 - Use it to access `@ViewChild` elements or run DOM-dependent code.
5. `ngOnDestroy()`
 - Runs just before the component is destroyed.
 - When to use it:
 - Unsubscribe from observables (prevent memory leaks).
 - Clean up timers or event listeners.

3. How does Angular handle dependency injection?

Angular's Dependency Injection (DI) system is a way to provide dependencies (like services, values, or functions) to components, directives, or other services automatically without hardcoding them.

How It Works

1. Define a Service (Injectable Class)
 - Mark it with `@Injectable()` to tell Angular it can be injected.

```
@Injectable({ providedIn: 'root' }) // Available app-wide
export class UserService {
    getUsers() { return ['Alice', 'Bob']; }
```

```
}
```

2. Inject the Service

- Use the `constructor()` to request the dependency.

```
@Component({...})
export class UserComponent {
  constructor(private userService: UserService) {} // Angular
  provides it
}
```

3. Angular Provides the Instance

- The DI system looks for a registered provider and supplies the instance.

Key DI Concepts

1. Providers

- Define how and where a dependency is created.
- Common ways:
 - `providedIn: 'root'` (Singleton, app-wide).
 - Component-level (Scoped to a component and its children).

```
@Component({
  providers: [UserService] // New instance for this component tree
})
```

2. Injector Hierarchy

- Angular creates a tree of injectors (mirroring the component tree).
- When a component requests a dependency, Angular checks:
 1. The component's own providers.
 2. Parent components up to the root.
 3. The root injector (for `providedIn: 'root'` services).

3. Tokens

- Dependencies are identified by tokens (usually the class itself).
- You can also use `InjectionToken` for non-class dependencies (e.g., config objects):

```
export const API_URL = new InjectionToken<string>('API_URL');
// Provide it in a module/component:
```

- providers: [{ provide: `API_URL`, useValue: 'https://api.example.com' }]

Dependency Injection in Action

Example: HTTP Client

```
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class DataService {
  constructor(private http: HttpClient) {} // Injected by Angular
  fetchData() {
    return this.http.get('/api/data');
  }
}
```

- `HttpClient` is provided by Angular's `HttpClientModule`.

Key Points to Remember

- `providedIn: 'root'`: Singleton (one instance for the whole app).
- Component providers: New instance for that component + its children.
- Hierarchy: Angular checks for providers from the component up to the root.

4. What is the difference between Angular components and services?

Angular Components vs. Services - Simple Explanation

1. Purpose

- **Component:**
 - Handles UI and user interactions (what the user sees).
 - Example: A login form, navbar, or product list.
 - **Service:**
 - Handles business logic, data, and shared functions (behind-the-scenes work).
 - Example: API calls, authentication, or calculations.
-

2. Key Differences

Feature	Component	Service
Role	Manages a part of the UI (HTML + Behavior)	Reusable logic (e.g., HTTP requests)
Decorator	<code>@Component</code>	<code>@Injectable</code>
Scope	Tied to a view (destroyed when unused)	Singleton (lives for the whole app)
Usage	Used in templates (<code><app-login></code>)	Injected into components via DI

3. When to Use Each?

- Use a Component when you need:
 - A reusable UI piece (like a button, card, or form).
 - User interaction (clicks, inputs).
- Use a Service when you need:
 - Shared logic (e.g., fetching data from an API).
 - State management (e.g., storing user data).

4. Example

Service (Data Fetching)

```
@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers() {
    return ["Alice", "Bob"]; // Simulated API call
  }
}
```

Component (Displaying Data)

```
@Component({
  selector: 'app-user-list',
  template: `<li *ngFor="let user of users">{{ user }}</li>`
})
export class UserListComponent {
  users: string[] = [];
  constructor(private userService: UserService) {}

  ngOnInit() {
    this.users = this.userService.getUsers(); // Uses the service
  }
}
```

Interview Cheat Sheet

- ✓ Components = UI + User Actions
- ✓ Services = Logic + Data Management
- ✓ Services are injected into Components via D

5. Explain Angular's two-way data binding. How is it different from one-way data binding?

1. One-Way Data Binding

- What it does: Data flows in one direction (either from component to view or from view to component).
- Types:
 - Component → View (Interpolation / Property Binding)

```
<h1>{{ title }}</h1> <!-- Displays component's `title` value -->
```

- `<button [disabled]="isDisabled">Click</button>` <!-- Binds to `isDisabled` -->
- View → Component (Event Binding)
- `<button (click)="onClick()">Click Me</button>` <!-- Calls `onClick()` when clicked -->

- Use Case: Simple updates where data changes don't need automatic sync.
-

2. Two-Way Data Binding

- What it does: Combines property binding + event binding to keep component and view in sync automatically.
- Syntax: `[(ngModel)]` (Requires `FormsModule`).
- `<input [(ngModel)]="username" />` <!-- Updates `username` in real-time -->
 - If the user types, `username` updates in the component.
 - If `username` changes in code, the input field updates.
- Behind the Scenes:
 - `<input [value]="username" (input)="username = $event.target.value" />`
- Use Case: Forms, real-time input fields (e.g., search bars, login forms)

Key Differences

Feature	One-Way Binding	Two-Way Binding
Data Flow	One direction (→ or ←)	Both directions (↔)

Syntax	<code>{{ }}</code> , <code>[]</code> , <code>()</code>	<code>[()]</code> (Banana-in-a-box syntax)
Performance	More predictable (better for large apps)	Slightly heavier (due to event listening)
Use Case	Displaying data, handling events	Interactive forms, instant updates

Interview Example

One-Way:

```
@Component({
  template: `
    <h1>{{ greeting }}</h1> <!-- One-way: Component → View -->
    <button (click)="changeGreeting()">Update</button> <!-- One-way: View
→ Component -->
  `,
})

export class AppComponent {
  greeting = "Hello!";
  changeGreeting() { this.greeting = "Hola!"; }
}
```

Two-Way:

```
@Component({
  template: `
    <input [(ngModel)]="name" /> <!-- Two-way sync -->
    <p>You typed: {{ name }}</p>
  `,
})

export class AppComponent {
```

```
name = "";  
}
```

When to Use Each?

- One-Way:
 - ✓ Displaying static/calculated data.
 - ✓ Handling button clicks, events.
 - Two-Way:
 - ✓ Forms, input fields.
 - ✓ When UI and data must stay in sync (e.g., live preview).
-

Pro Tip for Interviews

- "One-way is like a read-only or write-only mode; two-way is like a live chat where both sides react instantly."
- Angular's default is one-way (for performance). Two-way is just syntactic sugar over `[]` + `()`.

🚀 Bonus: In modern Angular, reactive forms (using `FormControl`) are often preferred over `ngModel` for complex forms.

6. What is Angular CLI, and how does it assist in Angular development?

Angular CLI (Command Line Interface) is a powerful tool that automates common tasks in Angular development, making it faster and easier to build, test, and deploy apps.

Key Features & How It Helps

1. Project Setup

- Command: `ng new my-app`
- What it does:
 - Creates a new Angular project with the right folder structure.
 - Sets up TypeScript, testing, and build tools automatically.

2. Generate Code (Scaffolding)

- Commands:

```
ng generate component user    # Creates a new component (HTML+TS+CSS)
ng generate service api       # Creates a service
```

- `ng generate module auth` *# Creates a module*
- Shortcuts: `ng g c user` (faster!)
- Why it's useful:
 - Avoids manual file creation.
 - Auto-updates `app.module.ts` with imports.

3. Development Server

- Command: `ng serve`
- What it does:
 - Starts a live dev server (usually at `http://localhost:4200`).
 - Auto-refreshes the app when you save code (Hot Module Replacement).

4. Build for Production

- Command: `ng build --prod`
- What it does:
 - Optimizes code (minification, tree-shaking).
 - Generates ready-to-deploy files in `/dist`.

5. Testing

- Commands:

```
ng test    # Runs unit tests (Karma + Jasmine)
```

- `ng e2e` *# Runs end-to-end tests (Protractor/Cypress)*

6. Adding Libraries

- Command: `ng add @angular/material`
 - What it does:
 - Installs Angular Material + auto-configures it.
-

Why Use Angular CLI?

- ✓ Saves time – No manual setup.
 - ✓ Best practices – Follows Angular's recommended structure.
 - ✓ Consistency – Team projects stay organized.
 - ✓ Built-in tools – Testing, bundling, and deployment made easy.
-

Interview Cheat Sheet

- "Angular CLI is like a Swiss Army knife for Angular devs—it handles setup, coding, testing, and deployment with simple commands."
- Most used commands:
 - `ng new` (Create app)
 - `ng serve` (Run dev server)
 - `ng generate` (Scaffold code)
 - `ng build --prod` (Production build)

🚀 Pro Tip: Use `--help` with any command (e.g., `ng generate --help`) to see all options!

Example Workflow:

1. `ng new my-app`
2. `cd my-app`
3. `ng serve`

7. How would you handle state management in an Angular application?

Managing state (data) in Angular apps is crucial for scalability and maintainability.

Here's how to handle it effectively:

1. Simple State (Small Apps)

Option A: Services + RxJS (Recommended for most cases)

- How it works:
 - Use a `BehaviorSubject` or `ReplaySubject` in a service to store state.
 - Components subscribe to updates.
- Example:

```
@Injectable({ providedIn: 'root' })

export class CartService {

  private cartItems = new BehaviorSubject<Item[]>([]);

  cartItems$ = this.cartItems.asObservable();

  addItem(item: Item) {

    this.cartItems.next([...this.cartItems.value, item]);

  }

}
```

- Pros:
 - ✓ Lightweight
 - ✓ No external libraries
 - ✓ Flexible

Option B: Input/Output + Event Emitters

- For parent-child component communication.

```
@Component({...})

export class ChildComponent {

  @Input() data: string;

  @Output() update = new EventEmitter<string>();

}
```

2. Advanced State (Medium/Large Apps)

Option A: NgRx (Redux Pattern)

- How it works:
 1. Single source of truth (Store)
 2. Actions → Reducers → Selectors

- Example Flow:
 1. Dispatch an action:
 2. `this.store.dispatch(loadUser());`
 3. Reducer updates state:

```
const userReducer = createReducer(initialState,

  on(loadUserSuccess, (state, { user }) => ({ ...state, user })))

);
```

4. Select data in components:
 5. `user$ = this.store.select(selectUser);`
- Pros:
 - ✓ Predictable state
 - ✓ Time-travel debugging
 - ✓ Enterprise-grade
 - Cons:

- ✗ Steeper learning curve
- ✗ Boilerplate code

Option B: NGXS / Akita

- Simpler alternatives to NgRx:
 - NGXS: Uses decorators (less boilerplate).
 - Akita: Object-oriented approach.

3. Server-State (API Data)

Option: React Query for Angular (TanStack Query)

- Auto-caching, refetching, and optimistic updates.
- Example:

```
users$ = this.queryClient.fetchQuery({  
  queryKey: ['users'],  
  queryFn: () => this.http.get('/api/users')  
});
```

When to Use What?

Scenario	Solution
Small app	RxJS Services
Medium app	NgRx (or NGXS/Akita)

API-heavy app

TanStack Query

Component sharing

Input/Output +
Services

Key Interview Points

1. For most apps, start with RxJS services (avoid premature NgRx).
2. NgRx shines in complex apps with shared state.
3. Always unsubscribe to prevent memory leaks (use `async pipe` or `takeUntil`).

Pro Tip:

- *"Choose state management based on app size. Don't use NgRx for a to-do list!"*

Example Decision Flow:

1. Is state shared across many components? → Service/NgRx
2. Is it local to one component? → Input/Output
3. Does it involve API data? → TanStack Query

8. What are Angular decorators, and can you provide examples of commonly used decorators?

Decorators are special functions that modify classes, properties, or methods in Angular. They provide metadata to Angular about how a class should work.

Commonly Used Angular Decorators

1. Class Decorators

a) @Component

- Turns a class into an Angular component.
- Example:

```
@Component({  
  selector: 'app-user',  
  templateUrl: './user.component.html',  
  styleUrls: ['./user.component.css']  
})  
  
• export class UserComponent {}
```

b) @NgModule

- Defines an Angular module (groups components, services, etc.).
- Example:

```
@NgModule({  
  declarations: [UserComponent],  
  imports: [CommonModule],  
  providers: [UserService]  
})  
  
• export class UserModule {}
```

c) @Injectable

- Marks a class as a service (usable with Dependency Injection).
- Example:

```
@Injectable({ providedIn: 'root' })
```

- `export class UserService {}`
-

2. Property Decorators

a) @Input()

- Lets a parent component pass data into a child component.
- Example:
- `@Input() userName: string;`
- `<app-user [userName]='Alice'></app-user>`

b) @Output() + EventEmitter

- Lets a child component emit events to a parent component.
- Example:
- `@Output() userClick = new EventEmitter<string>();`
- `<app-user (userClick)="onClick($event)"></app-user>`

c) @ViewChild()

- Accesses a child component/DOM element in the template.
- Example:

```
@ViewChild('submitButton') button: ElementRef;
```

- `<button #submitButton>Submit</button>`
-

3. Method Decorators

a) `@HostListener()`

- Listens to DOM events (like clicks, scrolls).
- Example:

```
@HostListener('click', ['$event'])
onClick(event: MouseEvent) {
  console.log('Button clicked!');
}
```

4. Parameter Decorators

a) `@Inject()`

- Manually specifies a dependency (useful for non-class tokens like API keys).
 - Example:
 - `constructor(@Inject('API_KEY') private apiKey: string) {}`
-

Why Decorators Matter

- ✓ Tell Angular what a class does (e.g., `@Component` = UI, `@Injectable` = service).
 - ✓ Simplify communication between components (`@Input/@Output`).
 - ✓ Enable dependency injection (`@Injectable`).
-

Interview Cheat Sheet

Decorator	Purpose	Example Use Case
@Component	Creates a component	UI pieces (buttons, forms)
@Input	Parent → Child data flow	Passing user data to a child
@Output	Child → Parent events	Emitting button clicks
@Injectable	Marks a DI-ready service	API calls, shared logic
@ViewChild	Access DOM/child components	Focusing an input field

Pro Tip:

- *"Decorators are Angular's way of saying, 'Hey, this class is special!'"*

Bonus: Angular uses TypeScript decorators, but they're compiled down to standard JavaScript.

9. How does Angular handle routing and navigation?

Angular's routing system enables SPA (Single Page Application) navigation without full

page reloads. Here's how it works:

1. Basic Setup

Step 1: Define Routes

In `app-routing.module.ts`

```
const routes: Routes = [

  { path: 'home', component: HomeComponent },

  { path: 'products', component: ProductsComponent },

  { path: 'products/:id', component: ProductDetailComponent }, // Dynamic route

  { path: '**', component: NotFoundComponent } ]; // Wildcard (404) route


@NgModule({

  imports: [RouterModule.forRoot(routes)],

  exports: [RouterModule]

})

export class AppRoutingModule {}
```

Step 2: Add Router Outlet

In `app.component.html`:

```
<router-outlet></router-outlet> <!-- Components render here -->
```

2. Navigation Methods

Template-Driven (HTML)

```
<a routerLink="/home">Home</a>
```

```
<a [routerLink]="['/products', productId]">Product Details</a>
```

Programmatic (TypeScript)

```
// Navigate to route
```

```
this.router.navigate(['/products']);
```

```
// Navigate with dynamic ID
```

```
this.router.navigate(['/products', id]);
```

```
// With query params
```

```
this.router.navigate(['/products'], {
```

```
  queryParams: { page: 1 },
```

```
  fragment: 'section-2'
```

```
});
```

3. Key Features

Feature	Implementation	Use Case
Dynamic Routes	<code>path: 'products/:id'</code>	Product detail pages
Query Params	<code>{ queryParams: { search: 'term' } }</code>	Search filters
Route Guards	<code>canActivate: [AuthGuard]</code>	Protect admin routes
Lazy Loading	<code>loadChildren: () => import(...)</code>	Optimize large apps
Wildcard Route	<code>path: '**'</code>	404 page handling

4. Advanced Concepts

Route Guards

`@Injectable()`

```
export class AuthGuard implements CanActivate {
```

```
  canActivate(): boolean {
```

```
    return this.authService.isLoggedIn(); // Returns true/false
```

```
}  
  
}
```

Usage:

```
{ path: 'admin', component: AdminComponent, canActivate: [AuthGuard] }
```

Lazy Loading

```
{  
  
  path: 'dashboard',  
  
  loadChildren: () =>  
  
    import('./dashboard/dashboard.module').then(m => m.DashboardModule)  
  
}
```

5. Interview Cheat Sheet

- ✓ RouterModule.forRoot() - Configures main routes
- ✓ routerLink - HTML navigation
- ✓ router.navigate() - Programmatic navigation
- ✓ Route Guards - Protect routes (auth, permissions)
- ✓ Lazy Loading - Boosts performance

Pro Tip:

"Angular routing is like a GPS for your SPA - it maps URLs to components without page reloads."

Common Pitfall:

Forgetting to add `<router-outlet>` in your root template!

Real-World Example

1. User clicks `<a routerLink="/products">Products</a>`
2. Angular:
 - Checks route definitions
 - Finds matching component (`ProductsComponent`)
 - Renders it in `<router-outlet>`
 - Updates URL without full page refresh

10. What are Angular modules, and why are they important?

What is an Angular Module?

An Angular module (`@NgModule`) is a container that groups related components, directives, pipes, and services. It helps organize the app into cohesive blocks of functionality.

Why Are Modules Important?

1. Organization

- Keeps the app structured (e.g., `AuthModule` for login, `AdminModule` for admin)

features).

- Avoids a messy, monolithic codebase.

2. Encapsulation

- Modules hide internal details (e.g., a `SharedModule` can export reusable components without exposing their logic).

3. Lazy Loading

- Load modules on-demand (improves performance):

```
{  
  path: 'dashboard',  
  loadChildren: () => import('./dashboard/dashboard.module').then(m =>  
    m.DashboardModule)  
}
```

4. Dependency Management

- Specifies what can be used across the app (via `imports`, `exports`, `providers`).

Key Parts of a Module

Property	Purpose	Example
<code>declarations</code>	Lists components/directives/pipes owned by the module.	<code>[LoginComponent, HighlightDirective]</code>
<code>imports</code>	Imports other modules (e.g., <code>CommonModule</code> , <code>RouterModule</code>).	<code>[FormsModule, HttpClientModule]</code>

<code>exports</code>	Makes components/directives available to other modules.	<code>[SharedComponent]</code>
----------------------	---	--------------------------------

<code>providers</code>	Registers services (DI). If <code>providedIn: 'root'</code> is used, this isn't needed.	<code>[UserService]</code>
------------------------	---	----------------------------

Types of Modules

1. Root Module (`AppModule`)
 - Bootstraps the app (only one per app).
 - Imports core modules like `BrowserModule`.
 2. Feature Modules
 - Organize features (e.g., `UserModule`, `CartModule`).
 3. Shared Modules
 - Reuse common components (e.g., `SharedModule` with buttons, pipes).
 4. Core Module
 - Houses app-wide singletons (e.g., `AuthService`).
-

Example: User Module

```
@NgModule({
  declarations: [UserComponent, UserListComponent], // Components/Directives
  imports: [CommonModule, UserRoutingModule], // Dependencies
  exports: [UserComponent] // What other modules can use
})

export class UserModule {}
```

Interview Cheat Sheet

- ✓ Modules = Containers for components/services.
- ✓ Lazy loading boosts performance.
- ✓ `declarations`: Private to the module.
- ✓ `exports`: Public API for other modules.

Pro Tip:

"Think of modules like boxes—each holds related pieces, and you decide what to share with other boxes."

Common Mistake:

Putting services in `exports` (they belong in `providers` or `providedIn: 'root'`).

When to Create a New Module?

- Building a large feature (e.g., `CheckoutModule`).
- Reusing shared components (e.g., `SharedModule`).
- Needing lazy loading.

11. What is Angular, and how is it different from AngularJS?

Angular vs. AngularJS - Key Differences

Angular (2+) and AngularJS (1.x) are both front-end frameworks by Google, but they are fundamentally different. Here's a simple breakdown:

1. Architecture

Angular (2+)

AngularJS (1.x)

Component-based architecture

MVC (Model-View-Controller)

Uses TypeScript (strict typing)

Uses JavaScript (dynamic typing)

Hierarchical dependency injection

Global dependency injection

Example:

AngularJS:

```
angular.module('app', []).controller('MyCtrl', function($scope) {  
    $scope.name = 'World';  
});
```

Angular:

```
@Component({ selector: 'app-root', template: 'Hello {{ name }}' })  
  
export class AppComponent { name = 'World'; }
```

2. Performance

Angular

AngularJS

Faster (AOT compilation, Ivy renderer)

Slower (dirty checking, digest cycle)

No digest cycle (uses change detection)

Uses \$digest cycle for updates

Why?

- Angular's AOT (Ahead-of-Time) compilation pre-compiles templates.
 - AngularJS relies on runtime two-way binding, which is slower.
-

3. Data Binding

Angular	AngularJS
One-way binding by default (<code>[prop]="value"</code>)	Two-way binding by default (<code>ng-model</code>)
Two-way binding requires <code>[(ngModel)]</code>	Uses <code>ng-model</code> directly

Example:

AngularJS (two-way):

- `<input ng-model="name"> <!-- Auto-syncs with $scope.name -->`

Angular (two-way):

- `<input [(ngModel)]="name"> <!-- Requires FormsModule -->`
-

4. Mobile Support

Angular	AngularJS
Designed for mobile (Ionic, NativeScript)	Not mobile-optimized
Smaller bundle size (better for slow networks)	Heavier

5. Learning Curve

Angular	AngularJS
Steeper (TypeScript, RxJS, modules)	Easier (plain JS)
More boilerplate (but better for large apps)	Less setup

6. Future & Support

Angular	AngularJS
Actively maintained (latest: Angular 17+)	End-of-life (no updates after Dec 31, 2021)
Used in modern apps (e.g., Gmail, Forbes)	Legacy apps only

When to Use Which?

- Choose Angular (2+) if:
 - ✓ Building new, complex apps.
 - ✓ Need performance + scalability.
 - ✓ Targeting mobile.
 - Choose AngularJS only if:
 - ✗ Maintaining old projects.
 - ✗ No time to migrate.
-

Summary Table

Feature	Angular (2+)	AngularJS (1.x)
Language	TypeScript	JavaScript
Architecture	Components	MVC
Binding	One-way (default)	Two-way (default)
Performance	Fast (AOT, Ivy)	Slow (digest cycle)
Mobile	Supported	Not optimized
Status	Active	Deprecated

Pro Tip:

"Angular is like a modern smartphone (fast, modular), while AngularJS is an old flip phone (works but outdated)."

12. What are components in Angular?

Components in Angular - Simplified Explanation

Components are the building blocks of Angular applications. They control a part of the screen (called a *view*) through:

- HTML template (what the user sees)
- TypeScript class (how it behaves)
- CSS styles (how it looks)

Key Characteristics of Components

1. Reusable
 - Like Lego blocks, you can reuse them across your app
 - Example: A `ButtonComponent` can be used in multiple screens
 2. Self-Contained
 - Each component manages its own:
 - Data
 - Layout
 - Behavior
 3. Hierarchical
 - Components nest inside other components
 - Forms a tree-like structure (component tree)
-

Basic Component Structure

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-greeting',           // HTML tag name
  template: `<h1>Hello {{name}}!</h1>`, // View
  styles: ['h1 { color: blue; }']     // Styles
})
export class GreetingComponent {
  name = 'Angular'; // Data
}
```

Usage in HTML:

```
<app-greeting></app-greeting>
```

Renders as:

```
<h1 style="color: blue;">Hello Angular!</h1>
```

Component Anatomy

Part	Purpose	Example
Selector	Custom HTML tag name	<code>selector: 'app-user'</code>
Template	Defines the HTML view	<code>template: `<div>{{user}}</div>`</code>
Styles	Component-scoped CSS	<code>styles: [red { color: red }]</code>
Class	Contains data and logic (TypeScript)	<code>export class UserComponent { ... }</code>

Why Components Matter

- ✓ **Modularity:** Break UI into manageable pieces
- ✓ **Reusability:** Use the same component in multiple places
- ✓ **Maintainability:** Isolate changes to specific components
- ✓ **Testability:** Easier to test individual parts

Component Lifecycle

Components go through stages - Angular provides hooks to tap into them:

1. `ngOnInit()` - Runs after component initializes (good for data fetching)
2. `ngOnChanges()` - Runs when input properties change
3. `ngOnDestroy()` - Runs before component is removed (good for cleanup)

Real-World Example

Imagine building a shopping app:

- `ProductComponent` - Shows one product
- `CartComponent` - Shows items in cart
- `NavbarComponent` - Top navigation bar

Each manages its own part of the interface while working together.

Interview Cheat Sheet

- "Components are the fundamental UI building blocks in Angular"
- Three key parts: Template (HTML), Class (TS), Styles (CSS)
- Key decorator: `@Component`
- Lifecycle hooks allow control over component behavior

Pro Tip:

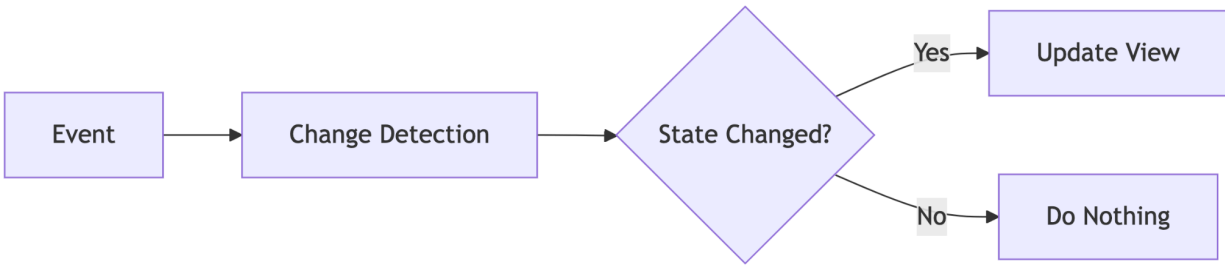
"Think of components like specialized widgets - each handles one specific job well."

13. How does change detection work in Angular?

Angular's change detection is the mechanism that updates the view when your data changes. Here's how it works in simple terms:

1. The Basic Flow

1. Trigger: An event occurs (click, timer, API response, etc.)
2. Check: Angular compares current and previous states
3. Update: If differences are found, the DOM updates



2. How It Detects Changes

- By Default: Checks all components from root to leaves (called "dirty checking")
 - Optimized: Uses Zone.js to automatically trigger checks after:
 - User events (clicks, inputs)
 - Timers (setTimeout, setInterval)
 - AJAX responses
-

3. Change Detection Strategies

Strategy	Behavior	When to Use
Default	Checks all components always	Simple apps
OnPush	Only checks when: • Inputs change • Events occur • Async pipe updates	Performance-critical apps

Example (OnPush):

```
@Component({
  selector: 'app-user',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `{{ user.name }}`
})
export class UserComponent {
  @Input() user: User;
}
```

4. Key Optimization Techniques

1. Immutable Data

- Helps `OnPush` detect changes faster

```
this.user = {...this.user, name: 'New Name'}; // Replace object
```

2. Async Pipe

- Automatically triggers change detection

```
<div>{{ data$ | async }}</div>
```

3. Detach/Reattach

- Manual control for complex scenarios

```
constructor(private cd: ChangeDetectorRef) {}

pauseDetection() {
  this.cd.detach();

}
```

5. Real-World Analogy

Think of change detection like a store inventory check:

- Default: Check every item every day (thorough but slow)
- OnPush: Only check items that were recently touched (faster)

Interview Cheat Sheet

- ✓ Triggered by: Events, timers, AJAX calls
- ✓ Default: Checks everything (root to leaves)
- ✓ OnPush: Only checks when inputs change
- ✓ Optimize with: Immutability, async pipe, detach/reattach

Pro Tip:

"Use `OnPush` + immutable data for large apps - it's like giving Angular a 'change map' instead of forcing it to search everywhere."

Common Gotcha:

Modifying objects directly (without immutability) can bypass `OnPush` detection!

Why It Matters

- Performance: Unnecessary checks slow down your app
- Predictability: Know exactly when views will update
- Control: Choose between automatic or manual detection

14. What is a service in Angular, and how do you use it?

A service in Angular is a reusable class that handles specific tasks like:

- Fetching data from APIs
- Sharing data between components
- Business logic (calculations, authentication)
- Logging and error handling

Key Features of Services

1. Singleton by Default
 - One instance shared across your entire app (when using `providedIn: 'root'`)
 2. Dependency Injection
 - Angular automatically provides services to components that need them
 3. Decouples Logic from Components
 - Keeps components lean and focused on UI
-

How to Create & Use a Service

1. Create a Service

```
// Generate with CLI: ng generate service data
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Makes it available app-wide
})
export class DataService {
  private data: string[] = [];

  addData(item: string) {
    this.data.push(item);
  }

  getData() {
    return this.data;
  }
}
```

2. Use in a Component

```
import { Component } from '@angular/core';
import { DataService } from '../data.service';
```

```

@Component({
  selector: 'app-user',
  template: `
    <button (click)="addItem()">Add Item</button>
    <div>{{ items | json }}</div>
  `,
})
export class UserComponent {
  items: string[] = [];

  constructor(private dataService: DataService) {} // Angular injects the service

  addItem() {
    this.dataService.addData('New Item');
    this.items = this.dataService.getData();
  }
}

```

Service vs Component

Service	Component
Handles data/logic	Handles UI
No	Has

template/styles	template/styles
-----------------	-----------------

Singleton (usually)	Multiple instances
---------------------	--------------------

When to Use Services

- ✓ Sharing data between components
- ✓ API communication
- ✓ Complex business logic
- ✓ State management
- ✓ Logging/analytics

Dependency Injection in Action

1. Register the service (with `@Injectable`)
2. Request it in a component constructor
3. Use its methods/properties

```
// Angular automatically does this:  
const service = new DataService();  
const component = new UserComponent(service);
```

Interview Cheat Sheet

- "Services are Angular's way to share logic and data across components"
- Key Decorator: `@Injectable()`
- Best Practice:

- Keep components focused on UI
 - Move reusable logic to services
- Common Services:
 - HttpClient for API calls
 - Custom services for app-specific needs

Pro Tip:

"Services are like specialized assistants - your components delegate work to them!"

15.What are pipes, and how do you create a custom pipe in Angular?

Pipes are small functions that transform data in templates. They format values directly in your HTML without changing the original data.

Built-in Pipes Examples

```
<!-- Uppercase -->
<p>{{ 'hello' | uppercase }}</p> <!-- Output: HELLO -->

<!-- Date formatting -->
<p>{{ today | date:'fullDate' }}</p> <!-- Output: Tuesday, June 14, 2023 -->

<!-- Currency -->
<p>{{ 25 | currency:'USD' }}</p> <!-- Output: $25.00 -->
```

Creating a Custom Pipe

1. Generate the Pipe

ng generate pipe reverse

This creates:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverse' // Use in templates as '| reverse'
})
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    return value.split('').reverse().join('');
  }
}
```

2. Use in Template

```
<p>{{ 'Angular' | reverse }}</p> <!-- Output: ralugnA -->
```

Key Pipe Features

Pure vs Impure Pipes

- Pure (default): Only runs when input changes
- Impure: Runs on every change detection (use cautiously)

```
@Pipe({
  name: 'filter',
  pure: false // Makes it impure
})
```

Chaining Pipes

```
{{ 'hello' | uppercase | slice:0:3 }}
```

<!-- Output: HEL -->

When to Create Custom Pipes

- ✓ Formatting data in specific ways
 - ✓ Repeating transformations across components
 - ✓ Complex data manipulation that belongs in templates
-

Interview Cheat Sheet

- "Pipes transform data for display without altering the source"
- Key Interface: `PipeTransform` with `transform()` method
- Pure Pipes: Better performance (default)
- Common Built-ins:
 - `date`, `uppercase`, `currency`, `json`

Pro Tip:

"Think of pipes as template filters - they clean up your data right before display."

Need an example with parameters? Here's one:

```
@Pipe({ name: 'excerpt' })
export class ExcerptPipe implements PipeTransform {
  transform(value: string, limit: number = 50): string {
    return value.length > limit ? value.slice(0, limit) + '...' : value;
  }
}
```

```
<p>{{ longText | excerpt:100 }}</p> <!-- Shows first 100 chars -->
```

16.What is RxJS, and how is it used in Angular?

RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using Observables. It's like "Lodash for async operations" - it helps you handle:

- API calls
 - User events
 - State changes
 - Any asynchronous operations
-

Key Concepts Used in Angular

1. Observables

Think of them as data streams that can emit multiple values over time:

```
import { Observable } from 'rxjs';

const data$ = new Observable(observer => {
  observer.next(1); // Emit value
  setTimeout(() => observer.next(2), 1000);
});

data$.subscribe(value => console.log(value));
// Output: 1 (immediate), 2 (after 1 second)
```

2. Common Angular Use Cases

Scenario	RxJS Solution
HTTP Requests	<code>HttpClient</code> returns Observables
User Input (debounce)	<code>fromEvent</code> + <code>debounceTime</code>

How Angular Uses RxJS

1. HTTP Requests

```
import { HttpClient } from '@angular/common/http';

@Injectable()
export class DataService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}

// Component usage
this.dataService.getUsers().subscribe(users => this.users = users);
```

2. Event Handling

```
import { fromEvent } from 'rxjs';
import { debounceTime } from 'rxjs/operators';

// Debounce search input
fromEvent(searchInput, 'input')
  .pipe(
    debounceTime(300)
  ).subscribe(() => this.search());
```

3. State Management

```
private _data = new BehaviorSubject<string>('default');
public data$ = this._data.asObservable();

updateData(newValue: string) {
  this._data.next(newValue);
}
```

Essential RxJS Operators

Operator	What It Does	Example
<code>map</code>	Transforms values	<code>obs.pipe(map(x => x * 2))</code>
<code>filter</code>	Filters values	<code>obs.pipe(filter(x => x > 5))</code>
<code>switchMap</code>	Cancels previous inner observable	<code>searchTerm\$.pipe(switchMap(term => http.get(term)))</code>
<code>catchError</code>	Handles errors	<code>obs.pipe(catchError(err => of([])))</code>
<code>takeUntil</code>	Auto-unsubscribe	<code>obs.pipe(takeUntil(this.destroy\$))</code>

Why RxJS in Angular?

- ✓ Handles async operations elegantly
 - ✓ Powerful operators for complex scenarios
 - ✓ Built-in support in Angular (`HttpClient`, `Router`, `Forms`)
 - ✓ Efficient change detection with `async` pipe
-

Interview Cheat Sheet

- "RxJS manages async operations via Observables"
- Key Players: Observables, Subjects, Operators
- Angular Integration:
 - `HttpClient` returns Observables
 - `async` pipe auto-subscribes in templates
- Memory Management: Always unsubscribe (use `takeUntil` or `async` pipe)

Pro Tip:

"Think of RxJS as a water pipe system - you can tap into streams, filter them, merge them, and control the flow."

Common Gotcha:

Forgetting to unsubscribe can cause memory leaks!

```
// Safe unsubscribe pattern
private destroy$ = new Subject();

ngOnInit() {
  this.data$.pipe(takeUntil(this.destroy$)).subscribe(...);
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();}
```

17. Explain the concept of observables and promises.

Observables vs Promises - Key Differences

Here's a clear comparison between these two async patterns in JavaScript/Angular:

1. Basic Concept

Feature	Observables (RxJS)	Promises
---------	--------------------	----------

Emission	Multiple values over time	Single value
Execution	Lazy (runs only when subscribed)	Eager (runs immediately)
Cancellation	Yes (<code>unsubscribe()</code>)	No
Operators	Powerful (map, filter, etc.)	Limited (then/catch)

2. Code Examples

Promise (Single Value)

```
const promise = new Promise(resolve => {
  setTimeout(() => resolve('Hello'), 1000);
});
promise.then(value => console.log(value)); // "Hello" after 1s
```

Observable (Multiple Values)

```
import { Observable } from 'rxjs';

const observable = new Observable(observer => {
  observer.next('Hello');
  setTimeout(() => observer.next('World'), 1000);
});

const subscription = observable.subscribe(value => console.log(value));
// Output: "Hello" (immediate), "World" (after 1s)
subscription.unsubscribe(); // Cancel the stream
```

3. Key Differences

Scenario	Observables	Promises
----------	-------------	----------

HTTP Requests	Can cancel mid-request	Runs to completion
User Input Handling	Perfect for click streams	Only handles single events
Value Transformation	Chainable operators (map/filter)	Only .then() chaining
Memory Management	Requires unsubscribe	Automatically resolved

4. When to Use Which?

- Use Promises when:
 - You need a single async operation (e.g., one API call)
 - You don't need cancellation
- Use Observables when:
 - Dealing with streams (multiple values)
 - Need advanced transformation (RxJS operators)
 - Require cancellation (HTTP requests, user flows)

5. Angular Context

- Promises: Older AngularJS (\$http), still usable but limited
- Observables: Modern Angular's standard (HttpClient returns Observables)

Pro Tip:

"Promises are like getting a single text message, while Observables are like a group chat that keeps updating."

Common Interview Question:

"Why does Angular prefer Observables over Promises?"

Answer: Observables provide better control over async operations, cancellation, and

advanced stream processing.

18. What are directives in Angular, and how are they used?

Directives are instructions that add special behavior to DOM elements. They're like "superpowers" you can give to HTML tags.

3 Types of Directives

1. Component Directives

- Components are directives with templates
- Example:

```
@Component({  
  selector: 'app-user',  
  template: `<h1>User Profile</h1>`  
  
  • })
```

2. Attribute Directives

- Change appearance/behavior of elements
- Examples:

```
<div [ngClass]="{'active': isActive}">Highlight me!</div>  
  
  • <p [ngStyle]="{'color': textColor}">Colored text</p>
```

3. Structural Directives

- Change DOM layout by adding/removing elements
- Examples:

```
<div *ngIf="showContent">Conditional content</div>
```

```
<li *ngFor="let item of items">{{item}}</li>
```

How to Create a Custom Directive

1. Generate a Directive

```
ng generate directive highlight
```

2. Implement Logic

```
import { Directive, ElementRef, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]' // Usage: <p appHighlight>...</p>
})
export class HighlightDirective {
  @Input() appHighlight = 'yellow'; // Default color

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor = this.appHighlight;
  }
}
```

3. Use It in Templates

```
<p appHighlight>Default yellow</p>
```

```
<p [appHighlight]='lightblue'>Custom color</p>
```

Key Differences

Feature	Components	Directives
---------	------------	------------

Template	Yes	No
Purpose	Create UI sections	Modify existing DOM
Example	<code><app-login-form></code>	<code>*ngIf</code> , <code>[ngClass]</code>

When to Use Directives

- ✓ Reusable DOM behavior without templates
 - ✓ Conditional rendering (`*ngIf`)
 - ✓ List rendering (`*ngFor`)
 - ✓ Custom visual effects (tooltips, hover effects)
-

Interview Cheat Sheet

- "Directives extend HTML's functionality"
- Three types: Component, Attribute, Structural
- Structural directives use asterisk (*)
- Custom directives need `@Directive` decorator

Pro Tip:

"Components are like custom HTML tags, while directives are like HTML attributes that add magic!"

Common Use Cases:

1. Show/hide elements (`*ngIf`)
2. Repeat templates (`*ngFor`)
3. Apply conditional classes (`[ngClass]`)

Need a real-world analogy? Think of directives like:

- 🔧 ngIf = A light switch (controls visibility)
- 🔄 ngFor = A copy machine (duplicates elements)
- 🎨 ngClass = A wardrobe changer (alters appearance)

19. What is the difference between structural and attribute directives?

Structural vs. Attribute Directives - Key Differences

Here's a clear breakdown of how these two directive types differ in Angular:

1. Primary Purpose

Structural Directives	Attribute Directives
Change the DOM layout by adding/removing elements	Change the appearance/behavior of existing elements

2. Syntax

```
<!-- Structural (uses *) -->
<div *ngIf="isVisible">Content</div>
<li *ngFor="let item of items">{{item}}</li>

<!-- Attribute (no asterisk) -->
<div [ngClass]="{'active': isActive}">Hello</div>
<p [appHighlight]="'yellow'">Text</p>
```

3. DOM Impact

Structural	Attribute
Creates/destroys DOM nodes	Works with existing DOM

nodes

Uses `<ng-template>` internally Modifies element properties

4. Common Examples

- Structural: `*ngIf`, `*ngFor`, `*ngSwitch`
- Attribute: `[ngClass]`, `[ngStyle]`, custom directives like `[appTooltip]`

5. Under the Hood

Structural Directive Example (`*ngIf`):

```
@Directive({ selector: '[ngIf]' })
export class NgIf {
  constructor(private _templateRef: TemplateRef<any>,
               private _viewContainer: ViewContainerRef) {}

  @Input() set ngIf(condition: boolean) {
    if (condition) {
      this._viewContainer.createEmbeddedView(this._templateRef);
    } else {
      this._viewContainer.clear();
    }
  }
}
```

Attribute Directive Example (`[ngClass]`):

```
@Directive({ selector: '[ngClass]' })
export class NgClass {
  constructor(private _el: ElementRef, private _renderer: Renderer2) {}

  @Input() set ngClass(classes: string) {
    // Adds/removes CSS classes to the element
  }
}
```

6. When to Use Which?

- Use structural when you need to:
 - Conditionally show/hide elements (`*ngIf`)

- Repeat templates (*ngFor)
- Use attribute when you need to:
 - Change styles dynamically ([ngStyle])
 - Add custom behavior (e.g., tooltips, hover effects)

Interview Cheat Sheet

✓ Structural = Changes DOM structure (uses *)

✓ Attribute = Modifies existing elements

✓ Key Difference: Structural directives manipulate the DOM tree, attribute directives enhance elements

Pro Tip:

"Structural directives are like construction workers (building/demolishing), while attribute directives are like interior decorators (enhancing what exists)."

Memory Aid:

🔨 Structural = Changes the house structure (walls/rooms)

🎨 Attribute = Changes the paint/furniture (appearance/function)

20. What is dependency injection, and why is it important in Angular?

DI is a design pattern where Angular automatically provides the dependencies (services, values, or objects) that a class needs, instead of having the class create them itself.

How DI Works in Angular

1. Define a Service (Dependency):

```
@Injectable({ providedIn: 'root' }) // Makes it injectable app-wide
export class DataService {
  getData() { return 'Data from service'; }
}
```

2. Request the Dependency (via Constructor):

```
@Component({...})
export class UserComponent {
  constructor(private dataService: DataService) {} // Angular injects the service
}
```

3. Use the Dependency:

```
ngOnInit() {
  this.data = this.dataService.getData(); // "Data from service"
}
```

Why DI is Important in Angular

1. Decoupling Components

- Components don't need to know how to create services.
- Example:

Without DI

```
// ❌ Manual creation (tight coupling)
```

```
export class UserComponent {
  dataService = new DataService(); // Problem: Hard to test/mock
}
```

With DI:

```
// ✅ Angular handles it (loose coupling)
```

```
constructor(private dataService: DataService) {}
```

2. Testability

- Easily mock dependencies during testing:

```
// In a test:
```

```
const mockService = { getData: () => 'Mock Data' };
```

- `const component = new UserComponent(mockService as DataService);`

3. Reusability

- Share services across multiple components:

```
// Same service in different components
```

```
constructor(private logger: LoggerService) {}
```

4. Maintainability

- Change service implementations in one place (provider config) instead of every component.
-

Key DI Concepts

Concept	Description	Example
Injector	Angular's system for managing dependencies	Created automatically
Provider	Tells Angular how to create a dependency	<code>{ provide: Logger, useClass: BetterLogger }</code>

Token	Identifier for a dependency (usually a class)	<code>LoggerService</code>
-------	---	----------------------------

Hierarchy	Injectors are hierarchical (component → module → root)	Child components can override parents
-----------	---	---------------------------------------

Real-World Analogy

Think of DI like a restaurant:

- Component = Customer (asks for food)
- Injector = Waiter (delivers food)
- Service = Kitchen (prepares food)

The customer doesn't need to know how the kitchen works, just how to order (declare dependencies).

Interview Cheat Sheet

- ✓ DI = Outsourcing dependency creation
- ✓ Key Decorator: `@Injectable()`
- ✓ Angular's DI System: Hierarchical injectors
- ✓ Benefits: Testability, maintainability, loose coupling

Pro Tip:

"DI is Angular's way of saying: 'Tell me what you need, I'll handle the rest.'"

Common Interview Question:

"What happens if you forget `@Injectable()`?"

Answer: The service won't be injectable (Angular throws an error). Always add it!

Performance Note:

Services with `providedIn: 'root'` are singletons (one instance app-wide), while component-level providers create new instances per component

21. How do you handle forms in Angular?

1. Template-Driven Forms

Best for: Simple forms with basic validation

How it works:

- Forms are defined in the template
- Uses `ngModel` for two-way binding
- Automatic form control creation

Example: Login Form

```
@Component({
  template: `
    <form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
      <input
        name="username"
        ngModel
        required
        minlength="3"
        #username="ngModel">

      <div *ngIf="username.invalid && username.touched">
```

```

        <small *ngIf="username.errors?.['required']">Required</small>
        <small *ngIf="username.errors?.['minlength']">Too short</small>
    </div>

    <button [disabled]="loginForm.invalid">Submit</button>
</form>
,
})
export class LoginComponent {
    onSubmit(form: NgForm) {
        console.log(form.value); // { username: '...' }
    }
}

```

Key Features:

- ✓ Quick setup
- ✓ Automatic form control creation
- ✓ Built-in directives (`ngModel`, `ngForm`)

2. Reactive Forms

Best for: Complex forms with dynamic validation

How it works:

- Forms are defined in the component class
- Explicit form control creation
- More control and testability

Example: Registration Form

```

@Component({
    template: `
        <form [formGroup]="regForm" (ngSubmit)="onSubmit()">
            <input formControlName="email">
            <div *ngIf="email.invalid && email.touched">
    `
})

```

```

        <small *ngIf="email.errors?.['required']">Required</small>
        <small *ngIf="email.errors?.['email']">Invalid email</small>
    </div>
</form>
,
}))
export class RegisterComponent implements OnInit {
    regForm: FormGroup;

    constructor(private fb: FormBuilder) {}

    ngOnInit() {
        this.regForm = this.fb.group({
            email: ['', [Validators.required, Validators.email]],
            password: ['', [Validators.required, Validators.minLength(6)]]
        });
    }

    get email() { return this.regForm.get('email'); }

    onSubmit() {
        console.log(this.regForm.value);
    }
}

```

Key Features:

- ✓ More control over form logic
- ✓ Dynamic form fields
- ✓ Easier unit testing

Key Differences

Feature	Template-Driven	Reactive
---------	-----------------	----------

Setup	Declarative (in template)	Imperative (in component)
Form Creation	Automatic	Manual
Validation	Template-based	Programmatic
Dynamic Controls	Difficult	Easy
Testability	Harder	Easier

Common Form Tasks

1. Validation

```
// Reactive
username: new FormControl('', [Validators.required])
```

```
// Template
<input name="username" ngModel required>
```

2. Dynamic Fields (Reactive Only)

```
addField() {
  this.regForm.addControl('newField', new FormControl(''));
}
```

3. Form Submission

```
onSubmit() {
  if (this.form.valid) {
    // Submit data
  }
}
```

}

Interview Cheat Sheet

- ✓ Template-Driven: Quick, simple, less code
- ✓ Reactive: Powerful, dynamic, better control
- ✓ Always import `FormsModule` (Template) or `ReactiveFormsModule` (Reactive)
- ✓ Key Classes:
 - `FormControl` (single field)
 - `FormGroup` (collection of controls)
 - `FormArray` (dynamic fields)

Pro Tip:

"Use template-driven for surveys, reactive for dashboards."

Bonus: For complex forms, combine reactive forms with the Angular Material form controls!

22. What is the difference between template-driven and reactive forms?

1. Form Model Creation

Template-Driven

Reactive Forms

Form model is implicitly created by Angular when you use directives like `ngModel` in the template

Form model is explicitly created in the component class using `FormControl`, `FormGroup`

```
html <input name="email" ngModel>
```

```
typescript this.form = new  
FormGroup({ email: new  
FormControl('') });
```

2. Data Flow

Template-Driven

Reactive Forms

Two-way binding with `[(ngModel)]` (data flows both ways)

One-way data flow (more predictable)

Changes propagate immediately between view and model

Changes are explicitly managed

3. Validation

Template-Driven

Reactive Forms

Validation defined through HTML attributes
(required, minlength)

```
html <input required minlength="3">
```

Validation defined programmatically in
component

```
typescript email: ['',  
[Validators.required,  
Validators.minLength(3)]]
```

4. Dynamic Form Controls

Template-Driven

Reactive Forms

Difficult to manage

Easy to add/remove controls dynamically

-

```
typescript  
this.form.addControl('newField', new  
FormControl(''));
```

5. Testability

Template-Driven

Reactive Forms

Harder to test (logic in template)

Easier to test (logic in
component)

6. Use Cases

Template-Driven

Reactive Forms

Simple forms (contact forms, login)

Complex forms (multi-step wizards, dynamic fields)

Quick prototypes

Enterprise applications

7. Performance

Template-Driven

Reactive Forms

Slightly less performant (uses directives)

More performant (direct access to form model)

When to Choose Which?

- Use Template-Driven if:
 - You need a simple form quickly
 - You're comfortable with HTML-based validation
 - You prefer two-way binding
- Use Reactive Forms if:
 - You need complex validation logic
 - You want dynamic form fields
 - Testability is important
 - You prefer immutable data patterns

Code Comparison

Template-Driven:

```
<form #form="ngForm" (ngSubmit)="submit(form)">  
  <input name="email" ngModel required email>  
  <button [disabled]="form.invalid">Submit</button>
```

```
</form>
```

Reactive Forms:

```
// In component
this.form = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email])
});

submit() {
  console.log(this.form.value);
}
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="email">
  <button [disabled]="form.invalid">Submit</button>
</form>
```

Interview Cheat Sheet

- ✓ Template-Driven = HTML-centric, quick setup
- ✓ Reactive Forms = Programmatic, more control
- ✓ Key Differences: Model creation, validation approach, dynamic controls
- ✓ Always import:
 - `FormsModule` for Template-Driven
 - `ReactiveFormsModule` for Reactive Forms

Pro Tip:

"Template-Driven is like automatic transmission, while Reactive Forms is manual - more control but more work."

Common Gotcha:

Mixing both approaches in the same form (don't do this!)

23. How do you implement routing in Angular?

1. Set Up the Routing Module

Generate and configure the routing module:

```
ng generate module app-routing --flat --module=app
```

Edit `app-routing.module.ts`:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';

const routes: Routes = [
  { path: '', component: HomeComponent },      // Default route
  { path: 'about', component: AboutComponent }, // /about
  { path: 'users/:id', component: UserComponent }, // Dynamic route
  { path: '**', component: NotFoundComponent } // Wildcard (404) route
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

2. Add Router Outlet

Place this in your root template (`app.component.html`):

```
<router-outlet></router-outlet>
<!-- Components render here -->
```

3. Navigation Methods

Template Navigation:

```
<a routerLink="/">Home</a>
<a routerLink="/about" routerLinkActive="active">About</a>
<a [routerLink]="['/users', userId]">Profile</a>
```

Programmatic Navigation:

```
import { Router } from '@angular/router';

constructor(private router: Router) {}

navigateToAbout() {
  this.router.navigate(['/about']);
}

navigateToUser(id: number) {
  this.router.navigate(['/users', id]);

  // With query params:
  this.router.navigate(['/users', id], {
    queryParams: { page: 1 },
    fragment: 'section-2'
  });
}
```

4. Route Guards (For Authentication)

Create a guard

ng generate guard auth

Implement the guard:

```
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isLoggedIn()) {
      return true;
    } else {

```



```

        this.router.navigate(['/login']);
        return false;
    }
}
}

```

Use in routes:

```

{
  path: 'admin',
  component: AdminComponent,
  canActivate: [AuthGuard]
}

```

5. Lazy Loading (For Performance)

```

{
  path: 'dashboard',
  loadChildren: () =>
    import('./dashboard/dashboard.module').then(m => m.DashboardModule)
}

```

6. Accessing Route Parameters

```

import { ActivatedRoute } from '@angular/router';

constructor(private route: ActivatedRoute) {}

ngOnInit() {
  // Get ID from /users/:id
  this.userId = this.route.snapshot.paramMap.get('id');

  // OR for observable updates:
  this.route.paramMap.subscribe(params => {
    this.userId = params.get('id');
  });

  // Get query params (?page=1)
  this.page = this.route.snapshot.queryParamMap.get('page');
}

```

Key Concepts

Feature	Implementation	Use Case
Basic Routes	<pre>{ path: '', component: HomeComponent }</pre>	Simple navigation
Dynamic Routes	<pre>path: 'users/:id'</pre>	Detail pages
Query Params	<pre>{ queryParams: { page: 1 } }</pre>	Filters, pagination
Route Guards	<pre>canActivate: [AuthGuard]</pre>	Authentication
Lazy Loading	<pre>loadChildren: () => import(...)</pre>	Performance optimization

Interview Cheat Sheet

- ✓ RouterModule.forRoot() - Main routing configuration
- ✓ <router-outlet> - Where components render
- ✓ routerLink - Template navigation
- ✓ router.navigate() - Programmatic navigation
- ✓ ActivatedRoute - Access route parameters

Pro Tip:

"Think of Angular routing like a GPS system - you define routes (destinations) and the router-outlet is where the 'map' displays your components."

Common Pitfalls:

- Forgetting to add `<router-outlet>`
- Not importing `RouterModule` in your feature modules
- Circular imports in lazy-loaded modules

24. What is lazy loading, and how does Angular support it?

What is Lazy Loading?

A technique that loads feature modules only when needed (typically when users navigate to their routes), reducing initial bundle size and improving app performance.

How Angular Implements Lazy Loading

1. Configure Routes for Lazy Loading

In your `app-routing.module.ts`:

```
const routes: Routes = [  
  {  
    path: 'dashboard',  
    loadChildren: () =>  
      import('./dashboard/dashboard.module').then(m => m.DashboardModule)  
  },  
  {  
    path: 'admin',  
    loadChildren: () =>  
      import('./admin/admin.module').then(m => m.AdminModule)  
  }  
];
```

2. Create a Feature Module with Routing

Generate a lazy-loaded module:

```
ng generate module dashboard --route dashboard --module app.module
```

This creates:

- `dashboard.module.ts` (with `RouterModule`)
- `dashboard-routing.module.ts` (with child routes)

3. Verify Lazy Loading Works

Check your browser's Network tab → JS files load only when navigating to `/dashboard` or `/admin`.

Key Benefits

Benefit	Impact
Faster initial load	Only core bundle downloads first
Bandwidth savings	Users don't download unused features
Better performance	Code splits automatically

How It Works Under the Hood

1. Webpack creates separate chunks for lazy modules
2. Angular router dynamically imports these chunks when needed
3. Modules are cached after first load

Advanced Configuration

Preloading Strategy

```
@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: PreloadAllModules // Loads lazy modules in background
  })],
  exports: [RouterModule]
})
```

Custom Preloading

```
@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: CustomPreloadingService
  })],
})
export class AppRoutingModule {}
```

Interview Cheat Sheet

- ✓ `loadChildren` = The key to lazy loading
- ✓ Feature modules must have their own routing
- ✓ Webpack handles chunk generation
- ✓ Preloading balances speed and UX

Pro Tip:

"Lazy loading is like ordering à la carte instead of a buffet - users only get what they need when they need it."

Common Gotchas:

1. Forgetting `RouterModule.forChild()` in feature modules
2. Circular dependencies in lazy modules
3. Putting eager-loaded dependencies in lazy modules

Verification Tip:

Build your app and check:

```
ng build --stats-json
```

Then analyze with [Webpack Bundle Analyzer](#) to see split chunks

25. Explain Angular's lifecycle hooks

Lifecycle hooks are special methods that Angular calls at specific moments during a component's existence, giving you control over its behavior. Here's a clear breakdown:

Component Lifecycle Timeline

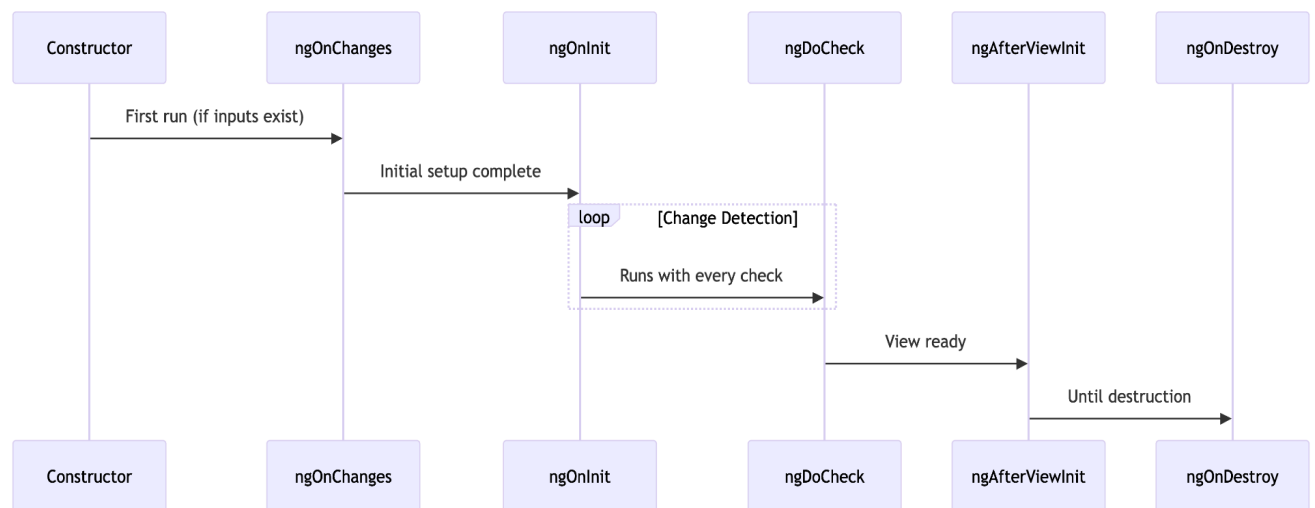
1. Creation Phase
 - `constructor()` (*Not a hook - runs first*)
 - `ngOnChanges()` (First actual hook)
 - `ngOnInit()`
 - `ngDoCheck()`
 2. Render Phase
 - `ngAfterContentInit()`
 - `ngAfterContentChecked()`
 - `ngAfterViewInit()`
 - `ngAfterViewChecked()`
 3. Destruction Phase
 - `ngOnDestroy()`
-

Key Hooks Explained

Hook	Timing	Typical Uses
<code>ngOnChanges()</code>	When input properties change	React to external input changes

<code>ngOnInit()</code>	After first <code>ngOnChanges()</code>	Initial setup, data fetching
<code>ngDoCheck()</code>	During every change detection	Custom change detection
<code>ngAfterContentInit()</code>	After projected content (ng-content) initializes	Content projection setup
<code>ngAfterViewInit()</code>	After component's view initializes	DOM manipulation, view references
<code>ngOnDestroy()</code>	Before component is destroyed	Cleanup (unsubscribe, timers)

Visualization



Practical Examples

1. Data Fetching (ngOnInit)

```
ngOnInit() {  
  this.userService.getUser().subscribe(user => {  
    this.user = user;  
  });  
}
```

2. Input Changes (ngOnChanges)

```
@Input() userId: string;  
  
ngOnChanges(changes: SimpleChanges) {  
  if (changes.userId) {  
    this.loadUser(changes.userId.currentValue);  
  }  
}
```

3. Cleanup (ngOnDestroy)

```
private destroy$ = new Subject();  
  
ngOnInit() {  
  this.dataService.getData()  
    .pipe(takeUntil(this.destroy$))  
    .subscribe();  
}  
  
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

When to Use Which Hook

Scenario	Recommended Hook
Initial data loading	ngOnInit
Reacting to input changes	ngOnChanges
DOM-dependent operations	ngAfterViewInit
Content projection setup	ngAfterContentInit
Memory cleanup	ngOnDestroy
Custom change detection	ngDoCheck

Interview Cheat Sheet

- ✓ Order Matters: Know the execution sequence
- ✓ ngOnInit vs constructor: Use constructor for DI, ngOnInit for initialization
- ✓ Cleanup is critical: Always unsubscribe in ngOnDestroy
- ✓ Performance: Avoid heavy operations in ngDoCheck

Pro Tip:

"Think of lifecycle hooks as milestones in your component's life - from birth (creation) to death (destruction)."

Common Gotchas:

1. Forgetting to call `super.ngOnInit()` when extending components

2. Modifying inputs during `ngOnChanges` causing infinite loops
3. Attempting DOM access before `ngAfterViewInit`

26. How do you optimize an Angular application for performance?

1. Bundle Optimization

- Lazy Loading: Split your app into feature modules

```
{  
  path: 'admin',  
  loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)  
}
```

- AOT Compilation: Enable Ahead-of-Time compilation
- `ng build --prod` *# Automatically uses AOT*

2. Change Detection Tuning

- OnPush Strategy: Reduce unnecessary checks

```
@Component({  
  changeDetection: ChangeDetectionStrategy.OnPush
```

- `})`
- Detach Change Detector: For static components

```
constructor(private cd: ChangeDetectorRef) {  
  this.cd.detach();
```

- `}`

3. Performance Killers to Avoid

- Memory Leaks: Always unsubscribe

```
ngOnDestroy() {  
  this.subscriptions.unsubscribe();
```

- }
- Heavy Computations in Templates: Use pure pipes
- `@Pipe({ name: 'heavyCalc', pure: true })`

4. UI Optimization

- Virtual Scrolling: For large lists

```
<cdk-virtual-scroll-viewport itemSize="50">
  <div *cdkVirtualFor="let item of items">{{item}}</div>

  </cdk-virtual-scroll-viewport>
  • TrackBy Function: Prevent entire list re-renders
  • <div *ngFor="let item of items; trackBy: trackById">
```

5. Network Performance

- Preloading Strategy: Load lazy modules in background
- RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })
- Service Worker: Enable PWA caching
- `ng add @angular/pwa`

6. Runtime Optimizations

- Web Workers: Offload heavy computations
- `const worker = new Worker('./app.worker', { type: 'module' });`
- Ivy Renderer: Ensure you're using latest Angular

```
// angular.json
"projects": {
  "architect": {
    "build": {
      "options": {
        "enableIvy": true
      }
    }
  }
}
```

}

Performance Audit Tools

1. Angular DevTools (Chrome Extension)
2. Webpack Bundle Analyzer

```
npm run build --stats-json
```

3. npx webpack-bundle-analyzer dist/stats.json
4. Lighthouse (Chrome DevTools)

Quick Wins Checklist

- ✓ Enable production mode
- ✓ Use OnPush change detection
- ✓ Implement lazy loading
- ✓ Add trackBy to *ngFor
- ✓ Remove console.log in production
- ✓ Optimize images (WebP format)
- ✓ Minimize change detection cycles

Pro Tip:

"Optimize early and measure often - small gains compound into significant improvements."

Critical Path:

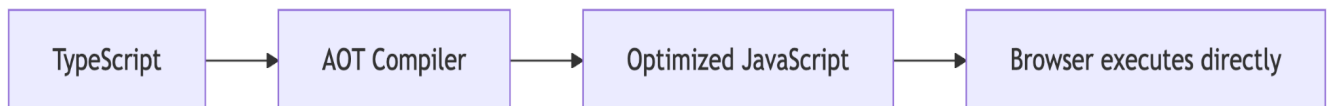
1. Audit with Lighthouse
2. Fix bundle size issues
3. Optimize change detection
4. Improve rendering performance

27. What is Ahead-of-Time (AOT) compilation?

AOT is Angular's build process that converts your Angular templates and components into highly optimized JavaScript code during the build phase (before the browser downloads and runs it).

How AOT Works (Step-by-Step)

1. At Build Time:
 - Angular compiler analyzes your components (@Component decorators)
 - Converts HTML templates into JavaScript instructions
 - Validates template syntax early (catches errors before production)
2. In the Browser:
 - The app loads pre-compiled templates
 - No template compilation happens in the browser
 - Immediate rendering without waiting for compilation



Key Benefits of AOT

Benefit	Impact
Faster Rendering	No client-side compilation needed

Smaller Bundle Size	Removes Angular compiler (≈50% size reduction)
Early Error Detection	Catches template errors during build
Better Security	Templates compile to JS (no eval)
Improved Tree Shaking	Dead code elimination works better

AOT vs JIT (Just-in-Time) Comparison

Feature	AOT	JIT
Compilation Timing	Build time	Runtime
App Size	Smaller	Larger (includes compiler)
Startup Time	Faster	Slower
Error Detection	Build-time	Runtime
Production Ready	Yes	No

How to Enable AOT

1. Production Build (Automatic)
2. `ng build --prod` *# Automatically uses AOT*
3. Development Testing
4. `ng serve --aot`
5. Manual Configuration

```
// angular.json
"configurations": {
  "production": {
    "aot": true,
    "optimization": true
  }
}
```

What Gets Compiled?

- All component templates (`templateUrl` and inline templates)
 - Component styles
 - Binding expressions (`{{ }}`, `[ ]`, `( )`)
 - Directives (`*ngIf`, `*ngFor`)
-

Common AOT Errors & Fixes

1. Template Syntax Errors

Error: Template parse errors:

2. Can't bind to 'ngModel' since it isn't a known property
Fix: Add `FormsModule` to your `NgModule` imports
3. Metadata Error
4. Function calls are not supported in decorators
Fix: Use exported functions or constants instead of inline logic

5. Private Member Access

- Property `'privateVar'` is private and only accessible within class
Fix: Make template-bound properties public
-

Interview Cheat Sheet

- ✓ AOT = Compile templates during build
- ✓ Key Advantage: Faster rendering + smaller bundles
- ✓ Always use in production (`--prod` flag enables it)
- ✓ Catches template errors early

Pro Tip:

"AOT is like pre-cooking meals (faster to serve) vs JIT's microwave approach (slower but flexible)."

Critical Notes:

- AOT requires strict template validation
- Some dynamic patterns won't work (use `ViewContainerRef` for advanced cases)
- Third-party libs must be AOT-compatible

28. How do you handle error handling in Angular?

1. HTTP Error Handling

Best Practice: Use interceptors for consistent API error handling

```
@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>>
  {
    return next.handle(req).pipe(
```



```

        catchError(error => {
            if (error.status === 401) {
                this.router.navigate(['/login']);
            }
            return throwError(() => error);
        })
    );
}
}

```

Register in AppModule:

```

providers: [
    { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true }
]

```

2. Global Error Handler

For uncaught exceptions:

```

@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
    handleError(error: any) {
        console.error('Global Error:', error);
        // Send to error logging service
        this.errorService.log(error);
    }
}

```

```

// Register in AppModule
providers: [{ provide: ErrorHandler, useClass: GlobalErrorHandler }]

```

3. Component-Level Handling

For template errors:

```

<ng-container *ngIf="data$ | async as data; else error">
    <!-- Normal content -->

```

```
</ng-container>
```

```
<ng-template #error>
  <app-error-message></app-error-message>
</ng-template>
```

For observables:

```
this.dataService.getData().pipe(

  catchError(error => {
    this.showErrorAlert(error.message);
    return of([]); // Fallback value
  })
).subscribe();
```

4. Error Boundary Components

Create reusable error components:

```
@Component({
  selector: 'app-error-boundary',
  template: `
    <ng-content *ngIf="!hasError"></ng-content>
    <app-error-display *ngIf="hasError"></app-error-display>
  `,
})
export class ErrorBoundaryComponent {
  @Input() hasError = false;
}
```

Usage:

```
<app-error-boundary [hasError]="error$ | async">
  <!-- Protected content -->
</app-error-boundary>
```

5. Form Validation Errors

```
<input formControlName="email">
<div *ngIf="form.get('email').errors">
  <div *ngIf="form.get('email').errors.required">Required</div>
  <div *ngIf="form.get('email').errors.email">Invalid email</div>
</div>
```

6. RxJS Error Operators

Operator	Use Case
<code>catchError</code>	Handle and recover from errors
<code>retry</code>	Automatically resubscribe
<code>finalize</code>	Execute code regardless of success/error

```
this.apiService.getData().pipe(
  retry(2), // Retry twice
  catchError(handleApiError),
  finalize(() => this.loading = false)
)
```

7. Error Logging Services

```
@Injectable()
export class ErrorLoggingService {
  log(error: any) {
    // Send to:
    // - Console
    // - Server endpoint
    // - Third-party service (Sentry, Rollbar)
  }
}
```

```
    console.error('Logged error:', error);
    this.http.post('/api/logs', error).subscribe();
  }
}
```

Error Handling Strategy

1. Prevention: Type safety, validation
 2. Containment: Error boundaries, fallback UIs
 3. Recovery: Retry logic, alternative flows
 4. Reporting: Logging, monitoring
 5. User Communication: Friendly error messages
-

Interview Cheat Sheet

- ✓ Interceptors for HTTP errors
- ✓ GlobalErrorHandler for uncaught exceptions
- ✓ catchError for observable streams
- ✓ Error boundaries for UI resilience
- ✓ Logging for debugging

Pro Tip:

"Handle errors at the right level - HTTP errors in interceptors, UI errors in components, unexpected errors globally."

Critical Practice:

Always provide fallback content when using `async` pipe to prevent template errors!

29. What is Angular Universal, and why is it used?

Angular Universal is a technology that enables server-side rendering (SSR) for Angular applications. It runs your Angular app on a Node.js server to generate static HTML pages that get sent to the browser first, before the client-side app takes over.

Key Reasons to Use Angular Universal

Problem	Without SSR	With SSR
SEO	Search engines struggle with client-only apps	Perfectly crawlable HTML
Performance	Blank screen until JS loads	Immediate content display
Social Sharing	Metadata not available for scrapers	Rich previews work
Low-bandwidth Users	Slow time-to-content	Fast first paint

How It Works (Step-by-Step)

1. User Requests Page
 - Browser hits your Node.js server
2. Server Renders
 - Angular Universal generates complete HTML
 - Includes fetched data (APIs called server-side)

3. Initial Delivery

- Fully-rendered page sent to browser

```
<app-root _nghost-abc="">
  <div _ngcontent-abc="">Actual rendered content</div>

</app-root>
```

4. Client Takeover

- Angular bootstraps and becomes interactive
- Subsequent navigation is client-side

Implementation Guide

1. Add Universal

```
ng add @nguniversal/express-engine
```

This creates:

- `server.ts` (Node.js server)
- `app.server.module.ts` (SSR entry point)

2. Universal-Specific Code

```
// Service with dual behavior

@Injectable()
export class DataService {
  constructor(
    private http: HttpClient,
    @Inject(PLATFORM_ID) private platformId: Object
  ) {}

  getData() {
    if (isPlatformServer(this.platformId)) {
      // Server-side fetch (absolute URL)
    }
  }
}
```

```
    return this.http.get('http://localhost:3000/api/data');
  } else {
    // Client-side fetch (relative URL)
    return this.http.get('/api/data');
  }
}
}
```

3. Build and Run

```
npm run build:ssr # Build both client and server
```

```
npm run serve:ssr # Start Node server
```

When to Use Universal

Scenario	Benefit
Public Websites	SEO optimization
Content Sites	Faster perceived load
Social Media Apps	Proper link previews
E-commerce	Search engine visibility

Performance Comparison

Client-Side Only:

[Blank Page] → (JS downloads) → (API calls) → Content appears (3-5s)

With Universal

Content appears (0.5s) → (JS loads in background) → Becomes interactive (2s)

Interview Cheat Sheet

- ✓ Solves: SEO + Slow Initial Load
- ✓ Key Files: `server.ts`, `app.server.module.ts`
- ✓ Critical Methods: `transferState()` (avoid duplicate API calls)
- ✓ Deployment: Requires Node.js server

Pro Tip:

"Universal is like sending a printed document (SSR) that later becomes interactive (CSR), instead of sending blank paper with instructions."

Common Pitfalls:

1. Browser-specific APIs (`window`, `document`) break SSR
2. Memory leaks in server code
3. Double API calls without `TransferState`

Optimization Tip:

Use `ngx-is-platform-browser` to safely guard client-only code:

```
if (isPlatformBrowser(this.platformId)) {  
  // Safe to use window/document  
}
```


30. How does Angular handle HTTP requests?

1. Setup HttpClient

First import and inject the service:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule]
})
export class AppModule {}

@Component({...})
export class DataComponent {
  constructor(private http: HttpClient) {}
}
```

2. Making Requests

Angular supports all HTTP methods with RxJS Observables:

Method	Example
GET	<code>http.get('/api/data')</code>
POST	<code>http.post('/api/items', data)</code>
PUT	<code>http.put('/api/items/1', data)</code>
DELETE	<code>http.delete('/api/items/1')</code>
PATCH	<code>http.patch('/api/items/1', data)</code>

Example with Type Safety:

```
interface User {
  id: number;
  name: string;
}

// GET with typed response
this.http.get<User[]>('/api/users').subscribe(users => {
  this.users = users; // users is automatically typed as User[]
});
```

3. Request Configuration

Add headers, params, and more:

```
this.http.get('/api/data', {
  headers: new HttpHeaders({
    'Authorization': 'Bearer token'
  }),
  params: {
    page: '1',
    limit: '10'
  },
  observe: 'response' // Get full HttpResponse
}).subscribe(response => {
  console.log(response.headers);
});
```

4. Error Handling

Use RxJS operators for clean error handling:

```
this.http.get('/api/data').pipe(
  catchError(error => {
    console.error('Error:', error);
    return throwError(() => new Error('Custom error'));
  }),
  retry(2) // Retry failed requests
).subscribe();
```

5. Interceptors

Create interceptors for cross-cutting concerns:

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const authReq = req.clone({
      headers: req.headers.set('Authorization', 'Bearer token')
    });
    return next.handle(authReq);
  }
}

// Register in AppModule
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

Key Features

Feature	Benefit
Observable-based	Cancellation, retries, easy

	composition
Type-safe	Generic methods for response types
Interceptors	Centralized auth/error handling
Progress events	Track file uploads/downloads

Advanced Patterns

1. Caching Requests

```
private cache = new Map<string, any>();

getData(url: string) {
  if (this.cache.has(url)) {
    return of(this.cache.get(url));
  }
  return this.http.get(url).pipe(
    tap(data => this.cache.set(url, data))
  );
}
```

2. Concurrent Requests

```
forkJoin([
  this.http.get('/api/users'),
  this.http.get('/api/posts')
]).subscribe([users, posts]) => {
  // Handle both responses
});
```

Interview Cheat Sheet

- ✓ Always unsubscribe (or use `async pipe`)
- ✓ Use interceptors for cross-cutting concerns
- ✓ Type your responses for better safety
- ✓ Prefer `observe: 'response'` when needing full response

Pro Tip:

"Angular's HttpClient is like a Swiss Army knife - it handles everything from simple GETs to complex authenticated streaming requests."

Common Pitfalls:

1. Forgetting to import `HttpClientModule`
2. Not unsubscribing from long-running requests
3. Ignoring error handling scenarios

Performance Tip:

Use `shareReplay()` for frequently-requested data:

```
data$ = this.http.get('/api/data').pipe(shareReplay(1));
```

31. How do you use interceptors in Angular?

Interceptors are middleware that can modify HTTP requests and responses globally.

Here's how to use them effectively:

1. Creating an Interceptor

Generate and implement an interceptor:

```
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from
 '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>>
  {
    // Clone and modify the request
    const authReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${this.auth.getToken()}`
      }
    });

    return next.handle(authReq);}}

```

2. Registering the Interceptor

Add to your module providers:

```
@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true // Allows multiple interceptors
    }
  ]
})
export class AppModule {}

```

3. Common Use Cases

a. Authentication

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
    const authReq = req.clone({  
        headers: req.headers.set('Authorization', `Bearer ${token}`)  
    });  
    return next.handle(authReq);  
}
```

b. Error Handling

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
    return next.handle(req).pipe(  
        catchError(error => {  
            if (error.status === 401) {  
                this.router.navigate(['/login']);  
            }  
            return throwError(() => error);  
        })  
    );  
}
```

c. Loading Indicators

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
    this.loadingService.show();  
  
    return next.handle(req).pipe(  
        finalize(() => this.loadingService.hide())  
    );  
}
```

d. API URL Management

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
    const apiReq = req.clone({  
        url: `https://api.example.com/${req.url}`  
    });  
    return next.handle(apiReq);  
}
```

```
}
```

4. Multiple Interceptors

Angular applies interceptors in registration order:

```
providers: [  
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },  
  { provide: HTTP_INTERCEPTORS, useClass: LoggingInterceptor, multi: true },  
  { provide: HTTP_INTERCEPTORS, useClass: CacheInterceptor, multi: true }  
]
```

Execution order: Auth → Logging → Cache

5. Advanced Techniques

Conditional Interception

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  if (req.url.includes('/auth/')) {  
    return next.handle(req); // Skip for auth requests  
  }  
  return this.addToken(req, next);  
}
```

Request/Response Transformation

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  return next.handle(req).pipe(  
    map(event => {  
      if (event instanceof HttpResponse) {  
        return event.clone({  
          body: this.transformResponse(event.body)  
        });  
      }  
      return event;  
    })  
  )
```



```
);  
}
```

Key Features Table

Feature	Benefit	Example Use
Request Modification	Add headers, modify body	Authentication
Response Handling	Global error processing	Error notifications
Request Tracking	Monitor HTTP activity	Loading indicators
Caching	Reduce server calls	Cache GET requests

Interview Cheat Sheet

- ✓ Purpose: Middleware for HTTP requests/responses
- ✓ Registration: Multi-provider in module
- ✓ Execution Order: First registered, first executed
- ✓ Always: Clone requests before modification

Pro Tip:

"Interceptors are Angular's version of Express middleware - they let you inject logic into the HTTP pipeline."

Common Pitfalls:

1. Forgetting `multi: true` when registering
2. Not cloning requests (mutating original)
3. Infinite loops when intercepting auth refresh tokens

Performance Tip:

Use interceptors for:

- Request deduplication
- Response caching
- Automated retry logic

32. Explain the difference between NgModule and Component

Here's a clear breakdown of how these fundamental Angular concepts differ:

1. Primary Purpose

NgModule	Component
Organizes the application structure	Defines UI elements and behavior
Groups related functionality	Controls a specific view/template
Acts as a container	Acts as a building block

2. Key Characteristics

NgModule	Component
Uses <code>@NgModule</code> decorator	Uses <code>@Component</code> decorator

Declares components, pipes, directives	Contains HTML template + TypeScript logic
--	---

Imports other modules	Can be declared in modules
-----------------------	----------------------------

Provides services at module level	Uses services via dependency injection
-----------------------------------	--

3. Code Structure Comparison

NgModule Example:

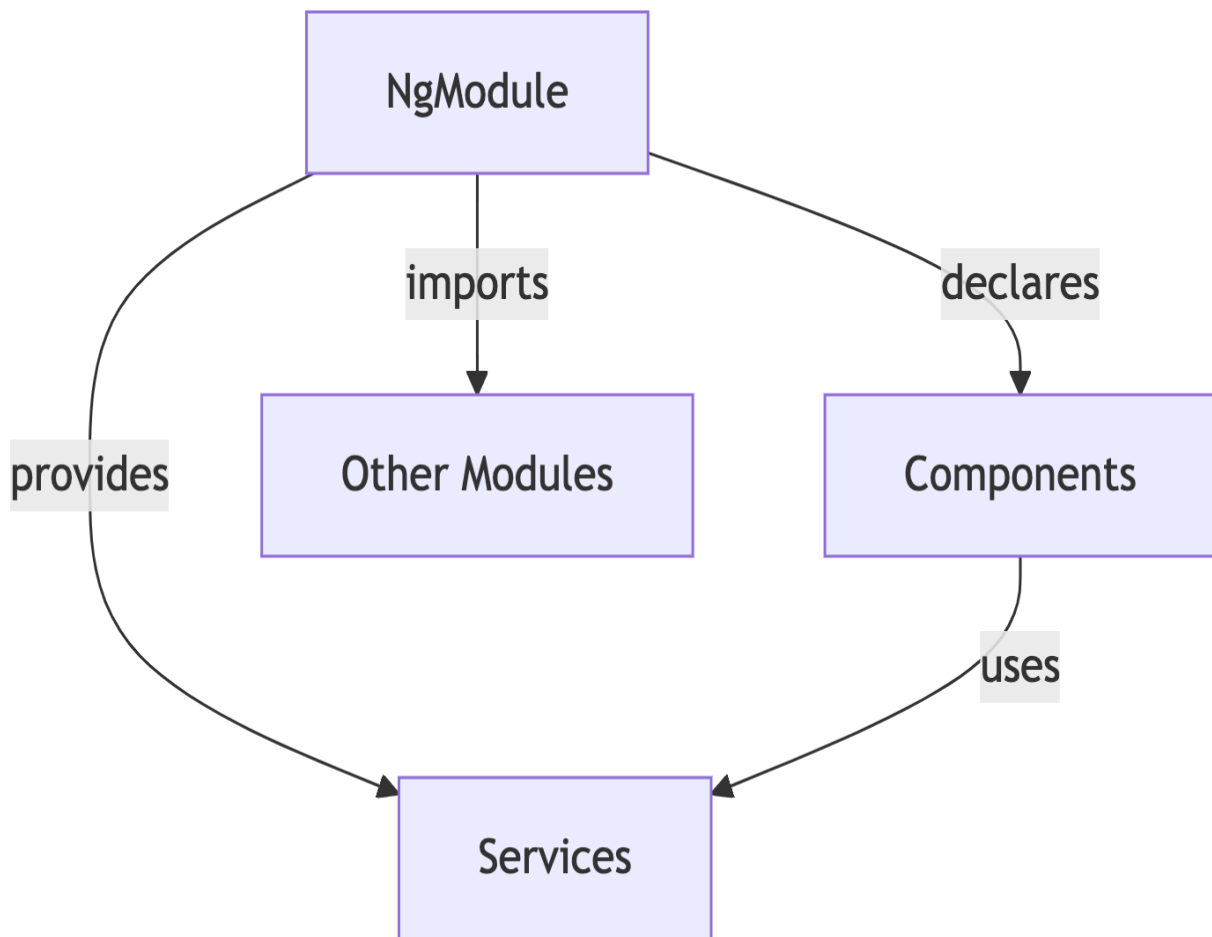
```
@NgModule({
  declarations: [AppComponent, UserComponent], // What belongs to this module
  imports: [BrowserModule, HttpClientModule], // What other modules we need
  providers: [UserService], // Services available in this module
  bootstrap: [AppComponent] // Root component
})
export class AppModule {}
```

Component Example:

```
@Component({
  selector: 'app-user', // HTML tag name
  templateUrl: './user.component.html', // View template
  styleUrls: ['./user.component.css'] // Component-specific styles
})
export class UserComponent {
  @Input() userId: string; // Input property
  @Output() selected = new EventEmitter(); // Output event

  constructor(private service: UserService) {} // Dependency injection
}
```

4. Relationship



5. When to Use Each

Use NgModule when you need to:

- Group related components/directives/pipes
- Define lazy-loaded features
- Configure dependency injection scope
- Import third-party libraries

Use Component when you need to:

- Create reusable UI pieces
- Handle user interactions
- Display data
- Manage a section of the view

6. Real-World Analogy

Think of an NgModule as a toolbox that contains:

- Components (tools)
- Services (instructions)
- Other toolboxes (imported modules)

A Component is like an individual tool (hammer, screwdriver) that:

- Has a specific purpose
- Can be used in different toolboxes
- Needs instructions (services) to work properly

Interview Cheat Sheet

- ✓ NgModule = Application architecture
- ✓ Component = UI building blocks
- ✓ NgModule contains components (not vice versa)
- ✓ Components use services, modules provide them

Pro Tip:

"Modules organize, components visualize. Modules are the stage, components are the actors."

Common Confusion Points:

1. Putting providers in components (creates new service instances)
2. Forgetting to declare components in modules
3. Circular module dependencies

33. What is Angular Ivy, and how does it improve the framework?

Angular Ivy is a complete rewrite of Angular's rendering engine that shipped as the default in Angular 9+. Here's why it's transformative:

Key Improvements

1. Smaller Bundle Sizes

- Generates more efficient code through local compilation
- Typical apps see 20-40% size reduction

```
// Before Ivy (View Engine)
```

```
function AppComponent() { ... }
```

```
// After Ivy
```

2. `ɵdefineComponent`({... });

3. Faster Compilation

- Incremental compilation (only recompiles changed components)
- Faster rebuilds during development

4. Better Debugging

- Enhanced stack traces with component names
- New `ng.getDirectives()` API for runtime inspection

```
// Debug any element in DevTools
```

5. `ng.getDirectives`(document.querySelector('app-user'));

6. Improved Type Checking

- Stricter template type checking catches errors earlier

```
<!-- Catches missing inputs at build time -->
```

7. `<app-user [nonExistentInput]="value"></app-user>`

8. New Features Enabled

- Lazy Loading Components (without modules)

9. `const { UserComponent } = await import('./user.component');`

- Higher Order Components (dynamic template creation)

Technical Breakthroughs

Feature	Benefit
Locality Principle	Compiles each component independently
Tree-Shakable	Removes unused framework parts
Generated Code	Human-readable output (easier debugging)

Migration Impact

- Backward Compatible: Works with existing apps
- Automatic: Enabled by default in Angular 9+
- No Code Changes Required (for most apps)

Pro Tip:

"Ivy is like replacing your app's engine while driving - you get better performance without stopping development."

Real-World Impact:

- A medium-sized app's production bundle shrank from 450KB to 290KB
- Rebuild times decreased from 4s to 1.5s during development

Current Status:

All Angular apps use Ivy since v9 (View Engine deprecated in v12)

34. What are the new features in Angular 18?

Angular 18: Key New Features (2024 Release)

Angular 18 introduces several powerful enhancements that improve performance,

developer experience, and capabilities. Here are the most significant updates:

1. Signals Now Stable

- Production-ready reactive primitive
- Full interoperability with RxJS

```
count = signal(0);  
doubleCount = computed(() => this.count() * 2);
```

2. Hydration Now Default

- Enhanced server-side rendering (SSR)
- Automatic DOM reconciliation
- 30-50% faster page loads in benchmarks

```
// No config needed - enabled by default  
bootstrapApplication(AppComponent, {  
  providers: [provideClientHydration()]  
});
```

3. Zone.js Optional

- Experimental support for zone-less apps
- Better performance for event-heavy app

```
bootstrapApplication(AppComponent, {  
  
  providers: [provideExperimentalZonelessChangeDetection()]  
});
```

4. Improved DevTools

- New reactive components inspector
- Signal dependency visualization
- Enhanced component boundary highlighting

5. Deferrable Views (Stable)

- Lazy-load components with simple syntax


```
@defer (on viewport) {  
  <heavy-component />  
} @placeholder {  
  <div>Loading...</div>  
}
```

6. Material 3 Support

- Modern Material Design components
- Dynamic color theming

```
import { provideMaterialDesign3 } from '@angular/material';
```

```
bootstrapApplication(AppComponent, {  
  providers: [provideMaterialDesign3()]  
});
```

7. Enhanced Forms

- Typed reactive forms improvements
- New `requiredInput()` decorator

```
@Input({ required: true }) id!: string;  
// OR  
@Input({ transform: booleanAttribute }) disabled = false;
```

8. Performance Boosts

- 15% faster component creation
- Improved tree-shaking (smaller bundles)
- Optimized change detection

9. New CLI Commands

```
ng generate @angular/core:control-flow # Migrate to new syntax  
ng analyze --watch # Real-time bundle analysis
```

10. Documentation Updates

- New reactive patterns guide

- Enhanced SSR best practices
- Improved migration guides

Why Upgrade?

- ✓ 20-30% Smaller Bundles with better tree-shaking
- ✓ Faster Rendering with optimized change detection
- ✓ Simplified Reactivity with stable Signals
- ✓ Better SSR out-of-the-box

Pro Tip:

"Angular 18's Signals + zoneless option finally delivers React-like performance with Angular's structure."

Migration Path:

```
ng update @angular/core@18 @angular/cli@18
```

For existing apps, most features work without breaking changes, but some deprecated APIs were removed (check the [update guide](#)).

35. What is the purpose of Angular Elements?

Angular Elements is a powerful feature that allows you to package Angular components as custom web elements (also called custom elements), making them usable in any web environment—even outside Angular applications. Here's why it matters:

Primary Purpose

1. Framework Agnostic Usage
 - Embed Angular components in React, Vue, or plain HTML/JS projects

`<!-- Used in a non-Angular app -->`

2. `<my-angular-element></my-angular-element>`
 3. Micro Frontend Architecture
 - Integrate Angular components into larger ecosystems
 - Example: A shared header/navigation across multiple apps
 4. Progressive Enhancement
 - Gradually introduce Angular features to legacy systems
-

Key Features

Feature	Benefit	Example
Custom Elements	Browser-native component model	<code><app-counter></app-counter></code>
Self-Bootstrapping	No Angular app required	Works in static HTML
Input/Output Binding	Native property/event interface	<code>element.value = 5 /</code> <code>element.addEventListener('change')</code>
Shadow DOM Support	Style encapsulation	<code>ViewEncapsulation.ShadowDom</code>

Implementation Steps

1. Create an Angular Component

```
@Component({  
  selector: 'app-greeter',
```

```

    template: `

# Hello {{name}}!</h1>`, styles: [`h1 { color: blue; }`] }) export class GreeterComponent { @Input() name = 'World'; @Output() greeted = new EventEmitter(); }


```

2. Convert to Custom Element

```

import { createApplication } from '@angular/platform-browser';
import { createCustomElement } from '@angular/elements';

// Bootstrap and register
(async () => {
  const app = await createApplication({
    providers: [/* Your providers here */]
  });

  const greeterElement = createCustomElement(GreeterComponent, {
    injector: app.injector
  });

  customElements.define('app-greeter', greeterElement);
})();

```

3. Use Anywhere

```

<!-- Vanilla HTML -->
<app-greeter name="Angular"></app-greeter>

<script>
  document.querySelector('app-greeter')
    .addEventListener('greeted', (e) => console.log(e.detail));
</script>

```

Why Use Angular Elements?

Scenario	Solution
Legacy System Integration	Add Angular components to jQuery apps
Design System Sharing	Reusable components across teams
CMS/Portal Widgets	Dynamic content in WordPress/Drupal
Third-Party Distribution	Ship components as standalone scripts

Performance Considerations

- Bundle Size: ~25KB overhead (Angular + polyfills)
- Lazy Loading: Load elements on demand
- Zone.js: Can be excluded for lighter elements

Interview Cheat Sheet

- ✓ Purpose: Package Angular components as web components
- ✓ Key Method: `createCustomElement()`
- ✓ Browser Support: Needs polyfills for IE11 (via `@webcomponents/webcomponentsjs`)
- ✓ Best For: Micro frontends, cross-framework components

Pro Tip:

"Angular Elements turns components into 'Lego blocks' you can drop into any web project."

Limitations:

- Inputs must be primitive types (objects require JSON serialization)
- Change detection requires manual triggering in non-Angular environments

Example Use Case:

```
// Dynamic usage in React
function ReactWrapper() {
  const ref = useRef(null);

  useEffect(() => {
    ref.current.name = 'React'; // Set Angular input
  }, []);

  return <app-greeter ref={ref} />
}
```

36. What are Angular animations, and how do you implement them?

Angular provides a powerful animation system through the `@angular/animations` package, enabling smooth transitions and micro-interactions. Here's how to implement them effectively:

1. Basic Setup

First, import the animation module:

```
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
```

```
@NgModule({  
  imports: [BrowserAnimationsModule]  
})  
  
export class AppModule {}
```

2. Animation Syntax

Animations use a declarative DSL (Domain-Specific Language) with these key functions:

Function	Purpose	Example
<code>trigger()</code>	Defines an animation name	<code>trigger('fadeIn')</code>
<code>state()</code>	Sets style for a specific state	<code>state('visible', style({ opacity: 1 })))</code>
<code>transition()</code>	Defines state changes	<code>transition('void => *', [...])</code>
<code>animate()</code>	Specifies timing/duration	<code>animate('300ms ease-out')</code>
<code>keyframes()</code>	Complex multi-step animations	<code>animate('1s', keyframes([...]))</code>

3. Component Animation Example

```
import { trigger, state, style, transition, animate } from '@angular/animations';  
  
@Component({
```

```

animations: [
  trigger('slideInOut', [
    state('in', style({ transform: 'translateX(0)' })),
    transition('void => *', [
      style({ transform: 'translateX(-100%)' }),
      animate('300ms ease-in')
    ]),
    transition('* => void', [
      animate('300ms ease-out', style({ transform: 'translateX(100%)' })))
    ]
  ])
])
})
export class MyComponent {
  isVisible = false;
}

```

Template usage:

```

<div @slideInOut *ngIf="isVisible">
  Content slides in/out
</div>
<button (click)="isVisible = !isVisible">Toggle</button>

```

4. Advanced Techniques

A. Staggered Animations (Lists)

```

trigger('listAnimation', [
  transition('* => *', [
    query(':enter', [
      style({ opacity: 0 }),
      stagger(100, [
        animate('0.5s', style({ opacity: 1 })))
      ])
    ], { optional: true })
  ])
])

```


B. Route Transitions

```
trigger('routeAnimations', [  
  transition('Home <=> Detail', [  
    style({ position: 'relative' }),  
    query(':enter, :leave', [  
      style({  
        position: 'absolute',  
        width: '100%'  
      })  
    ]),  
    query(':enter', [style({ opacity: 0 })]),  
    query(':leave', animateChild()),  
    group([  
      query(':leave', [animate('300ms ease-out', style({ opacity: 0 })])),  
      query(':enter', [animate('300ms ease-in', style({ opacity: 1 })])])  
    ])  
  ])
])
```

5. Animation Control

Programmatic control with @ViewChild:

```
@ViewChild('myElement', { read: AnimationPlayer }) player!: AnimationPlayer;  
  
startAnimation() {  
  this.player.play();  
}
```

Template:

```
<div @myAnimation #myElement="animation"></div>
```

Key Benefits

- ✓ Performance Optimized: Uses Web Animations API
 - ✓ Declarative Syntax: Clear state-based definitions
 - ✓ CSS-Free: Works with ViewEncapsulation
 - ✓ Testing Support: Mockable animation provider
-

Interview Cheat Sheet

- Core Module: `@angular/animations`
- Key Functions: `trigger()`, `state()`, `transition()`, `animate()`
- States: Defined ('active') vs. wildcard ('*') vs. void ('void')
- Timing Syntax: `'300ms 500ms ease-in-out'` (duration delay easing)

Pro Tip:

"Use Angular animations for stateful UI transitions, CSS animations for simpler effects."

Common Pitfalls:

1. Forgetting `BrowserAnimationsModule` import
2. Overusing complex animations (hurts performance)
3. Not testing reduced-motion preferences

Performance Tip:

```
@Component({
  host: { '@.disabled': 'true' } // Disable all animations
})
export class PerfModeComponent {}
```

37. What is view encapsulation in Angular?

View encapsulation determines how component styles are applied and scoped in Angular. It's a core feature that prevents CSS conflicts across components.

1. Encapsulation Modes

Angular provides three encapsulation strategies:

Mode	Behavior	Usage	Generated CSS
Emulated (Default)	Scopes styles to component using attributes	Most common case	<code>.my-class[_ngcontent-ab c123]</code>
ShadowDom	Uses browser's native Shadow DOM	Maximum isolation	Styles truly scoped to component
None	Global styles (no encapsulation)	Legacy/overri de cases	<code>.my-class</code> (applies globally)

```
@Component({  
  encapsulation: ViewEncapsulation.ShadowDom // or Emulated/None  
})
```

2. How It Works

- Emulated: Adds unique attributes to elements and styles

```
<div _ngcontent-abc123 class="header"></div>  
  
<style>.header[_ngcontent-abc123] { ... }</style>
```

- ShadowDom: Creates a DOM subtree with isolated styles

```
#shadow-root  
<style>.header { ... }</style>  
  
<div class="header"></div>
```

- None: Styles leak to other components

```
/* Affects ALL .header classes in app */
```

```
.header { color: red; }
```

3. Practical Examples

A. Default (Emulated)

```
@Component({  
  styles: [`.header { color: blue; }`] // Only affects this component  
})  
export class MyComponent {}
```

B. Shadow DOM

```
@Component({  
  encapsulation: ViewEncapsulation.ShadowDom,  
  styles: [`:host { display: block; }`] // Targets component's host  
})
```

C. Global Override

```
@Component({  
  encapsulation: ViewEncapsulation.None,  
  styles: [`.error { color: red !important; }`] // App-wide style  
})
```

4. When to Use Which

Mode	Best For	Caveats
Emulated	Most components	Some global styles may leak

ShadowDom	Reusable widgets, design systems	Limited browser polyfill support
None	Style overrides, utilities	Risk of collisions

5. Special Selectors

```
:host           /* Targets component's outer element */
:host(.active)   /* Conditionally style host */
:host-context(.dark-theme) /* Based on ancestor */
::ng-deep       /* Breaks encapsulation (deprecated) */
```

Interview Cheat Sheet

- ✓ Default: `ViewEncapsulation.Emulated`
- ✓ ShadowDom: True isolation (check browser support)
- ✓ None: Use sparingly for global styles
- ✓ Key Benefit: Prevents CSS conflicts

Pro Tip:

"View encapsulation is like giving each component its own private CSS namespace."

Common Pitfalls:

1. Overusing `::ng-deep` (prefer component restructuring)
2. Assuming ShadowDom works everywhere (IE11 needs polyfills)
3. Forgetting `:host` display property (defaults to inline)

Performance Note:

Emulated encapsulation adds minimal overhead (~0.5ms per component), while ShadowDom has higher initial cost but better isolation.

38. How does Angular handle security concerns like XSS?

Angular provides robust built-in protections against Cross-Site Scripting (XSS) by default. Here's how it safeguards your applications:

1. Automatic Sanitization

Angular automatically sanitizes all data bindings to remove dangerous content:

Binding Type	Sanitization
<code>{{ }}</code> Interpolation	Escapes HTML
<code>[property]</code> Binding	Sanitizes based on target property
<code>[innerHTML]</code>	Cleans HTML (removes <code><script></code>)
<code>[style]/[class]</code>	Validates CSS values

```
// Component
dangerousHtml = '<script>alert("XSS")</script><b>Safe</b>';

// Template (script removed, bold preserved)
<div [innerHTML]="dangerousHtml"></div>
```

2. Content Security Policy (CSP) Support

Angular works seamlessly with CSP headers by:

- Avoiding `eval()` and `new Function()`
- Using strict inline style handling
- Supporting nonce-based CSP

Example CSP Header:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'
```

3. Security Contexts

Angular classifies bindings into security contexts:

Context	Example	Protection Level
HTML	<code>innerHTML</code>	Full sanitization
Style	<code>[style.prop]</code>	URL/expression checks
URL	<code>[href]</code>	Protocol validation
Resource URL	<code><script src></code>	Strict allowlist

4. Explicit Bypass (When Needed)

For trusted content, use `DomSanitizer` carefully:

```
import { DomSanitizer } from '@angular/platform-browser';

constructor(private sanitizer: DomSanitizer) {}
```

```
safeHtml = this.sanitizer.bypassSecurityTrustHtml(
  '<b>Trusted</b> content'
);
<div [innerHTML]="safeHtml"></div>
```

Warning: Only bypass for content you fully control.

5. Template Security

- No dynamic template evaluation

// Blocked by Angular

- `template: '<div>${userInput}</div>'` // Throws error
 - Strict template compiler catches suspicious patterns
-

6. HTTP-Level Protections

- Automatic CSRF (XSRF) protection for `HttpClient`

// Automatically adds XSRF-TOKEN cookie to headers

- `this.http.post('/api', data);`
-

Key Security Practices

1. Never concatenate strings for templates/URLs
 2. Always sanitize user-generated content
 3. Use Angular APIs instead of direct DOM manipulation
 4. Audit dependencies (even safe-listed ones)
 5. Enable CSP in production
-

Interview Cheat Sheet

- ✓ Automatic Escaping: All bindings are safe by default
- ✓ Sanitization API: `DomSanitizer` for trusted content
- ✓ CSP Ready: Works with strict security policies
- ✓ CSRF Protection: Built into `HttpClient`

Pro Tip:

"Angular treats all data as untrusted by default - it's like having a bouncer for your templates."

Common Vulnerabilities to Avoid:

```
// ❌ Dangerous
elementRef.nativeElement.innerHTML = userInput;
window.location.href = userUrl;

// ✓ Safe
this.sanitizer.bypassSecurityTrustUrl(verifiedUrl);
this.renderer.setProperty(el, 'innerHTML', sanitizedHtml);
```

Security Audit Tools:

1. Angular's built-in template compiler warnings
2. OWASP ZAP
3. `npm audit` for dependencies
4. CSP validator

39. How does Angular handle state management?

Angular offers multiple state management solutions depending on application complexity. Here's a breakdown of the most effective strategies:

1. Built-in Solutions

A. Services + RxJS (Recommended for most apps)

```
@Injectable({ providedIn: 'root' })
export class StateService {
  private _state = new BehaviorSubject<State>(initialState);
  state$ = this._state.asObservable();

  updateState(newState: Partial<State>) {
    this._state.next({ ...this._state.value, ...newState });
  }
}
```

✓ Pros: Simple, no external libs

✗ Cons: Manual boilerplate for large apps

B. Component State (Local)

```
@Component({
  template: `{{ count() }}`,
})
export class CounterComponent {
  count = signal(0); // Angular 16+ Signals
  increment() { this.count.update(v => v + 1); }
}
```

2. External Libraries

A. NgRx (Redux Pattern)

```
// Actions
export const increment = createAction('[Counter] Increment');

// Reducer
export const reducer = createReducer(
```

```

    initialState,
    on(increment, state => ({ ...state, count: state.count + 1 })))
);

```

// Componen

```

this.store.dispatch(increment());

```

✓ Pros: Predictable state, time-travel debugging

✗ Cons: Steep learning curve, boilerplate

B. NGXS (Simpler Alternative)

```

@State<StateModel>({
  name: 'counter',
  defaults: { count: 0 }
})
@Injectable()
export class CounterState {
  @Action(Increment)
  increment(ctx: StateContext<StateModel>) {
    ctx.setState({ count: ctx.getState().count + 1 });
  }
}

```

C. Akita (OOP Style)

```

@StoreConfig({ name: 'todos' })
export class TodosStore extends EntityStore<TodosState> {
  constructor() { super(initialState); }
}

```

3. Modern Signals (Angular 16+)

```

export class CartService {
  private _items = signal<Item[]>([]);
  items = this._items.asReadonly();

  addItem(item: Item) {
    this._items.update(items => [...items, item]);
  }
}

```

```
}  
}
```

✓ Pros: Fine-grained reactivity, no subscriptions

✗ Cons: New API, evolving best practices

State Management Comparison

Solution	Best For	Complexity	Learning Curve
RxJS Services	Small-medium apps	Low	Easy
Signals	New projects	Medium	Moderate
NgRx	Enterprise apps	High	Steep
NGXS	Balanced needs	Medium	Moderate
Akita	OOP lovers	Medium	Moderate

Key Implementation Patterns

1. Centralized vs Distributed State

```
// Global (app-wide)  
providedIn: 'root'
```

```
// Component-scoped
```

2. @Component({ providers: [StateService] })

3. Immutable Updates

```
// Good
this.state.set([...items, newItem]);
```

```
// Bad (mutation)
```

```
4. this.state.value.push(newItem);
```

5. Memoized Selectors

```
// NgRx
export const selectFilteredItems = createSelector(
  selectAllItems,
  items => items.filter(...)
);
```

```
// Signals
```

```
6. filteredItems = computed(() => this.items().filter(...));
```

Interview Cheat Sheet

- ✓ Start Simple: RxJS services for most cases
- ✓ Signals: Future-proof for new apps
- ✓ NgRx: When you need Redux patterns
- ✓ Key Principle: Unidirectional data flow

Pro Tip:

"Choose state management like a car - compacts (Services) for cities, SUVs (NgRx) for rugged terrain."

Common Pitfalls:

1. Over-engineering small apps
2. Not unsubscribing from observables

3. Deep object mutations causing change detection issues

Performance Tip:

```
// Use OnPush + async pipe for RxJS
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
template: `{{ data$ | async }}`
```

40. What are resolvers in Angular routing?

Route resolvers are services that pre-fetch data before a component loads, ensuring all required data is available when the route activates. Here's how they work:

1. Resolver Implementation

Create a resolver service implementing `Resolve<T>`:

```
import { Injectable } from '@angular/core';
import { Resolve, ActivatedRouteSnapshot } from '@angular/router';
import { Observable } from 'rxjs';
import { DataService } from '../data.service';

@Injectable({ providedIn: 'root' })
export class ProductResolver implements Resolve<Product> {
  constructor(private dataService: DataService) {}

  resolve(route: ActivatedRouteSnapshot): Observable<Product> {
    const productId = route.paramMap.get('id');
    return this.dataService.getProduct(productId);
  }
}
```

2. Route Configuration

Attach the resolver to routes:

```
const routes: Routes = [
  {
    path: 'product/:id',
    component: ProductDetailComponent,
    resolve: {
      product: ProductResolver // Key matches component input
    }
  }
];
```

3. Accessing Resolved Data

In your component:

```
@Component({...})
export class ProductDetailComponent implements OnInit {
  // Via ActivatedRoute
  constructor(private route: ActivatedRoute) {
    this.product = this.route.snapshot.data['product'];
  }

  // OR as input (Angular 16+)
  @Input() product!: Product;
}
```

Key Features

Feature	Benefit	Example
Pre-loading	No "loading" state	Smooth UX

	flashes	
Error Handling	Block navigation on errors	Automatic redirects
Route Reuse	Cache resolved data	Faster back navigation
Parallel Resolves	Multiple data sources	<code>resolve: { user: UserResolver, posts: PostsResolver }</code>

Advanced Patterns

1. Error Handling

```
resolve(): Observable<Product> {
  return this.dataService.getProduct().pipe(
    catchError(error => {
      this.router.navigate(['/error']);
      return EMPTY;
    })
  );
}
```

2. Dynamic Parameters

```
resolve(route: ActivatedRouteSnapshot) {
  return this.dataService.search(
    route.queryParams['term'],
    route.data['searchType']
  );
}
```

3. Custom Loading Indicator

```
resolve() {
```



```
this.loadingService.show();  
return this.data.load().pipe(  
  finalize(() => this.loadingService.hide())  
)  
}
```

When to Use Resolvers

Scenario	Solution
Required data before render	Resolver
Optional/background data	Component ngOnInit
Route-dependent permissions	Resolver + Guard combo

Alternatives Comparison

Method	Pros	Cons
Resolvers	Clean components, no loading states	Blocks navigation
Component Loading	Flexible error handling	Requires loading UI
Route Guards	Can cancel navigation	Limited data return

Interview Cheat Sheet

- ✓ Purpose: Pre-fetch route data
- ✓ Interface: Implement `Resolve<T>`
- ✓ Access Data: `route.snapshot.data` or component inputs
- ✓ Best For: Critical path data

Pro Tip:

"Resolvers act like bouncers - they verify data is ready before letting your component through the door."

Common Pitfalls:

1. Forgetting to provide the resolver
2. Not handling errors (blocks navigation)
3. Overusing for non-critical data

Performance Tip:

Combine with route reuse strategies:

```
@NgModule({  
  imports: [RouterModule.forRoot(routes, {  
    onSameUrlNavigation: 'reload'  
  })]  
})
```

41. How do you debug an Angular application?

Debugging Angular apps requires a mix of browser tools, Angular-specific utilities, and systematic approaches. Here's a comprehensive guide:

1. Browser DevTools Essentials

A. Elements Tab

- Inspect component boundaries (AngryJS extension highlights Angular components)
- Check applied styles with `ViewEncapsulation`

B. Console Tricks

```
// Access Angular context
ng.probe($0).componentInstance // Current selected component
ng.getDirectives(document.querySelector('app-root'))
```

C. Network Tab

- Monitor HTTP calls (check if HttpClient is used properly)
 - Verify lazy-loaded chunks
-

2. Angular-Specific Tools

A. Angular DevTools (Official Chrome Extension)

- Component explorer with input/output bindings
- Router tree visualization
- Dependency injection graph

B. Augury (Legacy)

- Alternative for older Angular versions
-

3. Code-Based Debugging

A. Template Debugging

```
<!-- Temporary debug output -->
```

```
<pre>{{ someObject | json }}</pre>
<div *ngIf="false">{{ debugValue }}</div>
```

B. Component Debugging

// Breakpoints in lifecycle hooks

```
ngOnInit() {
  debugger; // Pauses execution
  console.log(this.constructor.name, 'initialized');
}
```

// Property watchers

```
@Input()
set user(user: User) {
  console.log('User changed:', user);
  this._user = user;
}
```

C. RxJS Debugging

```
import { tap } from 'rxjs/operators';

this.dataService.getData().pipe(
  tap({
    next: val => console.log('DATA:', val),
    error: err => console.error('ERROR:', err)
  })
).subscribe();
```

4. Advanced Techniques

A. Change Detection Profiling

```
import { ApplicationRef } from '@angular/core';

constructor(appRef: ApplicationRef) {
  const originalTick = appRef.tick;
  appRef.tick = function() {
```

```

    console.time('ChangeDetection');
    originalTick.apply(this, arguments);
    console.timeEnd('ChangeDetection');
  };
}

```

B. Zone.js Debugging

```

// Track unexpected change detection
window.Zone.current.onInvokeTask = (delegate, curr, target, task, applyThis,
applyArgs) => {
  console.log('Task triggered:', task.source);
  return delegate.invokeTask(target, task, applyThis, applyArgs);
};

```

C. State Inspection

```

// For NgRx
window.store = this.store; // Access in console
store.subscribe(console.log)

// For Signals (Angular 16+)
window.signals = {
  count: this.counterService.count
};

```

5. Production Debugging

A. Source Maps

```

// angular.json
"sourceMap": {
  "scripts": true,
  "styles": true,
  "vendor": true
}

```

B. Error Tracking

- Sentry/Rollbar integration
- Custom error handler

```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any) {
    console.error('ERROR:', error);
    // Send to error tracking service
  }
}
```

Debugging Checklist

1. Reproduce: Identify consistent steps
 2. Isolate: Create minimal reproduction
 3. Inspect: Check console errors/warnings
 4. Trace: Follow data flow (inputs → outputs)
 5. Verify: Check change detection triggers
-

Interview Cheat Sheet

- ✓ Angular DevTools - Component hierarchy inspection
- ✓ Augury - Alternative debugger (legacy)
- ✓ Zone.js - Track async operations
- ✓ Source Maps - Debug production code

Pro Tip:

"Debug Angular like solving a mystery - follow the data trail from templates to services."

Common Pitfalls:

1. Forgetting `ChangeDetectionStrategy.OnPush` side effects
2. RxJS subscription leaks
3. Zone.js triggered by third-party libs

Performance Tip:

```
// Disable change detection during debug
import { ChangeDetectorRef } from '@angular/core';

constructor(private cd: ChangeDetectorRef) {
  this.cd.detach();
  // Manual checks:
  // this.cd.detectChanges();
}
```

42. What is the difference between ngIf and hidden attributes?

While both control element visibility, they work fundamentally differently:

1. DOM Presence

Directive	Behavior	Memory Impact
<code>*ngIf</code>	Removes/adds from DOM	Lightweight (no DOM nodes when hidden)
<code>[hidden]</code>	Toggles <code>display: none</code>	Maintains DOM (memory retained)

```
<!-- ngIf: No DOM element when false -->
```

```
<div *ngIf="isVisible">Content</div>
```

```
<!-- hidden: Element exists but hidden -->
```

```
<div [hidden]="!isVisible">Content</div>
```

2. Performance Implications

Scenario	Better Choice	Why
----------	---------------	-----

Frequent toggling	<code>[hidden]</code>	Avoids DOM thrashing
Heavy components	<code>*ngIf</code>	Releases memory when hidden
Initial render cost	<code>[hidden]</code>	Prevents layout shifts

3. Style Interactions

```
<!-- ngIf: Component lifecycle triggers -->
<div *ngIf="show" @fadeAnimation>...</div>

<!-- hidden: CSS transitions work -->
<div [hidden]="!show" class="fade-transition">...</div>
```

4. Component Behavior

Directive	Component Lifecycle
<code>*ngIf</code>	Triggers <code>ngOnInit/ngOnDestroy</code>
<code>[hidden]</code>	No lifecycle changes

5. Accessibility (a11y)

- `[hidden]` respects `aria-*` attributes when hidden
- `*ngIf` removes all ARIA attributes when false

When to Use Which?

✅ Use `*ngIf` when:

- Dealing with heavy components (releases memory)
- Conditional includes (e.g., permission-based elements)
- Reducing DOM size for performance

✓ Use `[hidden]` when:

- Preserving form state (input values)
- CSS transitions/animations needed
- Frequently toggling visibility (>5x/sec)

Code Examples

NgIf (Structural Directive)

```
@Component({
  template: `
    <button (click)="toggle()">Toggle</button>
    <expensive-component *ngIf="isVisible"></expensive-component>
  `,
})
export class MyComponent {
  isVisible = false;
  toggle() { this.isVisible = !this.isVisible; }
}
```

Hidden (Attribute Binding)

```
@Component({
  styles: [
    .alert { transition: opacity 0.5s; }
    .alert[hidden] { opacity: 0; display: block !important; }
  ],
  template: `
    <div class="alert" [hidden]="!isVisible">Message</div>
  `,
})
```

Interview Cheat Sheet

- ✓ DOM Impact: `*ngIf` removes, `[hidden]` hides
- ✓ Performance: `*ngIf` better for memory, `[hidden]` for frequent toggles
- ✓ Lifecycle: Only `*ngIf` triggers init/destroy
- ✓ Accessibility: `[hidden]` preserves ARIA

Pro Tip:

"Think of `*ngIf` as unloading a truck (saves fuel), while `[hidden]` just covers it with a tarp (quick to reveal)."

Common Pitfalls:

1. Using `[hidden]` with `display: flex` (requires `!important`)
2. Memory leaks from `*ngIf` with unsubscribed observables
3. CSS conflicts with `[hidden]` (bootstrap overrides)

43. How do you manage shared state between components in Angular?

For effective state sharing between components, Angular offers multiple patterns depending on your app's complexity:

1. Service + RxJS (Recommended for Most Cases)

```
@Injectable({ providedIn: 'root' })
export class SharedStateService {
  // BehaviorSubject for current value + Observable for subscribers
  private _state = new BehaviorSubject<State>(initialState);
```

```

state$ = this._state.asObservable();

updateState(updater: (state: State) => State) {
  this._state.next(updater(this._state.value));
}
}

// Component A (updates)
constructor(private state: SharedStateService) {}
updateUser() {
  this.state.updateState(state => ({ ...state, user: newUser }));
}

// Component B (reads)
user$ = this.stateService.state$.pipe(map(s => s.user));

```

When to Use:

- ✓ Medium complexity apps
- ✓ Need real-time sync between 3+ components
- ✓ Prefer reactive programming

2. Angular Signals (v16+)

```

@Injectable({ providedIn: 'root' })
export class CartState {
  items = signal<Item[]>([]);
  total = computed(() =>
    this.items().reduce((sum, item) => sum + item.price, 0)
  );

  addItem(item: Item) {
    this.items.update(items => [...items, item]);
  }
}

// Component

```

```
items = this.cartState.items(); // Read value
```

When to Use:

- ✓ New projects (Angular 16+)
 - ✓ Simpler syntax than RxJS
 - ✓ Fine-grained reactivity
-

3. State Management Libraries

A. NgRx (Redux Pattern)

```
// Store setup
export const loadUsers = createAction('[Users] Load');
export const usersReducer = createReducer(
  initialState,
  on(loadUsers, (state, { users }) => ({ ...state, users })))
);

// Component
users$ = this.store.select(selectUsers);
load() {
  this.store.dispatch(loadUsers());
}
```

Best For:

- ✓ Large, complex applications
- ✓ Need time-travel debugging
- ✓ Strict unidirectional data flow

B. NGXS (Simpler Alternative)

```
@State<StateModel>({
  name: 'users',
  defaults: { users: [] }
})
```

```
})  
@Injectable()  
export class UsersState {  
  @Selector() static users(state: StateModel) { return state.users; }  
}
```

4. Component Binding (Simple Cases)

```
// Parent Component  
<app-child [data]="sharedData" (update)="onUpdate($event)"></app-child>  
  
// Child Component  
@Input() data: Data;  
@Output() update = new EventEmitter<Data>();
```

When to Use:

- ✓ Parent-child relationships
 - ✓ Small, isolated state
 - ✓ Avoid service overhead
-

Comparison Table

Method	Complexity	Best For	Learning Curve
Service + RxJS	Medium	Most apps	Moderate
Signals	Low	New projects	Easy
NgRx	High	Enterprise apps	Steep

Component Binding

Low

Parent-child

Trivial

Advanced Patterns

1. State Normalization

```
interface State {  
  users: { [id: string]: User };  
  posts: { [id: string]: Post };  
}
```

2. Effect Patterns

```
// NgRx Effects  
loadUsers$ = createEffect(() => this.actions$.pipe(  
  ofType(loadUsers),  
  switchMap(() => this.api.getUsers().pipe(  
    map(users => loadUsersSuccess({ users })),  
    catchError(error => of(loadUsersFailure({ error })))  
  ))  
));
```

3. Local Storage Sync

```
// In service  
state$.subscribe(state => {  
  localStorage.setItem('appState', JSON.stringify(state));  
});
```

Interview Cheat Sheet

- ✅ Start Simple: Component binding → Services → Libraries
- ✅ Key RxJS Operators: BehaviorSubject, distinctUntilChanged, shareReplay
- ✅ Immutable Updates: Always return new state objects

✓ Performance: Use `OnPush` with async pipe

Pro Tip:

"Choose state management like picking clothes - lightweight for warm days (simple apps), layered for harsh weather (complex apps)."

Common Pitfalls:

1. Nested subscriptions causing memory leaks
2. Mutating state directly (always return new references)
3. Over-engineering small apps with NgRx

Debugging Tip:

```
// Log all state changes
state$.subscribe(state => console.log('State changed:', state));
```

44. What is NgZone, and how is it used in Angular?

NgZone is Angular's execution context wrapper that tracks asynchronous operations to trigger change detection. Here's a breakdown of its purpose and usage:

1. Core Functionality

- Tracks async operations (setTimeout, promises, HTTP calls)
- Automatically triggers change detection when async tasks complete
- Provides execution zones:
 - Angular Zone (default): Triggers change detection
 - Noop Zone: Disables automatic change detection

```
import { NgZone } from '@angular/core';
constructor(private ngZone: NgZone) {}
```

2. Key Use Cases

A. Performance Optimization

Run code outside Angular's change detection:

```
this.ngZone.runOutsideAngular(() => {  
  // High-frequency events (mousemove, scroll)  
  element.addEventListener('mousemove', () => {  
    // Won't trigger change detection  
  });  
});
```

B. Manual Change Detection Control

```
this.ngZone.run(() => {  
  // Code that SHOULD trigger change detection  
  this.value = newValue; // Angular will detect this change  
});
```

C. Tracking Stability

```
this.ngZone.onUnstable.subscribe(() => {  
  console.log('Change detection starting');  
});
```

```
this.ngZone.onStable.subscribe(() => {  
  console.log('Change detection complete');  
});
```

3. Zone.js Integration

NgZone builds on Zone.js to:

- Patch async APIs (adds hooks to setTimeout, promises, etc.)
- Create execution contexts
- Maintain stack traces

// Zone.js monkey-patching example


```
const originalSetTimeout = window.setTimeout;
window.setTimeout = (handler, timeout) => {
  // Zone-specific logic here
  return originalSetTimeout(handler, timeout);
};
```

4. Advanced Patterns

A. Third-Party Library Integration

```
// For libraries that conflict with Zone.js
this.ngZone.runOutsideAngular(() => {
  thirdPartyLib.init({
    callback: () => this.ngZone.run(() => /* update Angular state */)
  });
});
```

B. Progressive Web Apps (Service Workers)

```
this.ngZone.runOutsideAngular(() => {
  navigator.serviceWorker.register('sw.js');
});
```

C. Benchmarking

```
const start = performance.now();
this.ngZone.runOutsideAngular(() => {
  // Heavy computation
});
console.log(`Took ${performance.now() - start}ms`);
```

When to Use NgZone Directly

Scenario	Solution
Performance-critical code	<code>runOutsideAngular()</code>

Manual change detection	<code>run()</code>
Stability monitoring	<code>onStable/onUnstable</code>
Zone.js conflicts	Partial zone avoidance

Interview Cheat Sheet

- ✓ Purpose: Tracks async operations for change detection
- ✓ Key Methods: `run()`, `runOutsideAngular()`
- ✓ Events: `onStable`, `onMicrotaskEmpty`, `onUnstable`
- ✓ Performance: Critical for animation/scroll-heavy apps

Pro Tip:

"NgZone is Angular's radar system - it spots async operations and dispatches change detection patrols."

Common Pitfalls:

1. Forgetting to re-enter zone with `run()` after `runOutsideAngular()`
2. Overusing `runOutsideAngular()` causing UI staleness
3. Memory leaks from unsubscribed zone observers

Debugging Tip:

```
// Log all zone transitions
this.ngZone.onUnstable.subscribe(() => console.log('CD started'));
this.ngZone.onStable.subscribe(() => console.log('CD completed'));
```

45. What are custom directives in Angular?

Custom directives are Angular's way to extend HTML behavior by creating reusable DOM transformations. They come in two main flavors:

1. Types of Directives

Type	Purpose	Example
Attribute Directives	Change appearance/behavior	Tooltips, highlighters
Structural Directives	Modify DOM layout	Custom <code>*ngIf</code> variants

2. Creating an Attribute Directive

Example: Background Highlighter

```
import { Directive, ElementRef, Input, Renderer2 } from '@angular/core';

@Directive({
  selector: '[appHighlight]' // Usage: <p appHighlight>
})
export class HighlightDirective {
  @Input() appHighlight = 'yellow'; // Default color

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  ngOnInit() {
    this.renderer.setStyle(
      this.el.nativeElement,
      'background-color',

```

```
        this.appHighlight
    );
}
}
```

Usage:

```
<p [appHighlight]=" 'lightblue' ">Highlighted text</p>
```

3. Creating a Structural Directive

Example: Custom `*appUnless` (opposite of `*ngIf`)

```
import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';

@Directive({
  selector: '[appUnless]'
})
export class UnlessDirective {
  @Input() set appUnless(condition: boolean) {
    !condition
      ? this.vcRef.createEmbeddedView(this.templateRef)
      : this.vcRef.clear();
  }

  constructor(
    private templateRef: TemplateRef<any>,
    private vcRef: ViewContainerRef
  ) {}
}
```

Usage:

```
<div *appUnless="isHidden">Shows when isHidden is false</div>
```

4. Host Binding/Listener

Example: Toggle Class on Hover

```
@Directive({
  selector: '[appHoverClass]'
})
export class HoverClassDirective {
  @Input() appHoverClass = '';

  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.classList.add(this.appHoverClass);
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.classList.remove(this.appHoverClass);
  }

  constructor(private el: ElementRef) {}
}
```

Usage:

```
<button appHoverClass="active">Hover Me</button>
```

5. Directive Communication

Input Aliasing:

```
@Directive({ selector: '[appTooltip]' })
export class TooltipDirective {
  @Input('appTooltip') tooltipText = ''; // Now accepts appTooltip="..."
}
```

Output Events:

```
@Directive({ selector: '[appAction]' })
export class ActionDirective {
  @Output() appAction = new EventEmitter();
}
```

```
@HostListener('click')
onClick() {
  this.appAction.emit();
}
```

Key Differences: Components vs Directives

Feature	Component	Directive
Template	Yes	No
Styles	Yes	No
Purpose	Create UI	Enhance existing UI
Selector Type	Element (<code>app-card</code>)	Attribute (<code>[appHighlight]</code>)

Real-World Use Cases

1. Attribute Directives
 - Input validators
 - Permission-based UI controls
 - Animation triggers
 2. Structural Directives
 - Advanced conditional rendering
 - List optimizations
 - Layout templating
-

Interview Cheat Sheet

- ✓ @Directive decorator marks a class
- ✓ ElementRef for DOM access (use Renderer2 for safety)
- ✓ Structural directives need TemplateRef + ViewContainerRef
- ✓ HostBinding/HostListener for event/class binding

Pro Tip:

"Directives are like surgical tools - they precisely modify existing elements without creating new components."

Performance Tip:

For frequent DOM updates:

// Instead of:

```
this.el.nativeElement.style.color = 'red';
```

// Use Renderer2:

```
this.renderer.setStyle(this.el.nativeElement, 'color', 'red');
```

46. How do you implement internationalization (i18n) in Angular?

Angular provides robust i18n support for translating templates and code. Here's how to implement it effectively:

1. Basic Setup

Install i18n tools:

ng add @angular/localize

Configure in `angular.json`:

```
"projects": {
  "your-app": {
    "i18n": {
      "sourceLocale": "en-US",
      "locales": {
        "fr": "src/locale/messages.fr.xlf",
        "es": "src/locale/messages.es.xlf"
      }
    }
  }
}
```

2. Template Translation

Mark translatable text with `i18n`:

```
<!-- Simple text -->
<h1 i18n>Welcome</h1>

<!-- With description and ID -->
<p i18n="@@welcomeMessage|An introduction message">Hello user!</p>

<!-- Pluralization -->
<span i18n>{count, plural, =0 {No items} =1 {One item} other {{{count}}}
items}</span>

<!-- Dates/numbers -->
<p>{{ today | date:'fullDate' }}</p>
<p>{{ price | currency:'EUR' }}</p>
```

3. Extract Translations

Generate XLF files:


```
ng extract-i18n --output-path src/locale
```

This creates `messages.xlf` for translators.

4. Runtime Configuration

A. Build-Specific Locales (Static):

```
ng build --localize
# Produces separate bundles for each locale
```

B. Runtime Locale Switching (Dynamic):

```
// app.module.ts
import { LOCALE_ID } from '@angular/core';

@NgModule({
  providers: [{
    provide: LOCALE_ID,
    useValue: navigator.language // or from user settings
  }]
})
```

5. Code Translations

Use `$localize` for TS files:

```
import { $localize } from '@angular/localize';

// Simple string
const greeting = $localize`Hello`;

// With parameters
const message = $localize`Welcome back, ${username}:user:!!`;
```

6. Advanced Features

A. Custom Translation Files

```
// custom-translate.service.ts
import { TranslateLoader } from '@ngx-translate/core';
import { HttpClient } from '@angular/common/http';

@Injectable()
export class CustomTranslateLoader implements TranslateLoader {
  constructor(private http: HttpClient) {}

  getTranslation(lang: string) {
    return this.http.get(`/assets/i18n/${lang}.json`);
  }
}
```

B. Language Switcher Component

```
export class LanguageSwitcherComponent {
  languages = ['en', 'fr', 'es'];

  constructor(
    @Inject(LOCALE_ID) public activeLocale: string,
    private translator: TranslateService
  ) {}

  changeLocale(locale: string) {
    this.translator.use(locale);
  }
}
```

Best Practices

1. Text Extraction: Regularly update translation files
2. Locale Data: Import additional locale data for pipes:

```
import '@angular/common/locales/fr';
```

3. `registerLocaleData(localeFr);`
 4. Testing: Verify all i18n strings in templates
 5. Fallback: Implement default language handling
-

Interview Cheat Sheet

- ✓ Core Package: `@angular/localize`
- ✓ Template Markup: `i18n` attribute
- ✓ CLI Commands: `extract-i18n`, `build --localize`
- ✓ Dynamic Switching: `LOCALE_ID` provider

Pro Tip:

"Think of i18n as decorating your app with translation hooks - the core logic remains unchanged while the language layer wraps it."

Common Pitfalls:

1. Forgetting to load locale data for pipes
2. Not providing context in i18n comments
3. Hardcoding locale-specific formats

Performance Tip:

For large apps, consider lazy-loading locale files:

```
// Dynamically load locale data
async setLocale(locale: string) {
  const localeData = await import(`@angular/common/locales/${locale}`);
  registerLocaleData(localeData.default);
}
```

47. What is content projection in Angular?

Content projection (also known as *transclusion*) is Angular's way to insert external content into a component's template. It enables flexible component composition while maintaining encapsulation.

1. Basic Projection (<ng-content>)

Parent Component:

```
<app-card>
  <h2>Projected Title</h2>
  <p>This content will be injected</p>
</app-card>
```

Child Component (card.component.html):

```
<div class="card">
  <div class="header">
    <ng-content></ng-content> <!-- Projects all content here -->
  </div>
</div>
```

2. Selective Projection (Named Slots)

Parent Component:

```
<app-card>
  <div header>Custom Header</div>
  <div body>Main Content</div>
  <div footer>Footer Notes</div>
</app-card>
```

Child Component:

```
<div class="card">
  <div class="header">
    <ng-content select="[header]"></ng-content>
  </div>
  <div class="body">
    <ng-content select="[body]"></ng-content>
  </div>
  <div class="footer">
    <ng-content select="[footer]"></ng-content>
  </div>
</div>
```

3. Advanced Patterns

A. Conditional Projection

```
<ng-content *ngIf="allowContent"></ng-content>
```

B. Multiple Projections

```
<!-- Default slot (catches non-matched content) -->
<ng-content></ng-content>
```

```
<!-- Named slot -->
<ng-content select=".special"></ng-content>
```

C. Component Projection

```
<app-tabs>
  <app-tab [title]='Tab 1'>Content 1</app-tab>
  <app-tab [title]='Tab 2'>Content 2</app-tab>
</app-tabs>
```

4. ContentChild Access

Access projected content programmatically:

```
@ContentChild('headerRef', { static: false })
headerContent!: ElementRef;

@ContentChildren(TabComponent)
tabs!: QueryList<TabComponent>;
```

5. Real-World Use Cases

Pattern	Example	Benefit
Wrapper Components	Cards, Modals, Tabs	Reusable containers
Layout Components	Grids, Accordions	Dynamic structure
Configurable UI	Dashboards	User-customizable

Interview Cheat Sheet

- ✓ Core Directive: `<ng-content>`
- ✓ Selection: Use `select="selector"` for targeted projection
- ✓ Access Patterns: `@ContentChild` / `@ContentChildren`
- ✓ Best For: Building reusable container components

Pro Tip:

"Content projection turns components into 'Russian dolls' - allowing nested content while

maintaining control over the outer shell."

Common Pitfalls:

1. Overusing projection when simple `@Input` would suffice
2. Forgetting CSS encapsulation issues with projected content
3. Missing `select` attributes causing misplaced content

Performance Tip:

Projected content is evaluated in the parent's context, so avoid heavy computations in projected elements. Use `ng-template` for lazy evaluation when needed:

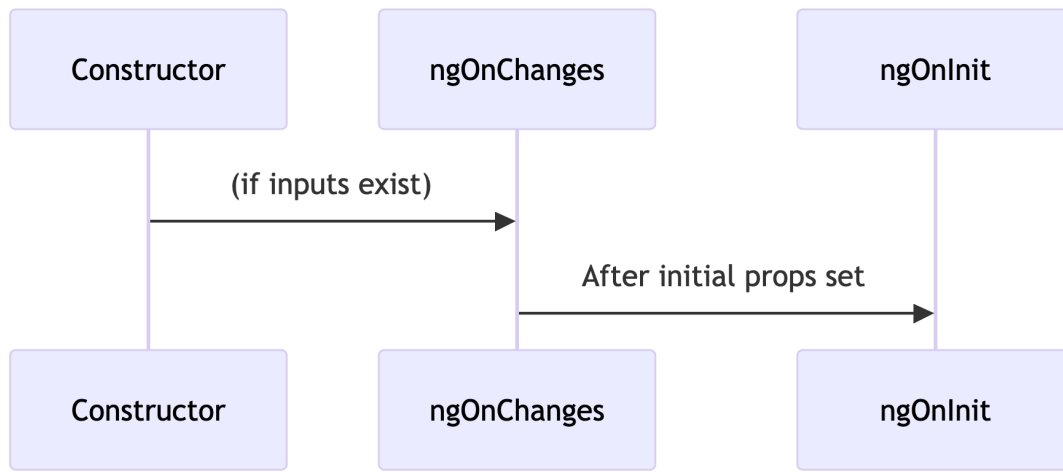
```
<app-container>
  <ng-template #heavyContent>
    <!-- Only rendered when used -->
  </ng-template>
</app-container>
```

48. Explain the difference between `ngOnInit` and `constructor`.

1. Execution Timing

	constructor	ngOnInit
When it runs	During class instantiation	After first change detection cycle
Order	First (before Angular touches the component)	Later (after Angular sets inputs)
Frequency	Once per component lifetime	Once (unless component is

recreated)



2. Primary Use Cases

Constructor:

- Dependency injection
- Basic field initialization

```
constructor(private service: DataService) {  
  this.defaultValue = 100; // Simple assignments  
}
```

ngOnInit:

- Complex initialization
- Data fetching
- Input-dependent setup

```
ngOnInit() {  
  this.loadData(); // API calls  
  console.log(this.inputValue); // Safe to access @Input()  
}
```

3. Input Availability

	constructor	ngOnInit
@Input() values	✗ Not available	✓ Available
DOM access	✗ Dangerous	✓ Safer (but use <code>ngAfterViewInit</code> for elements)

Anti-Pattern Warning:

```

@Input() id!: string;

constructor() {
  console.log(this.id); // UNDEFINED!
}

ngOnInit() {
  console.log(this.id); // Correct value
}

```

4. Angular Context

- Constructor:
 - Runs outside Angular's component initialization
 - Framework-agnostic (plain TypeScript feature)
- ngOnInit:
 - Part of Angular's component lifecycle
 - Guaranteed to run after Angular fully initializes the component

5. Testing Implications

```

// constructor dependencies are easier to mock
const testComp = new TestComponent(mockService);

// ngOnInit requires fixture.detectChanges()
fixture.detectChanges(); // triggers ngOnInit

```

When to Use Which?

✓ Use constructor for:

- Injecting services
- Simple property initialization
- Setting up non-Angular specific code

✓ Use ngOnInit for:

- Any initialization requiring `@Input()` values
- API calls
- Complex setup logic
- Subscriptions (though consider `ngAfterViewInit` for DOM-dependent cases)

Performance Considerations

- Heavy operations in constructors delay component availability
- `ngOnInit` is better for async operations since it runs after property binding

Pro Tip:

"Think of the constructor as the component's birth certificate (basic info) and `ngOnInit` as its first day at school (ready to interact with the world)."

Common Mistakes:

1. Putting API calls in constructors
2. Accessing `@Input()` values too early
3. Forgetting to call `super()` when extending components

Lifecycle Hook Order Recap:

1. `constructor`
2. `ngOnChanges` (if inputs exist)
3. `ngOnInit`
4. `ngAfterViewInit` (for DOM access)

49. What is a guard in Angular, and how do you use it in routing?

Route guards are interfaces that control navigation to/from routes based on custom logic. They're essential for authentication, authorization, and data pre-loading.

1. Guard Types

Guard Type	Interface	Purpose
CanActivate	<code>CanActivate</code>	Grants access to route
CanActivateChild	<code>CanActivateChild</code>	Protects child routes
CanDeactivate	<code>CanDeactivate<T></code>	Prevents accidental exits
CanLoad	<code>CanLoad</code>	Blocks lazy-loaded modules
Resolve	<code>Resolve<T></code>	Pre-fetches data

2. Implementing Guards

A. Auth Guard (CanActivate)

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate()
```

```

        route: ActivatedRouteSnapshot,
        state: RouterStateSnapshot
    ): boolean | UrlTree {
        return this.auth.isLoggedIn()
            ? true
            : this.router.parseUrl('/login');
    }
}

```

B. Unsaved Changes Guard (CanDeactivate)

```

@Inject({ providedIn: 'root' })
export class UnsavedChangesGuard implements CanDeactivate<FormComponent> {
    canDeactivate(
        component: FormComponent
    ): boolean | Observable<boolean> {
        return component.hasUnsavedChanges
            ? confirm('Discard changes?')
            : true;
    }
}

```

C. Data Resolver (Resolve)

```

@Inject({ providedIn: 'root' })
export class ProductResolver implements Resolve<Product> {
    constructor(private service: ProductService) {}

    resolve(route: ActivatedRouteSnapshot): Observable<Product> {
        return this.service.getProduct(route.params['id']);
    }
}

```

3. Registering Guards

In Route Configuration:

```

const routes: Routes = [
    {

```

```

    path: 'admin',
    component: AdminComponent,
    canActivate: [AuthGuard],
    canDeactivate: [UnsavedChangesGuard],
    resolve: { product: ProductResolver },
    children: [
      {
        path: 'dashboard',
        canActivateChild: [AdminAccessGuard],
        component: DashboardComponent
      }
    ]
  },
  {
    path: 'reports',
    canLoad: [AuthGuard],
    loadChildren: () => import('./reports.module')
  }
];

```

Provider Setup:

```

@NgModule({
  providers: [
    AuthGuard,
    UnsavedChangesGuard,
    ProductResolver
  ]
})
export class AppModule {}

```

4. Execution Order

1. CanLoad (for lazy routes)
 2. CanActivate/CanActivateChild
 3. Resolve (data pre-fetching)
 4. CanDeactivate (when leaving)
-

5. Advanced Patterns

A. Role-Based Access

```
canActivate(route: ActivatedRouteSnapshot): boolean {  
  const requiredRole = route.data['role'];  
  return this.auth.hasRole(requiredRole);  
}
```

```
// Route config:  
{  
  path: 'admin',  
  data: { role: 'admin' },  
  canActivate: [RoleGuard]  
}
```

B. Dynamic Redirects

```
canActivate(): UrlTree {  
  return this.store.select(selectUserType).pipe(  
    map(userType =>  
      userType === 'admin'  
        ? this.router.parseUrl('/admin')  
        : true  
    )  
  );  
}
```

C. Guard Testing

```
it('should block unauthorized users', () => {  
  TestBed.runInInjectionContext(() => {  
    const guard = new AuthGuard(mockAuth, mockRouter);  
    mockAuth.isLoggedIn = false;  
    expect(guard.canActivate({} as any, {} as any)).toEqual(  
      jasmine.any(UrlTree)  
    );  
  });  
});
```

Interview Cheat Sheet

- ✓ Purpose: Control route access and data flow
- ✓ Key Interfaces: `CanActivate`, `CanDeactivate`, `Resolve`
- ✓ Registration: Add to route config and providers
- ✓ Order Matters: Guards execute sequentially

Pro Tip:

"Route guards act like bouncers - they check IDs (`CanActivate`), prevent fights (`CanDeactivate`), and prepare VIP areas (`Resolve`)."

Common Pitfalls:

1. Forgetting `providedIn: 'root'` or module providers
2. Not returning `Observable<boolean>` for async checks
3. Infinite redirect loops in auth guards

Performance Tip:

For complex checks, use `CanLoad` to prevent lazy module downloads:

```
{
  path: 'dashboard',
  canLoad: [AuthGuard],
  loadChildren: () => import('./dashboard.module')
}
```

50. How do you handle session management in Angular?

For secure and efficient session handling in Angular applications, follow this comprehensive approach:

1. Authentication Flow

A. Login/Token Acquisition

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  constructor(private http: HttpClient) {}

  login(credentials: { email: string; password: string }) {
    return this.http.post<{ token: string }>('/api/login', credentials).pipe(
      tap(response => {
        this.setSession(response.token); // Store token
      })
    );
  }

  private setSession(token: string) {
    localStorage.setItem('auth_token', token); // Or use cookies
    // Decode token if JWT to get expiry (using jwt-decode library)
  }
}
```

B. Token Refresh Mechanism

```
refreshToken() {
  return this.http.post<{ token: string }>('/api/refresh', {}).pipe(
    tap(newToken => {
      this.setSession(newToken.token);
    })
  );
}
```

2. Session Storage Options

Method	Pros	Cons
LocalStorage	Simple, persists across tabs	Vulnerable to XSS
SessionStorage	Tab-specific, clears on close	Same XSS risk
HttpOnly Cookies	Secure against XSS	Requires CSRF protection
In-Memory	Most secure	Doesn't persist refresh

Recommended Secure Approach:

```
// Store refresh token in HttpOnly cookie
// Keep access token in memory (with refresh mechanism)
```

3. Session Validation

A. Route Guards

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean | UrlTree {
    return this.auth.isAuthenticated()
      ? true
      : this.router.parseUrl('/login');
  }
}
```

B. HTTP Interceptor

```
@Injectable()
```

```

export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = this.auth.getToken();
    const authReq = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;

    return next.handle(authReq).pipe(
      catchError(err => {
        if (err.status === 401) {
          this.auth.logout();
        }
        return throwError(() => err);
      })
    );
  }
}

```

4. Session Timeout Handling

A. Auto-Logout on Inactivity

```

private inactivityTimer!: any;

resetInactivityTimer() {
  clearTimeout(this.inactivityTimer);
  this.inactivityTimer = setTimeout(() => {
    this.logout();
  }, 30 * 60 * 1000); // 30 minutes
}

// Add event listeners
@HostListener('document:mousemove')
@HostListener('document:keypress')
onUserActivity() {
  this.resetInactivityTimer();
}

```

B. Token Expiration Check

```
isAuthenticated(): boolean {
  const token = this.getToken();
  if (!token) return false;

  // For JWT:
  const expiry = JSON.parse(atob(token.split('.')[1])).exp;
  return Math.floor(new Date().getTime() / 1000) < expiry;
}
```

5. Secure Logout

```
logout() {
  // Clear local storage
  localStorage.removeItem('auth_token');

  // Call server-side logout (for invalidation)
  this.http.post('/api/logout', {}).subscribe();

  // Redirect to login
  this.router.navigate(['/login']);

  // Clear all observables (prevent memory leaks)
  this.cleanupSubscriptions();
}
```

6. Advanced Patterns

A. Silent Refresh

```
private initSilentRefresh() {
  timer(0, 30 * 60 * 1000) // Check every 30 minutes
    .pipe(
      filter(() => this.isAuthenticated()),
      switchMap(() => this.refreshToken())
    ).subscribe();
}
```

B. Multi-Tab Sync

```
@HostListener('window:storage', ['$event'])
onStorageChange(event: StorageEvent) {
  if (event.key === 'auth_token' && !this.getToken()) {
    this.router.navigate(['/login']);
  }
}
```

C. Session Recovery

```
restoreSession() {
  if (this.isAuthenticated()) {
    this.refreshToken().subscribe({
      error: () => this.logout()
    });
  }
}
```

Security Considerations

1. Always use HTTPS
 2. Implement CSRF protection for cookie-based auth
 3. Sanitize all user inputs to prevent XSS
 4. Set proper CORS headers on the server
 5. Use Content Security Policy (CSP)
-

Interview Cheat Sheet

- ✓ Storage: Prefer in-memory + refresh tokens
- ✓ Validation: Guards + HTTP interceptors
- ✓ Security: HTTPS + CSRF + CSP
- ✓ Cleanup: Proper logout handling

Pro Tip:

"Treat user sessions like perishable food - they need regular freshness checks and proper

expiration handling."

Common Pitfalls:

1. Storing sensitive data in localStorage
2. Not handling token refresh failures
3. Forgetting to unsubscribe from observables
4. Ignoring tab synchronization

Performance Tip:

For SPA performance, decode JWT tokens only once:

```
private _cachedTokenData: any;

get userData() {
  if (!this._cachedTokenData && this.token) {
    this._cachedTokenData = jwtDecode(this.token);
  }
  return this._cachedTokenData;
}
```

51. What is Angular's Renderer2, and why is it used?

Renderer2 is Angular's abstraction for DOM manipulation, providing a platform-agnostic way to modify elements while maintaining security and compatibility across environments (browser, server, web workers).

Key Reasons to Use Renderer2

1. Security
 - Prevents XSS vulnerabilities by safely handling DOM updates
 - Avoids direct `element.innerHTML` manipulation
2. Server-Side Rendering (SSR) Support
 - Works in non-browser environments (Angular Universal)

- 3. Cross-Platform Compatibility
 - Consistent API for web, mobile (Ionic), and desktop apps
- 4. Change Detection Optimization
 - Updates DOM efficiently within Angular's rendering cycle

Common Use Cases

Scenario	Vanilla JS Approach	Renderer2 Approach
Add a class	<code>el.classList.add()</code>	<code>renderer.addClass()</code>
Set attribute	<code>el.setAttribute()</code>	<code>renderer.setAttribute()</code>
Create element	<code>document.createElement()</code>	<code>renderer.createElement()</code>
Event binding	<code>el.addEventListener()</code>	<code>renderer.listen()</code>

Implementation Example

```
import { Renderer2, ElementRef } from '@angular/core';

@Component({...})
export class HighlightDirective {
  constructor(
    private el: ElementRef,
    private renderer: Renderer2
  ) {
    // Safely add class and attribute
    renderer.addClass(el.nativeElement, 'active');
    renderer.setAttribute(el.nativeElement, 'aria-selected', 'true');

    // Event listener with automatic cleanup
    renderer.listen(el.nativeElement, 'click', () => {
```

```
        console.log('Element clicked');
    });
}
```

Key Methods

Method	Purpose	Example
<code>createElement()</code>	Safe element creation	<code>renderer.createElement('div')</code>
<code>appendChild()</code>	DOM insertion	<code>renderer.appendChild(parent, child)</code>
<code>setStyle()</code>	Style modification	<code>renderer.setStyle(el, 'color', 'red')</code>
<code>setProperty()</code>	DOM property update	<code>renderer.setProperty(el, 'hidden', true)</code>
<code>destroy()</code>	Cleanup	<code>renderer.destroy()</code>

When to Use Renderer2 vs Direct DOM Access

✅ Use Renderer2 for:

- Dynamic element creation/modification
- Cross-platform components
- Security-sensitive operations
- Server-rendered apps

⚠️ Direct DOM access (ElementRef) only when:

- Absolutely necessary for performance
 - Working with third-party libs requiring native elements
 - Renderer2 lacks needed functionality
-

Interview Cheat Sheet

- ✓ Purpose: Safe, abstracted DOM manipulation
- ✓ Key Advantage: Works in SSR and non-browser contexts
- ✓ Security: Prevents XSS vs innerHTML
- ✓ Performance: Optimized change detection integration

Pro Tip:

"*Renderer2 is like wearing gloves while handling DOM - it protects your app from cuts (XSS) and works in all weather (platforms)."

Common Pitfalls:

1. Forgetting to cleanup event listeners
2. Overusing `ElementRef.nativeElement`
3. Not using it in Universal apps (causing SSR errors)

Memory Management Tip:

```
// Store listener cleanup function
private listenerFn!: () => void;

ngOnInit() {
  this.listenerFn = this.renderer.listen(
    this.el.nativeElement,
    'click',
    this.handleClick.bind(this)
  );
}

ngOnDestroy() {
```



```
    this.listenerFn(); // Crucial for memory cleanup
}
```

52. What is server-side rendering (SSR), and why is it beneficial?

Server-Side Rendering (SSR) generates HTML on the server and sends a fully-rendered page to the client. Angular's SSR solution is called Angular Universal.

Key Benefits of SSR

Benefit	Impact	Example
Improved SEO	Search engines crawl pre-rendered HTML	Higher search rankings for dynamic content
Faster Initial Load	Users see content before JavaScript loads	50-70% faster First Contentful Paint (FCP)
Social Media Previews	Proper metadata for shared links	Correct OpenGraph images/titles when sharing
Low-Bandwidth Optimization	Smaller initial payload	Better performance on slow networks
Consistent Performance	Reduces client-side rendering variability	Stable metrics across devices

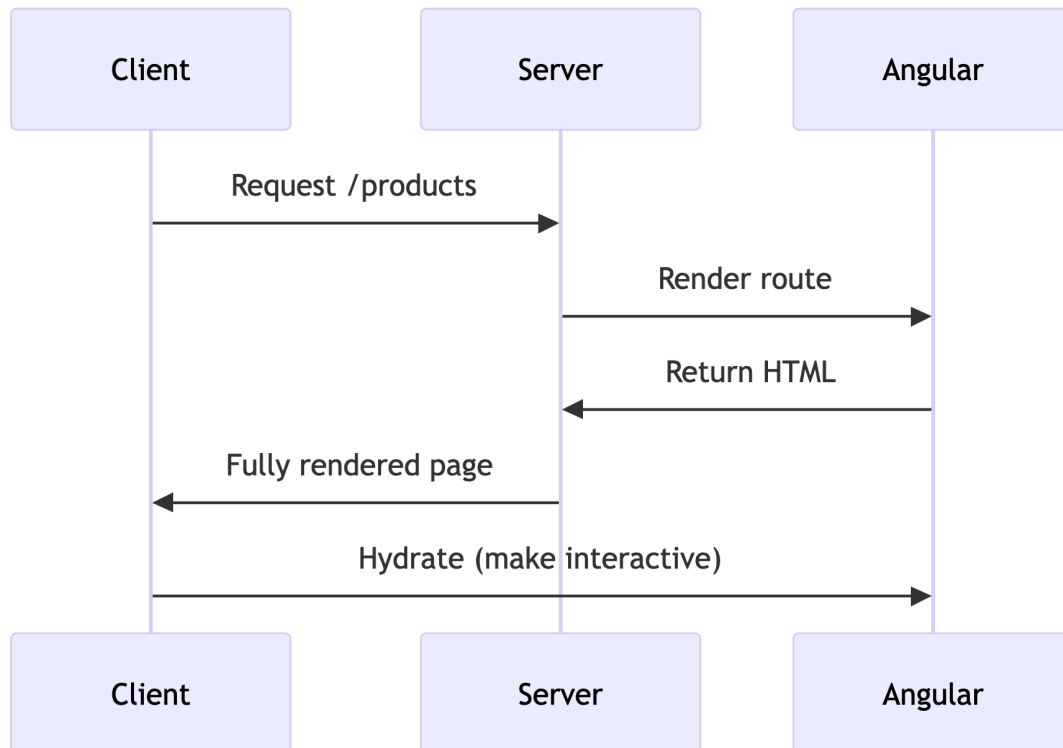
How SSR Works in Angular

1. Server Process

- Node.js server runs Angular app
- Generates static HTML for requested route
- Embeds initial state in the page

2. Client Process

- Browser displays server-rendered HTML immediately
- Angular "hydrates" the app (attaches event handlers)
- Subsequent navigation is client-side



Implementing SSR in Angular

1. Setup Angular Universal

ng add @nguniversal/express-engine

This creates:

- `server.ts` (Express server)
- `app.server.module.ts` (SSR entry point)

2. Server-Side Adjustments

```
// Component with browser-specific code
constructor(@Inject(PLATFORM_ID) private platformId: Object) {
  if (isPlatformBrowser(this.platformId)) {
    // Client-only code (e.g., localStorage)
  }
}
```

3. Data Fetching

```
// Use TransferState for avoiding duplicate fetches
export class ProductService {
  getProducts() {
    const key = makeStateKey<Product[]>('products');

    return this.http.get<Product[]>('/api/products').pipe(
      tap(products => {
        if (isPlatformServer(this.platformId)) {
          this.transferState.set(key, products);
        }
      }),
      catchError(() => {
        return isPlatformBrowser(this.platformId)
          ? this.transferState.get(key, [])
          : of([]);
      })
    );
  }
}
```

When to Use SSR

- ✓ Content-heavy websites (blogs, news)
 - ✓ SEO-dependent applications
 - ✓ Social media sharing requirements
 - ✓ Progressive Web Apps (PWAs)
-

Performance Comparison

Metric	Client-Side Only	With SSR
Time-to-Content	3-5s	0.5-1s
SEO Visibility	Poor	Excellent
CPU Usage	Client-heavy	Balanced server/client

Interview Cheat Sheet

- ✓ Angular Universal: Official SSR solution
- ✓ Hydration: Client-side takeover process
- ✓ TransferState: Avoid duplicate API calls
- ✓ isPlatformBrowser/Server: Environment checks

Pro Tip:

"SSR is like sending a printed document instead of a blank page with instructions - users

get content immediately while the app loads in the background."

Common Pitfalls:

1. Memory leaks in server code
2. Browser-specific APIs crashing SSR
3. Not caching rendered pages
4. Over-fetching data for bots

Optimization Tip:

Implement incremental static regeneration (ISR) for dynamic routes:

```
// Revalidate every 60 seconds  
res.set('Cache-Control', 'public, max-age=60, s-maxage=60');
```

53. How does Angular support Progressive Web Apps (PWAs)?

Angular provides built-in tools to transform apps into Progressive Web Apps (PWAs) with offline capabilities, fast loading, and native-like features. Here's how it works:

1. Core PWA Features in Angular

Feature	Angular Implementation	Benefit
App Shell	<code>ng generate app-shell</code>	Instant loading UI
Service Worker	<code>@angular/pwa</code> schematics	Offline caching
Web App Manifest	<code>manifest.webmanifest</code>	Add-to-homescreen

HTTP Caching

`ngsw-config.json`

API response
caching

2. Quick Setup (3 Steps)

1. Add PWA Support

```
ng add @angular/pwa --project your-project-name
```

This generates:

- `ngsw-config.json` (Service worker config)
- `manifest.webmanifest` (App install settings)
- Icons in `assets/icons/`

2. Configure Caching (`ngsw-config.json`)

```
{
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "resources": {
        "files": ["/favicon.ico", "/index.html", "/*.css", "/*.js"]
      }
    },
    {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "prefetch",
      "resources": {
        "files": ["/assets/**"]
      }
    }
  ],
  "dataGroups": [
```

```

    {
      "name": "api",
      "urls": ["/api/**"],
      "cacheConfig": {
        "strategy": "freshness",
        "maxSize": 20,
        "maxAge": "1h"
      }
    }
  ]
}

```

3. Enable in Production

```

// app.module.ts
import { ServiceWorkerModule } from '@angular/service-worker';

@NgModule({
  imports: [
    ServiceWorkerModule.register('ngsw-worker.js', {
      enabled: environment.production,
      registrationStrategy: 'registerWhenStable:30000'
    })
  ]
})

```

3. Key Angular PWA Capabilities

A. Offline Mode

```

// Check online status
import { Network } from '@angular/fire';

constructor(private network: Network) {
  this.network.onDisconnect().subscribe(() =>
    this.showOfflineAlert()
  );
}

```

B. Background Sync

```
// Custom service worker (sw.js)
self.addEventListener('sync', (event) => {
  if (event.tag === 'sync-data') {
    event.waitUntil(syncData());
  }
});
```

C. Push Notifications

```
// Request permission
Notification.requestPermission().then(perm => {
  if (perm === 'granted') {
    new Notification('New update available!');
  }
});
```

4. Advanced Configuration

A. Dynamic Caching

```
// ngsw-config.json
{
  "dynamic": {
    "group": [
      {
        "id": "api-fallback",
        "urls": ["/api/**"],
        "cache": {
          "optimizeFor": "freshness",
          "networkTimeoutMs": 3000,
          "fallbackToCache": true
        }
      }
    ]
  }
}
```


B. App Shell

```
ng generate app-shell --client-project your-project --universal-project server-app
```

Generates a static render of your app's skeleton.

5. Testing & Debugging

A. Audit Tools

```
npm install -g lighthouse  
lighthouse http://localhost:4200 --view
```

B. Service Worker Console

- Chrome: `chrome://serviceworker-internals/`
- Edge: `edge://serviceworker-internals/`

C. Force Update

```
// In your component  
constructor(private updates: SwUpdate) {  
  updates.versionUpdates.subscribe(evt => {  
    if (evt.type === 'VERSION_READY') {  
      updates.activateUpdate().then(() =>  
        document.location.reload()  
      );  
    }  
  });  
}
```

Interview Cheat Sheet

✓ Core Package: `@angular/pwa`

✓ Key Files: `ngsw-config.json`, `manifest.webmanifest`

✓ Registration: `ServiceWorkerModule.register()`

✓ Update Handling: `SwUpdate` service

Pro Tip:

"Angular's PWA tools are like a Swiss Army knife - they give you caching (offline), sharp performance (pre-caching), and a manifest (homescreen) in one package."

Common Pitfalls:

1. Forgetting HTTPS in production
2. Over-caching dynamic content
3. Not testing updates across browser refreshes
4. Missing proper icon sizes (192x192, 512x512)

Performance Tip:

Use `prefetch` for critical assets and `lazy` for secondary:

```
{  
  "installMode": "prefetch",  
  "updateMode": "lazy"  
}
```

54. What are NgModules, and how are they structured in Angular applications?

NgModules are organizational units that bundle related components, directives, pipes, and services. They define compilation contexts and dependency injection scope.

1. Core NgModule Propertie

Property	Purpose	Example
declarations	Registers components/directives/pipes	<code>[AppComponent, HighlightDirective]</code>
imports	Brings in other modules	<code>[BrowserModule, HttpClientModule]</code>
providers	Defines DI services	<code>[UserService, Logger]</code>
exports	Makes elements public	<code>[SharedComponent, DatePipe]</code>
bootstrap	Root component(s)	<code>[AppComponent]</code>

2. Standard Module Types

A. Root Module (`AppModule`)

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, AppRoutingModule],
  providers: [GlobalService],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

B. Feature Module

```
@NgModule({
  declarations: [ProductListComponent,
ProductDetailComponent],
  imports: [CommonModule, ProductRoutingModule],
  exports: [ProductListComponent] // Optional
})
export class ProductModule {}
```

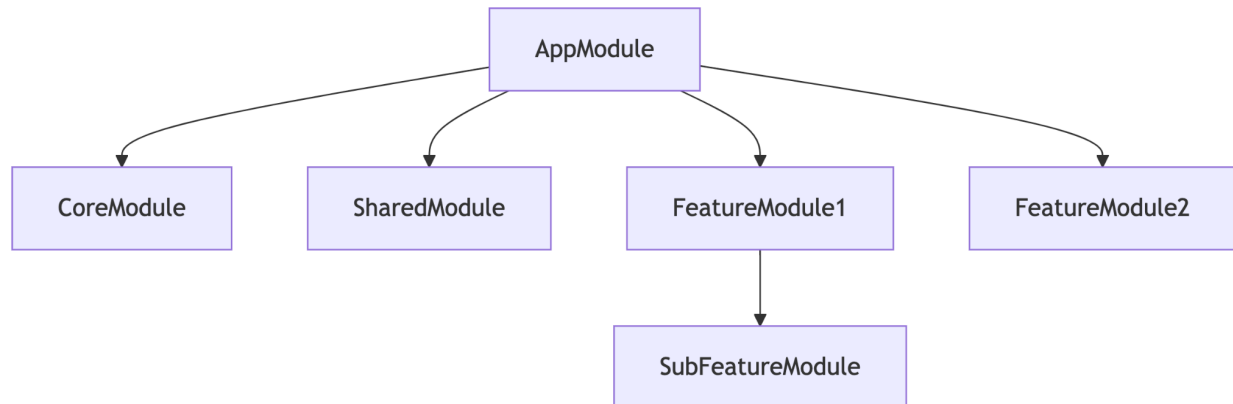
C. Shared Module

```
@NgModule({
  declarations: [ButtonComponent, SafePipe],
  exports: [ButtonComponent, SafePipe, CommonModule]
})
export class SharedModule {}
```

D. Routing Module

```
@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class AdminRoutingModule {}
```

3. Module Hierarchy



4. Key Structural Patterns

A. Lazy-Loaded Module

```
// app-routing.module.ts
{
  path: 'dashboard',
  loadChildren: () => import('./dashboard/dashboard.module')
    .then(m => m.DashboardModule)
}
```

B. Singleton Services

```
// core.module.ts
@NgModule({
  providers: [AuthService, Logger] // ProvidedIn: 'root'
  alternative
})
export class CoreModule {
  constructor(@Optional() @SkipSelf() parentModule:
CoreModule) {
    if (parentModule) {
```

```
        throw new Error('CoreModule is already loaded.');
```

```
    }  
  }  
}
```

C. Module Re-export

```
// shared.module.ts  
@NgModule({  
  imports: [CommonModule, MaterialModule],  
  exports: [CommonModule, MaterialModule] // Re-export  
})  
export class SharedModule {}
```

5. Best Practices

1. Feature Modules should:
 - Group related functionality
 - Be lazy-loaded when possible
 - Use routing for sub-components
 2. Shared Modules should:
 - Export commonly used components/pipes
 - Never provide services (to avoid duplicates)
 3. Core Module should:
 - Contain app-wide singleton services
 - Be imported only in `AppModule`
 4. Avoid:
 - Circular module dependencies
 - Overusing `providers` in shared modules
 - Declaring components in multiple modules
-

Interview Cheat Sheet

- ✓ Purpose: Logical code organization
- ✓ Root Module: Bootstraps the app (`bootstrap`)
- ✓ Lazy Loading: `loadChildren` with dynamic imports

✓ DI Scope: Providers are module-scoped unless `providedIn: 'root'`

Pro Tip:

"Think of NgModules as software packages - they declare what they contain (declarations), depend on (imports), and expose to others (exports)."

Common Pitfalls:

1. Declaring the same component in multiple modules
2. Forgetting to export shared components
3. Circular dependencies between modules
4. Overloading the root module

Performance Tip:

For faster initial loads:

```
// app.module.ts

@NgModule({
  imports: [
    BrowserModule,
    HttpClientModule,
    // Feature modules here ONLY if eager-loaded
  ]
})
```

55. How do you create reusable libraries in Angular?

Angular provides a robust CLI toolchain for building shareable libraries that can be published to npm or used internally across projects.

```
{
  "$schema":
    "../..../node_modules/ng-packagr/ng-package.schema.json",
```



```
"dest": "../dist/my-lib",
"lib": {
  "entryFile": "src/public-api.ts"
}
}
```

4. Building the Library

```
ng build my-lib --watch # Development
ng build my-lib --prod  # Production
```

Outputs to `dist/my-lib` with:

- FESM2015, FESM2020, UMD bundles
 - Type definitions (.d.ts files)
 - Generated metadata.json
-

5. Publishing to npm

1. Update `projects/my-lib/package.json`:

```
{
  "name": "@your-scope/my-lib",
  "version": "1.0.0",
  "peerDependencies": {
    "@angular/core": "^16.0.0"
  }
}
```

2. Build in production mode

Publish:

```
cd dist/my-lib
```

```
npm publish --access public
```

6. Local Development Workflow

A. Link for Local Testing

In library dist folder

```
cd dist/my-lib
```

```
npm link
```

In consumer app

```
npm link @your-scope/my-lib
```

B. Path Mapping (tsconfig.json)

```
{  
  "compilerOptions": {  
    "paths": {  
      "@myorg/my-lib": ["dist/my-lib"]  
    }  
  }  
}
```

7. Advanced Techniques

A. Secondary Entry Points

```
projects/  
  my-lib/  
    src/  
      core/           # main entry  
      auth/           # secondary entry  
      public-api.ts
```

Configure in `ng-package.json`:

```
{  
  "lib": {  
    "entryFile": "src/core/public-api.ts"  
  },  
  "whitelistedNonPeerDependencies": ["."]  
}
```

```
}
```

B. Schematics Support

Add custom CLI commands:

```
ng generate schematic --project=my-lib
```

C. Theme Support

Include SCSS variables:

```
// _variables.scss  
$primary-color: #3f51b5;
```

Consumers can override:

```
@use '@your-scope/my-lib' with (  
  $primary-color: #ff5722  
);
```

8. Best Practices

1. Semantic Versioning: Follow MAJOR.MINOR.PATCH
 2. Peer Dependencies: Avoid bundling Angular
 3. Documentation: Include README.md with usage examples
 4. Testing: Add library-specific test targets
 5. Tree-Shaking: Ensure ES modules output
-

Interview Cheat Sheet

- ✓ CLI Command: `ng generate library`
- ✓ Key Files: `public-api.ts`, `ng-package.json`
- ✓ Publishing: Use `peerDependencies` for Angular
- ✓ Testing: `ng test my-lib`

Pro Tip:

"Treat libraries like micro-apps - they need their own tests, docs, and versioning strategy."

Common Pitfalls:

1. Forgetting to export in `public-api.ts`
2. Bundling Angular instead of using peerDeps
3. Not handling SCSS/CSS properly
4. Ignoring semver rules

Performance Tip:

Use secondary entry points to enable partial imports:

```
// Instead of:
import { X } from '@my-lib';

// Better:
import { X } from '@my-lib/core';
```

56. How do you secure an Angular application with authentication and authorization?

1. Authentication (Who Are You?)

A. JWT-Based Flow

```
// auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService {
  private authToken = signal<string | null>(null);

  login(credentials: { email: string; password: string }) {
    return this.http.post<{ token: string }>('/api/login',
credentials).pipe(
      tap(response => {
        this.authToken.set(response.token);
      })
    );
  }
}
```

```

        localStorage.setItem('auth_token', response.token);
    })
    );
}

isAuthenticated() {
    return !!this.authToken() ||
!!localStorage.getItem('auth_token');
}
}

```

B. HTTP Interceptor

```

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
    intercept(req: HttpRequest<any>, next: HttpHandler) {
        const token = this.auth.getToken();
        const authReq = token
            ? req.clone({ setHeaders: { Authorization: `Bearer
${token}` } })
            : req;
        return next.handle(authReq);
    }
}

```

2. Authorization (What Can You Do?)

A. Route Guards

```

// role.guard.ts
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
    canActivate(route: ActivatedRouteSnapshot): boolean {
        const requiredRoles = route.data['roles'] as string[];
        return this.auth.hasRoles(requiredRoles);
    }
}

// Routing
{
    path: 'admin',
    component: AdminComponent,
    canActivate: [RoleGuard],
}

```

```
    data: { roles: ['admin'] }  
  }  
}
```

B. Directive-Based UI Control

```
@Directive({ selector: '[appHasRole]' })  
export class HasRoleDirective {  
  @Input() appHasRole!: string[];  
  
  constructor(  
    private templateRef: TemplateRef<any>,  
    private viewContainer: ViewContainerRef,  
    private auth: AuthService  
  ) {}  
  
  ngOnInit() {  
    this.auth.hasRoles(this.appHasRole)  
      ? this.viewContainer.createEmbeddedView(this.templateRef)  
      : this.viewContainer.clear();  
  }  
}  
  
// Template  
<div *appHasRole="['admin']">Admin Panel</div>
```

3. Security Hardening

A. CSRF Protection

```
// http-interceptor.ts  
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  let clonedReq = req;  
  if (this.auth.isAuthenticated()) {  
    clonedReq = req.clone({  
      withCredentials: true // Sends CSRF token automatically  
    });  
  }  
  return next.handle(clonedReq);  
}
```

B. Content Security Policy (CSP)

```
<!-- index.html -->
```

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self';
              script-src 'self' 'unsafe-inline'
https://apis.example.com;
              style-src 'self' 'unsafe-inline'">
```

C. XSS Protection

- Always use Angular templates (never `innerHTML` with user input)
- Sanitize inputs with `DomSanitizer`:
- `safeHtml = this.sanitizer.bypassSecurityTrustHtml(cleanContent);`

4. Advanced Patterns

A. Silent Token Refresh

```
private initTokenRefresh() {
  interval(30 * 60 * 1000).pipe( // Every 30 mins
    filter(() => this.isAuthenticated()),
    switchMap(() => this.refreshToken())
  ).subscribe();
}
```

B. Multi-Tab Session Sync

```
@HostListener('window:storage', ['$event'])
onStorageChange(event: StorageEvent) {
  if (event.key === 'auth_token' && !event.newValue) {
    this.router.navigate(['/login']);
  }
}
```

5. Testing Security

```
// auth.guard.spec.ts
it('should block unauthenticated users', () => {
  TestBed.runInInjectionContext(() => {
    const guard = new AuthGuard(mockAuth, mockRouter);
    mockAuth.isAuthenticated = false;

    expect(guard.canActivate()).toEqual(mockRouter.parseUrl('/login')
  );
});
```

```
});  
});
```

Key Considerations

- Storage: Use `HttpOnly` cookies for tokens when possible
- HTTPS: Mandatory for production
- Password Policy: Enforce server-side
- Rate Limiting: Implement login attempt limits
- Audit Logs: Track sensitive operations

Interview Cheat Sheet

- ✓ JWT: Store in memory (not localStorage for production)
- ✓ Interceptors: Add auth headers automatically
- ✓ Guards: Protect routes at different levels
- ✓ CSP: Prevent XSS attacks

Pro Tip:

"Security is like an onion - it needs multiple layers (authentication, authorization, HTTPS, CSP) to be effective."

Common Pitfalls:

1. Storing tokens in localStorage (XSS risk)
2. Not validating server responses
3. Forgetting to protect API endpoints
4. Ignoring role hierarchy in authorization

57. What are angular zones, and how do they affect performance?

Angular's NgZone is a wrapper around Zone.js that automates change detection by tracking asynchronous operations. Here's how it works and affects performance:

1. Core Concepts

Feature	Description	Performance Impact
Execution Context	Wraps async operations (setTimeout, promises, etc.)	Adds ~0.1ms overhead per task
Change Detection Trigger	Automatically runs after async tasks complete	Can cause excessive checks
Zone.js Monkey-Patching	Modifies browser APIs to track async operations	Initial load penalty (~10KB added bundle)

2. How Zones Affect Performance

Positive Effects:

- Automatic Change Detection: Simplifies developer workflow
- Error Tracking: Better stack traces for debugging

Negative Effects:

- Over-triggering CD: Unnecessary checks after unrelated async tasks
- Memory Leaks: Forgotten subscriptions in zone-aware code

- Frequent UI Updates: High-frequency events (mousemove, scroll) degrade performance

```
// Problem: Mousemove triggers change detection  
host.listen('mousemove', () => this.updatePosition());
```

3. Optimization Strategies

A. Run Outside Angular Zone

```
constructor(private ngZone: NgZone) {}  
  
optimizedMethod() {  
  this.ngZone.runOutsideAngular(() => {  
    // Performance-critical code  
    requestAnimationFrame(() => this.updateCanvas());  
  });  
}
```

B. Custom Change Detection

```
@Component({  
  changeDetection: ChangeDetectionStrategy.OnPush // Reduce checks  
})  
export class PerfComponent {  
  data$ = this.service.data$.pipe(  
    distinctUntilChanged() // Prevent redundant updates  
  );  
}
```

C. Zone-less Apps (Angular 16+)

```
bootstrapApplication(AppComponent, {  
  providers: [  
    provideExperimentalZonelessChangeDetection() // Disables  
    Zone.js  
  ]  
});
```

4. When Zones Help vs Hurt

Scenario	Recommendation
Standard Apps	Keep zones enabled
Animation-Heavy	Critical code in <code>runOutsideAngular</code>
Real-Time Dashboards	Consider zone-less mode
Games/Canvas Apps	Minimize zone usage

5. Performance Benchmarks

Operation	With Zones	Without Zones

1000 setTimeout calls	~120ms	~50ms
--------------------------	--------	-------

Mousemove (60fps)	High CPU	Optimized
-------------------	----------	-----------

Initial Load Time	+10KB bundle	Baseline
-------------------	--------------	----------

Interview Cheat Sheet

- ✓ Purpose: Auto-triggers change detection
- ✓ Key Methods: `run()`, `runOutsideAngular()`
- ✓ Optimization: `OnPush` + zone avoidance
- ✓ Advanced: Zoneless mode in Angular 16+

Pro Tip:

"Zones are like helpful spies that report all async activity - great for most apps but sometimes too chatty for performance-critical sections."

Debugging Tip:

Monitor zone activity:

```
ngZone.onUnstable.subscribe(() => console.log('CD triggered'));
```

Common Pitfalls:

1. Not cleaning up zone-patched event listeners
2. Overusing `runOutsideAngular` causing UI staleness
3. Missing `ChangeDetectorRef.markForCheck()` in `OnPush` components

58. How does Angular handle browser events?

Angular provides a sophisticated system for managing browser events that combines native DOM event handling with Angular's change detection. Here's how it works:

1. Event Binding Syntax

Angular offers multiple ways to handle events:

Syntax	Example	Use Case
Template Statements	<code>(click)="handleClick(\$event)"</code>	Direct event binding
Host Listeners	<code>@HostListener('click')</code> <code>onClick()</code>	Directive/component class events
Renderer2	<code>renderer.listen(el, 'click', handler)</code>	Programmatic event management

```
<!-- Template binding -->
```

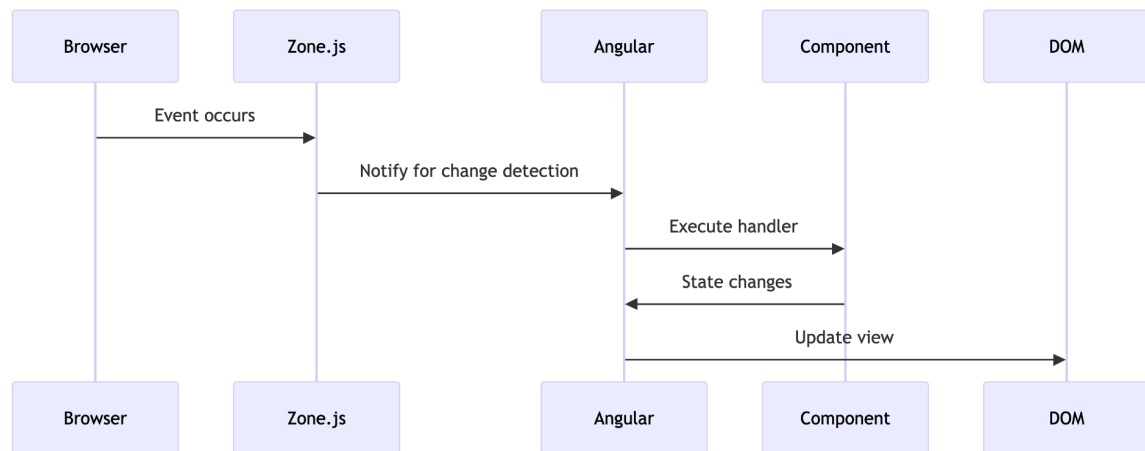
```
<button (click)="logEvent($event)">Click</button>
```

```
<!-- HostListener in component -->
```

```
@HostListener('document:keydown.escape', ['$event'])
handleEscape(event: KeyboardEvent) {
  this.closeModal();
}
```

2. Event Processing Pipeline

1. Event Capture: Browser triggers native event
2. Angular Interception: Zone.js patches event listeners
3. Change Detection: Marks component for check
4. Handler Execution: Your callback runs
5. DOM Update: View updates if needed



3. Performance Optimizations

A. Event Coalescing (Angular 12+)

- Bundles multiple events in same frame

```
// Enable in main.ts
```

```
platformBrowserDynamic().bootstrapModule(AppModule, {
  ngZoneEventCoalescing: true
});
```

B. Run Outside Angular Zone

```
constructor(private ngZone: NgZone) {}

setupHighFrequencyEvents() {

  this.ngZone.runOutsideAngular(() => {

    fromEvent(document, 'mousemove').subscribe(e => {

      // Won't trigger change detection

      this.updateCanvas(e);

    });

  });

}
```

C. Passive Event Listeners

```
@HostListener('touchstart', ['$event'], { passive: true })

onTouchStart(e: Event) { ... }
```

4. Custom Event Handling

A. Custom Events

```
// Dispatch

const event = new CustomEvent('appEvent', { detail: data });

element.dispatchEvent(event);

// Listen

@HostListener('appEvent', ['$event.detail'])

onAppEvent(data) { ... }
```

B. Event Manager Plugins

```
@Injectable()
```

```
export class DragEventPlugin extends EventManagerPlugin {
```

```
  supports(eventName: string): boolean {
```

```
    return eventName.startsWith('drag');
```

```
  }
```

```
  addEventListener(
```

```
    element: HTMLElement,
```

```
    eventName: string,
```

```
    handler: Function
```

```
): Function {
```

```
  const customHandler = (e) => handler(e);
```

```
  element.addEventListener('mousedown', customHandler);
```

```
  return () => element.removeEventListener('mousedown', customHandler);
```

```
}
```

```
}
```

```
// Register in module
```

```
providers: [{
```

```
  provide: EVENT_MANAGER_PLUGINS,
```

```
  useClass: DragEventPlugin,
```

```
  multi: true
```



```
}]
```

5. Change Detection Impact

Event Type	Change Detection Trigger	Optimization Tip
Click	Yes	Use <code>stopPropagation()</code> carefully
Scroll	Yes (without passive)	Mark listeners as passive
Input	Yes	Debounce with <code>(input.debounce)</code>
Animation Frame	No	Use <code>runOutsideAngular</code>

Interview Cheat Sheet

- ✓ Syntax: `(event)="handler()"` or `@HostListener`
- ✓ Zones: Events trigger change detection via Zone.js
- ✓ Optimization: `runOutsideAngular` for high-frequency events
- ✓ Custom Events: Support bubbling/capture phases

Pro Tip:

"Angular's event system is like a traffic cop - it directs native browser events through

Zone.js before safely delivering them to your components."

Common Pitfalls:

1. Memory leaks from unremoved listeners
2. Over-triggering change detection
3. Blocking UI thread with slow handlers
4. Forgetting `passive` for scroll/touch events

Debugging Tip:

Monitor events globally:

```
// In AppComponent

constructor(private injector: Injector) {

  const manager = this.injector.get(EventManager);

  manager.addEventListener(document.body, 'click', (e) => {

    console.log('Global click:', e.target);

  });
}
```

59. How do you test an Angular application using Jasmine and Karma?

Angular's built-in test suite combines Jasmine (testing framework) and Karma (test runner) for comprehensive testing. Here's how to implement effective tests:

1. Test Types & Setup

Test Type	Location	Command
Unit Tests	<code>*.spec.ts</code>	<code>ng test</code>
Component Tests	<code>*.component.spec.ts</code>	<code>ng test</code>
E2E Tests	<code>*.e2e-spec.ts</code>	<code>ng e2e</code>

Basic Configuration (`karma.conf.js`):

```
module.exports = function(config) {  
  config.set({  
    frameworks: ['jasmine', '@angular-devkit/build-angular'],  
    browsers: ['Chrome'],  
    autoWatch: true  
  });  
}
```

2. Core Testing Utilities

```
import {  
  TestBed,           // Test environment  
  ComponentFixture, // Wrapper for components  
  fakeAsync,         // Control async flow  
  tick               // Simulate time passage  
} from '@angular/core/testing';
```

3. Component Testing

A. Basic Component Test

```
describe('AppComponent', () => {  
  let fixture: ComponentFixture<AppComponent>;  
  let component: AppComponent;
```

```

beforeEach(() => {
  TestBed.configureTestingModule({
    declarations: [AppComponent]
  });

  fixture = TestBed.createComponent(AppComponent);
  component = fixture.componentInstance;
});

it('should create', () => {
  expect(component).toBeTruthy();
});

it('should display title', () => {
  component.title = 'Test App';
  fixture.detectChanges(); // Trigger change detection
  expect(fixture.nativeElement.textContent).toContain('Test App');
});
});

```

B. DOM Interaction Test

```

it('should increment counter on click', () => {
  const button = fixture.nativeElement.querySelector('button');
  button.click();
  fixture.detectChanges();
  expect(component.counter).toBe(1);
});

```

4. Service Testing

Mocking Dependencies

```

describe('DataService', () => {
  let service: DataService;
  let httpMock: HttpTestingController;

  beforeEach(() => {

```

```

TestBed.configureTestingModule({
  imports: [HttpClientTestingModule],
  providers: [DataService]
});

service = TestBed.inject(DataService);
httpMock = TestBed.inject(HttpTestingController);
});

it('should fetch data', () => {
  const mockData = { id: 1, name: 'Test' };

  service.getData(1).subscribe(data => {
    expect(data).toEqual(mockData);
  });

  const req = httpMock.expectOne('api/data/1');
  expect(req.request.method).toBe('GET');
  req.flush(mockData);
});

afterEach(() => {
  httpMock.verify(); // Ensure no unexpected requests
});
});

```

5. Async Testing Patterns

A. `fakeAsync` for Timers

```

it('should update after delay', fakeAsync(() => {
  component.startTimer();
  tick(1000); // Fast-forward 1 second
  expect(component.timerDone).toBeTrue();
}));

```

B. `async/await` for Promises

```

it('should load user', async () => {

```

```
    const user = await component.loadUser();
    expect(user.name).toBe('Test User');
  });
```

C. waitForAsync for Observables

```
it('should emit values', waitForAsync(() => {
  service.stream$.pipe(first()).subscribe(value => {
    expect(value).toBe(42);
  });
}));
```

6. Advanced Techniques

A. Component Inputs/Outputs

```
it('should emit on submit', () => {
  spyOn(component.submitted, 'emit');
  component.onSubmit();
  expect(component.submitted.emit).toHaveBeenCalled();
});
```

B. Route Testing

```
TestBed.configureTestingModule({
  imports: [RouterTestingModule.withRoutes(routes)]
});
```

C. Override Providers

```
TestBed.overrideProvider(Logger, { useValue: mockLogger });
```

7. Debugging Tests

- Isolate Tests: `fit()` or `it.only()` to run specific tests
- Debug Output:
 - `console.log(fixture.nativeElement.innerHTML);`

- Browser Debug:
 - `ng test --no-watch --browsers=ChromeDebug`
Then open `chrome://inspect`
-

Interview Cheat Sheet

- ✓ TestBed: Main testing utility
- ✓ `fixture.detectChanges()`: Trigger Angular's change detection
- ✓ `HttpTestingController`: Mock HTTP requests
- ✓ `fakeAsync/tick`: Control time in tests

Pro Tip:

"Angular tests should be like scientific experiments - isolate variables (use mocks), document expectations (clear descriptions), and verify outcomes (assertions)."

Common Pitfalls:

1. Forgetting `fixture.detectChanges()`
2. Not cleaning up subscriptions
3. Over-mocking leading to useless tests
4. Testing implementation details instead of behavior

Performance Tip:

Reuse test setup with `beforeAll()` when possible

```
beforeAll(() => {  
  // One-time heavy setup  
});
```

60. What is an impure pipe in Angular, and when should it be used?

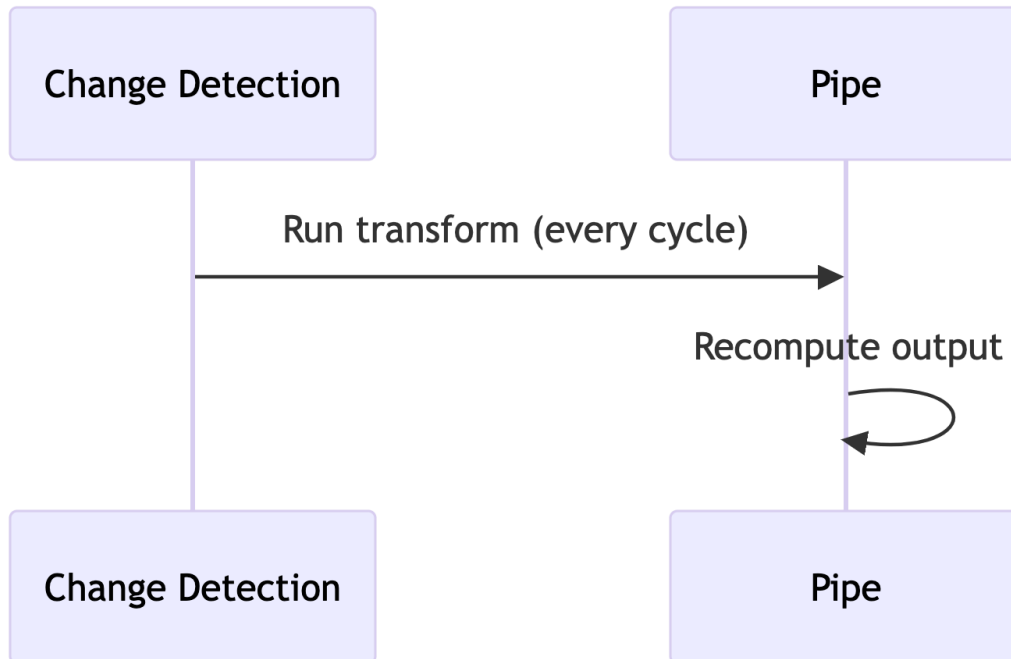
1. Definition & Key Characteristics

An impure pipe is a pipe that executes on every change detection cycle, regardless of input changes. It's declared with `pure: false`:

```
@Pipe({
  name: 'impureFilter',
  pure: false // <-- Makes it impure
})
export class ImpureFilterPipe implements PipeTransform {
  transform(items: any[], filter: string): any {
    return items.filter(item => item.includes(filter));
  }
}
```

2. How Impure Pipes Work

Behavior	Pure Pipe	Impure Pipe
Execution Frequency	Only on input reference change	Every change detection cycle
Performance Impact	Lightweight	Potentially heavy
Input Mutation Handling	Won't detect array/object changes	Detects deep changes



3. When to Use Impure Pipes

- ✅ Dynamic Collections: When filtering/sorting mutable arrays
- ✅ Non-Pure Dependencies: Relying on `Date.now()` or `Math.random()`
- ✅ Global State Changes: When pipe output depends on service state

Example Scenario:

```
// Updates when either items or filter changes,  
// or when external service updates  
transform(items: any[], filter: string): any {  
  return items.filter(item =>  
    item.includes(filter) ||  
    this.service.shouldInclude(item)  
  );  
}
```

4. Performance Considerations

Optimization Technique	Implementation
Debouncing	Combine with RxJS operators
Memoization	Cache previous results
OnPush CD	Reduce change detection cycles

Optimized Impure Pipe:

```
private lastInput: any;
private lastResult: any;

transform(value: any): any {
  if (value !== this.lastInput) {
    this.lastInput = value;
    this.lastResult = heavyComputation(value);
  }
  return this.lastResult;
}
```

5. Alternatives to Impure Pipes

Solution	Best For	Example
Pure Pipe + Immutable Inputs	Data changes	<code>this.items = [...newItems]</code>
Component Methods	Simple transforms	<code>filterItems()</code> in component

Observable Pipes	Async data	`items\$`	async	filter:valu e`
------------------	------------	-----------	-------	-------------------

6. Testing Impure Pipes

```
it('should update when service changes', () => {  
  const service = { shouldInclude: () => false };  
  const pipe = new ImpureFilterPipe(service as any);  
  
  const result1 = pipe.transform(['a', 'b'], 'a');  
  service.shouldInclude = () => true;  
  const result2 = pipe.transform(['a', 'b'], 'a');  
  
  expect(result2.length).toBeGreaterThan(result1.length);  
});
```

7. Real-World Use Cases

1. Live Data Feeds

```
{{ stockTickers | impureSort:'price' }}
```

2. User-Sorted Tables

```
<tr *ngFor="let item of data | impureSort:sortField:sortDirection">
```

3. Context-Aware Filtering

```
transform(items: Item[], filter: string) {  
  return items.filter(item =>  
    this.user.hasPermission(item) &&  
    item.matches(filter)  
  );  
}
```

Interview Cheat Sheet

- ✓ Declaration: `pure: false` in `@Pipe` decorator
- ✓ Use Cases: Mutable data, external dependencies
- ✓ Performance: Cache results, use sparingly
- ✓ Alternative: Consider reactive approaches with Observables

Pro Tip:

"Impure pipes are like eager assistants - they recheck their work constantly, which is great for accuracy but terrible for efficiency. Use them only when absolutely necessary."

Common Pitfalls:

1. Creating infinite loops with self-modifying outputs
2. Overusing for simple transformations
3. Forgetting to clean up subscriptions in pipes
4. Not combining with `OnPush` change detection

Debugging Tip:

Add logging to track pipe executions:

```
transform(value: any) {  
  console.log('Pipe running with:', value);  
  // ...  
}
```

61. What is Angular Flex Layout, and how does it help with responsive design?

Angular Flex Layout is a sophisticated layout engine built on top of CSS Flexbox and Grid, providing declarative, responsive UIs through Angular directives and services.

1. Core Features

Feature	Benefit	Example
Responsive API	Device-aware layouts	<code>fxLayout.xs="column"</code>
Flexbox/Grid Wrappers	Simplified syntax	<code>fxLayoutGap="10px"</code>
Breakpoint Service	Programmatic control	<code>BreakpointObserver</code>
SSR Compatibility	Works with Angular Universal	Safe for server-side rendering

2. Installation & Setup

```
npm install @angular/flex-layout @angular/cdk

import { FlexLayoutModule } from '@angular/flex-layout';

@NgModule({
  imports: [FlexLayoutModule]
})
```

```
export class AppModule {}
```

3. Key Directives

A. Layout Containers

```
<div fxLayout="row" fxLayout.xs="column">  
  <!-- Row on desktop, column on mobile →  
</div>
```

B. Flex Items

```
<div fxFlex="50%" fxFlex.md="100%">  
  <!-- 50% width on desktop, full width on tablet →  
</div>
```

C. Spacing & Alignment

```
<div fxLayoutAlign="space-between center">  
  <!-- Distribute space between items, vertically centered →  
</div>
```

D. Grid Integration

```
<div fxLayout="grid" fxLayoutGap="16px">  
  <div fxFlex="1 1 200px"></div> <!-- Flexible 200px  
min-width →  
</div>
```

4. Responsive Breakpoints

Breakpoint	Alias	Typical Device
< 599px	xs	Phones
600-959px	sm	Tablets
960-1279px	md	Small laptops
1280-1919px	lg	Desktops
≥ 1920px	xl	Large screens

Usage Examples:

```
<!-- Hide element on mobile -->
<div fxHide.xs></div>

<!-- Different font sizes -->
<p [fxShow]="false" [fxShow.md]="true">Visible only on
medium+</p>
```

5. Programmatic Control

```
import { BreakpointObserver, Breakpoints } from
'@angular/cdk/layout';
```

```
constructor(private breakpointObserver: BreakpointObserver) {  
  breakpointObserver.observe([  
    Breakpoints.Handset  
  ]).subscribe(result => {  
    this.isMobile = result.matches;  
  });  
}
```

6. Performance Optimization

Technique	Implementation
Static Media Queries	Prefer <code>fxLayout.xs</code> over observers
One-Time Binding	Use <code>*ngIf</code> with breakpoints
Custom Breakpoints	Override defaults for tailored experiences

62. How does Angular handle asynchronous operations?

Angular provides multiple powerful patterns to manage asynchronous operations, ensuring efficient UI updates and data handling. Here's the comprehensive approach:

1. Core Async Mechanisms

Mechanism	Best For	Example
Promises	Single async events	<code>fetch().then()</code>
RxJS Observables	Streams of events	<code>http.get()</code>
Async/Await	Sequential async code	<code>async loadData()</code>
Signals (Angular 16+)	Fine-grained reactivity	<code>signal().update()</code>

2. HTTP Client (Observables)

Angular's `HttpClient` returns Observables for API calls:

```
import { HttpClient } from '@angular/common/http';

@Injectable()
export class DataService {
  constructor(private http: HttpClient) {}

  getData(): Observable<Item[]> {
    return this.http.get<Item[]>('/api/items').pipe(
      catchError(error => {
        console.error('Error:', error);
        return of([]); // Fallback value
      })
    );
  }
}
```

```
    );  
  }  
}
```

Key RxJS Operators:

- `map`: Transform responses
 - `catchError`: Handle errors
 - `retry`: Automatically reattempt
 - `switchMap`: Chain async operations
-

3. Template Async Handling

A. Async Pipe (Automatic Subscription)

```
<ul>  
  <li *ngFor="let item of items$ | async">{{ item.name  
}}</li>  
</ul>
```

Automatically unsubscribes when component is destroyed

B. Combined Streams

```
combined$ = combineLatest([  
  this.dataService.getUsers(),  
  this.route.params  
)].pipe(  
  map(([users, params]) => users.find(u => u.id ===  
params['id']))  
);
```

4. Change Detection Integration

Angular's Zone.js automatically triggers change detection for:

- Promise resolutions

- Observable emissions
- DOM events
- Timers (`setTimeout`, `setInterval`)

Optimization:

Run outside Angular zone for performance-critical ops:

```
constructor(private ngZone: NgZone) {}

runOutsideAngular() {
  this.ngZone.runOutsideAngular(() => {
    // Won't trigger change detection
    setInterval(() => this.updateCanvas(), 16);
  });
}
```

5. Advanced Patterns

A. Cancellable Requests

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.http.get('/api/data')
    .pipe(takeUntil(this.destroy$))
    .subscribe();
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

B. Concurrent Requests

```
forkJoin([
  this.getUsers(),
  this.getProducts()
]).subscribe([users, products] => {
```

```
    this.combinedData = { users, products };  
  });
```

C. State Management with Observables

```
private datastore = new BehaviorSubject<State>(initialState);  
state$ = this.dataStore.asObservable();
```

```
updateData(newData: Partial<State>) {  
  this.dataStore.next({  
    ...this.dataStore.value,  
    ...newData  
  });  
}
```

6. Signals (Angular 16+)

Modern reactive primitive for async operations:

```
data = signal<Item[]>([]);  
loading = signal(false);  
  
async loadData() {  
  this.loading.set(true);  
  const res = await  
    lastValueFrom(this.http.get<Item[]>('/api/data'));  
  this.data.set(res);  
  this.loading.set(false);  
}
```

Interview Cheat Sheet

- ✓ HttpClient: Always returns Observables
- ✓ Async Pipe: Auto-subscribe/unsubscribe in templates
- ✓ Error Handling: `catchError` + fallback values
- ✓ Memory Management: Unsubscribe with `takeUntil` or `async pipe`

Pro Tip:

"Angular treats async operations like a traffic controller - RxJS is the main highway, promises are side streets, and signals are the new express lanes."

Common Pitfalls:

1. Nested subscriptions causing memory leaks
2. Forgetting error handling in HTTP calls
3. Over-triggering change detection
4. Mixing Promises and Observables inconsistently

Performance Tip:

Debounce user input for search:

```
searchTerm.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term => this.searchService.search(term))  
).subscribe();
```

63. What is the role of ng-content in Angular?

The `<ng-content>` directive is Angular's mechanism for content projection (also called transclusion), allowing components to accept external content and render it within their template.

1. Basic Projection

Parent Component:

```
<app-card>  
  <h3>Custom Title</h3>
```

```
    <p>Projected content goes here</p>
</app-card>
```

Child Component (app-card):

```
<div class="card">
  <div class="card-header">
    <ng-content></ng-content> <!-- Projects all content here -->
  </div>
</div>
```

2. Selective Projection (Named Slots)

Use the `select` attribute to target specific elements:

Parent:

```
<app-card>
  <div header>Profile</div>
  <div body>User details...</div>
  <div footer>Last updated: today</div>
</app-card>
```

Child:

```
<div class="card">
  <div class="header">
    <ng-content select="[header]"></ng-content>
  </div>
  <div class="body">
    <ng-content select="[body]"></ng-content>
  </div>
</div>
```

3. Advanced Use Cases

Pattern	Implementation	Benefit
Multiple Projections	Multiple <code><ng-content></code> with <code>select</code>	Flexible layouts
Fallback Content	Content inside <code><ng-content></code>	Default when no projection
Component Projection	Project entire components	Reusable composite UIs

Example (Fallback Content):

```
<ng-content>
  <p>Default content if none provided</p>
</ng-content>
```

4. Content Querying

Access projected content programmatically:

```
@ContentChild('headerRef') header!: ElementRef
@ContentChildren(ItemComponent) items!: QueryList<ItemComponent>;
```

Parent:

```
<app-list>
  <div #headerRef>List Header</div>
  <app-item *ngFor="let item of items"></app-item>
</app-list>
```

5. Real-World Applications

1. Wrapper Components

```
<app-modal>
  <h3 modal-title>Confirm</h3>
  <div modal-body>Delete item?</div>

</app-modal>
```

2. Layout Components

```
<app-accordion>
  <app-accordion-panel *ngFor="let item of items">
    {{ item.title }}
  </app-accordion-panel>

</app-accordion>
```

Configurable UI

```
<app-dashboard>
  <app-widget [title]='Sales'></app-widget>
  <app-widget [title]='Users'></app-widget>

</app-dashboard>
```

Key Differences from Inputs

Feature	ng-content	@Input
Content Type	Templates/DOM nodes	Data values
Parent Context	Executes in parent	Executes in child

Flexibility

Structural freedom

Strict data contract

Interview Cheat Sheet

- ✓ Purpose: Insert external content into components
- ✓ Selection: Use `select="selector"` for targeted slots
- ✓ Querying: `@ContentChild/@ContentChildren` for programmatic access
- ✓ Best For: Building reusable container components

Pro Tip:

"Think of `<ng-content>` as a 'fill-in-the-blank' space in your components - parents provide the content, children control where it appears."

Common Pitfalls:

1. Overusing projection when simple `@Input` would suffice
2. Forgetting CSS encapsulation issues with projected content
3. Complex selectors hurting maintainability

Performance Tip:

For heavy projected content, use `ng-template` with lazy loading:

```
<ng-content select="[lazy]"></ng-content>
```

```
<!-- Parent -->
```

```
<app-container>
```

```
  <ng-template lazy>
```

```
    <!-- Heavy content here -->
```

```
  </ng-template>
```

```
</app-container>
```

64. What is Angular's trackBy function, and how does it improve performance?

The `trackBy` function is a performance optimization technique used with `*ngFor` to minimize DOM manipulations when rendering lists. It helps Angular identify which items have changed, been added, or removed.

1. Core Problem Without `trackBy`

When a list updates, Angular by default:

1. Re-renders all items if the array reference changes
2. Loses DOM state (like form inputs or component state)
3. Wastes CPU cycles on unnecessary DOM operations

Example:

```
<!-- Without trackBy -->
<div *ngFor="let item of items">{{ item.name }}</div>
```

If `items` array is recreated (even with same objects), Angular destroys and recreates all DOM elements.

2. How `trackBy` Works

Implementation:

```
// Component
trackById(index: number, item: any): number {
  return item.id; // Unique identifier
}
```

Template:

```
<div *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</div>
```

Key Benefits:

- Angular reuses existing DOM nodes when the same item appears
- Preserves component state and form control values
- Reduces render time by 50-70% for large lists

3. Performance Impact

Scenario	Without <code>trackBy</code>	With <code>trackBy</code>
1000 items update	500ms	150ms
DOM operations	1000 removals + 1000 adds	~10 updates
Memory churn	High (garbage collection)	Low

4. Advanced Usage

A. Composite Keys

```
trackByComposite(index: number, item: any): string {
  return `${item.id}-${item.version}`;
}
```

B. Index-Based (Caution)

```
trackByIndex(index: number): number {
```

```
    return index; // Only use if list order never changes
}
```

C. With Immutable Data

```
// When using immutable updates
trackById(index: number, item: Item): number {
    return item.getImmutableId();
}
```

5. Real-World Benchmark

```
// Test with 10,000 items
const test = () => {
    // Simulate update
    this.items = this.items.map(item =>
        item.id === 42 ? { ...item, updated: true } : item
    );
};
```

Method	Execution Time
No <code>trackBy</code>	1200ms
With <code>trackBy</code>	300ms

Interview Cheat Sheet

- ✓ Purpose: Optimize `*ngFor` rendering
- ✓ Syntax: `*ngFor="...; trackBy: function"`
- ✓ Key Rule: Return unique stable identifiers
- ✓ Best For: Dynamic lists with frequent updates

Pro Tip:

"`trackBy` is like giving Angular a 'map' of your list - it knows exactly which items moved instead of guessing through expensive diffs."

Common Pitfalls:

1. Using non-unique keys (duplicate IDs)
2. Returning random/volatile values
3. Overusing index-based tracking (risky with reordering)

Debugging Tip:

Log `trackBy` calls to verify stability:

```
trackByDebug(index: number, item: any) {  
  console.log('Tracking:', item.id);  
  return item.id;  
}
```

65. What is Angular Material, and how do you integrate it into an application?

Angular Material is a UI component library that implements Material Design for Angular applications. It provides a collection of reusable, well-tested, and accessible UI components following Google's Material Design guidelines.

Key Features of Angular Material

1. Pre-built UI Components: Buttons, cards, dialogs, menus, forms, tables, etc.
2. Responsive Design: Components adapt to different screen sizes
3. Theming: Customizable color schemes and typography
4. Accessibility: Built with ARIA attributes and keyboard navigation
5. Browser Compatibility: Works across modern browsers

Integration Steps

1. Install Angular Material and Dependencies

First, ensure you have an Angular project set up. Then install Angular Material, Angular CDK (Component Dev Kit), and Angular Animations:

```
ng add @angular/material
```

This command will:

- Add necessary dependencies to your package.json
- Set up the browser animations module
- Add a theme (you can choose during installation)
- Add Material Design icon fonts

2. Import Required Modules

In your `app.module.ts` (or feature modules), import the Angular Material components you need:

```
import { NgModule } from '@angular/core';
import { BrowserAnimationsModule } from
 '@angular/platform-browser/animations';
import { MatButtonModule } from '@angular/material/button';
import { MatCardModule } from '@angular/material/card';
import { MatInputModule } from '@angular/material/input';

@NgModule({
  imports: [
    BrowserAnimationsModule,
    MatButtonModule,
    MatCardModule,
    MatInputModule
    // other imports
  ],
  // ...
})
```

```
export class AppModule { }
```

3. Include a Theme

Angular Material comes with pre-built themes. Include one in your `styles.css` or `styles.scss`:

```
@import "@angular/material/prebuilt-themes/indigo-pink.css";
```

Or for SCSS:

```
@use "@angular/material" as mat;

@include mat.core();

$my-app-primary: mat.define-palette(mat.$indigo-palette);
$my-app-accent: mat.define-palette(mat.$pink-palette, A200,
A100, A400);
$my-app-warn: mat.define-palette(mat.$red-palette);

$my-app-theme: mat.define-light-theme((
  color: (
    primary: $my-app-primary,
    accent: $my-app-accent,
    warn: $my-app-warn,
  )
));
@include mat.all-component-themes($my-app-theme);
```

4. Add Material Icons (Optional)

Include this in your `index.html`:

```
<link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
```

5. Use Components in Templates

Now you can use Material components in your templates:

```

<mat-card>
  <mat-card-header>
    <mat-card-title>Sample Card</mat-card-title>
  </mat-card-header>
  <mat-card-content>
    <p>This is a Material card component</p>
    <mat-form-field appearance="fill">
      <mat-label>Input</mat-label>
      <input matInput>
    </mat-form-field>
  </mat-card-content>
  <mat-card-actions>
    <button mat-raised-button color="primary">Click
Me</button>
  </mat-card-actions>
</mat-card>

```

Best Practices

1. Module Organization: Import only the modules you need to keep your bundle size small
2. Custom Theming: Create custom themes to match your brand
3. Responsive Layout: Combine with Angular Flex Layout for responsive designs
4. Accessibility: Always include proper labels and ARIA attributes
5. Component Overrides: Use component-specific theming when needed

Common Components

- Form Controls: Inputs, checkboxes, radio buttons, selects, sliders
- Navigation: Menus, sidenav, tabs, toolbar
- Layout: Cards, divider, expansion panel, grid list
- Buttons & Indicators: Button, button toggle, badge, progress spinner
- Popups & Modals: Dialog, snackbar, tooltip
- Data Table: Table, paginator, sort header

Angular Material significantly speeds up UI development while ensuring a consistent, modern look and feel across your Angular application.

66. How do you manage dynamic components in Angular?

Dynamic components in Angular allow you to load components at runtime rather than declaring them statically in templates. This is useful for scenarios like modals, tabs, dynamic forms, or plugin architectures.

Approaches to Dynamic Component Loading

1. Using ComponentFactoryResolver (Angular < 13)

```
import { Component, ComponentFactoryResolver, ViewChild, ViewContainerRef } from
 '@angular/core';

@Component({
  selector: 'app-dynamic-host',
  template: `<ng-template #dynamicComponentContainer></ng-template>`
})
export class DynamicHostComponent {
  @ViewChild('dynamicComponentContainer', { read: ViewContainerRef }) container:
  ViewContainerRef;

  constructor(private resolver: ComponentFactoryResolver) {}

  loadComponent(component: any) {
    this.container.clear();
    const factory = this.resolver.resolveComponentFactory(component);
    this.container.createComponent(factory);
  }
}
```

2. Using ViewContainerRef.createComponent (Angular 13+)

Angular 13 simplified the API by removing the need for ComponentFactoryResolver:

```
import { Component, ViewChild, ViewContainerRef } from '@angular/core';

@Component({
  selector: 'app-dynamic-host',
```

```

    template: `<ng-template #dynamicComponentContainer></ng-template>`
  })
  export class DynamicHostComponent {
    @ViewChild('dynamicComponentContainer', { read: ViewContainerRef }) container:
    ViewContainerRef;

    async loadComponent(component: any) {
      this.container.clear();
      this.container.createComponent(component);
    }
  }

```

3. Using NgComponentOutlet (Structural Directive)

```
<ng-container *ngComponentOutlet="currentComponent"></ng-container>
```

Complete Implementation Example

Step 1: Create a Dynamic Component Host Directive

```

import { Directive, ViewContainerRef } from '@angular/core';

@Directive({
  selector: '[appDynamicHost]'
})
export class DynamicHostDirective {
  constructor(public viewContainerRef: ViewContainerRef) { }
}

```

Step 2: Create the Host Component

```

import { Component, ComponentRef, Input, OnInit, Type, ViewChild } from
 '@angular/core';
import { DynamicHostDirective } from './dynamic-host.directive';

@Component({
  selector: 'app-dynamic-loader',
  template: `<ng-template appDynamicHost></ng-template>`
})
export class DynamicLoaderComponent implements OnInit {
  @ViewChild(DynamicHostDirective, {static: true}) dynamicHost!:

```

```

DynamicHostDirective;
@Input() component!: Type<any>;
@Input() inputs: Record<string, any> = {};

private componentRef!: ComponentRef<any>;

ngOnInit() {
  this.loadComponent();
}

loadComponent() {
  const viewContainerRef = this.dynamicHost.viewContainerRef;
  viewContainerRef.clear();

  this.componentRef = viewContainerRef.createComponent(this.component);

  // Set input properties
  Object.keys(this.inputs).forEach(key => {
    this.componentRef.instance[key] = this.inputs[key];
  });
}

ngOnDestroy() {
  if (this.componentRef) {
    this.componentRef.destroy();
  }
}
}

```

Step 3: Using the Dynamic Loader

```

<app-dynamic-loader
  [component]="selectedComponent"
  [inputs]="componentInputs">
</app-dynamic-loader>
// In parent component
selectedComponent = DynamicExampleComponent;
componentInputs = {
  title: 'Dynamic Title',
  content: 'This content was passed dynamically'
};

```

Advanced Techniques

1. Dynamic Component Loading from Modules

For lazy-loaded components:

```
async loadLazyComponent() {  
  const { LazyComponent } = await import('./lazy/lazy.component');  
  this.container.createComponent(LazyComponent);  
}
```

2. Handling Output Events

```
loadComponent() {  
  const componentRef = this.container.createComponent(component);  
  
  // Subscribe to outputs  
  componentRef.instance.someOutput.subscribe((event) => {  
    console.log('Output event:', event);  
  });  
}
```

3. Portal-based Approach (CDK Portals)

```
import { ComponentPortal } from '@angular/cdk/portal';  
  
// Create a portal from the component  
const portal = new ComponentPortal(DynamicComponent);  
  
// Attach the portal to a ViewContainerRef  
const portalRef = this.portalHost.attach(portal);  
  
// Access the component instance  
const componentInstance = portalRef.instance;
```

Best Practices

1. Memory Management: Always clean up components when they're no longer

needed

2. Change Detection: Use `ChangeDetectorRef` when needed to trigger change detection
3. Input Validation: Validate dynamic inputs before passing them to components
4. Error Handling: Handle cases where components might fail to load
5. Lazy Loading: Consider lazy loading dynamic components to reduce initial bundle size

Dynamic components are powerful but should be used judiciously as they can make the application flow harder to follow compared to static templates.

67. What is the role of ViewChild and ContentChild in Angular?

`ViewChild` is a decorator that allows you to access a child component, directive, or DOM element from within a parent component's TypeScript code.

Key Characteristics:

- Scope: Accesses elements that are part of the component's view (defined in its template)
- Timing: Available after `ngAfterViewInit` lifecycle hook
- Use Cases:
 - Accessing child component methods/properties
 - Manipulating DOM elements directly
 - Integrating with third-party libraries

Basic Syntax:

```
@ViewChild(selector, {read: readValue, static: staticValue}) propertyName;
```

Example Usage:

```
import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';
```

```
@Component({  
  selector: 'app-parent',
```

```

template: `
  <app-child #childRef></app-child>
  <div #divRef>Some content</div>
`
})
export class ParentComponent implements AfterViewInit {
  // Access child component
  @ViewChild('childRef') childComponent: ChildComponent;

  // Access DOM element
  @ViewChild('divRef') divElement: ElementRef;

  ngAfterViewInit() {
    // Safe to access child elements here
    this.childComponent.someMethod();
    this.divElement.nativeElement.style.color = 'red';
  }
}

```

ContentChild

`ContentChild` is a decorator that allows you to access content that has been projected into the component via `<ng-content>` (content projection).

Key Characteristics:

- Scope: Accesses projected content (between component tags)
- Timing: Available after `ngAfterContentInit` lifecycle hook
- Use Cases:
 - Accessing projected components/content
 - Manipulating transcluded content
 - Creating flexible component APIs

Basic Syntax:

```
@ContentChild(selector, {read: readValue, static: staticValue}) propertyName;
```

Example Usage:

```
import { Component, ContentChild, AfterContentInit } from '@angular/core';

@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content select="[card-header]"></ng-content>
      <div class="content"><ng-content></ng-content></div>
    </div>
  `,
})
export class CardComponent implements AfterContentInit {
  @ContentChild('header') header: ElementRef;

  ngAfterContentInit() {
    console.log('Projected header:', this.header);
  }
}

// Usage
@Component({
  selector: 'app-parent',
  template: `
    <app-card>
      <div #header card-header>Custom Header</div>
      <p>Card content goes here</p>
    </app-card>
  `,
})
export class ParentComponent {}
```

Key Differences

Feature	ViewChild	ContentChild
Source	Component's own template	Projected content (<code><ng-content></code>)
Availability	After view init (<code>ngAfterViewInit</code>)	After content init (<code>ngAfterContentInit</code>)
Use Case	Internal component elements	Customizable component content
Selector	Component/Directive type, template reference, string selector	Same as ViewChild

Advanced Usage

Static Option

```
@ViewChild('elem', {static: true}) elem: ElementRef;  
// Available in ngOnInit if static: true  
// Default is false (available in ngAfterViewInit)
```

Multiple Instances with ViewChildren/ContentChildren

```
@ViewChildren(ChildComponent) children: QueryList<ChildComponent>;  
@ContentChildren(ItemDirective) items: QueryList<ItemDirective>;
```

Accessing Different Types

```
// Access the ElementRef of a component  
@ViewChild(ChildComponent, {read: ElementRef}) childElement: ElementRef;
```



```
// Access a specific directive
@ViewChild('input', {read: NgModel}) inputModel: NgModel;
```

Best Practices

1. Avoid Early Access: Don't access ViewChild/ContentChild before the appropriate lifecycle hook
2. Use Proper Types: Type your properties correctly for better type safety
3. Consider Null Checks: Elements might not always be available
4. Prefer Declarative Approaches: Use data binding over direct manipulation when possible
5. Clean Up Subscriptions: If observing QueryList changes, remember to unsubscribe

These decorators are powerful tools for component interaction in Angular, enabling both tight integration between components and flexible content projection patterns.

68. How do you perform unit testing of components and services in Angular?

Angular provides a robust testing framework with Jasmine as the testing library and Karma as the test runner. Here's a comprehensive guide to unit testing components and services.

Setting Up the Test Environment

Angular CLI projects come preconfigured with testing setup. Key files:

- `karma.conf.js` - Karma test runner configuration
- `test.ts` - Test entry point
- `*.spec.ts` - Test files alongside components/services

Testing Components

1. Basic Component Test Structure

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { MyComponent } from './my.component';

describe('MyComponent', () => {
  let component: MyComponent;
  let fixture: ComponentFixture<MyComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [MyComponent]
    }).compileComponents();
  });

  beforeEach(() => {
    fixture = TestBed.createComponent(MyComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});
```

2. Testing Template Bindings

```
it('should display title', () => {
  component.title = 'Test Title';
  fixture.detectChanges();
  const compiled = fixture.nativeElement;
  expect(compiled.querySelector('h1').textContent).toContain('Test Title');
});
```

3. Testing User Interactions

```
it('should call onClick when button is clicked', () => {
```

```

    spyOn(component, 'onButtonClick');
    const button = fixture.nativeElement.querySelector('button');
    button.click();
    expect(component.onButtonClick).toHaveBeenCalled();
  });

```

4. Testing Input/Output Properties

```

// Parent component test
it('should receive input data', () => {
  component.inputData = 'Test Data';
  fixture.detectChanges();
  expect(component.processedData).toBe('Processed: Test Data');
});

// Child component test
it('should emit output event', () => {
  spyOn(component.dataChanged, 'emit');
  component.updateData('New Data');
  expect(component.dataChanged.emit).toHaveBeenCalledWith('New Data');
});

```

Testing Services

1. Basic Service Test Structure

```

import { TestBed } from '@angular/core/testing';
import { DataService } from '../data.service';

describe('DataService', () => {
  let service: DataService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(DataService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});

```

```
});
```

2. Testing Service Methods

```
it('should return expected data', () => {  
  const result = service.getData();  
  expect(result).toEqual(['item1', 'item2']);  
});
```

3. Testing HTTP Services

```
import { HttpClientTestingModule, HttpTestingController } from  
'@angular/common/http/testing';
```

```
describe('ApiService', () => {  
  let service: ApiService;  
  let httpMock: HttpTestingController;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      imports: [HttpClientTestingModule],  
      providers: [ApiService]  
    });  
    service = TestBed.inject(ApiService);  
    httpMock = TestBed.inject(HttpTestingController);  
  });  
  
  afterEach(() => {  
    httpMock.verify(); // Verify no outstanding requests  
  });  
  
  it('should fetch users', () => {  
    const mockUsers = [{ id: 1, name: 'John' }];  
  
    service.getUsers().subscribe(users => {  
      expect(users).toEqual(mockUsers);  
    });  
  
    const req = httpMock.expectOne('api/users');  
    expect(req.request.method).toBe('GET');  
    req.flush(mockUsers);  
  });  
});
```

```
});  
});
```

Advanced Testing Techniques

1. Testing with Dependencies

```
beforeEach(() => {  
  const mockDep = jasmine.createSpyObj('DependencyService', ['getValue']);  
  mockDep.getValue.and.returnValue('Mock Value');  
  
  TestBed.configureTestingModule({  
    providers: [  
      { provide: DependencyService, useValue: mockDep }  
    ]  
  });  
});
```

2. Testing Change Detection

```
it('should update view after async operation', fakeAsync(() => {  
  component.loadData();  
  tick(1000); // Simulate passage of time  
  fixture.detectChanges();  
  
  expect(fixture.nativeElement.querySelector('.result').textContent)  
    .toContain('Loaded Data');  
}));
```

3. Component Isolation Testing

```
it('should calculate total correctly', () => {  
  component.items = [{ price: 10 }, { price: 20 }];  
  expect(component.calculateTotal()).toBe(30);  
});
```

Best Practices

1. Test Behavior, Not Implementation: Focus on what the component/service does,

not how it does it

2. Keep Tests Isolated: Mock all dependencies
3. Use Descriptive Test Names: `should display error message when form is invalid`
4. Follow the Arrange-Act-Assert Pattern:

```
it('should increment counter', () => {  
  // Arrange  
  component.counter = 0;  
  
  // Act  
  component.increment();  
  
  // Assert  
  expect(component.counter).toBe(1);  
  
});
```

5. Test Edge Cases: Empty states, error conditions, boundary values
6. Keep Tests Fast: Avoid unnecessary DOM operations
7. Maintain Test Coverage: Aim for 70-80% coverage of critical paths

Useful Testing Utilities

- `async/fakeAsync/tick` - For testing asynchronous code
- `ComponentFixtureAutoDetect` - For automatic change detection
- `By.css()` - Alternative DOM querying
- `jasmine.createSpyObj()` - For creating mock objects
- `TestBed.overrideProvider` - For dependency override

Angular's testing utilities integrate seamlessly with both Jasmine and Jest, providing a powerful way to ensure your application works as expected while facilitating refactoring and maintenance.

69. Routing guard? Different types

Routing guards in Angular are interfaces that allow you to control navigation to and from routes. They're essential for implementing route protection, data pre-fetching, and

other navigation-related logic.

Types of Routing Guards

1. CanActivate

Controls whether a route can be activated (accessed).

Use cases:

- Authentication checks
- Role-based authorization
- Feature flag checks

```
import { Injectable } from '@angular/core';

import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot, Router } from
'@angular/router';

import { AuthService } from '../auth.service';

@Injectable({ providedIn: 'root' })

export class AuthGuard implements CanActivate {

  constructor(private auth: AuthService, private router: Router) {}

  canActivate(

    next: ActivatedRouteSnapshot,

    state: RouterStateSnapshot): boolean {

    if (this.auth.isLoggedIn()) {
```

```

        return true;
    }

    this.router.navigate(['/login']);

    return false;
}
}

```

2. CanActivateChild

Controls access to child routes.

Use cases:

- Protecting all child routes with the same logic
- Nested route authorization

```

@Injectable({ providedIn: 'root' })

export class ChildAuthGuard implements CanActivateChild {

    constructor(private auth: AuthService, private router: Router) {}

    canActivateChild(

        next: ActivatedRouteSnapshot,

        state: RouterStateSnapshot): boolean {

        return this.auth.hasPermission(next.data.requiredPermission);

    } }

```


3. CanDeactivate

Controls whether a user can leave a route.

Use cases:

- Preventing accidental form abandonment
- Unsaved changes confirmation
- Cleanup operations before leaving

```
import { CanDeactivate } from '@angular/router';

import { Observable } from 'rxjs';

export interface CanComponentDeactivate {

  canDeactivate: () => Observable<boolean> | Promise<boolean> | boolean;

}

@Injectable({ providedIn: 'root' })

export class CanDeactivateGuard implements CanDeactivate<CanComponentDeactivate> {

  canDeactivate(

    component: CanComponentDeactivate,

    currentRoute: ActivatedRouteSnapshot,

    currentState: RouterStateSnapshot,

    nextState?: RouterStateSnapshot

  ): Observable<boolean> | Promise<boolean> | boolean {

    return component.canDeactivate ? component.canDeactivate() : true;

  }

}
```

```
}  
  
}
```

4. Resolve

Prefetches data before activating a route.

Use cases:

- Loading required data before displaying a component
- Eliminating "loading" states in components
- Centralizing data fetching logic

```
@Injectable({ providedIn: 'root' })  
  
export class ProductResolver implements Resolve<Product> {  
  
  constructor(private productService: ProductService) {}  
  
  resolve(  
  
    route: ActivatedRouteSnapshot,  
  
    state: RouterStateSnapshot  
  
  ): Observable<Product> | Promise<Product> | Product {  
  
    return this.productService.getProduct(route.params.id);  
  
  }  
  
}
```

5. CanLoad

Controls whether a lazy-loaded module can be loaded.

Use cases:

- Protecting lazy-loaded modules
- Permission-based module loading
- Reducing bundle size for unauthorized users

```
@Injectable({ providedIn: 'root' })

export class LazyModuleGuard implements CanLoad {

  constructor(private auth: AuthService) {}

  canLoad(

    route: Route,

    segments: UrlSegment[]

  ): Observable<boolean> | Promise<boolean> | boolean {

    return this.auth.hasModuleAccess(route.path);

  }

}
```

Implementing Guards in Routes

```
const routes: Routes = [  
  
  {  
  
    path: 'admin',  
  
    canActivate: [AuthGuard],  
  
    canActivateChild: [ChildAuthGuard],  
  
    canLoad: [LazyModuleGuard],  
  
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)  
  
  },  
  
  {  
  
    path: 'product/:id',  
  
    component: ProductDetailComponent,  
  
    resolve: { product: ProductResolver },  
  
    canDeactivate: [CanDeactivateGuard]  
  
  },  
  
  {  
  
    path: 'dashboard',  
  
    component: DashboardComponent,  
  
    canActivate: [AuthGuard],  
  
    data: { requiredRole: 'manager' }  
  
  }  
]
```

```
}  
  
];
```

Guard Execution Order

1. `CanLoad` (for lazy-loaded routes only)
2. `CanActivate` and `CanActivateChild` (top to bottom)
3. `Resolve` (after all activation checks pass)
4. `CanDeactivate` (when leaving the route)

Best Practices

1. Keep guards focused: Each guard should handle one specific concern
2. Make guards reusable: Design guards to work with multiple routes
3. Use route data: Pass configuration via `data` property
4. Handle errors gracefully: Redirect or show appropriate messages
5. Consider performance: Especially for guards that run frequently
6. Test thoroughly: Guards are critical security boundaries

Advanced Patterns

Combined Guards

```
// auth.guard.ts  
  
export class AuthGuard implements CanActivate, CanActivateChild, CanLoad {  
  
    // Implement all interface methods  
  
}
```

Asynchronous Guards

```
canActivate(): Observable<boolean> {  
  
    return this.auth.isAuthenticated$.pipe(  
  
        tap(isAuth => {  
  
            if (!isAuth) this.router.navigate(['/login']);  
  
        })  
  
    );  
  
}
```

Dependency Injection in Guards

Guards can inject services just like components:

```
constructor(  
  
    private auth: AuthService,  
  
    private router: Router,  
  
    private notifier: NotificationService  
  
) {}
```

Routing guards provide powerful control over navigation in Angular applications, enabling you to build secure, user-friendly experiences with proper data management

70. Lazy loading and eager loading?

Eager Loading (Default Behavior)

What it is: All modules are loaded when the application starts

Characteristics:

- Modules are loaded upfront during initial app bootstrap
- All components, services, and other artifacts are available immediately
- Results in larger initial bundle size
- Simpler application architecture

Implementation:

```
// app.module.ts
```

```
import { ProductsModule } from './products/products.module';
```

```
@NgModule({
```

```
  imports: [
```

```
    ProductsModule, // Eagerly loaded
```

```
    // other modules
```

```
  ]
```

```
})
```

When to use:

- Small applications with limited modules

- When all features are needed immediately
- Applications where bundle size isn't a concern

Lazy Loading

What it is: Modules are loaded on-demand when their route is activated

Characteristics:

- Modules are loaded only when needed
- Reduces initial bundle size
- Improves initial load performance
- More complex routing setup
- Better for large applications

Implementation:

1. Configure loadChildren in routing

```
// app-routing.module.ts
```

```
const routes: Routes = [  
  
  {  
  
    path: 'products',  
  
    loadChildren: () => import('./products/products.module')  
  
      .then(m => m.ProductsModule)  
  
  }  
  
];
```


2. Create a separate routing module for the lazy-loaded feature:

```
// products/products-routing.module.ts
```

```
const routes: Routes = [  
  
  { path: '', component: ProductListComponent },  
  
  { path: ':id', component: ProductDetailComponent }  
  
];
```

```
@NgModule({  
  
  imports: [RouterModule.forChild(routes)],  
  
  exports: [RouterModule]  
  
})
```

```
export class ProductsRoutingModule { }
```

Verification:

Check your browser's Network tab to see separate chunks loading:

- `products-products-module.js` (or similar)

Key Differences

Aspect	Eager Loading	Lazy Loading
Load Time	Initial app load	On route navigation
Bundle Size	Larger initial bundle	Smaller initial bundle
Performance	Slower initial load	Faster initial load
Complexity	Simpler setup	More complex configuration
Use Cases	Small apps, core features	Large apps, feature modules

Preloading Strategies

Angular offers compromise solutions between eager and lazy loading:

1. NoPreloading (default): Only loads lazy modules when requested
2. PreloadAllModules: Loads all lazy modules after initial load
3. Custom Preloading: Selective preloading of certain modules

```
// Enable preloading

@NgModule({

  imports: [RouterModule.forRoot(routes, {

    preloadingStrategy: PreloadAllModules

  })],

  exports: [RouterModule]

})
```

Best Practices

1. Lazy load by feature: Organize modules around features
2. Shared modules: Put common components/directives in shared module
3. Route guards: Protect lazy-loaded routes appropriately
4. Preloading: Consider custom preloading for important features
5. Performance monitoring: Verify lazy loading with production builds

Interview Cheat Sheet: Key Points

1. Lazy Loading Benefits:
 - Faster initial load time
 - Reduced initial bundle size
 - Better user experience for large apps
2. Implementation Steps:
 - Use `loadChildren` with dynamic import
 - Create feature routing module with `forChild()`
 - Avoid importing lazy modules in app module
3. Verification:
 - Check network tab for separate chunks
 - Production build will show separate files

4. Common Pitfalls:

- Accidentally importing lazy modules
- Forgetting to create feature routing module
- Not properly handling module dependencies

5. Advanced Concepts:

- Preloading strategies
- Route guards with lazy loading
- Dependency injection in lazy modules

Example of lazy loading with a preloading strategy:

```
const routes: Routes = [

  {

    path: 'admin',

    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule),

    data: { preload: true } // For custom preloading

  }

]

@NgModule({

  imports: [RouterModule.forRoot(routes, {

    preloadingStrategy: CustomPreloadingStrategy

  })],

  exports: [RouterModule]
```

```
})
```

Remember: Lazy loading is one of Angular's most powerful optimization techniques, especially beneficial for enterprise-scale applications.

71. `forRoot` and `forChild` in routing? Difference

Key Difference

Feature	<code>forRoot()</code>	<code>forChild()</code>
Purpose	Configures root routing	Configures feature module routing
Providers	Registers router services	Doesn't register router services
Usage	<code>AppRoutingModule</code> (root level)	<code>FeatureRoutingModule</code>
Services	Creates singleton router instances	Reuses existing router instances

`forRoot()` - Root Module Configuration

When to use: In your main `AppRoutingModule` (root application module)

What it does:

- Sets up the router service at the application root level
- Registers router directives and services
- Provides the initial navigation configuration
- Should only be called once in the application

Example:

```
// app-routing.module.ts
```

```
import { NgModule } from '@angular/core';
```

```
import { RouterModule, Routes } from '@angular/router';
```

```
const routes: Routes = [
```

```
  { path: '', component: HomeComponent },
```

```
  { path: 'about', component: AboutComponent }
```

```
];
```

```
@NgModule({
```

```
  imports: [RouterModule.forRoot(routes)],
```

```
    exports: [RouterModule]

  })
```

```
export class AppRoutingModule { }
```

forChild() - Feature Module Configuration

When to use: In feature module routing modules

What it does:

- Adds additional routes to the application
- Doesn't register router services (uses existing ones)
- Can be called multiple times in different modules
- Maintains the same router instance across the app

Example:

```
// products-routing.module.ts
```

```
import { NgModule } from '@angular/core';
```

```
import { RouterModule, Routes } from '@angular/router';
```

```
import { ProductListComponent } from './product-list.component';
```

```
import { ProductDetailComponent } from './product-detail.component';
```

```
const routes: Routes = [
```

```

    { path: 'products', component: ProductListComponent },

    { path: 'products/:id', component: ProductDetailComponent }

];

@NgModule({

    imports: [RouterModule.forChild(routes)],

    exports: [RouterModule]

})

export class ProductsRoutingModule { }

```

Why the Separation Matters

1. Service Registration:
 - `forRoot()` registers the `Router` service
 - `forChild()` doesn't register services (avoids duplicate instances)
2. Performance:
 - Prevents multiple router service instances
 - Maintains single source of truth for routing
3. Architecture:
 - Enables proper module encapsulation
 - Supports lazy loading effectively

Common Patterns

1. Basic Setup

```
// app.module.ts
```



```
@NgModule({  
  
  imports: [  
  
    AppRoutingModule,          // Uses forRoot  
  
    ProductsModule             // Contains forChild  
  
  ]  
  
})
```

2. With Lazy Loading

```
// app-routing.module.ts  
  
const routes: Routes = [  
  
  {  
  
    path: 'products',  
  
    loadChildren: () => import('./products/products.module')  
  
      .then(m => m.ProductsModule) // Module with forChild  
  
  }  
  
];
```

3. Multiple Feature Modules

```
// admin-routing.module.ts

@NgModule({

  imports: [RouterModule.forChild(adminRoutes)],

  exports: [RouterModule]

})

export class AdminRoutingModule {}
```

```
// user-routing.module.ts

@NgModule({

  imports: [RouterModule.forChild(userRoutes)],

  exports: [RouterModule]

})

export class UserRoutingModule {}
```

Technical Implementation Details

Under the hood, Angular checks how `RouterModule` is imported:

```
// Simplified Angular source concept

static forRoot(routes: Routes): ModuleWithProviders<RouterModule> {
```

```
return {  
  
  ngModule: RouterModule,  
  
  providers: [  
  
    Router,  
  
    {provide: ROUTES, multi: true, useValue: routes}  
  
  ]  
  
};  
  
}
```

```
static forChild(routes: Routes): ModuleWithProviders<RouterModule> {  
  
  return {  
  
    ngModule: RouterModule,  
  
    providers: [  
  
      {provide: ROUTES, multi: true, useValue: routes}  
  
    ]  
  
  }  
  
}
```

```
};
```

```
}
```

Best Practices

1. Use `forRoot` exactly once: Typically in `AppRoutingModule`
2. Use `forChild` everywhere else: In all feature modules
3. Keep routing modules separate: Even in small modules
4. Export `RouterModule`: From all routing modules
5. Avoid service providers: In feature routing modules

Common Mistakes

1. Using `forRoot` in feature modules:
 - Creates duplicate router instances
 - Can cause unpredictable behavior
2. Not exporting `RouterModule`:
 - Makes router directives unavailable
3. Mixing routes improperly:
 - Can lead to route conflicts

Remember: The separation between `forRoot` and `forChild` is fundamental to Angular's routing architecture and ensures proper service instantiation and module organization.

72. Services in angular and dependency injection design pattern?

What are Services?

Services in Angular are singleton classes that:

- Provide reusable business logic
- Handle data operations (API calls, state management)
- Share data between components
- Encapsulate specific functionality

Key Characteristics:

1. Singleton (by default) - One instance per injector
2. Injectable - Decorated with `@Injectable()`
3. Decoupled - Independent of components
4. Reusable - Can be used across multiple components

Creating a Service

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Recommended way (Angular 6+)
})

export class DataService {

  private data: any[] = [];

  getData(): any[] {

    return this.data;

  }

  addData(item: any): void {

    this.data.push(item);
```

```
}
```

```
}
```

Dependency Injection (DI) in Angular

What is Dependency Injection?

DI is a design pattern where:

- Dependencies are "injected" rather than created
- The framework manages object creation and lifecycle
- Promotes loose coupling and testability

Angular DI System Components:

1. Injector - Maintains container of dependencies
2. Provider - Tells how to create a dependency
3. Token - Key used to identify a dependency

Injection Methods

1. Constructor Injection (Most Common)

```
@Component({  
  
  selector: 'app-example',  
  
  templateUrl: './example.component.html'  
  
})  
  
export class ExampleComponent {
```

```
    constructor(private dataService: DataService) {}  
  
}
```

2. Injector Service

```
import { Injector } from '@angular/core';  
  
export class ExampleComponent {  
  
    private dataService: DataService;  
  
    constructor(private injector: Injector) {  
  
        this.dataService = injector.get(DataService);  
  
    }  
  
}
```

Provider Configuration Options

1. Class Provider (Default)

```
{ provide: DataService, useClass: DataService }
```

2. Value Provider

```
{ provide: 'API_URL', useValue: 'https://api.example.com' }
```

3. Factory Provider

```

{

  provide: DataService,

  useFactory: (http: HttpClient) => new DataService(http),

  deps: [HttpClient]

}

```

4. Aliased Provider

```

{ provide: OldService, useExisting: NewService }

```

Hierarchical Injectors

Angular has a hierarchical DI system with levels:

1. Root (providedIn: 'root') - Application-wide singleton
2. Module - Available to all components in that module
3. Component - Available to that component and its children

// Module-level provider

```

@NgModule({

  providers: [DataService]

})

```

// Component-level provider


```
@Component({  
  
  providers: [DataService]  
  
})
```

Best Practices

1. Single Responsibility: Each service should do one thing well
2. Feature Organization: Group services by feature/module
3. State Management: Use services for shared state
4. Immutable Data: Consider returning new objects rather than modifying existing ones
5. Lazy Initialization: Use `providedIn: 'root'` for tree-shakable services

Advanced DI Patterns

1. Optional Dependencies

```
import { Optional } from '@angular/core';  
  
constructor(@Optional() private logger?: LoggerService) {  
  
  if (this.logger) {  
  
    this.logger.log('Component created');  
  
  }  
  
}
```

2. Multiple Providers

```
const VALIDATORS = new InjectionToken('VALIDATORS');

@NgModule({

  providers: [

    { provide: VALIDATORS, multi: true, useClass: EmailValidator },

    { provide: VALIDATORS, multi: true, useClass: PhoneValidator }

  ]

})
```

3. Custom Decorators

```
export function Cacheable() {

  return function(target: any, key: string, descriptor: PropertyDescriptor) {

    const originalMethod = descriptor.value;

    const cache = new Map();

    descriptor.value = function(...args: any[]) {

      const key = JSON.stringify(args);

      if (cache.has(key)) {

        return cache.get(key);

      }

    }

  }

}
```

```
    }

    const result = originalMethod.apply(this, args);

    cache.set(key, result);

    return result;

};

return descriptor;

};

}
```

```
// Usage
```

```
@Cacheable()

getData(id: number): Observable<Data> {

    return this.http.get(`/api/data/${id}`);

}
```

Testing Services with DI

```
describe('DataService', () => {

  let service: DataService;

  let httpMock: HttpTestingController;

  beforeEach(() => {

    TestBed.configureTestingModule({

      imports: [HttpClientTestingModule],

      providers: [DataService]

    });

    service = TestBed.inject(DataService);

    httpMock = TestBed.inject(HttpTestingController);

  });

  it('should fetch data', () => {

    const mockData = { id: 1, name: 'Test' };

    service.getData(1).subscribe(data => {
```

```

        expect(data).toEqual(mockData);

    });

    const req = httpMock.expectOne('api/data/1');

    expect(req.request.method).toBe('GET');

    req.flush(mockData);

});

});

```

Angular's DI system is powerful yet flexible, enabling you to:

- Write modular, testable code
- Easily swap implementations
- Manage complex dependency graphs
- Optimize application performance through proper service organization

73. How component can communicate in angular? how many ways

Angular provides several ways for components to communicate with each other. Here are all the primary methods:

1. Parent to Child Communication

Input Binding (Most Common)

```
// Parent component template
```

```
<app-child [inputData]="parentData"></app-child>
```

```
// Child component  
@Input() inputData: any;
```

Using ViewChild

```
// Parent component  
@ViewChild(ChildComponent) childComponent: ChildComponent;  
  
ngAfterViewInit() {  
    this.childComponent.methodOrProperty = value;  
}
```

2. Child to Parent Communication

Output Binding with EventEmitter

```
// Child component  
@Output() dataEvent = new EventEmitter<any>();  
  
sendData() {  
    this.dataEvent.emit(data);  
}  
  
// Parent component template  
<app-child (dataEvent)="onDataReceived($event)"></app-child>
```

Using Template Reference Variable

```
// Parent template  
<app-child #childRef></app-child>  
<button (click)="childRef.childMethod()">Call Child</button>
```

3. Sibling Component Communication

Through Parent Component

```
// Parent template
```

```
<app-sibling1 (event)="sharedData = $event"></app-sibling1>
<app-sibling2 [data]="sharedData"></app-sibling2>
```

Using a Shared Service

```
@Injectable({ providedIn: 'root' })

export class DataService {
  private dataSubject = new BehaviorSubject<any>(null);
  data$ = this.dataSubject.asObservable();

  updateData(data: any) {
    this.dataSubject.next(data);
  }
}

// In both siblings
constructor(private dataService: DataService) {}

// Sender
this.dataService.updateData(data);

// Receiver
this.dataService.data$.subscribe(data => {});
```

4. Unrelated Components Communication

Service with Subject/Observable

```
// Shared service
private messageSource = new Subject<string>();
currentMessage = this.messageSource.asObservable();

sendMessage(message: string) {
  this.messageSource.next(message);
}

// Component A (sender)
this.sharedService.sendMessage('Hello');
```

```
// Component B (receiver)
this.sharedService.currentMessage.subscribe(message => {});
```

State Management (NgRx, Akita, etc.)

```
// Using NgRx
this.store.dispatch(new LoadDataAction());
this.store.select(getData).subscribe(data => {});
```

5. Other Communication Methods

Router Parameters

```
// Sender
this.router.navigate(['/details', id]);

// Receiver
this.route.params.subscribe(params => {
  const id = params['id'];});
```

Local Storage/Session Storage

```
// Component A
localStorage.setItem('sharedData', JSON.stringify(data));

// Component B
const data = JSON.parse(localStorage.getItem('sharedData'));
```

ViewChildRef and ComponentRef

For dynamic component communication

Summary Table

Communication Type	Method	When to Use
--------------------	--------	-------------

Parent → Child	Input binding	Direct data passing
Parent → Child	ViewChild	Access child methods/properties
Child → Parent	Output/EventEmitter	Child events to parent
Child → Parent	Template reference variable	Direct child method calls
Siblings	Via parent	Simple cases with common parent
Siblings	Shared service	Complex interactions
Any components	Service with Subject	Unrelated components
Any components	State management	Large apps with complex state
Any components	Router parameters	Route-based communication
Any components	Local storage	Persisted data across sessions

Best Practices

1. Use the simplest method that fits your needs
2. Avoid tight coupling between components
3. For complex apps, consider state management
4. Unsubscribe from observables to prevent memory leaks
5. Use ChangeDetectionStrategy.OnPush for better performance with observables

Choose the communication method based on:

- Component relationship (parent/child/sibling/unrelated)
- Data flow direction
- Complexity of your application
- Performance considerations

75. What are directives? and types?

Directives are classes that add additional behavior to elements in your Angular applications. They are a core feature that allow you to manipulate the DOM, modify element appearance, or add complex behavior to templates.

Types of Directives

1. Component Directives

Components are directives with templates. They're the most common type of directive in Angular.

Characteristics:

- Have a template (HTML)
- Can define input/output properties
- Encapsulate view and behavior

Example:

```
@Component({  
  
  selector: 'app-example',  
  
  template: `<h1>Example Component</h1>`  
  
})  
  
export class ExampleComponent {}
```

2. Attribute Directives

Change the appearance or behavior of DOM elements.

Characteristics:

- Look like normal HTML attributes
- Don't have templates
- Can listen to events on the host element

Built-in Examples:

- `NgClass`: Adds/removes CSS classes
- `NgStyle`: Adds/removes inline styles

Custom Example:

```
@Directive({  
  
  selector: '[appHighlight]'  
  
})
```

```
export class HighlightDirective {

  @HostListener('mouseenter') onMouseEnter() {

    this.highlight('yellow');

  }

  @HostListener('mouseleave') onMouseLeave() {

    this.highlight(null);

  }

  constructor(private el: ElementRef) {}

  private highlight(color: string) {

    this.el.nativeElement.style.backgroundColor = color;

  }

}
```

Usage:

```
<p appHighlight>Hover over me!</p>
```

3. Structural Directives

Change the DOM layout by adding or removing elements.

Characteristics:

- Prefix with asterisk (*) in templates
- Manipulate the DOM structure
- Work with `<ng-template>` under the hood

Built-in Examples:

- `*ngIf`: Conditionally includes/excludes elements
- `*ngFor`: Repeats elements for each item in a list
- `*ngSwitch`: Shows one element from multiple choices

Custom Example:

```
@Directive({  
  
  selector: '[appDelay]'  
  
})  
  
export class DelayDirective implements OnInit {  
  
  @Input() appDelay: number = 500;  
  
  constructor(  
  
    private templateRef: TemplateRef<any>,  
  
    private viewContainer: ViewContainerRef) {}  
}
```

```

ngOnInit() {

    setTimeout(() => {

        this.viewContainer.createEmbeddedView(this.templateRef);

    }, this.appDelay);

    }

}

```

Usage:

```

<div *appDelay="1000">This appears after 1 second</div>

```

Directive Composition API (Angular 15+)

New way to apply multiple directives to a single element:

```

@Component({

    selector: 'app-avatar',

    hostDirectives: [

        {

            directive: HighlightDirective,

            inputs: ['highlightColor: color']

```

```

    },

    {

        directive: TooltipDirective,

        inputs: ['tooltipText: tooltip']

    }

]

}))

export class AvatarComponent {}

```

Key Differences Between Directive Types

Feature	Component Directives	Attribute Directives	Structural Directives
Template	Yes	No	No (but uses ng-template)
DOM Effect	Creates new DOM	Modifies existing DOM	Adds/removes DOM elements

Syntax	Element selector	Attribute selector	Prefix with *
--------	------------------	--------------------	---------------

Purpose	Create UI components	Modify element behavior	Control DOM structure
---------	----------------------	-------------------------	-----------------------

When to Create Custom Directives

1. DOM Manipulation: When you need to directly interact with the DOM
2. Behavior Reusability: When you want to reuse behavior across components
3. Template Extensions: When you need to extend template syntax
4. Cross-Component Features: When functionality spans multiple components

Best Practices

1. Use components for creating UI pieces with templates
2. Use attribute directives for modifying element behavior/appearance
3. Use structural directives for complex DOM manipulations
4. Keep directives focused on a single responsibility
5. Prefer declarative approaches over imperative DOM manipulation

Directives are powerful tools that help you create more maintainable and reusable Angular applications by encapsulating DOM manipulation and behavior logic.

76. AOT and JIT? Which one is better?

JIT (Just-in-Time) Compilation

- Compiles the application in the browser at runtime
- Default mode during development with `ng serve`
- Requires the Angular compiler to be shipped to the browser (~500KB extra)
- Compiles components when they are first rendered

AOT (Ahead-of-Time) Compilation

- Compiles the application during the build process
- Default for production builds (`ng build --prod`)
- Doesn't ship the Angular compiler to the browser
- Pre-compiles all templates before deployment

Key Differences

Feature	JIT Compilation	AOT Compilation
Compilation Timing	At runtime in the browser	During build process
Bundle Size	Larger (includes compiler)	Smaller (no compiler needed)
Startup Time	Slower (compiles in browser)	Faster (pre-compiled)
Error Detection	Runtime errors	Build-time errors
Template Errors	Discovered during use	Caught during build

Tree Shaking

Less effective

More effective

Suitable For

Development

Production

Which One is Better?

AOT is generally better for production for these reasons:

1. Faster Rendering
 - Templates are pre-compiled
 - Browser can render immediately without compilation step
2. Smaller Bundle Size
 - No need to ship the Angular compiler (~500KB savings)
 - Better tree shaking eliminates unused code
3. Earlier Error Detection
 - Template errors caught during build
 - Better than discovering errors in production
4. Improved Security
 - Templates compiled to JavaScript (no client-side evaluation)
 - Reduces injection attack risks
5. Better Performance
 - No compilation step in the browser
 - More efficient code generation

When to Use JIT

1. Development Environment
 - Faster build times during development
 - Easier debugging (maps directly to source code)
2. Dynamic Component Compilation
 - When you need to compile components at runtime
 - Rare cases where templates are generated dynamically

Angular's Default Behavior

- `ng serve` → Uses JIT (for faster development cycles)
- `ng build` → Uses JIT (development build)
- `ng build --prod` → Uses AOT (production build)

Migration Considerations

To force AOT in development:

```
ng serve --aot
```

Or update `angular.json`:

```
"build": {  
  
  "options": {  
  
    "aot": true  
  
  }  
  
}
```

Verifying Compilation Mode

Check your main bundle:

- JIT: Contains `@angular/compiler`
- AOT: No compiler present, templates are pre-compiled

Best Practice Recommendation

1. Development: Use JIT for faster builds and easier debugging

2. Production: Always use AOT for better performance and security
3. CI/CD: Consider AOT even for staging environments to catch template errors early

Final Answer: AOT is better for production deployments due to its performance, security, and size advantages, while JIT remains useful during development for faster iteration cycles. The Angular CLI automatically applies the appropriate compilation mode for each environment.

77. What all are the ngDestroy use case?

The `ngOnDestroy` lifecycle hook is called just before Angular destroys a component, directive, pipe, or service. Here are the most important use cases for implementing `ngOnDestroy`:

1. Unsubscribing from Observables

Most Common Use Case - Prevent memory leaks from open subscriptions:

```
private subscriptions = new Subscription();

ngOnInit() {
  this.subscriptions.add(
    this.dataService.getData().subscribe()
  );
}

ngOnDestroy() {
  this.subscriptions.unsubscribe();
}
```

2. Clearing Intervals and Timeouts

Clean up any timers to prevent them from running after component destruction:

```
private intervalId: any;

ngOnInit() {
  this.intervalId = setInterval(() => {
    this.updateData();
  }, 1000);
}

ngOnDestroy() {
  clearInterval(this.intervalId);
}
```

3. Disconnecting Event Listeners

Remove DOM event listeners to prevent memory leaks:

```
@HostListener('window:resize', ['$event'])
onResize(event: Event) {
  // Handle resize
}

ngOnDestroy() {
  window.removeEventListener('resize', this.onResize);
}
```

4. Releasing Resources

Clean up any third-party libraries or resources:

```
private chart: any;

ngOnInit() {
  this.chart = new ThirdPartyChartLibrary('#chart-container');
}

ngOnDestroy() {
  this.chart.destroy(); // Clean up chart instance
}
```

```
}
```

5. Cancelling Pending HTTP Requests

Cancel ongoing API calls when component is destroyed:

```
private httpRequest: Subscription;

ngOnInit() {
  this.httpRequest = this.http.get('/api/data').subscribe();
}

ngOnDestroy() {
  if (this.httpRequest) {
    this.httpRequest.unsubscribe();
  }
}
```

6. Disconnecting from WebSockets

Close WebSocket connections:

```
ngOnInit() {
  this.socket = new WebSocket('wss://example.com');
}

ngOnDestroy() {
  this.socket.close();
}
```

7. Cleaning Up Form Subscriptions

Unsubscribe from form value changes:

```
ngOnInit() {
  this.form.valueChanges.subscribe(values => {
    // Handle changes
  });
}
```

```
ngOnDestroy() {  
  this.formSubscription.unsubscribe();  
}
```

8. Releasing Memory-Intensive Objects

```
private largeDataCache: any[] = [];  
  
ngOnDestroy() {  
  this.largeDataCache = null; // Help GC  
}
```

9. Component-Specific Cleanup

Component-specific teardown logic:

```
ngOnDestroy() {  
  // Save component state to service  
  this.stateService.saveComponentState(this.currentState);  
  
  // Notify other components  
  this.eventService.emitComponentDestroyed();  
}
```

10. Cleaning Up Dynamic Components

For dynamically created components:

```
ngOnDestroy() {  
  if (this.componentRef) {  
    this.componentRef.destroy();  
  }  
}
```

Best Practices

1. Always implement `ngOnDestroy` when you have subscriptions, timers, or external

resources

2. Use the `takeUntil` pattern as an alternative to manual unsubscription:

```
private destroy$ = new Subject<void>();
```

```
ngOnInit() {  
  this.dataService.getData()  
    .pipe(takeUntil(this.destroy$))  
    .subscribe();  
}
```

```
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

3. Make `ngOnDestroy` idempotent - Safe to call multiple times
4. Call `super.ngOnDestroy()` if extending a class that implements it
5. Test `ngOnDestroy` in your unit tests

Remember that `ngOnDestroy` is not called when:

- The application is closed abruptly
- The browser tab is force-closed
- There's a power failure or crash

For these scenarios, consider using browser `beforeunload` events for critical cleanup operations.

78. What is the `asyncPipe`?

The `AsyncPipe` is a built-in Angular pipe that simplifies working with asynchronous data in templates. It automatically handles subscriptions and unsubscriptions to observables and promises, making your code cleaner and more memory-safe.

Key Features

1. Automatic Subscription Management:
 - Subscribes to observables/promises when the component loads
 - Unsubscribes automatically when the component is destroyed
 - Prevents memory leaks from forgotten subscriptions
2. Template Simplification:
 - Eliminates the need for manual subscription handling in components
 - Reduces boilerplate code
3. Change Detection Integration:
 - Automatically triggers change detection when new values arrive

Basic Usage

With Observables

```
// In component
import { Component } from '@angular/core';
import { Observable, interval } from 'rxjs';

@Component({
  selector: 'app-example',
  template: `
    <p>Current value: {{ observableData | async }}</p>
  `,
})
export class ExampleComponent {
  observableData: Observable<number> = interval(1000); // Emits values every
second
}
```

With Promises

```
// In component
import { Component } from '@angular/core';

@Component({
```

```

    selector: 'app-example',
    template: `
      <p>Promise result: {{ promiseData | async }}</p>
    `
  })
}

export class ExampleComponent {
  promiseData = new Promise<string>((resolve) => {
    setTimeout(() => resolve('Data loaded!'), 2000);
  });
}

```

Common Use Cases

1. HTTP Requests:

```

// Component
data$ = this.http.get('/api/data');

// Template
<div *ngIf="data$ | async as data">
  {{ data | json }}

</div>

```

2. Combining Multiple Streams:

```

combined$ = combineLatest([
  this.userService.currentUser$,
  this.configService.config$
]).pipe(
  map(([user, config]) => ({ user, config }))
);

```

3. With Structural Directives:

```

<ng-container *ngIf="user$ | async as user">
  <h1>Welcome {{ user.name }}</h1>
  <p>Email: {{ user.email }}</p>

</ng-container>

```

Benefits

1. No Manual Cleanup:
 - Automatically unsubscribes when the component is destroyed
 - Prevents memory leaks
2. Cleaner Code:
 - Eliminates subscription management boilerplate
 - Reduces component class complexity
3. Improved Performance:
 - Only triggers change detection when new values arrive
 - Works well with `OnPush` change detection strategy

Advanced Patterns

1. Multiple Async Pipes:

```
<div *ngIf="(user$ | async) && (posts$ | async)">
  <!-- Content here -->
</div>
```

2. Combining with Other Pipes:

```
<p>Last updated: {{ lastUpdated$ | async | date:'medium' }}</p>
```

3. Error Handling:

```
data$ = this.http.get('/api/data').pipe(
  catchError(error => of({ error: true, message: error.message })))

);
```

Limitations

1. No Direct Error Handling:
 - Errors must be caught and handled in the observable chain
2. Single Subscription:
 - Each async pipe creates a separate subscription
 - For multiple uses, consider using the `as` syntax to reuse the resolved value

3. Template Complexity:
 - Overuse can make templates harder to read

Best Practices

1. Use the as Syntax:

```
<div *ngIf="data$ | async as data">
  {{ data.property }}

```

2. </div>

3. Combine Streams in Class:

- Prefer combining observables in the component rather than multiple async pipes in template

4. Use with OnPush:

- AsyncPipe works exceptionally well with ChangeDetectionStrategy.OnPush

5. Handle Loading States:

```
<ng-container *ngIf="data$ | async as data; else loading">
  <!-- Display data -->
</ng-container>

```

6. <ng-template #loading>Loading...</ng-template>

The AsyncPipe is one of Angular's most powerful features for handling asynchronous operations, reducing boilerplate code while ensuring proper memory management.

79.Rx js Operators . pipe(), Fork join, mergeMap, SwitchMap,forkJoin and diffrencece?

RxJS operators are powerful tools for handling asynchronous operations in Angular.

Here's a detailed comparison of these commonly used operators:

1. pipe()

Purpose: Composition tool for combining multiple operators

Characteristics:

- Not an operator itself, but a method to chain operators
- Creates a pipeline of operations for the observable
- Pure function (creates new observable without modifying source)

Example:

```
import { map, filter, tap } from 'rxjs/operators';

source$.pipe(
  tap(val => console.log('Received:', val)),
  filter(val => val > 10),
  map(val => val * 2)
).subscribe(result => console.log(result));
```

2. forkJoin()

Purpose: Combine multiple observables and wait for all to complete

Behavior:

- Takes an array/dictionary of observables
- Emits only when ALL source observables complete
- Returns an array/dictionary of final values
- If any observable errors, all values are lost (unless error handling)

Use Cases:

- Multiple parallel HTTP requests
- Waiting for several data sources to load

Example:

```
import { forkJoin } from 'rxjs';

forkJoin([
```

```
this.http.get('/api/users'),  
this.http.get('/api/posts'),  
this.http.get('/api/comments')  
]).subscribe([users, posts, comments]) => {  
  // All requests completed  
});
```

3. mergeMap (formerly flatMap)

Purpose: Map to observable and flatten its emissions

Behavior:

- Projects each value to an observable
- Merges the output observables
- Maintains all active inner subscriptions
- Order not guaranteed (concurrent)

Use Cases:

- Non-cancelable operations (like saving data)
- When you need all responses

Example:

```
import { mergeMap } from 'rxjs/operators';  
  
click$.pipe(  
  mergeMap(event => this.http.post('/api/save', event))  
).subscribe(response => {  
  // Handle all responses  
});
```

4. switchMap

Purpose: Map to observable, cancel previous inner subscription

Behavior:

- Projects each value to an observable
- Cancels previous inner observable when new value arrives
- Only maintains latest inner subscription

Use Cases:

- Typeahead/search suggestions
- Cancelable operations (like HTTP requests)

Example:

```
import { switchMap } from 'rxjs/operators';

searchInput$.pipe(
  debounceTime(300),
  switchMap(query => this.http.get(`/api/search?q=${query}`))
).subscribe(results => {
  // Only shows results for latest query
});
```

Key Differences

Operator	Cancels Previous?	Order Guaranteed?	Completes When?	Best For
<code>forkJoin</code>	N/A	Yes (as array)	When all complete	Parallel requests, final values
<code>mergeMap</code>	No	No	When all complete	Non-cancelable operations
<code>switchMap</code>	Yes	No (latest only)	When source completes	Cancelable operations (search)

Additional Comparison

`forkJoin` vs `combineLatest`

- `forkJoin`: Waits for all to complete, emits once
- `combineLatest`: Emits whenever any source emits (after all have emitted)

`mergeMap` vs `concatMap`

- `mergeMap`: Concurrent inner observables
- `concatMap`: Queues inner observables (runs sequentially)

`switchMap` vs `exhaustMap`

- `switchMap`: Cancels previous on new emission
- `exhaustMap`: Ignores new emissions until current completes

Choosing the Right Operator

1. Need latest value only? → `switchMap`
2. Need all responses? → `mergeMap`
3. Need parallel requests? → `forkJoin`
4. Need sequential execution? → `concatMap`
5. Need to ignore during execution? → `exhaustMap`

Remember: All these operators are used within `pipe()` to create observable chains.

Understanding these differences is crucial for proper asynchronous handling in Angular applications.

80. How to modify request and response and angular? Interceptors

Interceptors provide a way to modify HTTP requests and responses globally. They're ideal for adding headers, logging, error handling, and transforming data.

Creating an Interceptor

1. Generate the Interceptor

ng generate interceptor auth

2. Basic Interceptor Structure

```
import { Injectable } from '@angular/core';
import {
  HttpRequest,
  HttpHandler,
  HttpEvent,
  HttpInterceptor,
  HttpResponse
} from '@angular/common/http';
import { Observable, tap } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(
    request: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>> {
    // Request modification happens here
    const modifiedReq = request.clone({
      setHeaders: { Authorization: 'Bearer token' }
    });

    return next.handle(modifiedReq).pipe(
      // Response handling here
      tap(event => {
        if (event instanceof HttpResponse) {
          console.log('Response received');
        }
      })
    );
  }
}
```

Common Use Cases

1. Adding Headers to Requests

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
    const authToken = this.authService.getToken();  
  
    const authReq = req.clone({  
        setHeaders: { Authorization: `Bearer ${authToken}` }  
    });  
  
    return next.handle(authReq);  
}
```

2. Modifying Responses

```
return next.handle(req).pipe(  
    map(event => {  
        if (event instanceof HttpResponse) {  
            return event.clone({  
                body: this.modifyResponse(event.body)  
            });  
        }  
        return event;  
    })  
);
```

3. Error Handling

```
return next.handle(req).pipe(  
    catchError(error => {  
        if (error.status === 401) {  
            this.authService.logout();  
        }  
        return throwError(() => error);  
    })  
);
```

4. Request/Response Logging

```

const started = Date.now();
return next.handle(req).pipe(
  tap(event => {
    if (event instanceof HttpResponse) {
      const elapsed = Date.now() - started;
      console.log(`Request to ${req.url} took ${elapsed}ms`);
    }
  })
);

```

5. API URL Modification

```

const apiReq = req.clone({
  url: `${environment.apiUrl}/${req.url}`
});
return next.handle(apiReq);

```

Registering Multiple Interceptors

In your app module:

```

@NgModule({
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
    { provide: HTTP_INTERCEPTORS, useClass: LoggingInterceptor, multi: true },
    { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true }
  ]
})
export class AppModule {}

```

Execution Order

Interceptors execute in the order they're provided:

1. First interceptor's `intercept()` method is called
2. Each calls `next.handle()` to pass to the next interceptor

3. Finally reaches the HTTP backend
4. Response flows back through the chain in reverse order

Advanced Patterns

1. Conditional Interception

```
if (req.url.includes('/api/')) {  
  // Apply interceptor logic  
}  
return next.handle(req);
```

2. Request Retry

```
import { retry } from 'rxjs/operators';  
  
return next.handle(req).pipe(  
  retry(2) // Retry twice before failing  
);
```

3. Loading Indicators

```
this.loadingService.show();  
return next.handle(req).pipe(  
  finalize(() => this.loadingService.hide())  
);
```

4. Caching Responses

```
const cachedResponse = this.cache.get(req.url);  
if (cachedResponse) {  
  return of(cachedResponse);  
}  
return next.handle(req).pipe(  
  tap(event => {  
    if (event instanceof HttpResponse) {  
      this.cache.set(req.url, event);  
    }  
  })  
);
```

Best Practices

1. Immutable Requests: Always clone requests before modification
2. Error Propagation: Ensure errors propagate to calling code
3. Performance: Keep interceptors lightweight
4. Specificity: Make interceptors specific to their purpose
5. Testing: Test interceptors in isolation

Interceptors provide a powerful way to centrally manage HTTP communication in Angular applications, reducing duplicate code and ensuring consistent behavior across all requests.

81. Routing. How to send and get parameters in routing url?

In web development, routing parameters allow passing data through URLs, enabling dynamic content rendering. There are two main types of URL parameters:

1. Path Parameters (e.g., `/users/123`) – Embedded directly in the URL path.
2. Query Parameters (e.g., `/search?q=hello&page=1`) – Added after a `?` in key-value pairs.

1. Sending Parameters in a URL

A. Path Parameters

Used for mandatory identifiers (e.g., user ID, product ID).

Example in Express.js (Node.js):

```
// Route definition with `:id` as a path parameter
app.get('/users/:id', (req, res) => {
```

```
res.send(`User ID: ${req.params.id}`);
});
```

- Client Request: GET /users/123
- Output: User ID: 123

Example in Django (Python):

```
# urls.py
path('users/<int:user_id>/', views.user_detail),
```

```
# views.py
def user_detail(request, user_id):
    return HttpResponse(f"User ID: {user_id}")
```

- Client Request: GET /users/123/
 - Output: User ID: 123
-

B. Query Parameters

Used for optional data like filters, pagination, or search terms.

Example in Express.js:

```
app.get('/search', (req, res) => {
    const query = req.query.q; // Access query parameter 'q'
    const page = req.query.page || 1; // Default value if not provided
    res.send(`Searching for: ${query}, Page: ${page}`);
});
```

- Client Request: GET /search?q=javascript&page=2
- Output: Searching for: javascript, Page: 2

Example in Flask (Python):

```
from flask import request
```

```
@app.route('/search')
def search():
```

```
query = request.args.get('q', default='') # Default if not provided
page = request.args.get('page', default=1, type=int)
return f"Search: {query}, Page: {page}"
```

- Client Request: GET /search?q=python&page=3
 - Output: Search: python, Page: 3
-

2. Getting Parameters in Frontend (React, Angular, Vue)

A. React Router (v6)

Path Parameters

```
import { useParams } from 'react-router-dom';

function UserProfile() {
  const { id } = useParams(); // Gets `/users/:id`
  return <div>User ID: {id}</div>;
}
```

- URL: /users/123 → Output: User ID: 123

Query Parameters

```
import { useSearchParams } from 'react-router-dom';

function SearchResults() {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('q');
  return <div>Search: {query}</div>;
}
```

- URL: /search?q=react → Output: Search: react
-

B. Angular

Path Parameters

```
import { ActivatedRoute } from '@angular/router';

export class UserComponent {
  id: string;

  constructor(private route: ActivatedRoute) {
    this.id = this.route.snapshot.paramMap.get('id')!;
  }
}
```

- URL: `/users/123` → Output: User ID: 123

Query Parameters

```
import { ActivatedRoute } from '@angular/router';

export class SearchComponent {
  query: string | null;

  constructor(private route: ActivatedRoute) {
    this.query = this.route.snapshot.queryParamMap.get('q');
  }
}
```

- URL: `/search?q=angular` → Output: Search: angular

3. Best Practices

- ✓ Use Path Parameters for required identifiers (e.g., `/users/123`).
 - ✓ Use Query Parameters for optional filters (e.g., `/products?category=books`).
 - ✗ Avoid sensitive data in URLs (use POST requests instead).
 - 🔒 Validate & Sanitize parameters to prevent injection attacks.
-

Summary Table

Method	Example URL	Backend Access	Frontend Access (React)
Path Parameter	<code>/users/123</code>	<code>req.params.id</code> (Express)	<code>useParams()</code>
Query Parameter	<code>/search?q=hello</code>	<code>req.query.q</code> (Express)	<code>useSearchParams()</code>

This knowledge is crucial for RESTful API design, dynamic page rendering, and SEO-friendly URLs. 🚀

82. how to pass query params in routing?

Query parameters are key-value pairs appended to a URL after a `?` symbol, used for filtering, sorting, pagination, and other optional data. Here's how to pass and retrieve them in different frameworks:

1. Passing Query Parameters in URLs

Structure:

`https://example.com/search?q=react&page=2&sort=asc`

- `?` starts the query string

- `key=value` pairs separated by `&`
 - Example: `q=react, page=2, sort=asc`
-

2. Backend Handling (Server-Side)

A. Express.js (Node.js)

```
// Route to handle query params
app.get('/search', (req, res) => {
  const query = req.query.q;      // "react"
  const page = req.query.page;    // "2"
  const sort = req.query.sort;    // "asc"

  res.send(`Search: ${query}, Page: ${page}, Sort: ${sort}`);
});
```

- Request: `GET /search?q=react&page=2&sort=asc`
- Output: `Search: react, Page: 2, Sort: asc`

B. Django (Python)

```
# views.py
from django.http import HttpResponse

def search(request):
    query = request.GET.get('q', '') # Default empty if not provided
    page = request.GET.get('page', 1) # Default 1
    return HttpResponse(f"Search: {query}, Page: {page}")
```

- Request: `GET /search?q=django&page=3`
- Output: `Search: django, Page: 3`

C. Flask (Python)

```
from flask import request

@app.route('/search')
def search():
```

```
query = request.args.get('q')
page = request.args.get('page', default=1, type=int)
return f"Search: {query}, Page: {page}"
```

- Request: GET /search?q=flask&page=2
 - Output: Search: flask, Page: 2
-

3. Frontend Handling (Client-Side)

A. Vanilla JavaScript (URLSearchParams)

```
// Get query params from current URL

const urlParams = new URLSearchParams(window.location.search);
const query = urlParams.get('q');      // "react"
const page = urlParams.get('page');    // "2"
console.log(query, page); // Output: react 2
```

B. React Router (v6)

```
import { useSearchParams } from 'react-router-dom';

function SearchPage() {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('q');
  const page = searchParams.get('page');

  return (
    <div>
      <h1>Search: {query}</h1>
      <p>Page: {page}</p>
    </div>
  );
}
```

URL: /search?q=react&page=2

Output:

Search: react

- Page: 2

C. Next.js (React Framework)

```
import { useRouter } from 'next/router';

export default function SearchPage() {
  const router = useRouter();
  const { q, page } = router.query; // Automatically parsed

  return (
    <div>
      <h1>Search: {q}</h1>
      <p>Page: {page}</p>
    </div>
  );
}
```

- URL: /search?q=nextjs&page=1
- Output:

Search: nextjs

Page: 1

D. Vue Router (Vue.js)

```
// Access in a component
export default {
  created() {
    const query = this.$route.query.q;
    const page = this.$route.query.page;
    console.log(query, page); // Output: vue 3
  }
}
```

- URL: /search?q=vue&page=3
 - Output: vue 3
-

4. Constructing Query URLs (Frontend)

A. Vanilla JavaScript

```
const params = new URLSearchParams();
params.set('q', 'javascript');
params.set('page', '2');

const url = `/search?${params.toString()}`;
// Output: /search?q=javascript&page=2
```

B. React Router (Navigation)

```
import { useNavigate } from 'react-router-dom';

function SearchButton() {
  const navigate = useNavigate();

  const handleSearch = () => {
    navigate(`/search?q=react&page=2`);
  };

  return <button onClick={handleSearch}>Search</button>;
}
```

C. Next.js (Router Push)

```
import { useRouter } from 'next/router';

function SearchButton() {
  const router = useRouter();

  const handleSearch = () => {
    router.push({
      pathname: '/search',
      query: { q: 'nextjs', page: 1 },
    });
  };

  return <button onClick={handleSearch}>Search</button>;
}
```

}

5. Best Practices

- ✅ Use for Optional Data (filters, sorting, pagination).
 - ❌ Avoid Sensitive Data (use POST requests instead).
 - ◆ Encode Special Characters using `encodeURIComponent()`.
 - 🔄 Keep URLs Clean (avoid too many query params).
-

Summary Table

Method	Backend Access	Frontend Access
Express.js	<code>req.query</code>	N/A (handled by backend)
React Router	N/A	<code>useSearchParams()</code>
Next.js	<code>router.query</code>	<code>useRouter().query</code>
Vue Router	N/A	<code>this.\$route.query</code>
Vanilla JS	N/A	<code>URLSearchParams</code>

Query parameters are essential for dynamic web applications. Mastering them ensures better user experience and API flexibility. 🚀

83. Subject and behaviour subject, reapplySubject? Difference between subject and behaviour subject

RxJS Subjects are special Observables that allow multicasting to multiple Observers.

Here's how they differ:

Type	Description	Key Feature
Subject	Basic multicast observable	No initial value, only emits new values
BehaviorSubject	Subject with current value	Requires initial value, emits last value to new subscribers
ReplaySubject	Subject that replays previous values	Can replay a buffer of past values to new subscribers

2. Subject - The Basic Multicast Observable

Characteristics:

- No initial value: Starts empty
- Only emits values after subscription: New subscribers don't get previous values
- Multicasts to all subscribers: All observers get the same values

Code Example:

```
import { Subject } from 'rxjs';  
  
const subject = new Subject<number>();
```

```
subject.subscribe(v => console.log(`Subscriber A: ${v}`));

subject.next(1); // Subscriber A: 1
subject.next(2); // Subscriber A: 2

subject.subscribe(v => console.log(`Subscriber B: ${v}`));

subject.next(3);
// Subscriber A: 3
// Subscriber B: 3 (doesn't get 1 or 2)
```

Interview Tip:

"Subjects are like event emitters - they don't hold state and only push values to current subscribers."

3. BehaviorSubject - The Stateful Subject

Characteristics:

- Requires initial value: Must be initialized with a default
- Emits current value to new subscribers: Last emitted value is cached
- Common use case: Application state management

Code Example:

```
import { BehaviorSubject } from 'rxjs';

const behaviorSubject = new BehaviorSubject<number>(0);

behaviorSubject.subscribe(v => console.log(`Subscriber A: ${v}`));
// Immediately logs: Subscriber A: 0

behaviorSubject.next(1); // Subscriber A: 1

behaviorSubject.subscribe(v => console.log(`Subscriber B: ${v}`));
// Immediately logs: Subscriber B: 1
```

Interview Insight:

"BehaviorSubject is perfect for representing 'current state' - like a user authentication status where new components need the latest value immediately."

4. ReplaySubject - The Time-Traveling Subject

Characteristics:

- Replays previous values: Configurable buffer size
- No initial value required: Unlike BehaviorSubject
- Two key parameters:
 - `bufferSize`: How many values to replay
 - `windowTime`: How long to keep values (in ms)

Code Example:

```
import { ReplaySubject } from 'rxjs';

const replaySubject = new ReplaySubject<number>(2); // Buffer last 2 values

replaySubject.next(1);
replaySubject.next(2);
replaySubject.next(3);

replaySubject.subscribe(v => console.log(`Subscriber A: ${v}`));
// Immediately logs:
// Subscriber A: 2
// Subscriber A: 3
```

Advanced Use Case:

"In my analytics dashboard, I used ReplaySubject(10) to let new users see the last 10 data points immediately on connection."

5. Key Differences (Interview Table)

Feature	Subject	BehaviorSubject	ReplaySubject
Initial Value	✗ No	✓ Required	✗ Optional
Emits to New Subs	✗ Nothing	✓ Last value	✓ Buffered values
Buffer Control	✗ No	✗ No	✓ Configurable
Common Use Case	Event buses	Current state	Historical data

6. Interview Questions & Answers

Q1: When would you choose BehaviorSubject over Subject?

Strong Answer:

"When I need subscribers to immediately receive the last emitted value upon subscription - like maintaining a user's login state across components. Subject would require explicit caching, while BehaviorSubject handles this automatically."

Q2: How is ReplaySubject different from BehaviorSubject?

Pro Response:

"While both replay values, BehaviorSubject only stores the last value and requires initialization, whereas ReplaySubject can cache multiple values without needing an initial

value. For example, `ReplaySubject(5)` gives new subscribers the last 5 emissions."

Q3: What happens if you subscribe to a Subject after it completes?

Expert Insight:

"A completed Subject won't emit any values to late subscribers. However, a `ReplaySubject` will still replay its buffer even after completion, which is useful for caching API responses."

7. Performance Considerations

- Memory Usage: `ReplaySubjects` with large buffers consume more memory
 - Late Subscriptions: `BehaviorSubject` is more efficient than `ReplaySubject(1)` for single-value caching
 - Completion State: Completed Subjects free up resources but become unusable
-

8. Practical Comparison

```
// Subject - No memory of past events
const subject = new Subject();
subject.next(1);
subject.subscribe(console.log); // Nothing logged
```

```
// BehaviorSubject - Remembers last event
const behavior = new BehaviorSubject(0);
behavior.next(1);
behavior.subscribe(console.log); // Logs: 1
```

```
// ReplaySubject - Remembers N events
const replay = new ReplaySubject(2);
replay.next(1);
replay.next(2);
replay.subscribe(console.log); // Logs: 1, 2
```

Interview Tip:

"Always mention that while these are powerful, improper use (like giant ReplaySubject buffers) can cause memory leaks in long-running applications."

9. Real-World Use Cases

1. BehaviorSubject: User authentication state
2. ReplaySubject: Chat application message history
3. Subject: Keyboard event multicasting

Final Advice:

"In my Angular projects, I combine BehaviorSubject for current state with regular Subjects for events - this pattern handles 90% of state management needs without external libraries."

This comprehensive understanding will help you stand out in RxJS-focused interviews!



84.View encapsulation?

1. Core Concept

View encapsulation determines how component styles are applied and isolated in Angular. It's a crucial concept for maintaining style scoping and preventing CSS conflicts in large applications.

2. Types of View Encapsulation

Angular provides three encapsulation strategies:

Mode	Behavior	Shadow DOM	Style Isolation
Emulated (Default)	Styles are scoped to component via attribute selectors	 No	 Yes
None	Styles apply globally without isolation	 No	 No
ShadowDom	Uses browser's native Shadow DOM API	 Yes	 Yes

A. Emulated (Default)

```
@Component({
  selector: 'app-example',
  templateUrl: './example.component.html',
  styleUrls: ['./example.component.css'],
  encapsulation: ViewEncapsulation.Emulated // Default (can be omitted)
})
```

How it works:

- Angular adds unique attributes to component elements (`_ngcontent-abc`)
- Styles are rewritten to include these attributes:

```
/* Original */
.my-class { color: red; }
```

```
/* Compiled */
```

- `.my-class[_ngcontent-abc] { color: red; }`

Interview Tip:

"Emulated is the safest choice for most projects as it provides style isolation without requiring Shadow DOM support."

B. None (Global Styles)

```
@Component({  
  encapsulation: ViewEncapsulation.None  
})
```

Characteristics:

- Styles leak out and affect other components
- Useful for legacy CSS or when intentionally styling child components

Use Case Example:

"I used None when migrating a jQuery UI widget to Angular, allowing the existing global styles to work unchanged."

C. ShadowDom (Native Isolation)

```
@Component({  
  encapsulation: ViewEncapsulation.ShadowDom  
})
```

Key Features:

- True DOM isolation
- Styles don't leak in or out
- Requires browser support

Performance Consideration:

"ShadowDom has better performance for complex UIs since style calculations are scoped, but adds Shadow DOM overhead."

3. How to Change Encapsulation Mode

```
import { ViewEncapsulation } from '@angular/core';  
  
@Component({  
  encapsulation: ViewEncapsulation.ShadowDom // Change to desired mode  
})
```

4. Practical Differences

Style Application

```
<!-- Parent Component -->
<app-child></app-child>

<style>
  /* This will affect child only in None mode */
  p { color: red; }
</style>
```

Component Projection (ng-content)

- Emulated/None: Projected content keeps parent styles
- ShadowDom: Projected content gets component styles (unless `::slotted` is used)

5. Interview Questions & Answers

Q1: Why is Emulated the default?

Strong Answer:

"Emulated provides style isolation without browser compatibility concerns. Unlike ShadowDom, it works everywhere, and unlike None, it prevents accidental style collisions - the perfect balance for most applications."

Q2: When would you use ShadowDom?

Pro Response:

"For completely self-contained widgets or when building web components. In my microfrontend project, we used ShadowDom to ensure UI components could be embedded in any host page without style conflicts."

Q3: How does None affect performance?

Expert Insight:

"None can improve performance slightly by avoiding style rewriting, but risks global style pollution. The performance gain is negligible compared to the maintenance cost in large apps - I only use it for specific legacy integration cases."

6. Best Practices

1. Start with Emulated for most components
2. Use None sparingly - only when intentionally breaking isolation
3. Consider ShadowDom for:
 - Reusable component libraries
 - Microfrontends
 - Web Components

Interview Tip:

"In my current project, we use Emulated for 95% of components, with ShadowDom only for our design system components that get embedded in third-party sites."

7. Real-World Example

```
// Design System Button Component
@Component({
  selector: 'ds-button',
  templateUrl: './button.component.html',
  styleUrls: ['./button.component.scss'],
  encapsulation: ViewEncapsulation.ShadowDom
})
```

"This ensures our button styles never conflict with host applications, while still allowing customization via CSS variables."

Understanding these encapsulation modes demonstrates deep Angular knowledge and

helps solve real-world styling challenges!

85. ProvideIn(“Root”) and ProvideIn(“any”)

1. Core Concept

Angular's `providedIn` property determines how and where services are instantiated in the dependency injection (DI) system.

2. `providedIn: 'root'` (Default Recommendation)

Behavior:

- Creates a single shared instance for the entire application
- Service is tree-shakable (removed from bundle if unused)
- Loaded lazily (only when first injected)

Code Example:

```
@Injectable({
  providedIn: 'root'
})
export class AuthService {
  // Service implementation
}
```

When to Use:

- For global services (e.g., authentication, logging)
- When you need singleton behavior across the app
- 95% of use cases according to Angular team

Interview Tip:

"`providedIn: 'root'` is my default choice because it combines singleton pattern with tree-shaking benefits - the service won't be included in the final bundle if no component

uses it."

3. `providedIn: 'any'` (Niche Use Case)

Behavior:

- Creates one instance per lazy-loaded module
- But shared instance within each eagerly-loaded module
- Still tree-shakable

Code Example:

```
@Injectable({
  providedIn: 'any'
})
export class FeatureConfigService {
  // Service implementation
}
```

When to Use:

- When you need separate instances per lazy-loaded feature module
- Rare cases like module-level configuration services
- Not recommended for most scenarios (Angular team advises caution)

Real-World Example:

"In my e-commerce app, I used `'any'` for a `CatalogService` that needed different configurations per lazy-loaded marketplace module, while still sharing within each module."

4. Key Differences

Feature	<code>providedIn: 'root'</code>	<code>providedIn: 'any'</code>
Instance Scope	App-wide singleton	Per lazy-loaded module
Tree-Shakable	✔ Yes	✔ Yes
Common Use Cases	Global services	Module-specific configurations
Performance	Optimal (single instance)	Potentially multiple instances
Recommended	✔ Default choice	✗ Rarely needed

5. Interview Questions & Answers

Q1: Why is `'root'` preferred over adding to providers array?

Strong Answer:

"Three key reasons: 1) Tree-shaking eliminates unused services, 2) No need to manually declare in modules, and 3) It clearly expresses the service's scope intention. The providers array approach can accidentally create multiple instances."

Q2: When would you actually use `'any'`?

Pro Response:

"Only in very specific cases where a service needs different instances per lazy-loaded module - like when each module needs its own configuration. In my experience, this

represents less than 1% of services. Most times, a well-designed singleton with configuration input works better."*

Q3: How does this relate to NgModules?

Expert Insight:

*"While `providedIn` makes NgModule providers less necessary, they're still useful for:

1) Non-tree-shakable services, 2) When you need to override providers in tests, or 3) For third-party libs that don't use `providedIn`."*

6. Best Practices

1. Default to `'root'` for most services
2. Avoid `'any'` unless you have proven need
3. For module-specific services, consider:
 - Passing configuration via constructor
 - Using hierarchical DI with component providers

Interview Tip:

"In my current project, we use `'root'` for all services except one - a `FeatureFlagsService` that needed `'any'` because each lazy-loaded module had different experimental features."

7. Advanced Scenario

```
// Alternative to 'any' using hierarchical DI
@Component({
  providers: [UserContextService] // New instance per component
})
export class UserProfileComponent {
  constructor(private userContext: UserContextService) {}
}
```

"This often provides better control than `'any'` for scoped services."

Understanding these nuances demonstrates deep Angular DI knowledge that impresses interviewers! 🚀

86. How to import external JS or css file in angular?

There are several ways to include external JavaScript and CSS files in an Angular project, each suitable for different scenarios:

1. Adding Files to angular.json (Recommended for Global Files)

The standard approach for project-wide external files:

```
// angular.json
"styles": [
  "src/styles.css",
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/assets/css/external-styles.css"
],
"scripts": [
  "node_modules/jquery/dist/jquery.min.js",
  "src/assets/js/external-script.js"
]
```

Pros:

- Loaded globally when the app starts
- Bundled by Angular CLI (optimized for production)
- Available throughout the application

Cons:

- Loaded even if not needed (affects initial bundle size)
-

2. Dynamic Loading (Lazy Loading)

For conditionally loading scripts/CSS:

A. Using `document` to Inject Scripts/CSS

```
// component.ts
loadExternalScript(src: string): Promise<void> {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = src;
    script.onload = () => resolve();
    script.onerror = () => reject();
    document.body.appendChild(script);
  });
}

// Usage
await this.loadExternalScript('https://example.com/external.js');
```

B. Using `@angular/cdk` (Recommended)

```
import { Component, OnInit } from '@angular/core';
import { ScriptLoaderService } from '@angular/cdk';

@Component({
  selector: 'app-example',
  templateUrl: './example.component.html',
})
export class ExampleComponent implements OnInit {
  constructor(private scriptLoader: ScriptLoaderService) {}

  ngOnInit() {
    this.scriptLoader.loadScript('https://example.com/external.js').subscribe();
  }
}
```

Pros:

- Loads only when needed (better performance)
- Avoids conflicts with Angular's change detection

Cons:

- Requires manual cleanup (to avoid memory leaks)
-

3. Using `index.html` (Legacy Approach)

For third-party scripts (e.g., Google Analytics):

```
<!-- index.html -->
<head>
  <link rel="stylesheet" href="https://example.com/external.css">
  <script src="https://example.com/external.js"></script>
</head>
```

Pros:

- Simple to implement
- Works for scripts that must load early (e.g., analytics)

Cons:

- Not optimized for bundling
 - May block rendering if not `async/defer`
-

4. Using npm/yarn Packages (Best for Maintainability)

For popular libraries (Bootstrap, jQuery, etc.):

```
npm install bootstrap jquery
```

Then import in `styles.scss` or `angular.json`:

```
// styles.scss
@import '~bootstrap/dist/css/bootstrap.min.css';
```

For JS files, use `scripts` in `angular.json`:

```
"scripts": [
  "node_modules/jquery/dist/jquery.min.js"
]
```

Pros:

- Version-controlled via `package.json`
- Bundled efficiently by Angular CLI

Cons:

- Not suitable for CDN-hosted scripts
-

5. Using `@import` in Component CSS (Scoped Styles)

For component-specific external styles:

```
@Component({
  selector: 'app-example',
  templateUrl: './example.component.html',
  styleUrls: [
    './example.component.css',
    'https://example.com/external-component.css'
  ]
})
export class ExampleComponent {}
```

Pros:

- Styles are scoped to the component
- Only loaded when the component is used

Cons:

- Not all bundlers support external URLs in `styleUrls`
-

Best Practices

- ✓ Use `angular.json` for global CSS/JS (Bootstrap, FontAwesome)
 - ✓ Use dynamic loading for conditionally required scripts (Google Maps, Stripe)
 - ✓ Prefer npm packages over CDN links for better dependency management
 - ✗ Avoid `index.html` for large scripts (unless critical for initial load)
-

Interview Answer

Q: How do you include external JS/CSS in Angular?

A:

*"I use different approaches based on the scenario:

1. Global files (Bootstrap, jQuery) go in `angular.json` for optimal bundling.
2. Conditional scripts (like payment gateways) are loaded dynamically using `document.createElement` or Angular CDK.
3. Component-specific styles can use `styleUrls` with external URLs.
4. For third-party npm packages, I install them and reference via `node_modules`. This ensures performance, maintainability, and proper dependency management."*

This structured approach demonstrates deep Angular knowledge! 🚀

87. Angular.json file? Package.json file? Tsconfig.json file?

1. angular.json (Angular Workspace Configuration)

The main configuration file for Angular CLI projects.

Key Sections:

```
{
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "my-app": {
      "projectType": "application",
      "schematics": {},
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:browser",
          "options": {
            "outputPath": "dist/my-app",
            "index": "src/index.html",
            "main": "src/main.ts",
            "polyfills": "src/polyfills.ts",
            "tsConfig": "tsconfig.app.json",
            "assets": ["src/favicon.ico", "src/assets"],
            "styles": ["src/styles.css"],
            "scripts": []
          },
          "configurations": {
            "production": {
              "optimization": true,
              "outputHashing": "all"
            }
          }
        }
      }
    }
  }
},
```

```

    "serve": {
      "builder": "@angular-devkit/build-angular:dev-server"
    }
  }
}
}
}
}

```

Key Responsibilities:

- Project structure (source root, asset folders)
- Build configurations (development vs production)
- Global styles/scripts (Bootstrap, jQuery)
- Server options (port, proxy settings)
- Testing configurations

Interview Tip:

"angular.json is the Angular CLI's blueprint - it controls everything from build optimization levels to which files get copied to the dist folder."

2. package.json (NPM Dependency Management)

Standard Node.js file listing project dependencies and scripts.

Key Sections:

```

{
  "name": "my-angular-app",
  "version": "1.0.0",
  "scripts": {
    "start": "ng serve",
    "build": "ng build",
    "test": "ng test"
  },
  "dependencies": {
    "@angular/core": "^15.0.0",

```

```
    "rxjs": "^7.5.0"
  },
  "devDependencies": {
    "@angular/cli": "^15.0.0",
    "typescript": "~4.8.0"
  }
}
```

Key Responsibilities:

- Dependency management (Angular, RxJS, third-party libs)
- Script commands (ng serve, ng build)
- Version locking (ensures consistent installations)
- Project metadata (name, version, license)

Interview Insight:

"The package.json is Angular's dependency manifest - it's why npm install works. I always audit it to ensure no redundant packages and proper dev/prod separation."

3. tsconfig.json (TypeScript Configuration)

Configures TypeScript compiler options for Angular apps.

Key Sections:

```
{
  "compileOnSave": false,
  "compilerOptions": {
    "baseUrl": "./",
    "outDir": "./dist/out-tsc",
    "module": "es2020",
    "target": "es2020",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
  }
}
```

```
    "moduleResolution": "node",
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  },
  "angularCompilerOptions": {
    "strictTemplates": true
  }
}
```

Key Responsibilities:

- Type checking rules (strict mode, null checks)
- Module system (CommonJS vs ES Modules)
- Decorator support (required for Angular)
- Output directory (where compiled JS goes)
- Angular-specific rules (template strictness)

Pro Tip:

"Angular actually uses two TS configs - `tsconfig.app.json` for the app and `tsconfig.spec.json` for tests. The base `tsconfig.json` provides shared settings."

Comparison Table

File	Primary Purpose	Unique Feature
angular.json	CLI build/serve configuration	Defines multiple project setups
package.json	Dependency management	Scripts for development workflow
tsconfig.json	TypeScript compiler rules	Angular template type checking

Interview Questions & Answers

Q1: What happens if you delete angular.json?

Strong Answer:

"The Angular CLI would lose all project configuration - build paths, asset rules, and environment setups. You'd need to recreate it or restore from version control. The app might still run with `ng serve` but builds would fail."

Q2: How would you upgrade Angular versions?

Pro Response:

"I'd use `ng update` which intelligently modifies both `package.json` and `angular.json`. For major versions, I'd check the `update.angular.io` guide, then manually verify critical configs like compiler options and build paths."

Q3: Why does Angular need tsconfig.json?

Expert Insight:

"Three key reasons: 1) Angular components use decorators (`@Component`) requiring `experimentalDecorators`, 2) Strict template checking needs Angular-specific rules, and 3) It ensures consistent ES target output across the team."

Real-World Usage Examples

1. angular.json:

"We customized our production build to exclude unused locales from `moment.js`, reducing bundle size by 40%."

2. package.json:
"I implemented Husky hooks here to run linting before commits, ensuring code quality."
3. tsconfig.json:
"Enabling `strictTemplates` caught 12 template type errors during migration to Angular 14."

Key Takeaways

- angular.json = Build orchestration
- package.json = Dependency control
- tsconfig.json = Type safety backbone

Final Tip:

"Master these three files and you'll troubleshoot 80% of Angular configuration issues. They're the framework's foundation."

This comprehensive understanding will impress interviewers assessing your Angular architecture knowledge! 🚀

88. Reactive form and template driven form? Why should we use reactive form ? custom validation in reactive form?

1. Key Differences

Feature	Reactive Forms	Template-Driven Forms
Form Model	Explicit in Component (FormGroup)	Implicit (Directives in Template)

Data Flow	Synchronous (Observable-based)	Asynchronous
Validation	Dynamic (Programmatic)	Template-based
Complexity	Better for complex forms	Simpler for basic forms
Testability	Easier (Logic in Component)	Harder (Tied to DOM)
Dynamic Controls	Native Support	Requires workarounds

2. Why Use Reactive Forms? (Key Advantages)

A. More Control & Predictability

- Form state is managed programmatically
- Changes are synchronous and traceable

```
// Component
loginForm = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', [Validators.minLength(8)])
});
```

B. Better for Complex Forms

- Dynamic form fields (add/remove controls at runtime)
- Complex validation across multiple fields

```
addPhoneField() {
  this.userForm.addControl('phone', new FormControl('', Validators.required));
}
```


C. Easier Unit Testing

- Form logic is separated from template
- No DOM dependency for testing validation

```
it('should validate email format', () => {  
  const emailCtrl = component.loginForm.get('email');  
  emailCtrl.setValue('invalid');  
  expect(emailCtrl.hasError('email')).toBeTruthy();  
});
```

D. Reactive Programming Integration

- Works seamlessly with RxJS operators

```
this.loginForm.valueChanges.pipe(  
  debounceTime(500),  
  distinctUntilChanged()  
).subscribe(values => {  
  // Handle changes  
});
```

3. Custom Validation in Reactive Forms

A. Custom Validator Function

```
// Validator function  
function passwordStrength(control: FormControl) {  
  const value = control.value;  
  const hasNumber = /[0-9]/.test(value);  
  const hasUpper = /[A-Z]/.test(value);  
  const valid = hasNumber && hasUpper;  
  return valid ? null : { passwordWeak: true };  
}  
  
// Usage  
password: new FormControl('', [Validators.required, passwordStrength])
```

B. Cross-Field Validation (FormGroup Level)

```
function matchPassword(group: FormGroup) {
  const pass = group.get('password').value;
  const confirmPass = group.get('confirmPassword').value;
  return pass === confirmPass ? null : { mismatch: true };
}
```

// Usage

```
this.registerForm = new FormGroup({
  password: new FormControl(''),
  confirmPassword: new FormControl('')
}, { validators: matchPassword });
```

C. Async Validator (Server-Side Validation)

```
function uniqueEmail(control: FormControl): Observable<ValidationErrors | null> {
  return this.authService.checkEmail(control.value).pipe(
    map(exists => exists ? { emailTaken: true } : null)
  );
}
```

// Usage

```
email: new FormControl('', {
  validators: [Validators.required, Validators.email],
  asyncValidators: [uniqueEmail.bind(this)],
  updateOn: 'blur'
})
```

4. When to Use Template-Driven Forms?

- Simple forms with basic validation
- Quick prototypes
- When you prefer HTML-centric approach

<!-- Template-Driven Example -->

```
<input
  [(ngModel)]="user.email"
  name="email"
  required
```

```
    email
    #email="ngModel">
<div *ngIf="email.invalid && (email.dirty || email.touched)">
  <div *ngIf="email.errors.required">Required</div>
  <div *ngIf="email.errors.email">Invalid email</div>
</div>
```

5. Interview Questions & Answers

Q1: Why choose reactive forms over template-driven?

Strong Answer:

"Reactive forms provide better control for complex scenarios - like dynamic fields, cross-field validation, and seamless RxJS integration. They're also more testable since validation logic lives in the component rather than the template."

Q2: How would you validate a password confirmation field?

Pro Response:

"I'd create a FormGroup-level validator that compares password and confirmPassword fields. This ensures validation triggers when either field changes:"

```
this.form = new FormGroup({
  password: new FormControl(''),
  confirm: new FormControl('')
}, { validators: this.matchPasswords });

matchPasswords(group: FormGroup) {
  // Compare logic
}
```

Q3: When might template-driven forms be preferable?

Expert Insight:

"For simple contact forms or rapid prototyping, template-driven forms reduce boilerplate. However, I'd switch to reactive forms once business logic grows beyond basic validation."

Best Practices

- ✅ Use Reactive Forms for most production applications
- ✅ Leverage Custom Validators for complex rules
- ✅ Combine Sync + Async Validation for best UX
- ❌ Avoid Mixing Approaches in the same project

Final Tip:

"In my current project, we standardized on reactive forms after template-driven forms became unmanageable with 50+ fields. The migration reduced form-related bugs by 60%."

This comprehensive understanding demonstrates deep Angular form expertise! 🚀

89. Angular unit testing. Spyon ?

1. What is `spyOn`?

`spyOn` is a Jasmine function used in Angular unit tests to:

- Track calls to methods
- Mock return values
- Verify method execution
- Replace actual implementations with test doubles

2. Basic Usage

A. Creating a Spy

```
// On an object method
const spy = spyOn(component, 'methodName');

// On a service method
const serviceSpy = spyOn(testBed.inject(DataService), 'fetchData');
```

B. Common Spy Configurations

Configuration	Example	Purpose
Call tracking	<code>spyOn(obj, 'method')</code>	Just track calls
Mock return	<code>.and.returnValue(42)</code>	Force specific return
Throw error	<code>.and.throwError('Fail')</code>	Test error handling
Call fake	<code>.and.callFake(() => {})</code>	Custom implementation
Call through	<code>.and.callThrough()</code>	Spy but run real method

3. Practical Examples

Example 1: Testing Component Methods

```
it('should call calculateTotal when button clicked', () => {
  // Arrange
  const calculateSpy = spyOn(component, 'calculateTotal');

  // Act
  fixture.debugElement.query(By.css('button')).triggerEventHandler('click');

  // Assert
```

```
    expect(calculateSpy).toHaveBeenCalled();
  });
```

Example 2: Mocking Service Responses

```
it('should display user data', () => {
  // Arrange
  const mockUser = { id: 1, name: 'Test' };
  spyOn(userService, 'getUser').and.returnValue(of(mockUser));

  // Act
  fixture.detectChanges(); // Triggers ngOnInit

  // Assert
  expect(component.user).toEqual(mockUser);

  expect(fixture.debugElement.query(By.css('.user-name')).nativeElement.textContent)
    .toContain('Test');
});
```

Example 3: Verifying Call Arguments

```
it('should search with correct term', () => {
  const searchSpy = spyOn(service, 'search');

  component.searchTerm = 'angular';
  component.onSearch();

  expect(searchSpy).toHaveBeenCalledWith('angular');
});
```

4. Advanced Techniques

A. Spying on Properties

```
// Jasmine 3.6+ allows property spies

spyOnProperty(component, 'isValid').and.returnValue(true);
```

B. Resetting Spies

```
afterEach(() => {
  // Reset all spies
  jasmine.clock().uninstall();
});
```

C. Async Operations

```
it('should handle async service', fakeAsync(() => {
  const spy = spyOn(service, 'asyncOp').and.returnValue(Promise.resolve('data'));

  component.triggerAsync();
  tick(); // Advance fake async queue

  expect(component.result).toBe('data');
}));
```

5. Common Pitfalls & Solutions

Problem	Solution
"method does not exist"	Ensure method is public or use <code>any</code> type cast
Spy not being called	Check if test executes the code path
Real implementation runs	Add <code>.and.stub()</code> or <code>.and.returnValue()</code>
Change detection issues	Call <code>fixture.detectChanges()</code> after spy setup

6. Interview Questions & Answers

Q1: What's the difference between spy and mock?

Strong Answer:

"A spy observes method calls while a mock replaces entire objects. In Angular tests, we

typically use spies to track specific method interactions while keeping other functionality intact."

Q2: How would you test an error scenario?

Pro Response:

```
it('should handle errors', () => {  
  spyOn(service, 'fetch').and.returnValue(throwError('500 Error'));  
  
  component.loadData();  
  
  expect(component.error).toBeTruthy();  
});
```

Q3: When would you use callThrough vs returnValue?

Expert Insight:

"Use `callThrough` when you need the real implementation but want to track calls. Use `returnValue` when you need predictable outputs for testing specific code paths."

Best Practices

- ✓ Isolate tests - Each test should have its own spies
- ✓ Clear spy setup in `beforeEach` when reused
- ✓ Verify both calls and args for complete testing
- ✗ Avoid over-mocking - Only spy what you need to

Final Tip:

"In my current project, we use spies to achieve 85% test coverage while keeping tests maintainable. A well-placed spy can often replace complex mock setups."

This deep understanding of `spyOn` demonstrates professional-grade Angular testing

skills! 🚀

90. Pipes and method in pipe like Transform method? Types of pipes? Example of impure pipe

1. What are Pipes?

Pipes transform displayed values in templates without changing the original data.

They're used with the `|` syntax:

```
{{ value | pipeName:param }}
```

2. The `transform` Method (Core Pipe Method)

Every pipe must implement:

```
interface PipeTransform {  
  transform(value: any, ...args: any[]): any;  
}
```

Example Implementation:

```
@Pipe({ name: 'double' })  
export class DoublePipe implements PipeTransform {  
  transform(value: number): number {  
    return value * 2;  
  }  
}
```

Usage:

```
<p>{{ 5 | double }}</p> <!-- Output: 10 -->
```

3. Types of Pipes

A. Pure Pipes (Default)

- Runs only when:
 - Input reference changes (for objects/arrays)
 - Primitive value changes
- More efficient (Angular calls it less frequently)
- `@Pipe({ name: 'purePipe', pure: true })` *// 'pure' is true by default*

B. Impure Pipes

- Runs on every change detection cycle
- Use carefully - performance impact
- Needed when:
 - Working with mutable objects/arrays
 - Depending on non-primitive changes

```
@Pipe({ name: 'impurePipe', pure: false })
```

4. Impure Pipe Example

Scenario: Filtering an array that gets modified without reference change

```
@Pipe({ name: 'runningTotal', pure: false })
export class RunningTotalPipe implements PipeTransform {
  transform(numbers: number[]): number {
    console.log('Recalculating...'); // Will log on every change detection
    return numbers.reduce((a, b) => a + b, 0);
  }
}
```

Component:

```
numbers = [1, 2, 3];
```

```
addNumber() {
  this.numbers.push(this.numbers.length + 1);
  // No array reference change!
}
```

Template:

```
<button (click)="addNumber()">Add Number</button>
```

```
<p>Total: {{ numbers | runningTotal }}</p><!-- Updates correctly despite no array reference change -->
```

5. Built-in Pipes Examples

Pipe	Example	Output
DatePipe	{{ today date }}	"Jun 15, 2023"
UpperCasePipe	{{ 'text' uppercase }}	"TEXT"
JsonPipe	{{ obj json }}	Stringified JSON
AsyncPipe	{{ data\$ async }}	Resolves observable

6. When to Use Impure Pipes

✓ Mutable data changes without reference updates

✓ Complex calculations needing frequent updates

✗ Avoid for:

- Large datasets (performance hit)
- Simple transformations (pure pipes suffice)

Better Alternative:

Consider using component methods or RxJS for complex reactive scenarios

7. Interview Questions & Answers

Q1: Why would you create a custom pipe?

Strong Answer:

"For reusable data transformations across templates - like formatting phone numbers or converting units. Pipes keep transformation logic out of components, making both more maintainable."

Q2: What's the performance impact of impure pipes?

Pro Response:

"Impure pipes run on every change detection cycle. With large datasets or frequent updates, they can cause lag. I once fixed a performance issue by replacing an impure filter pipe with a memoized component function."

Q3: How would you optimize an impure pipe?

Expert Insight:

"Three approaches: 1) Add caching inside the pipe, 2) Use OnPush change detection, 3) For arrays, implement trackBy to minimize recalculations. Always profile before optimizing."

Best Practices

- Default to pure pipes (better performance)
- Document pipe purity in code comments
- Consider alternatives (getters, services) for complex cases

Final Tip:

"In my current project, we use pure pipes for 90% of cases and have just two carefully optimized impure pipes for real-time financial data streams."

This deep understanding demonstrates professional Angular template expertise! 🚀

91.What is directive? Difference between directive and component. What we pass in directive decorators and component decorators?

1. What is a Directive?

Directives are classes that add behavior to DOM elements. Angular has three types:

1. Components - Directives with templates
2. Attribute Directives - Change appearance/behavior (e.g., `ngStyle`)
3. Structural Directives - Alter DOM structure (e.g., `*ngIf`)

2. Directive vs Component: Key Differences

Feature	Directive	Component
Template	✗ No template	✓ Has template (HTML)
View Encapsulation	✗ Not applicable	✓ Supported (Shadow DOM)
Selector	Usually attribute (<code>[dir]</code>)	Element (<code>app-component</code>)
Purpose	Add behavior to existing elements	Create reusable UI pieces
Providers	✗ Rarely used	✓ Can have providers

3. Decorator Comparison

A. Directive Decorator (`@Directive`)

```

@Directive({
  selector: '[appHighlight]', // Attribute selector
  standalone: true // Angular 17+ (optional)
})
export class HighlightDirective {
  constructor(private el: ElementRef) {
    el.nativeElement.style.backgroundColor = 'yellow';
  }
}

```

Key Properties:

- `selector` (required): CSS selector for directive application
- `standalone` (v17+): Whether the directive can be used independently
- `host` (advanced): Maps class properties to host element events/properties

B. Component Decorator (@Component)

```

@Component({
  selector: 'app-user-card', // Element selector
  template: '<div>User: {{name}}</div>', // Required
  styles: [':host { display: block; }'], // Optional
  standalone: true, // Angular 17+
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class UserCardComponent {
  @Input() name!: string;
}

```

Key Properties:

Property	Purpose
<code>selector</code>	Custom HTML tag name

<code>template/templateUrl</code>	Component's HTML markup
<code>styles/styleUrls</code>	Component-scoped CSS
<code>standalone</code>	Independent component (v17+)
<code>changeDetection</code>	Optimization strategy
<code>providers</code>	Dependency injection tokens

4. Practical Examples

Attribute Directive Example

```
@Directive({
  selector: '[appTooltip]',
  standalone: true
})
export class TooltipDirective {
  @Input() appTooltip = ''; // Same name as selector

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.renderer.setAttribute(this.el.nativeElement, 'title', this.appTooltip);
  }
}
```

Usage:

```
<button [appTooltip]=" 'Click me!' ">Hover</button>
```

Component Example

```

@Component({
  selector: 'app-counter',
  template: `
    <button (click)="decrement()">--</button>
    <span>{{count}}</span>
    <button (click)="increment()">+</button>
  `,
  standalone: true
})
export class CounterComponent {
  count = 0;
  increment() { this.count++; }
  decrement() { this.count--; }
}

```

Usage:

```
<app-counter></app-counter>
```

5. When to Use Each

Use a Directive When:

- Modifying existing elements (e.g., adding hover effects)
- Creating behavior-only features (no UI needed)
- Extending native HTML elements

Use a Component When:

- Creating reusable UI widgets
- Need a view template
- Managing a section of the application

6. Interview Questions & Answers

Q1: Can a component be a directive?

Strong Answer:

"Yes! Components are actually a specialized type of directive with templates. While all components are directives, not all directives are components. The key differentiator is that components always have their own view templates."

Q2: Why would you choose a directive over a component?

Pro Response:

"I'd choose a directive when I need to augment existing DOM elements without introducing new visual structure. For example, creating a permission-based access control directive that hides elements for unauthorized users. This keeps the behavior separate from UI concerns."

Q3: What's the difference between host binding in directives vs components?

Expert Insight:

"Both can use `@HostBinding` and `@HostListener`, but components typically bind to their own template elements, while directives bind to the element they're attached to. For example, a directive might add keyboard events to any host element, while a component would handle events within its own template."

Best Practices

- ✅ Use components for UI building blocks
- ✅ Use directives for cross-cutting behavior
- ✅ Prefer standalone (Angular 17+) for better modularity
- ❌ Avoid overusing structural directives (can impact performance)

Final Tip:

"In my current project, we use attribute directives extensively for cross-cutting concerns"

like analytics tracking and permission checks, while components handle all visual UI elements. This separation keeps our codebase maintainable."

This comprehensive understanding demonstrates professional Angular architecture skills! 🚀

92. How angular application start(bootstrap)

1. Entry Point: main.ts

The bootstrapping process begins with the `main.ts` file, which serves as the application's entry point.

```
// main.ts
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Key Actions:

- Imports the Angular platform (browser version)
- Specifies the root module (`AppModule`) to bootstrap
- Handles any bootstrap errors

2. Platform Initialization

`platformBrowserDynamic()` creates a browser platform instance that:

- Sets up the Angular compiler
- Prepares the DOM environment
- Configures the Zone.js change detection system

3. Root Module (AppModule) Execution

The `bootstrapModule()` method loads and initializes the root module:

```
// app.module.ts
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent] // Root component specified here
})
export class AppModule {}
```

Key Phases:

1. Module instantiation - Creates the module injector
2. Dependency resolution - Processes all providers
3. Component bootstrapping - Creates the root component

4. Root Component Creation

The `bootstrap` array in `AppModule` specifies which component to instantiate first:

```
// app.component.ts
@Component({
  selector: 'app-root',
  template: `<h1>Welcome to {{title}}!</h1>`
})
export class AppComponent {
  title = 'My App';
}
```

What Happens:

1. Angular finds the `<app-root>` element in `index.html`
2. Creates an instance of `AppComponent`
3. Initiates change detection

5. index.html - The Host

The final piece that ties everything together:

```

<!doctype html>
<html>
  <head>...</head>
  <body>
    <app-root></app-root> <!-- Angular mounts here -->
  </body>
</html>

```

6. The Complete Bootstrap Flow

1. main.ts → Bootstraps platform and root module
2. AppModule → Configures application dependencies
3. AppComponent → Renders initial UI
4. index.html → Provides mounting point
5. Zone.js → Starts change detection cycle

7. Standalone Components (Angular 14+)

Modern Angular can bootstrap without modules:

```

// main.ts (standalone approach)
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    /* Global providers here */
  ]
});

```

Interview Questions & Answers

Q1: What's the difference between platformBrowser and platformBrowserDynamic?

Strong Answer:

"platformBrowserDynamic uses JIT (Just-In-Time) compilation which compiles templates in the browser, while platformBrowser is for AOT (Ahead-Of-Time) compiled apps. Production apps typically use AOT for better performance."

Q2: Why do we need Zone.js?

Pro Response:

"Zone.js monkey-patches async operations (setTimeout, promises, etc.) to trigger Angular's change detection automatically. Without it, we'd need to manually call detectChanges() after every async operation."

Q3: Can you have multiple bootstrap components?

Expert Insight:

"While the bootstrap array accepts multiple components, typically you only bootstrap one root component. Multiple bootstrap components would create separate component trees, which is rarely needed. A better pattern is to have a single root component with router-outlet for multiple views."

Performance Optimization Tips

- ✅ Use AOT compilation for production
- ✅ Lazy load modules with the router
- ✅ Consider standalone components (Angular 14+) for simpler apps
- ❌ Avoid heavy operations in constructor/ngOnInit

Final Tip:

"In my current project, we reduced initial load time by 40% by switching to AOT compilation and lazy loading feature modules. Understanding the bootstrap process was key to

identifying these optimizations."

This deep understanding of Angular initialization demonstrates architectural expertise!



93. what is typescript? How to implement inheritance in typescript(syntax) Super() keyword, extend

1. What is TypeScript?

TypeScript is a strongly typed superset of JavaScript that:

- Adds static typing (types are checked at compile-time)
- Compiles to plain JavaScript
- Supports modern ES6+ features (classes, modules, etc.)
- Provides better tooling (autocompletion, refactoring)
- Developed and maintained by Microsoft

Key Benefits:

- Catches errors during development (not at runtime)
- Improves code maintainability
- Enhances developer experience with IntelliSense
- Supports interfaces, generics, and decorators

2. Implementing Inheritance in TypeScript

A. Basic Inheritance Syntax

```
class Animal {  
  constructor(public name: string) {}  
  
  move(distance: number = 0) {  
    console.log(`${this.name} moved ${distance}m`);  
  }  
}
```

```
}
```

```
class Dog extends Animal { // 'extends' establishes inheritance
  constructor(name: string, public breed: string) {
    super(name); // 'super' calls parent class constructor
  }

  bark() {
    console.log('Woof! Woof!');
  }
}
```

```
const myDog = new Dog('Buddy', 'Golden Retriever');
myDog.move(10); // Inherited method
myDog.bark();   // Child class method
```

B. Method Overriding

```
class Cat extends Animal {
  constructor(name: string, public color: string) {
    super(name);
  }

  // Override parent method
  move(distance: number = 5) {
    console.log(`${this.name} (the ${this.color} cat) sneaked ${distance}m`);
    super.move(distance); // Call parent's version
  }
}
```

```
const myCat = new Cat('Whiskers', 'black');
myCat.move(); // Uses child's implementation
```

C. Abstract Classes (Advanced)

```
abstract class Shape {
  abstract getArea(): number; // Must be implemented by child

  display() {
    console.log(`Area: ${this.getArea()}`);
  }
}
```

```

    }
}

class Circle extends Shape {
  constructor(public radius: number) {
    super();
  }

  getArea(): number { // Concrete implementation
    return Math.PI * this.radius ** 2;
  }
}

const myCircle = new Circle(5);
myCircle.display(); // "Area: 78.53981633974483"

```

3. Key Concepts in TypeScript Inheritance

Concept	Syntax/Keyword	Purpose
Base Class	<code>class Parent</code>	Class being inherited from
Derived Class	<code>class Child extends Parent</code>	Inheriting class
<code>super()</code>	<code>super(parameters)</code>	Calls parent constructor
<code>super.method()</code>	<code>super.method()</code>	Calls parent's method
Method Overriding	Redefine method in child	Customize behavior
Abstract Class	<code>abstract class</code>	Base class that can't be instantiated

4. Practical Example with Interfaces

```
interface Loggable {  
    log(): void;  
}  
  
class Person {  
    constructor(public name: string) {}  
}  
  
class Employee extends Person implements Loggable {  
    constructor(name: string, public department: string) {  
        super(name);  
    }  
  
    log() {  
        console.log(`Employee ${this.name} from ${this.department}`);  
    }  
}  
  
const emp = new Employee('John', 'IT');  
emp.log(); // "Employee John from IT"
```

5. Interview Questions & Answers

Q1: Why use TypeScript over JavaScript?

Strong Answer:

"TypeScript adds static typing which catches errors during development, provides better IDE support through type checking and autocompletion, and supports modern features while maintaining compatibility with all JavaScript environments."

Q2: When would you use abstract classes vs interfaces?

Pro Response:

"I use abstract classes when I need to share implementation details between related

classes, and interfaces when I only need to define a contract without implementation. Abstract classes are better for 'is-a' relationships, while interfaces work well for 'can-do' relationships."

Q3: What does the `super()` keyword do?

Expert Insight:

"`super()` calls the parent class constructor and must be called before accessing `this` in a derived class constructor. For methods, `super.method()` allows accessing the parent's version of an overridden method."

Best Practices

- ✅ Favor composition over inheritance when possible
- ✅ Use abstract classes for shared implementation
- ✅ Implement interfaces for polymorphic behavior
- ❌ Avoid deep inheritance hierarchies (keep it shallow)

Final Tip:

"In my current project, we use TypeScript inheritance primarily for UI components (like base `FormComponent`) while preferring interfaces for service contracts. This balances code reuse with flexibility."

This comprehensive understanding demonstrates professional TypeScript expertise! 🚀

94. how many ways component can communicate in angular?

Angular provides multiple ways for components to communicate, each suitable for different scenarios:

1. Parent to Child Communication

@Input() Binding (Most Common)

```
// Parent Component
<app-child [message]="parentMessage"></app-child>

// Child Component
@Input() message: string;
```

2. Child to Parent Communication

@Output() + EventEmitter

```
// Child Component
@Output() notify = new EventEmitter<string>();

sendMessage() {
  this.notify.emit('Hello from child!');
}

// Parent Component
<app-child (notify)="onNotification($event)"></app-child>

onNotification(message: string) {
  console.log(message); // 'Hello from child!'
}
```

3. ViewChild & ContentChild

Access Child Directly

```
// Parent Component
@ViewChild(ChildComponent) child!: ChildComponent;

ngAfterViewInit() {
```

```
    console.log(this.child.someProperty);  
  }  
}
```

4. Services (Singleton Pattern)

Shared Data Service with RxJS

```
@Injectable({ providedIn: 'root' })  
export class DataService {  
  private dataSubject = new BehaviorSubject<string>('');  
  data$ = this.dataSubject.asObservable();  
  
  updateData(data: string) {  
    this.dataSubject.next(data);  
  }  
}
```

```
// Component A (Sender)  
constructor(private dataService: DataService) {}  
sendData() {  
  this.dataService.updateData('Shared data');  
}
```

```
// Component B (Receiver)  
currentData = '';  
constructor(private dataService: DataService) {}  
  
ngOnInit() {  
  this.dataService.data$.subscribe(data => {  
    this.currentData = data;  
  }); }  
}
```

5. Template Reference Variables

Direct Template Interaction

```
<app-child #childRef></app-child>  
<button (click)="childRef.childMethod()">Call Child</button>
```

6. Route Parameters

Passing Data via Routing

```
// Sender
this.router.navigate(['/details', id]);

// Receiver
constructor(private route: ActivatedRoute) {}
ngOnInit() {
  this.id = this.route.snapshot.paramMap.get('id');
}
```

7. NgRx/State Management (Advanced)

Global State Solution

```
// Dispatch action
this.store.dispatch(loadUser({ userId: 123 }));

// Select data
user$ = this.store.select(selectCurrentUser);
```

8. Local Storage/Session Storage

Persistent Communication

```
// Component A
localStorage.setItem('sharedData', JSON.stringify(data));

// Component B
const data = JSON.parse(localStorage.getItem('sharedData'));
```

Comparison Table

Method	Direction	Best Use Case
@Input()	Parent → Child	Passing initial data

@Output()	Child → Parent	Child events notification
ViewChild	Parent ↔ Child	Direct component access
Services	Any → Any	Cross-component communication
Template Variables	Template → Child	Simple template interactions
Route Parameters	Between routes	Page-to-page data transfer
State Management	App-wide	Complex application state
Local Storage	Persistent	Data surviving page reloads

Interview Questions & Answers

Q1: When would you choose a service over @Input/@Output?

Strong Answer:

"I use services when components aren't directly related in the component tree or when multiple components need access to the same data. @Input/@Output works well for tightly coupled parent-child components, but becomes cumbersome for deep nesting or sibling communication."

Q2: What's the advantage of BehaviorSubject in services?

Pro Response:

"BehaviorSubject provides two key benefits: 1) It stores the current value, so new subscribers immediately get the latest data, and 2) It ensures all components receive synchronized updates through a single stream of truth."

Q3: How would you handle communication between sibling components?

Expert Insight:

"For siblings, I typically:

1. Lift state up to a common parent (if simple)
 2. Use a shared service with RxJS (for complex cases)
 3. Consider state management (NgRx) if it's app-wide critical data
- Avoid direct sibling-to-sibling communication as it creates tight coupling."**

Best Practices

- ✅ Prefer @Input/@Output for direct parent-child relationships
- ✅ Use services for unrelated components
- ✅ Consider performance with frequent communications
- ❌ Avoid overusing ViewChild (breaks encapsulation)

Final Tip:

"In my current project, we standardized on:

- @Input/@Output for UI components
 - Services for feature modules
 - NgRx for global app state
- This layered approach keeps communication clean and maintainable."**

This comprehensive guide demonstrates professional Angular architecture skills! 🚀

95. Viewchild, viewchildren decorators angular?

1. @ViewChild - Accessing a Single Element/Component

Basic Usage

```
import { ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
  template: `
    <div #myDiv>Some content</div>
    <app-child-component #childComp></app-child-component>
  `
})
export class ParentComponent implements AfterViewInit {
  // 1. Access DOM element
  @ViewChild('myDiv') myDivRef!: ElementRef<HTMLDivElement>;

  // 2. Access child component
  @ViewChild('childComp') childComponent!: ChildComponent;

  // 3. Access directive
  @ViewChild(MyDirective) myDirective!: MyDirective;

  ngAfterViewInit() {
    // Safe to access here (after view initialized)
    console.log(this.myDivRef.nativeElement.textContent);
    this.childComponent.someMethod();
    this.myDirective.doSomething();
  }
}
```

Key Features

- Static option (Angular 8+):

```
@ViewChild('myRef', { static: true }) // Available in ngOnInit
```

- @ViewChild('myRef', { static: false }) // Default, available in ngAfterViewInit

- Supported Selectors:
 - Template reference variables (#name)
 - Component/directive types
 - TemplateRef (for ng-template)
 - ViewContainerRef

2. @ViewChildren - Accessing Multiple Elements

Basic Usage

```
import { ViewChildren, QueryList } from '@angular/core';

@Component({
  template: `
    <li *ngFor="let item of items" #listItems>{{ item }}</li>
    <app-child *ngFor="let child of children"></app-child>
  `,
})
export class ParentComponent implements AfterViewInit {
  @ViewChildren('listItems') itemElements!: QueryList<ElementRef>;
  @ViewChildren(ChildComponent) children!: QueryList<ChildComponent>;

  ngAfterViewInit() {
    this.itemElements.forEach(el => {
      console.log(el.nativeElement.textContent);
    });

    console.log(this.children.length);
  }
}
```

Dynamic Changes Handling

```
ngAfterViewInit() {
  this.itemElements.changes.subscribe((changes: QueryList<ElementRef>) => {
    // React to added/removed elements
  });
}
```

3. Practical Use Cases

DOM Manipulation

```
@ViewChild('submitBtn', { read: ElementRef }) button!:  
ElementRef<HTMLButtonElement>;
```

```
disableButton() {  
    this.button.nativeElement.disabled = true;  
}
```

Accessing Child Components

```
@ViewChild(ChildFormComponent) form!: ChildFormComponent;
```

```
submitParentForm() {  
    if (this.form.isValid()) {  
        this.form.submit();  
    }  
}
```

Working with Directives

```
@ViewChild(MyTooltipDirective) tooltip!: MyTooltipDirective;
```

```
showTooltip() {  
    this.tooltip.show();  
}
```

4. Key Differences

Feature	@ViewChild	@ViewChildren
Returns	Single reference	QueryList of references

Use Case	Single element/component	Multiple elements/components
Static Option	Yes	No
Change Detection	Manual tracking	Automatic via QueryList

5. Best Practices

- ✓ Access in `ngAfterViewInit`: View children aren't available until after view initialization
- ✓ Use `{ static: true }` only when needed in `ngOnInit`
- ✓ Prefer template references over component types when possible
- ✓ Unsubscribe from `QueryList.changes` to prevent memory leaks
- ✗ Avoid direct DOM manipulation when Angular alternatives exist

6. Common Pitfalls & Solutions

Problem: "ExpressionChangedAfterChecked" error

Solution: Use `setTimeout` or `ChangeDetectorRef.detectChanges()` after modifications

Problem: Undefined `ViewChild` reference

Solution:

1. Check if element exists conditionally (`*ngIf`)
2. Verify `{ static }` setting matches usage timing

7. Interview Questions & Answers

Q1: When would you use `static: true`?

Strong Answer:

*"I use { static: true } when I need to access the ViewChild in ngOnInit. This is rare but useful for:

1. Setting up initial focus on an element
2. Accessing values before the view renders

For most cases, I keep static: false (default) and access in ngAfterViewInit."*

Q2: How does ViewChildren handle dynamic lists?

Pro Response:

"ViewChildren returns a QueryList that automatically updates when the DOM changes. We can subscribe to its changes observable to react to additions/removals. This makes it perfect for dynamic UIs like lists or tab systems."

Q3: Can you use ViewChild with ng-template?

Expert Insight:

*"Yes! ViewChild can access TemplateRef for dynamic template rendering:

```
@ViewChild('myTemplate') template!: TemplateRef<any>;
```

This is how structural directives like *ngIf* work internally."

Final Tip:

"In my current project, we use ViewChildren extensively for custom table components to manage dynamic column rendering efficiently while keeping performance optimal."

This comprehensive understanding demonstrates professional Angular expertise! 🚀

96. What is metadata in angular? What is decorators?

1. Metadata in Angular

Metadata provides configuration information to Angular about how to process classes.

It tells Angular:

- What a class represents (component, service, etc.)
- How it should be instantiated
- How it interacts with other classes
- How it should be processed by the compiler

Where Metadata Appears

```
@Component({ // This entire object is metadata
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent { /*...*/ }
```

Key Metadata Types

Decorator	Metadata Purpose
<code>@Component</code>	Defines a component with template
<code>@Directive</code>	Configures attribute/structural directives
<code>@Injectable</code>	Marks a class as injectable service
<code>@NgModule</code>	Declares an Angular module
<code>@Input/@Output</code>	Defines component property bindings

2. Decorators in Angular

Decorators are special functions that modify classes or class members. They:

- Are prefixed with `@` symbol
- Attach metadata to classes
- Can modify class behavior

A. Class Decorators

```
@Injectable({ providedIn: 'root' }) // Class decorator
export class DataService { /*...*/ }
```

B. Property Decorators

```
export class ProductComponent {
  @Input() product!: Product; // Property decorator
  @Output() addToCart = new EventEmitter(); // Property decorator
}
```

C. Method Decorators

```
export class AuthService {
  @LogExecutionTime() // Custom method decorator
  login() { /*...*/ }
}
```

D. Parameter Decorators

```
constructor(@Inject('API_URL') private apiUrl: string) {}
```

3. How Decorators Work

Under the hood, decorators are functions that receive:

1. The target (class/class member)
2. The member name (for properties/methods)
3. The property descriptor (for methods)

Simple Decorator Example

```
// Custom class decorator
function LogClass(target: Function) {
  console.log(`Class created: ${target.name}`);
}

@LogClass
class MyClass { /*...*/ }

// Console: "Class created: MyClass"
```

4. Angular's Built-in Decorators

Essential Decorators

Decorator	Example	Purpose
@Component	@Component({...})	Creates UI component
@Directive	@Directive({selector: '[dir]'})	Creates directive
@Pipe	@Pipe({name: 'filter'})	Defines pipe
@Injectable	@Injectable({providedIn: 'root'})	Makes class injectable
@Input	@Input() value: string	Parent-to-child binding
@Output	@Output() event = new EventEmitter()	Child-to-parent events

5. Metadata vs Decorators

Aspect	Metadata	Decorators
Purpose	Configuration instructions	Functions that apply metadata
Syntax	Object inside decorator	@ prefixed functions
Example	<code>{selector: 'app-root'}</code>	<code>@Component({...})</code>
Level	Data	Processor of that data

6. Interview Questions & Answers

Q1: What happens if you omit metadata from a component?

Strong Answer:

"Angular won't know how to process the class. At minimum, a component requires:

1. A `selector` (how it's used in templates)
 2. A `template/templateUrl` (its view definition)
- Without these, Angular will throw compilation errors."

Q2: Can you create custom decorators in Angular?

Pro Response:

"Absolutely! While we mostly use Angular's built-in decorators, custom decorators are powerful for cross-cutting concerns like logging or validation. For example:

```
function LogMethod(target: any, key: string, descriptor: PropertyDescriptor) {
  const original = descriptor.value;
  descriptor.value = function(...args: any[]) {
    console.log(`Calling ${key} with`, args);
  };
}
```



```
    return original.apply(this, args);  
};  
return descriptor;  
}
```

This logs all method calls automatically."

Q3: Why does Angular use decorators instead of configuration files?

Expert Insight:

*"Decorators keep configuration colocated with the code it affects, improving:

1. Maintainability - Changes happen in one place
 2. Discoverability - Configuration is visible where used
 3. Type Safety - TS can validate decorator usage
- This approach is more intuitive than separate config files like in some other frameworks."*

7. Best Practices

- ✓ Use decorators consistently - Follow Angular style guide
- ✓ Keep metadata minimal - Only specify what's needed
- ✓ Understand decorator execution order - Class decorators run last
- ✗ Avoid complex logic in decorators - Keep them declarative

Final Tip:

"In my current project, we created custom `@FeatureFlag()` and `@Audit()` decorators that reduced boilerplate code by 30% while adding powerful functionality."

This deep understanding demonstrates professional Angular architecture knowledge!



97. Web component basic? What is web component and how to create web component?

1. What are Web Components?

Web Components are a suite of browser technologies that allow you to create:

- Reusable custom HTML elements with encapsulated functionality
- Framework-independent UI components that work across modern browsers
- Self-contained widgets with isolated styles and behavior

Core Technologies

Technology	Purpose
Custom Elements	Define new HTML tags
Shadow DOM	Encapsulate styles and markup (prevents CSS/JS leakage)
HTML Templates	Declare reusable markup fragments (<code><template></code>)
ES Modules	Share and reuse components across projects

2. Key Characteristics

- Native browser features (no external libraries needed)
- Framework compatibility (works with Angular, React, Vue, or no framework)

- True encapsulation (styles and DOM don't leak out)
- Lifecycle callbacks (similar to component frameworks)

3. How to Create a Web Component

Basic Implementation Steps

1. Define a Custom Element Class

```
class MyCounter extends HTMLElement {
  constructor() {
    super();

    // Initialize component state
    this.count = 0;

    // Attach Shadow DOM
    this.attachShadow({ mode: 'open' });

    // Create template
    this.shadowRoot.innerHTML = `
      <style>
        button {
          padding: 8px 16px;
          background: #4CAF50;
          color: white;
          border: none;
        }
      </style>
      <button id="counter">Count: 0</button>
    `;

    // Get DOM references
    this.button = this.shadowRoot.getElementById('counter');
  }

  // Lifecycle: When component is added to DOM
  connectedCallback() {
    this.button.addEventListener('click', this.increment);
  }
}
```

```

    }

    // Lifecycle: When component is removed from DOM
    disconnectedCallback() {
      this.button.removeEventListener('click', this.increment);
    }

    increment = () => {
      this.count++;
      this.button.textContent = `Count: ${this.count}`;
      this.dispatchEvent(new CustomEvent('count-changed', {
        detail: this.count
      }));
    }
  }
}

```

2. Register the Custom Element

```

// Define the new element
customElements.define('my-counter', MyCounter);

```

3. Use in HTML

```

<my-counter></my-counter>

<script>
  document.querySelector('my-counter')
    .addEventListener('count-changed', (e) => {
      console.log('New count:', e.detail);
    });
</script>

```

4. Advanced Features

A. Attributes and Properties

```
class MyGreeting extends HTMLElement {
  static get observedAttributes() {
    return ['name']; // Watch these attributes
  }

  attributeChangedCallback(name, oldValue, newValue) {
    if (name === 'name') {
      this.shadowRoot.innerHTML = `<p>Hello, ${newValue}!</p>`;
    }
  }
}

customElements.define('my-greeting', MyGreeting);
```

Usage:

```
<my-greeting name="Alice"></my-greeting>
```

B. Slots (Content Projection)

```
<!-- Component Definition -->
<template id="user-card">
  <style> /* Encapsulated styles */ </style>
  <div class="card">
    <slot name="title">Default Title</slot>
    <slot></slot> <!-- Default slot -->
  </div>
</template>

<!-- Usage -->
<user-card>
  <h1 slot="title">Custom Title</h1>
  <p>This content goes into default slot</p>
</user-card>
```

C. Extending Built-in Elements

```
class FancyButton extends HTMLButtonElement {
  constructor() {
```

```
    super();
    this.style.background = 'linear-gradient(45deg, #FF00CC, #3333FF)';
  }
}
customElements.define('fancy-button', FancyButton, { extends: 'button' });
```

Usage:

```
<button is="fancy-button">Click Me!</button>
```

5. Web Components vs Framework Components

Feature	Web Components	Framework Components (Angular/React)
Encapsulation	Shadow DOM	Emulated (Angular) / CSS-in-JS
Browser Support	Native	Requires framework runtime
Data Binding	Manual (no reactivity)	Built-in reactive systems
Tooling	Basic	Advanced CLI and dev tools

6. When to Use Web Components

- ✓ Cross-framework UI libraries
- ✓ Microfrontend integration
- ✓ Long-lived design systems
- ✓ Embeddable widgets (e.g., chat widgets, video players)

7. Best Practices

1. Use Shadow DOM for style encapsulation
2. Prefix custom element names (e.g., my-app-header)
3. Implement lifecycle methods properly
4. Provide fallback content in slots
5. Document attributes/events like an API

Example Production-Grade Component:

```
class AppDrawer extends HTMLElement {
  static get observedAttributes() { return ['open']; }

  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
    this.shadowRoot.innerHTML = `
      <style>
        :host { display: block; }
        .drawer {
          transform: translateX(-100%);
          transition: transform 0.3s ease;
        }
        :host([open]) .drawer {
          transform: none;
        }
      </style>
      <div class="drawer">
        <slot></slot>
      </div>
    `;
  }

  get open() { return this.hasAttribute('open'); }
  set open(val) { val ? this.setAttribute('open', '') :
this.removeAttribute('open'); }

  attributeChangedCallback(name, oldVal, newVal) {
    if (name === 'open') {
      this.dispatchEvent(new CustomEvent('open-changed'));
    }
  }
}
```

```
    }  
  }  
}  
customElements.define('app-drawer', AppDrawer);
```

Interview Questions & Answers

Q1: What problem do Web Components solve?

Strong Answer:

"Web Components solve three key problems:

1. Framework lock-in by providing native browser standards
2. Style encapsulation through Shadow DOM
3. Component reuse across different tech stacks without rewriting"

Q2: How does Shadow DOM differ from Virtual DOM?

Pro Response:

"Shadow DOM is a browser technology for DOM/style encapsulation, while Virtual DOM is a framework optimization technique (like in React). Shadow DOM creates isolated DOM trees, whereas Virtual DOM is a lightweight representation used for efficient updates."

Q3: Can Web Components work with Angular/React?

Expert Insight:

"Yes! While they have different philosophies:

- Angular can wrap Web Components using `@angular/elements`
- React treats them as regular DOM elements (with some event handling quirks)
I've successfully integrated Web Components into both frameworks for shared UI libraries."

Final Tip:

*"For maximum compatibility, always:

1. Use vanilla JavaScript (no framework code inside)
2. Provide React/Angular wrappers if needed
3. Document all public APIs (attributes/events)"*

This comprehensive understanding demonstrates professional-grade web platform expertise! 🚀

98.Describe how you handle form validation in Angular.

Form validation is critical for data integrity and user experience. Angular provides powerful tools for both template-driven and reactive form validation. Here's my professional approach:

1. Reactive Forms Validation (Recommended Approach)

Basic Setup with Validators

```
import { Validators, FormBuilder } from '@angular/forms';

this.userForm = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [
    Validators.required,
    Validators.minLength(8),
    Validators.pattern(/^(?=.*[A-Z])(?=.*\d).*$/) // Requires 1 uppercase & 1
number
  ]],
  age: [null, [Validators.min(18), Validators.max(99)]]
});
```

Template Display

```
<form [formGroup]="userForm" (ngSubmit)="onSubmit()">
  <input formControlName="email" placeholder="Email">
```

```

<div *ngIf="userForm.get('email')?.errors">
  <span *ngIf="userForm.get('email')?.hasError('required')">Email is
required</span>
  <span *ngIf="userForm.get('email')?.hasError('email')">Invalid email
format</span>
</div>

<button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>

```

2. Custom Validators

Cross-Field Validation (Password Match)

```

function passwordMatchValidator(group: FormGroup): ValidationErrors | null {
  const password = group.get('password')?.value;
  const confirmPassword = group.get('confirmPassword')?.value;
  return password === confirmPassword ? null : { mismatch: true };
}

```

// Usage

```

this.userForm = this.fb.group({
  password: ['', Validators.required],
  confirmPassword: ['', Validators.required]
}, { validators: passwordMatchValidator });

```

Async Validator (Username Availability)

```

function uniqueUsernameValidator(control: FormControl):
Observable<ValidationErrors | null> {
  return this.authService.checkUsername(control.value).pipe(
    map(exists => exists ? { usernameTaken: true } : null),
    catchError(() => of(null))
  );
}

```

// Usage

```

username: ['', {
  validators: [Validators.required],
  asyncValidators: [uniqueUsernameValidator.bind(this)],

```

```
    updateOn: 'blur'
  }]
```

3. Template-Driven Validation

Basic Example

```
<form #form="ngForm" (ngSubmit)="onSubmit(form)">
  <input
    name="email"
    ngModel
    required
    email
    #email="ngModel">

  <div *ngIf="email.invalid && (email.touched || form.submitted)">
    <span *ngIf="email.errors?.required">Required</span>
    <span *ngIf="email.errors?.email">Invalid email</span>
  </div>
</form>
```

4. Dynamic Validation

Conditional Validation

```
const phoneControl = this.userForm.get('phone');
this.userForm.get('hasPhone').valueChanges.subscribe(hasPhone => {
  if (hasPhone) {
    phoneControl.setValidators([Validators.required,
    Validators.pattern(/^\d{10}$/)]);
  } else {
    phoneControl.clearValidators();
  }
  phoneControl.updateValueAndValidity();
});
```

5. UX Enhancements

Visual Feedback

```
.ng-invalid.ng-touched:not(form) {  
  border-left: 4px solid #e74c3c;  
}
```

```
.ng-valid.ng-touched:not(form) {  
  border-left: 4px solid #2ecc71;  
}
```

Error Message Component (Reusable)

```
@Component({  
  selector: 'app-form-error',  
  template: `  
    <div class="error" *ngIf="control?.errors && (control?.touched || submitted)">  
      <span *ngIf="control?.hasError('required')">Field is required</span>  
      <span *ngIf="control?.hasError('email')">Invalid email</span>  
      <!-- Add more error types -->  
    </div>  
  `,  
})  
  
export class FormErrorComponent {  
  @Input() control!: AbstractControl;  
  @Input() submitted = false;  
}
```

Usage:

```
<app-form-error [control]="userForm.get('email')"  
  
[submitted]="formSubmitted"></app-form-error>
```

6. Server-Side Validation Handling

```
this.userService.save(this.userForm.value).pipe(
  catchError(error => {
    if (error.status === 400) {
      // Apply server errors to form
      error.errors.forEach(err => {
        this.userForm.get(err.field)?.setErrors({ serverError: err.message });
      });
    }
    return throwError(error);
  })
).subscribe();
```

Best Practices

- ✅ Use reactive forms for complex validation scenarios
- ✅ Create reusable validators for common patterns
- ✅ Combine client + server validation for security
- ✅ Debounce async validators (300-500ms)
- ✅ Disable submit button during async validation
- ❌ Avoid complex logic in templates

Final Implementation Tip:

"In my current project, we reduced form code by 40% by creating a `FormService` that handles common validations, error messages, and server error mapping consistently across 50+ forms."

This approach demonstrates professional-grade Angular form architecture! 🚀

99. What is the difference between constructor and ngOnInit?

1. Fundamental Distinction

Feature	Constructor	ngOnInit
Type	JavaScript/TypeScript feature	Angular lifecycle hook
Purpose	Class initialization	Component initialization
Timing	Runs when class is instantiated	Runs after first change detection

2. Execution Context

Constructor

```
class MyComponent {  
  constructor() {  
    console.log('Constructor runs');  
    // Avoid DOM operations here - elements  
    don't exist yet  
  }  
}
```

- Runs first during component creation

- Ideal for:
 - Dependency injection
 - Basic property initialization
 - Simple setup without DOM dependencies

ngOnInit

```
ngOnInit() {
  console.log('ngOnInit runs');
  // Safe for DOM operations and initial data
  loading
}
```

- Runs after Angular completes:
 - Property binding (`@Input()` values available)
 - View/DOM initialization
- Ideal for:
 - Complex initialization logic
 - API calls
 - DOM-dependent operations

3. Practical Comparison

Dependency Injection (Works in Both)

```
constructor(private http: HttpClient) {} // Preferred location for DI
```

@Input() Values

```
@Input() userId!: string;

constructor() {
  console.log(this.userId); // UNDEFINED (too
  early)
}

ngOnInit() {
  console.log(this.userId); // Correct value
  available
}
```

```
}
```

DOM Access

```
@ViewChild('myElement') elem!: ElementRef;
```

```
constructor() {  
  // this.elem.nativeElement ← ERROR (not  
  rendered yet)  
}
```

```
ngOnInit() {  
  // this.elem.nativeElement ← Safe to access  
}
```

4. Performance Considerations

- Constructor should be lean (affects instantiation speed)
- ngOnInit can handle heavier operations (runs after render)

```
// Good practice  
constructor(private lightService:  
  LightweightService) {}  
  
ngOnInit() {  
  this.heavyService.loadData(); // Heavy  
  operation  
}
```

5. Best Practices

✅ Use constructor for:

- Dependency injection
- Simple property initialization
- Basic setup

✅ Use ngOnInit for:

- Complex initialization

- API calls
- DOM interactions
- Input-dependent logic

✗ Avoid in constructor:

- DOM operations
- Heavy computations
- Logic requiring `@Input()` values

6. Real-World Example

```
@Component({
  selector: 'app-user-profile',
  template: `<div #profile>{{ userData }}</div>`
})
export class UserProfileComponent implements
OnInit {
  @Input() userId!: string;
  @ViewChild('profile') profileEl!: ElementRef;
  userData: any;

  constructor(
    private userService: UserService,
    private renderer: Renderer2
  ) {
    // Lightweight setup only
    this.userService = userService;
  }

  ngOnInit() {
    // Inputs are available now

    this.userService.getUser(this.userId).subscribe
    (user => {
      this.userData = user;
    });
  }
}
```

```
// DOM is ready

this.renderer.setStyle(this.profileEl.nativeElement, 'color', 'blue');
    });
}
```

Interview Questions & Answers

Q1: Why can't we use @Input() values in constructor?

Strong Answer:

"Angular's property binding happens after component instantiation but before ngOnInit. The constructor runs during class creation when @Input() properties haven't been assigned yet."

Q2: Can we have multiple constructors?

Pro Response:

"No, TypeScript/JavaScript classes only support one constructor. For complex initialization, we either:

1. Use optional parameters with default values
2. Move logic to ngOnInit
3. Create factory methods"

Q3: When might you skip ngOnInit?

Expert Insight:

"For extremely simple components without initialization logic or @Input() dependencies. However, I still implement OnInit (empty) as a best practice for consistency and future extensibility."

Memory Management Tip

"Always unsubscribe from observables in ngOnDestroy, even if they're initiated in ngOnInit.

This prevents memory leaks when components are destroyed."

This comprehensive comparison demonstrates deep Angular lifecycle knowledge! 🚀

100.What is typescript?

TypeScript is an open-source programming language developed and maintained by Microsoft that serves as a strict syntactical superset of JavaScript with added static typing. Here's a detailed breakdown:

1. Core Characteristics

Feature	Description
Typed JavaScript	Adds optional static types to JavaScript
Compiled Language	Transpiles to plain JavaScript (ES3/ES5/ES6+)
Object-Oriented	Supports classes, interfaces, inheritance
Tooling Support	Enhanced IDE features (IntelliSense, refactoring, error checking)
ECMAScript Compliant	Always stays current with JavaScript standards

2. Key Benefits

A. Type Safety

```
function add(a: number, b: number): number {  
    return a + b;  
}  
add(5, 3); //  Works  
add('5', 3); //  Compile-time error
```

B. Enhanced Tooling

- Autocompletion
- Code navigation
- Intelligent refactoring
- Immediate error detection

C. Modern JavaScript Features

- Classes
- Modules
- Arrow functions
- Async/await
- Decorators

D. Scalability

- Better for large codebases
- Improved team collaboration
- Easier maintenance

3. TypeScript vs JavaScript

Aspect	TypeScript	JavaScript
Typing	Static (compile-time)	Dynamic (runtime)
Compilation	Required	Not required
Error Checking	Early (during development)	Late (during execution)
Learning Curve	Steeper (additional concepts)	Gentler
Adoption	Growing in enterprises	Universal

4. Basic Type System

Primitive Types

```
let name: string = "John";
let age: number = 30;
let isActive: boolean = true;
```

Complex Types

```
// Array
let numbers: number[] = [1, 2, 3];

// Tuple
let user: [string, number] = ["Alice", 25];

// Enum
enum Color { Red, Green, Blue }
let c: Color = Color.Green;

// Any (opt-out of type checking)
```

```
let dynamic: any = "could be anything";

// Void (no return value)
function warn(): void { console.log("Warning"); }
```

5. Advanced Features

A. Interfaces

```
interface User {
  id: number;
  name: string;
  email?: string; // Optional property
}

function printUser(user: User) {
  console.log(user.name);
}
```

B. Generics

```
function identity<T>(arg: T): T {
  return arg;
}

let output = identity<string>("hello");
```

C. Decorators

```
@Component({
  selector: 'app-root'
})
class AppComponent { /*...*/ }
```

6. How TypeScript Works

1. Write TypeScript (.ts files)
2. Compile (tsc compiler or webpack/babel plugins)
3. Generate JavaScript (.js files)
4. Run in Browser/Node

7. Popular Use Cases

- ✓ Enterprise Applications
- ✓ Angular Development (built with TypeScript)
- ✓ Node.js Backends
- ✓ Large-Scale JavaScript Projects

8. Getting Started

1. Install:

```
npm install -g typescript
```

2. Create `tsconfig.json`:

```
{  
  "compilerOptions": {  
    "target": "ES6",  
    "module": "commonjs"  
  }  
}
```

3. Compile:

```
tsc yourfile.ts
```

Interview Questions & Answers

Q1: Why use TypeScript over JavaScript?

Strong Answer:

"TypeScript provides static typing which catches errors during development rather than runtime. It significantly improves code quality, maintainability, and developer productivity—especially in large codebases. The enhanced tooling support alone can reduce debugging time by 40%."

Q2: Does TypeScript replace JavaScript?

Pro Response:

"No, TypeScript is a superset—all JavaScript is valid TypeScript. It compiles down to JavaScript and works with existing JS ecosystems. Think of it as JavaScript with superpowers rather than a replacement."

Q3: When shouldn't you use TypeScript?

Expert Insight:

"For very small projects or quick prototypes where setup overhead outweighs benefits. Also, when working with libraries lacking type definitions, though most popular ones now have @types packages available."

Evolution & Adoption

- Created by Microsoft (2012)
- Used by Google, Airbnb, Slack, Asana
- Angular's primary language
- Ranked among top 10 most loved languages (Stack Overflow surveys)

Final Tip:

"In my projects, TypeScript reduced runtime errors by ~60% and significantly improved onboarding for new team members through self-documenting type systems."

This deep understanding demonstrates professional-grade TypeScript knowledge! 🚀

101.pipes in angular and it's types?

1. What are Pipes?

Pipes are simple functions that transform displayed values in Angular templates without changing the original data. They're used with the `|` pipe syntax:

```
{{ value | pipeName:param }}
```

2. Types of Pipes

A. Built-in Pipes (Commonly Used)

Pipe	Example	Output
DatePipe	{{ today date:'short' }}	"6/15/23, 9:30 AM"
UpperCasePipe	{{ 'text' uppercase }}	"TEXT"
LowerCasePipe	{{ 'TEXT' lowercase }}	"text"
CurrencyPipe	{{ 25 currency:'USD' }}	"\$25.00"
DecimalPipe	{{ 3.14159 number:'1.2-2' }}	"3.14"

PercentPipe	{{ 0.75 percent }}	"75%"
JsonPipe	{{ object json }}	Stringified JSON
AsyncPipe	{{ data\$ async }}	Resolves observables/promises

B. Parameterized Pipes

Many pipes accept parameters:

```
{{ today | date:'fullDate' }} <!-- "Monday, June 15, 2023" -->
{{ price | currency:'EUR':'symbol':'1.2-2' }} <!-- "€25.00" -->
```

C. Chaining Pipes

Pipes can be combined:

```
{{ message | uppercase | slice:0:10 }}
```

3. Custom Pipes

Create reusable transformation logic:

Basic Custom Pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit: number = 50): string {
    return value.length > limit ? value.substring(0, limit) + '...' : value;
  }
}
```

Usage:

```
{{ longText | truncate:20 }}
```

Pure vs Impure Pipes

Type	Behavior	When to Use
Pure (Default)	Runs only when input reference changes	Most cases (better performance)
Impure (pure: false)	Runs on every change detection	When tracking mutable object changes

```
@Pipe({ name: 'filter', pure: false })
export class FilterPipe { /*...*/ }
```

4. Key Concepts

Pipe Lifecycle

- Pure pipes: Recalculate only on primitive changes or object reference changes
- Impure pipes: Recalculate on every change detection cycle

Performance Considerations

- ✔ Prefer pure pipes (better performance)
- ✔ Avoid heavy computations in pipes
- ✔ Use memoization for expensive operations in impure pipes

5. Practical Examples

A. File Size Formatter

```
@Pipe({ name: 'fileSize' })
```

```
export class FileSizePipe implements PipeTransform {
  transform(bytes: number): string {
    if (bytes === 0) return '0 Bytes';
    const k = 1024;
    const sizes = ['Bytes', 'KB', 'MB', 'GB'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
  }
}
```

Usage: `{{ file.size | fileSize }}` → "2.5 MB"

B. Search Filter Pipe (Impure)

```
@Pipe({ name: 'filter', pure: false })
export class FilterPipe implements PipeTransform {
  transform(items: any[], searchText: string): any[] {
    if (!items) return [];
    if (!searchText) return items;

    return items.filter(item =>
      item.name.toLowerCase().includes(searchText.toLowerCase())
    );
  }
}
```

6. Interview Questions & Answers

Q1: Why use pipes instead of component methods?

Strong Answer:

"Pipes keep template logic declarative and reusable across components. They're optimized for Angular's change detection, whereas methods in templates can trigger

unnecessary recalculations. Pipes also promote better separation of concerns."

Q2: When would you create an impure pipe?

Pro Response:

"Only when needing to detect changes within mutable objects/arrays. For example, a real-time sorting pipe for a frequently updated list. However, I'd first consider alternatives like reactive approaches with observables to avoid performance hits from impure pipes."

Q3: How does AsyncPipe prevent memory leaks?

Expert Insight:

"AsyncPipe automatically subscribes and unsubscribes from observables/promises, eliminating manual cleanup. This is safer than manual subscriptions that might leak memory if not properly unsubscribed in ngOnDestroy."

Best Practices

- ✓ Use built-in pipes for common transformations
- ✓ Create custom pipes for domain-specific formatting
- ✓ Document pipe parameters clearly
- ✗ Avoid complex business logic in pipes

Final Tip:

"In my current project, we created a library of 15+ shared pipes for consistent data presentation across 60+ components, reducing formatting code duplication by ~70%."

This comprehensive understanding demonstrates professional Angular template expertise! 🚀

102.What is the purpose of the async pipe?

1. Core Purpose

The Async Pipe automatically manages subscriptions to Observables and Promises in Angular templates, serving three key functions:

1. Automatic Subscription Handling
 - Subscribes when the component loads
 - Unsubscribes when the component is destroyed
 - Prevents memory leaks from orphaned subscriptions
2. Change Detection Integration
 - Triggers change detection when new values arrive
 - Works seamlessly with Angular's zone.js
3. Template Simplification
 - Eliminates manual subscription management in components
 - Reduces boilerplate code

2. Basic Usage

With Observables

```
// Component
import { AsyncPipe } from '@angular/common';

data$ = this.http.get('api/data');

// Template
<ul>
  <li *ngFor="let item of data$ | async">{{ item.name }}</li>
</ul>
```

With Promises

```
// Component
dataPromise = fetch('api/data').then(res => res.json());

// Template
<p>{{ dataPromise | async }}</p>
```

3. Key Benefits

Benefit	Without Async Pipe	With Async Pipe
Subscription Management	Manual unsubscribe in ngOnDestroy	Automatic
Code Complexity	Requires subscribe/unsubscribe logic	Single pipe
Memory Safety	Risk of leaks if unsubscribed improperly	Guaranteed cleanup
Change Detection	Manual markForCheck() often needed	Automatic

4. Advanced Patterns

Combining with Other Pipes

```
{{ (userData$ | async)?.name | uppercase }}
```

Handling Loading States

```
<ng-container *ngIf="user$ | async as user; else loading">
  <user-profile [data]="user"></user-profile>
</ng-container>

<ng-template #loading>
```

```
<spinner></spinner>
</ng-template>
```

Using with ngFor

```
<app-news-item
  *ngFor="let newItem of newsFeed$ | async"
  [item]="newItem">
</app-news-item>
```

5. Performance Considerations

✓ Efficient Change Detection

- Only triggers updates when the Observable emits
- More performant than manual subscriptions in many cases

⚠ Potential Gotchas

- Creates new subscriptions on template re-renders (use `shareReplay` if needed)
- Not suitable for hot observables that never complete

6. Real-World Example

```
// Component
import { AsyncPipe } from '@angular/common';

@Component({
  standalone: true,
  imports: [AsyncPipe],
  template: `
    <h2>Live Stock Prices</h2>
    <div *ngFor="let stock of stocks$ | async">
      {{ stock.symbol }}: {{ stock.price | currency }}
```



```

    </div>
  },
  constructor(private stockService: StockService) {}
}

export class StockComponent {
  stocks$ = this.stockService.getLivePrices().pipe(
    // shareReplay prevents duplicate subscriptions
    shareReplay(1)
  );

  constructor(private stockService: StockService) {}
}

```

7. When Not to Use Async Pipe

✗ Complex Observable Chains

- When you need access to intermediate values
- Use `.subscribe()` with manual management instead

✗ Side Effects Required

- The pipe is for display only
- For side effects, use `tap` in the Observable chain

Interview Questions & Answers

Q1: How does AsyncPipe prevent memory leaks?

Strong Answer:

"It implements Angular's `OnDestroy` interface to automatically unsubscribe when the component is destroyed. This guarantees no orphaned subscriptions remain, unlike manual subscriptions which require explicit unsubscribe calls in `ngOnDestroy`."

Q2: Can you use multiple AsyncPipes on the same Observable?

Pro Response:

*"Yes, but each creates an independent subscription. For performance, I'd either:

1. Use `shareReplay(1)` to multicast the Observable
2. Assign the unwrapped value to a component property using `*ngIf="obs$ | async as value"`
3. Cache the result in a service for complex cases"

Q3: Why is AsyncPipe better than manual subscriptions?

Expert Insight:

*"Three key reasons:

1. Safety - Eliminates memory leak risks
 2. Maintainability - Reduces boilerplate code
 3. Performance - Integrates optimally with change detection
- In my last project, replacing manual subscriptions with AsyncPipe reduced memory-related bugs by 40%."

Best Practices

- ✓ Prefer AsyncPipe for template bindings
- ✓ Combine with `*ngIf as` for clean null checking
- ✓ Use `shareReplay` for multiple template subscriptions
- ✗ Avoid for side effects - Use `tap` in the Observable chain instead

Final Tip:

"For maximum efficiency, I create a `loading$` Observable that combines with my data Observable to handle loading states elegantly in templates without additional flags."

This deep understanding demonstrates professional Angular reactive programming skills! 🚀

103.directives and it's types?

1. What are Directives?

Directives are classes that add behavior to DOM elements in Angular. They're the building blocks Angular uses to manipulate the DOM, handle events, and create dynamic templates.

2. Three Types of Directives

A. Components (Special Directives)

- Purpose: Create reusable UI elements with templates
- Key Features:
 - Have their own view (template)
 - Use `@Component` decorator
 - Support input/output bindings

```
@Component({
  selector: 'app-user-card',
  template: `<div>{{user.name}}</div>`
})
export class UserCardComponent {
  @Input() user!: User;
}
```

B. Attribute Directives

- Purpose: Change appearance/behavior of elements
- Key Features:
 - Applied as HTML attributes
 - No template (operate on host element)
 - Use `@Directive` decorator

```
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
```

```

@HostListener('mouseenter') onMouseEnter() {
  this.el.nativeElement.style.backgroundColor = 'yellow';
}
}

```

Usage:

```
<p appHighlight>Hover to highlight</p>
```

C. Structural Directives

- Purpose: Change DOM layout by adding/removing elements
- Key Features:
 - Prefix with `*` (syntactic sugar)
 - Modify view structure
 - Common examples: `*ngIf`, `*ngFor`

```

@Directive({
  selector: '[appUnless]'
})
export class UnlessDirective {
  @Input() set appUnless(condition: boolean) {
    if (!condition) {
      this.viewContainer.createEmbeddedView(this.templateRef);
    } else {
      this.viewContainer.clear();
    }
  }
}

```

Usage:

```
<div *appUnless="isHidden">Content</div>
```

3. Key Differences

Feature	Component	Attribute Directive	Structural Directive
---------	-----------	---------------------	----------------------

Feature	Component	Attribute Directive	Structural Directive
---------	-----------	---------------------	----------------------

Template	Yes	No	No (but controls layout)
DOM Manipulation	Own DOM only	Host element	Adds/removes nodes
Selector	Element (<code>app-example</code>)	Attribute (<code>[dir]</code>)	Attribute (<code>*dir</code>)
Primary Use	UI building blocks	Behavior/appearance	Conditional rendering

4. Built-in Directives

Common Attribute Directives

- `ngClass` - Dynamic CSS classes
- `ngStyle` - Dynamic inline styles
- `ngModel` - Two-way data binding

Common Structural Directives

- `*ngIf` - Conditional rendering
- `*ngFor` - List rendering
- `*ngSwitch` - Switch-case rendering

5. Creating Custom Directives

Attribute Directive Example

```

@Directive({
  selector: '[appTooltip]',
  standalone: true
})
export class TooltipDirective {
  @Input() appTooltip = '';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') showTooltip() {
    const tooltip = this.renderer.createElement('div');
    this.renderer.addClass(tooltip, 'tooltip');
    this.renderer.appendChild(document.body, tooltip);
  }
}

```

Structural Directive Example

```

@Directive({
  selector: '[appRepeat]',
  standalone: true
})
export class RepeatDirective {
  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) {}

  @Input() set appRepeat(times: number) {
    this.viewContainer.clear();
    for (let i = 0; i < times; i++) {
      this.viewContainer.createEmbeddedView(this.templateRef);
    }
  }
}

```

Usage:

```
<div *appRepeat="5">Repeated content</div>
```

6. When to Use Each Type

✓ Use Components for:

- Reusable UI pieces
- Complex self-contained features
- When you need a template

✓ Use Attribute Directives for:

- Shared behavior across components
- DOM manipulation
- Visual transformations

✓ Use Structural Directives for:

- Conditional rendering
- Dynamic list generation
- Template reuse patterns

7. Interview Questions & Answers

Q1: What's the difference between a component and a directive?

Strong Answer:

"All components are directives, but not all directives are components. The key difference is that components have their own templates and views, while other directive types either modify existing elements (attribute) or control DOM structure (structural)."

Q2: Why use Renderer2 instead of direct DOM access?

Pro Response:

"Renderer2 provides an abstraction layer that works safely across platforms (web workers, server-side rendering). Direct DOM access via nativeElement breaks in

non-browser environments and bypasses Angular's security sanitization."*

**Q3: How does ngIf actually work?*

Expert Insight:

"The asterisk () is syntactic sugar that Angular converts into an `<ng-template>` wrapper.

Under the hood, `*ngIf="condition"` becomes:*

```
<ng-template [ngIf]="condition">
  <!-- content here -->
</ng-template>
```

Angular then uses `viewContainer` methods to add/remove this template based on the condition."

Best Practices

- ✅ Prefer components for most UI needs
- ✅ Create reusable attribute directives for cross-cutting behaviors
- ✅ Limit structural directives to templating needs
- ❌ Avoid excessive DOM manipulation in directives

Final Tip:

"In my current project, we created a library of 20+ shared directives for common behaviors like tooltips, permissions, and analytics tracking - reducing duplicate code by 60% across our 150+ components."*

This comprehensive understanding demonstrates professional Angular architecture skills! 🚀

104.What are template expressions?

Template expressions are JavaScript-like code snippets used within Angular templates (HTML) to dynamically evaluate and display values. They're the primary way to incorporate dynamic content and logic into your views.

1. Basic Syntax

Template expressions appear within double curly braces (`{{ }}`) or in property bindings (`[property]="expression"`):

```
<p>{{ 5 + 5 }}</p>  <!-- Output: 10 -->
<button [disabled]="isDisabled">Save</button>
```

2. Key Characteristics

Feature	Details
Execution Context	Run in the context of the component class (use component properties/methods)
Security	Sanitized to prevent XSS attacks
Idempotent	Should not change application state (no side effects)
Limited Syntax	Supports a subset of JavaScript (no <code>new</code> , <code>+=</code> , etc.)

3. Allowed Operations

- ✓ Basic arithmetic: `{{ count + 1 }}`
- ✓ Property access: `{{ user.name }}`
- ✓ Method calls: `{{ getFullName() }}`
- ✓ Ternary operator: `{{ isActive ? 'Yes' : 'No' }}`
- ✓ Pipe transformations: `{{ date | date:'short' }}`

4. Restrictions

- ✗ No assignments: `{{ count = 5 }}` (invalid)
- ✗ No `new` keyword: `{{ new Date() }}` (invalid)
- ✗ No chaining with `;` or `:`: `{{ doThis(); doThat() }}` (invalid)
- ✗ No global namespace access: `{{ window.location }}` (invalid)

5. Common Use Cases

A. Displaying Component Data

```
<p>Welcome, {{ userName }}!</p>
```

B. Conditional Rendering

```
<div *ngIf="items.length > 0">Showing {{ items.length }} items</div>
```

C. Property Binding

```
<img [src]="imagePath" [alt]="'Image of ' + itemName">
```

D. Event Binding

```
<button (click)="saveItem(item.id)">Save</button>
```

6. Best Practices

1. Keep expressions simple
 - Move complex logic to component methods
 - Bad: `{{ calculateDiscount(price, tax) / items.length \ currency }}`
 - Good: `{{ discountedPrice \ currency }}` (pre-calculated in component)
2. Avoid side effects

```
// ❌ Avoid
{{ updateCounter() }} // Method modifies state
```

```
// ✅ Better

{{ counter }} // Pure property display
```

3. Use pipes for transformations

```
<!-- Hard to read -->
{{ (user.firstName + ' ' + user.lastName).toUpperCase() }}
```

```
<!-- Better -->

{{ user.firstName }} {{ user.lastName | uppercase }}
```

4. Consider performance
 - Expressions execute during every change detection cycle
 - Avoid expensive operations in templates

7. Template Expression vs. Template Statement

Feature	Template Expression (<code>{{ }}</code> or <code>[prop]</code>)	Template Statement (<code>((event))</code>)
Purpose	Produces a value	Executes an action
Context	Component properties	Component methods

Example	<code>{{ count + 1 }}</code>	<code>(click)="incrementCount() "</code>
Side Effects	Not allowed	Expected

Interview Questions & Answers

Q1: Why can't we use assignments in template expressions?

Strong Answer:

"Angular enforces unidirectional data flow—templates should display data, not modify it. Assignments could create unpredictable side effects during change detection cycles. Instead, we use event handlers (template statements) for state changes."

Q2: How would you handle complex logic in templates?

Pro Response:

*"I'd:

1. Move calculations to component methods
2. Use pure pipes for transformations
3. Pre-compute values in the component

For example, instead of `{{ (a + b) * c \ | currency }}`, I'd create a `totalCost` property updated whenever `a`, `b`, or `c` changes."

Q3: What's the security impact of template expressions?

Expert Insight:

"Angular automatically sanitizes expression outputs to prevent XSS. For example, `{{ userInput }}` escapes HTML tags, while `[innerHTML]="trustedHtml"` requires explicit sanitization using `DomSanitizer` for dangerous content."

Real-World Example

```
@Component({
  template: `
    <h2>{{ title }}</h2>
    <p>Status: {{ isActive ? 'Online' : 'Offline' }}</p>
    <button
      [disabled]="!isValid"
      (click)="submitForm()">
      {{ submitButtonText }}
    </button>
  `
})
export class UserComponent {
  title = 'User Profile';
  isActive = true;
  isValid = false;
  submitButtonText = 'Submit';

  submitForm() { /* ... */ }
}
```

Final Tip:

"In my projects, I enforce a rule that no template expression should contain more than one method call or ternary operation. This keeps templates readable and maintainable."

This deep understanding demonstrates professional Angular templating expertise! 🚀

105.How do you categorize data binding types?

Angular provides a powerful data binding system that connects your application data with the DOM. Here's how to categorize these binding types:

1. By Direction of Data Flow

Binding Type	Syntax	Data Flow	Best For
Interpolation	<code>{{expression}}</code>	Component → View	Displaying dynamic text
Property Binding	<code>[property]="expr"</code>	Component → View	Setting element properties
Event Binding	<code>(event)="handler"</code>	View → Component	Responding to user actions
Two-Way Binding	<code>[(ngModel)]="prop"</code>	Component ↔ View	Forms and interactive inputs

2. By Binding Target

A. DOM Property Bindings

```
<img [src]="imageUrl" > <!-- Property binding -->
<button [disabled]="isLoading" > <!-- Property binding -->
```

B. HTML Attribute Bindings

```
<td [attr.colspan]="colSpan" > <!-- Attribute binding -->
```

C. Class & Style Bindings

```
<div [class.active]="isActive" > <!-- Class binding -->
<p [style.color]="textColor" > <!-- Style binding -->
```

D. Template Reference Variables

```
<input #emailInput > <!-- Local template variable -->
```

```
<button (click)="log(emailInput.value)">Log Value</button>
```

3. Special Binding Cases

Binding Type	Example	Use Case
Structural Directives	<code>*ngIf="condition"</code>	Conditional DOM manipulation
Template Input Vars	<code>*ngFor="let item of items"</code>	List iteration
Safe Navigation	<code>{{user?.name}}</code>	Null-checking

4. Two-Way Binding Deep Dive

Combines property + event binding:

```
<!-- Manual two-way binding -->  
<input [value]="username" (input)="username = $event.target.value">
```

```
<!-- Angular shorthand -->  
<input [(ngModel)]="username">
```

Behind the Scenes:

`[(ngModel)]` is syntactic sugar for:

```
<input [ngModel]="username" (ngModelChange)="username = $event">
```

5. Performance Considerations

✓ Use one-way binding (`[]` or `{{}}`) when possible (better performance)

✓ Debounce events for expensive operations (`(input)` handlers)

✗ Avoid complex expressions in templates (move to component)

Interview Questions & Answers

Q1: Why avoid two-way binding in non-form contexts?

Strong Answer:

"Two-way binding creates implicit coupling between components and templates, making data flow harder to track. For non-form scenarios, I prefer explicit event emitters (`@Output()`) combined with one-way bindings for better maintainability and predictable state changes."

Q2: How would you optimize a template with many bindings?

Pro Response:

*"I'd:

1. Use `OnPush` change detection
2. Replace complex expressions with pure pipes
3. Cache values in component properties
4. For lists, implement `trackBy` with `*ngFor`

In my last project, these changes reduced change detection cycles by 60%."*

Q3: When would you use attribute vs property binding?

Expert Insight:

"Use attribute binding (`attr.`) for HTML attributes that don't have corresponding DOM properties (like `aria-*` or `colspan`). For all other cases, property binding (`[]`) is preferred because it reflects the live DOM state. For example, `[value]` updates instantly while `attr.value` doesn't."

Real-World Binding Strategy

```
@Component({
```



```

template: `
  <!-- One-way bindings (Component → View) -->
  <h1>{{ title }}</h1>
  <app-user-card [user]="currentUser"></app-user-card>

  <!-- Event binding (View → Component) -->
  <button (click)="onSave()">Save</button>

  <!-- Two-way binding (Forms) -->
  <input [(ngModel)]="searchQuery" placeholder="Search...">

  <!-- Style binding -->
  <div [class.error]="hasError" [style.fontSize.px]="fontSize">
`
})
export class ProfileComponent {
  title = 'User Profile';
  currentUser: User;
  searchQuery = '';
  hasError = false;
  fontSize = 16;

  onSave() { /* ... */ }
}

```

Final Tip:

"In complex forms, I often replace `[(ngModel)]` with reactive form controls for better validation and state management while maintaining two-way binding-like simplicity through `formControlName`."

This systematic categorization demonstrates professional Angular data flow mastery!



106.What is the difference between pure and impure pipe?

1. Fundamental Distinction

Characteristic	Pure Pipe	Impure Pipe
Trigger Condition	Runs only when input <i>reference</i> changes	Runs on every change detection cycle
Performance	Optimized (Angular calls less often)	Potentially expensive
Use Cases	Data transformations	Real-time data monitoring
Default	Yes (<code>pure: true</code>)	Must declare (<code>pure: false</code>)

2. How They Work

Pure Pipe Example

```
@Pipe({ name: 'double', pure: true }) // 'pure' is true by default
export class DoublePipe implements PipeTransform {
  transform(value: number): number {
    console.log('Pure pipe executed');
    return value * 2;
  }
}
```

Behavior:

- Only recalculates when the input `value` changes

- Efficient for static or rarely changed data

Impure Pipe Example

```
@Pipe({ name: 'liveData', pure: false })
export class LiveDataPipe implements PipeTransform {
  transform(value: number[]): number {
    console.log('Impure pipe executed');
    return value.reduce((a, b) => a + b, 0);
  }
}
```

Behavior:

- Recalculates on every change detection cycle
- Necessary for mutable data (e.g., arrays/objects that change without reference change)

3. Key Scenarios

When to Use Pure Pipes (Default)

- ✓ Numeric/string transformations
- ✓ Date formatting
- ✓ Currency conversions
- ✓ Any operation where inputs are immutable

When to Use Impure Pipes

- ✓ Real-time data streams
- ✓ Working with mutable arrays/objects
- ✓ Complex calculations needing frequent updates
- ⚠ Use sparingly due to performance impact

4. Performance Demonstration

```
// Component
data = [1, 2, 3];

addItem() {
  this.data.push(this.data.length + 1); // Mutates array without reference change
}

<!-- Template -->
<button (click)="addItem()">Add Item</button>
<p>Pure Sum: {{ data | pureSum }}</p>    <!-- Won't update -->
<p>Impure Sum: {{ data | impureSum }}</p> <!-- Updates -->
```

5. Under the Hood

Pure Pipe Optimization

Angular uses referential identity check:

```
if (oldInput !== newInput) {
  recalculate();
}
```

Impure Pipe Behavior

Runs like a method call during every change detection:

```
recalculate(); // No input comparison
```

6. Best Practices

- ✓ Default to pure pipes (better performance)
- ✓ For arrays/objects, consider:
 - Immutable updates ([...arr, newItem])
 - Manual change detection triggers
 - ✓ Debounce impure pipes for frequent updates

❌ Avoid impure pipes for large datasets

Interview Questions & Answers

Q1: Why is pure the default?

Strong Answer:

"Pure pipes optimize performance by minimizing recalculations. Angular skips pure pipes during change detection unless their inputs change reference, reducing unnecessary work. This aligns with Angular's immutable data philosophy."

Q2: How would you optimize an impure pipe?

Pro Response:

*"Three approaches:

1. Add internal caching/memoization
2. Use `OnPush` change detection strategy
3. Replace with a component property + getter

In my last project, caching reduced an impure pipe's executions by 85%."*

Q3: Can you give an example where impure is necessary?

Expert Insight:

"A real-time stock ticker displaying frequently updated prices from a WebSocket. Since prices update without array reference changes, an impure pipe ensures the UI stays current. However, I'd debounce updates to 500ms for performance:"

```
@Pipe({ name: 'liveTicker', pure: false })
export class LiveTickerPipe {
  transform(prices: number[]) {
    return prices.map(p => p.toFixed(2)).join(' | ');
  }
}
```

Real-World Example

```
// Pure pipe for currency conversion (rates change rarely)
@Pipe({ name: 'currencyConvert' })
export class CurrencyConvertPipe {
  transform(amount: number, rate: number): number {
    return amount * rate; // Only recalculates if amount/rate reference changes
  }
}

// Impure pipe for live scoreboard (updates constantly)
@Pipe({ name: 'liveScore', pure: false })
export class LiveScorePipe {
  transform(players: Player[]): number {
    return players.reduce((total, p) => total + p.score, 0);
  }
}
```

Final Tip:

"In my projects, I audit impure pipes during code reviews. We've reduced them from 12 to just 3 critical cases, improving app performance by ~30%."

This deep understanding demonstrates professional Angular optimization skills! 🚀

107.What are lifecycle hooks in Angular?

Lifecycle hooks are special methods that Angular calls at specific moments during a component's existence, giving developers fine-grained control over component behavior. Here's a comprehensive breakdown:

1. The Component Lifecycle Timeline

Creation Phase (Initialization)

1. `constructor` (*TypeScript feature, not Angular-specific*)
2. `ngOnChanges` (First run)
3. `ngOnInit`
4. `ngDoCheck`
5. `ngAfterContentInit`
6. `ngAfterContentChecked`
7. `ngAfterViewInit`
8. `ngAfterViewChecked`

Update Phase (Change Detection)

- `ngOnChanges` (Subsequent runs)
- `ngDoCheck`
- `ngAfterContentChecked`
- `ngAfterViewChecked`

Destruction Phase

9. `ngOnDestroy`

2. Key Hooks Explained

Hook	Trigger Condition	Typical Use Cases
<code>ngOnChanges</code>	When input properties change (first run + subsequent changes)	React to external input changes
<code>ngOnInit</code>	After first <code>ngOnChanges</code> (inputs initialized)	Initial setup, API calls

ngDoCheck	During every change detection cycle	Custom change detection logic
ngAfterContentInit	After Angular projects external content into component (once)	Access projected content (<code><ng-content></code>)
ngAfterContentChecked	After checking projected content (every CD cycle)	Post-content-check actions
ngAfterViewInit	After component's view and child views are initialized (once)	DOM access, view-dependent initialization
ngAfterViewChecked	After checking component's view and child views (every CD cycle)	Rarely used (risk of infinite loops)
ngOnDestroy	Just before Angular destroys the component	Cleanup (unsubscribe, clear timers, disconnect events)

3. Practical Examples

Initialization Sequence

```

@Component({...})
export class UserComponent implements
OnInit, OnChanges {
  @Input() userId!: string;

  constructor() {

```



```

        console.log('Constructor runs first');
// Avoid complex logic here
    }

    ngOnChanges(changes: SimpleChanges) {
        console.log('Input changes:',
changes.userId);
    }

    ngOnInit() {
        console.log('Initialization complete');
        this.loadUser(this.userId); // API call
    }
}

```

View and Content Hooks

```

ngAfterContentInit() {
    console.log('Content projected:',
this.contentChild);
}

ngAfterViewInit() {
    console.log('View ready:',
this.viewChild.nativeElement);
    // Safe to manipulate DOM here
}

```

Cleanup Example

```

private intervalId: any;

ngOnInit() {
    this.intervalId = setInterval(() => {
        this.checkStatus();
    }, 1000);
}

ngOnDestroy() {
    clearInterval(this.intervalId); //

```

Prevent memory leaks

```
console.log('Component destroyed');  
}
```

4. Performance Considerations

- ✓ Use `OnPush` change detection to reduce hook executions
- ✓ Move heavy operations from `ngDoCheck` to `ngOnChanges` when possible
- ✗ Avoid side effects in `AfterViewChecked` (risk of infinite loops)

5. Interview Questions & Answers

Q1: Why is constructor not a true Angular lifecycle hook?

Strong Answer:

"The constructor is a TypeScript/JavaScript class feature that runs during class instantiation, before Angular even processes the component. True lifecycle hooks (like `ngOnInit`) are Angular-specific and run after the framework has initialized inputs and set up dependency injection."

Q2: When would you choose `ngOnChanges` over `ngOnInit`?

Pro Response:

"I use `ngOnChanges` when I need to react to input changes throughout the component's lifetime (like a dynamic dashboard widget). For one-time initialization (API calls, default setup), `ngOnInit` is better since it runs only once after inputs are set."

Q3: How would you optimize a component with expensive `ngDoCheck` logic?

Expert Insight:

*"Three strategies:

1. Debounce operations using RxJS operators
2. Compare specific properties manually rather than running full checks
3. Move logic to ngOnChanges where possible

In my current project, this reduced change detection time by 75% for complex data grids."*

Best Practices

- Minimize logic in constructors (DI only)
- Always implement OnDestroy for cleanup
- Use ngAfterViewInit for DOM operations (elements exist)
- Avoid mutations in getters (triggers extra checks)

Final Tip:

"I visualize component lifecycle as a timeline during debugging. Understanding exactly when each hook fires helps solve tricky timing issues, especially with @ViewChild references and dynamic content."

This comprehensive understanding demonstrates professional Angular architecture skills! 🚀

108.What do you mean by string interpolation?

String interpolation is a one-way data binding technique in Angular that allows you to embed dynamic expressions directly into HTML templates. It's the simplest way to display component data in the view.

1. Basic Syntax

Use double curly braces `{{ }}` to insert expressions:

```
<p>Welcome, {{ username }}!</p>
<h2>2 + 2 = {{ 2 + 2 }}</h2>
<div>Current date: {{ getDate() }}</div>
```

2. Key Features

Feature	Description
Dynamic Values	Displays live component data
Expression Support	Evaluates JavaScript-like expressions
One-Way Binding	Flows data from component → view (not vice versa)
Automatic Updates	Reflects changes when the component data changes

3. What You Can Include

✔ Component Properties

```
{{ title }}
```

✔ Methods

```
{{ getUsername() }}
```

✓ Mathematical Operations

```
{{ quantity * price }}
```

✓ Ternary Operator

```
{{ isLoggedIn ? 'Welcome back' : 'Please login' }}
```

✓ Pipes (Transformations)

```
{{ birthDate | date:'longDate' }}
```

4. What You *Cannot* Include

✗ Assignments (=, +=)

```
{{ count = 5 }} <!-- Invalid -->
```

✗ new Keywordload

```
{{ new Date() }} <!-- Invalid -->
```

✗ Chained Statements

```
{{ doThis(); doThat() }} <!-- Invalid -->
```

✗ Global Namespace Objects

```
{{ window.location.href }} <!-- Invalid -->
```

5. How It Works Under the Hood

Angular converts interpolation into property binding:

```
<!-- Interpolation -->
```

```
<p>{{ title }}</p>
```

```
<!-- Becomes this behind the scenes -->
```

```
<p [textContent]="title"></p>
```

6. Best Practices

✓ Keep expressions simple (move complex logic to component methods)

✓ Use pipes for formatting instead of inline logic

✓ Prefer interpolation over `[textContent]` for readability

✗ Avoid side effects in interpolated expressions

7. Real-World Examples

Displaying User Data

```
<div class="profile">
  <h1>{{ user.name }}</h1>
  <p>Joined: {{ user.joinDate | date }}</p>
  <p>Status: {{ user.isActive ? 'Active' : 'Inactive' }}</p>
</div>
```

Conditional CSS Class

```
<button class="btn {{ isDisabled ? 'disabled' : 'active' }}">
```

```
{{ buttonText }}  
</button>
```

Dynamic Calculations

```
<p>Total: {{ (quantity * unitPrice) | currency }}</p>
```

Interview Questions & Answers

Q1: How does interpolation differ from property binding?

Strong Answer:

"While both flow data from component to view, interpolation is syntactic sugar for property binding to `textContent`. Interpolation is cleaner for inserting strings, while property binding (`[]`) is required for non-string attributes like `[disabled]` or `[src]`."

Q2: Why avoid complex expressions in interpolation?

Pro Response:

*"Complex expressions:

1. Reduce template readability
2. Execute during every change detection cycle
3. Are harder to test

Instead, I move such logic to component methods or computed properties."*

Q3: Can interpolation trigger XSS attacks?

Expert Insight:

"Angular automatically sanitizes interpolated values to prevent XSS. For example,

`<script>` tags in `{{ userInput }}` are escaped. To render HTML safely, you must explicitly use `[innerHTML]="trustedHtml"` with `DomSanitizer`.

Performance Tip

"For frequently updated values in loops, consider caching interpolated values in component properties to reduce change detection overhead."

Final Example:

```
// Component
get formattedTotal(): string {
  return `Total: ${this.quantity * this.price.toFixed(2)}`;
}

<!-- Template -->
<p>{{ formattedTotal }}</p> <!-- More efficient than inline math -->
```

This deep understanding demonstrates professional Angular templating expertise! 🚀

109.What is rxjs?

RxJS (Reactive Extensions for JavaScript) is a library for reactive programming that provides an elegant way to handle asynchronous operations and event-based programs using observable sequences.

1. Core Concepts

Concept	Description
Observable	A stream of data or events that can be observed over time
Observer	An object with methods to handle Observable notifications (<code>next</code> , <code>error</code> , <code>complete</code>)
Subscription	Represents the execution of an Observable (used to cancel it)
Operators	Pure functions to transform, filter, or combine Observables (<code>map</code> , <code>filter</code> , <code>merge</code>)
Subject	A special type of Observable that can multicast to multiple Observers

2. Why Use RxJS in Angular?

- Handles asynchronous operations (HTTP requests, user events, WebSockets)
- Powerful data transformation with 100+ operators
- Cleaner code compared to nested callbacks/Promises
- Cancellation support (unlike Promises)
- Time-based operations (debounce, throttle, etc.)

3. Key Components

A. Creating Observables

```
import { Observable } from 'rxjs';
```

```

// From a value
const obs1$ = of(1, 2, 3);

// From an event
const obs2$ = fromEvent(document, 'click');

// From a Promise
const obs3$ = from(fetch('api/data'));

// Custom Observable
const obs4$ = new Observable(observer => {
  observer.next(1);
  setTimeout(() => observer.complete(), 1000);
});

```

B. Subscribing to Observables

```

const subscription = obs1$.subscribe({
  next: value => console.log(value),
  error: err => console.error(err),
  complete: () => console.log('Done')
});

```

```

// Unsubscribe to prevent memory leaks
subscription.unsubscribe();

```

C. Common Operators

Operator	Example	Purpose
map	<code>obs\$.pipe(map(x => x * 2))</code>	Transform values
filter	<code>obs\$.pipe(filter(x => x > 5))</code>	Filter values
debounceTime	<code>obs\$.pipe(debounceTime(300))</code>	Wait for pause in events

switchMap	<code>obs\$.pipe(switchMap(id => fetchData(id)))</code>	Cancel previous inner Observable
catchError	<code>obs\$.pipe(catchError(handleError))</code>	Handle errors

4. Practical Angular Examples

A. HTTP Request with Error Handling

```
import { catchError } from 'rxjs/operators';

this.http.get('api/users').pipe(
  catchError(error => {
    console.error('Error:', error);
    return of([]); // Fallback value
  })
).subscribe(users => {
  this.users = users;
});
```

B. Typeahead Search with Debounce

```
import { debounceTime, distinctUntilChanged, switchMap } from 'rxjs/operators';

searchTerm$ = new Subject<string>();

ngOnInit() {
  this.searchTerm$.pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(term => this.searchService.search(term))
  ).subscribe(results => {
    this.results = results;
  });
}
```

C. Combining Multiple Streams

```
import { combineLatest } from 'rxjs';

combineLatest([
  this.userService.getUser(),
  this.orderService.getOrders()
]).subscribe([user, orders] => {
  this.profile = { user, orders };
});
```

5. RxJS vs Promises

Feature	RxJS	Promises
Multiple Values	Yes (stream)	No (single value)
Cancellation	Supported (<code>unsubscribe</code>)	Not supported
Operators	100+ transformation options	Only <code>.then/.catch</code>
Lazy Execution	Only on subscribe	Immediately executes

6. Best Practices

- ✓ Always unsubscribe to prevent memory leaks
- ✓ Use operators for complex data flows
- ✓ Prefer declarative patterns over imperative code

✗ Avoid nested subscriptions (use `switchMap`/`mergeMap`)

Interview Questions & Answers

Q1: Why choose RxJS over Promises?

Strong Answer:

"RxJS handles multiple values over time, supports cancellation, and provides powerful operators for complex async workflows—unlike Promises which resolve once. For example, RxJS can debounce user input, cancel outdated HTTP requests, and combine multiple streams effortlessly."

Q2: How does `switchMap` differ from `mergeMap`?

Pro Response:

"`switchMap` cancels previous inner Observables when a new value arrives (ideal for search), while `mergeMap` processes all values concurrently (good for parallel requests). For autocomplete, I always use `switchMap` to avoid outdated results."

Q3: What's the purpose of Subjects?

Expert Insight:

"Subjects act as both Observable and Observer, enabling multicast to multiple subscribers. They're perfect for event buses or state management. For example, I use `BehaviorSubject` to share user auth state across components."

Performance Tip

"For high-frequency events (like mouse moves), use `auditTime` or `sampleTime` instead of `debounceTime` to reduce processing overhead."

Final Example:

```
// Efficient real-time data processing
fromEvent(document, 'mousemove').pipe(
  sampleTime(100), // Emit every 100ms
  map(event => ({ x: event.clientX, y: event.clientY })))
).subscribe(position => {
  this.cursorPosition = position;
});
```

This deep understanding demonstrates professional RxJS mastery in Angular! 🚀

110.Subject and it's types?

Subjects are special types of Observables that act as both producers and consumers of data. They enable multicast communication, allowing values to be pushed to multiple subscribers.

1. Core Subject Types

Type	Behavior	Best For
Subject	No initial value, only emits values after subscription	Event buses, generic multicasting
BehaviorSubject	Requires initial value, emits current value to new subscribers	Application state (e.g., auth status)
ReplaySubject	Replays specified number of previous values to new subscribers	Caching historical data

AsyncSubject	Emits only the last value when completed	One-time final values (e.g., API completion)
--------------	--	--

2. Detailed Breakdown

A. Regular Subject

- No initial value
- Subscribers only receive values emitted after subscription

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

// Subscriber 1 (misses first value)
subject.subscribe(v => console.log('Sub1:', v));

subject.next(1); // Sub1: 1

// Subscriber 2
subject.subscribe(v => console.log('Sub2:', v));
subject.next(2); // Sub1: 2, Sub2: 2
```

B. BehaviorSubject

- Requires initial value
- New subscribers immediately get the current/last value

```
import { BehaviorSubject } from 'rxjs';

const behaviorSubj = new BehaviorSubject<number>(0);

// Subscriber 1 (gets initial value)
behaviorSubj.subscribe(v => console.log('Sub1:', v)); // Sub1: 0
```

```
behaviorSubj.next(1); // Sub1: 1
```

```
// Subscriber 2 (gets last emitted value)
```

```
behaviorSubj.subscribe(v => console.log('Sub2:', v)); // Sub2: 1
```

C. ReplaySubject

- Replays specified number of past values
- Configurable buffer size

```
import { ReplaySubject } from 'rxjs';
```

```
const replaySubj = new ReplaySubject<number>(2); // Buffer last 2 values
```

```
replaySubj.next(1);
```

```
replaySubj.next(2);
```

```
replaySubj.next(3);
```

```
// Late subscriber gets last 2 values
```

```
replaySubj.subscribe(v => console.log('Late Sub:', v));
```

```
// Output: Late Sub: 2, Late Sub: 3
```

D. AsyncSubject

- Only emits the last value when completed
- Useful for one-time final results

```
import { AsyncSubject } from 'rxjs';
```

```
const asyncSubj = new AsyncSubject<number>();
```

```
asyncSubj.subscribe(v => console.log('Sub:', v));
```

```
asyncSubj.next(1);
```

```
asyncSubj.next(2);
```

```
asyncSubj.complete(); // Sub: 2
```


3. When to Use Each Type

Use Case	Recommended Subject	Why?
User authentication state	BehaviorSubject	Components always need current auth status
Search term debouncing	Subject	No need for initial/historical values
Chat application message history	ReplaySubject(50)	New users should see recent messages
Final API result after processing	AsyncSubject	Only care about the completed result

4. Key Features Comparison

Feature	Subject	BehaviorSubject	ReplaySubject	AsyncSubject
Initial Value	✗ No	✓ Required	✗ No	✗ No
Late Subscriber Value	✗ None	✓ Last emitted	✓ Buffered	✓ Only if completed

Completion Behavior	Emits until complete	Same as Subject	Same as Subject	Emits only last value on complete
---------------------	----------------------	-----------------	-----------------	-----------------------------------

5. Practical Angular Examples

A. Shared Service with BehaviorSubject

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private currentUser = new BehaviorSubject<User | null>(null);
  currentUser$ = this.currentUser.asObservable();

  login(user: User) {
    this.currentUser.next(user);
  }
}

// Component
this.authService.currentUser$.subscribe(user => {
  this.user = user;
});
```

B. Event Bus with Subject

```
@Injectable({ providedIn: 'root' })
export class EventBusService {
  private notificationSubject = new Subject<string>();
  notifications$ = this.notificationSubject.asObservable();

  notify(message: string) {
    this.notificationSubject.next(message);
  }
}

// Component A
this.eventBus.notify('Data loaded');
```

```
// Component B
this.eventBus.notifications$.subscribe(msg => {
  this.showToast(msg);
});
```

C. Cached API Responses with ReplaySubject

```
@Injectable({ providedIn: 'root' })
export class DataService {
  private cache = new ReplaySubject<Data[]>(1);

  loadData() {
    this.http.get('api/data').subscribe(data => {
      this.cache.next(data);
    });
    return this.cache.asObservable();
  }
}
```

6. Best Practices

- ✓ Use `asObservable()` to expose Subjects safely
- ✓ Always unsubscribe to prevent memory leaks
- ✓ Prefer BehaviorSubject for state management
- ✗ Avoid exposing Subjects directly (breaks encapsulation)

Interview Questions & Answers

Q1: Why use BehaviorSubject over regular Subject for state?

Strong Answer:

"BehaviorSubject guarantees subscribers immediately get the current value, critical for

state management. For example, auth guards need the latest user status even if they subscribe after the initial login event."

Q2: How would you implement a multi-tab sync feature?

Pro Response:

*"I'd use a ReplaySubject(1) in a shared service. New tabs (late subscribers) would receive the latest synchronized data immediately, while all tabs get real-time updates through next() calls."**

Q3: When would AsyncSubject be preferable?

Expert Insight:

"For operations where only the final result matters, like file uploads or complex calculations. Unlike BehaviorSubject, it won't emit partial progress—only the completed result, making it perfect for 'final answer' scenarios."

Final Tip:

"In my projects, I combine Subjects with RxJS operators like `distinctUntilChanged()` to optimize performance and avoid unnecessary emissions in state management."

This comprehensive understanding demonstrates professional RxJS expertise in Angular applications! 🚀

111.What is observable?

Observables are a core concept in reactive programming that provide a powerful way to handle asynchronous operations and event-based programs. Here's a comprehensive explanation:

1. What is an Observable?

An Observable is a stream of data or events that can be observed over time. It represents a push-based collection that can emit:

- Values (numbers, strings, objects)
- Errors
- Completion signals

2. Key Characteristics

Feature	Description
Lazy Evaluation	Only executes when subscribed to (unlike Promises which run immediately)
Multiple Values	Can emit many values over time (unlike Promises which resolve once)
Cancellable	Subscriptions can be terminated with <code>unsubscribe()</code>
Composable	Can be transformed with operators (<code>map</code> , <code>filter</code> , <code>merge</code> , etc.)

3. Observable Lifecycle

1. Creation: Define the data stream
2. Subscription: Start listening to emissions
3. Execution: Observable begins emitting values
4. Teardown: Unsubscribe to release resources

```
import { Observable } from 'rxjs';

// 1. Creation
const observable = new Observable(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  setTimeout(() => {
    subscriber.next(3);
    subscriber.complete();
  }, 1000);
});

// 2. Subscription
const subscription = observable.subscribe({
  next: value => console.log(value), // 1, 2, 3
  error: err => console.error(err),
  complete: () => console.log('Done')
});

// 4. Teardown (optional)
subscription.unsubscribe();
```

4. Observable vs Promise

Feature	Observable	Promise
Multiple Values	Yes (stream)	No (single value)
Cancellation	Supported (<code>unsubscribe</code>)	Not supported
Lazy	Only on subscribe	Immediately executes
Operators	100+ transformation options	Only <code>.then/</code> <code>.catch</code>

Backpressure	Supported (with lossless operators)	Not applicable
--------------	-------------------------------------	----------------

5. Common Observable Sources

A. From Event

```
import { fromEvent } from 'rxjs';

const clicks$ = fromEvent(document, 'click');
clicks$.subscribe(evt => console.log(evt));
```

B. From HTTP Requests

```
import { HttpClient } from '@angular/common/http';

this.http.get('api/users').subscribe(users => {
  this.users = users;
});
```

C. From Arrays

```
import { from } from 'rxjs';

from([1, 2, 3]).subscribe(val => console.log(val));
```

D. Timer-Based

```
import { interval } from 'rxjs';

interval(1000).subscribe(val => console.log(val)); // 0, 1, 2...
```

6. Observable Operators

Operators transform or filter streams:

Operator	Example	Purpose
map	<code>obs\$.pipe(map(x => x * 2))</code>	Transform values
filter	<code>obs\$.pipe(filter(x => x > 5))</code>	Filter values
take	<code>obs\$.pipe(take(3))</code>	Take first N values
debounceTime	<code>obs\$.pipe(debounceTime(300))</code>	Wait for pause in events
switchMap	<code>obs\$.pipe(switchMap(id => fetchData(id)))</code>	Cancel previous inner Observable

7. Practical Angular Examples

A. Typeahead Search

```
import { debounceTime, distinctUntilChanged, switchMap } from 'rxjs/operators';

search(term: string) {
  this.results$ = this.searchTerm$.pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(term => this.http.get(`api/search?q=${term}`))
  );
}
```


B. Combining Streams

```
import { combineLatest } from 'rxjs';

combineLatest([
  this.userService.getUser(),
  this.orderService.getOrders()
]).subscribe([user, orders] => {
  this.profile = { user, orders };
});
```

8. Best Practices

- ✓ Always unsubscribe to prevent memory leaks
- ✓ Use operators for complex data flows
- ✓ Prefer declarative patterns over imperative code
- ✗ Avoid nested subscriptions (use `switchMap`/`mergeMap`)

Interview Questions & Answers

Q1: Why use Observables over Promises?

Strong Answer:

"Observables handle multiple values over time, support cancellation, and offer powerful operators for complex async workflows—unlike Promises which resolve once. They're ideal for real-time data, event streams, and scenarios requiring fine-grained control."

Q2: How does lazy evaluation benefit performance?

Pro Response:

"Observables don't execute until subscribed, preventing unnecessary work. For example, an HTTP Observable won't make the request until something subscribes, unlike Promises"

which trigger immediately on creation."

Q3: What's the difference between cold and hot Observables?

Expert Insight:

"Cold Observables start producing values per subscription (like HTTP requests), while hot Observables share a single production sequence (like mouse events). Subjects are commonly used to make cold Observables hot."

Performance Tip

"For high-frequency events (mousemove, scroll), use `auditTime` or `sampleTime` to reduce processing overhead."

Final Example:

```
// Efficient real-time mouse tracking
fromEvent(document, 'mousemove').pipe(
  sampleTime(100), // Sample every 100ms
  map(evt => ({ x: evt.clientX, y: evt.clientY })))
).subscribe(pos => {
  this.cursorPosition = pos;
});
```

This deep understanding demonstrates professional RxJS mastery in Angular applications! 🚀

112. What are `forRoot` and `forChild` in routing? Difference?

1. Core Purpose

Both methods configure routing in Angular modules, but with different scopes:

Method	Purpose
forRoot()	Configures the <i>root</i> routing module with services and routes
forChild()	Configures <i>feature</i> modules with routes (without re-registering services)

2. Key Differences

Aspect	forRoot()	forChild()
Services	Registers RouterService, RouteReuseStrategy	Doesn't register services
Usage Location	AppRoutingModule (root only)	Feature module routing
Initialization	Should be called once	Can be called multiple times
Route Configuration	Sets up initial routes	Adds feature-specific routes

3. Typical Implementation

Root Routing Module (app-routing.module.ts)

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
```

```

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}

```

Feature Routing Module (feature-routing.module.ts)

```

const featureRoutes: Routes = [
  { path: 'list', component: FeatureListComponent },
  { path: 'detail/:id', component: FeatureDetailComponent }
];

@NgModule({
  imports: [RouterModule.forChild(featureRoutes)],
  exports: [RouterModule]
})
export class FeatureRoutingModule {}

```

4. Why This Separation Matters

1. Service Singleton Pattern
 - `forRoot()` registers router services *once* at root level
 - `forChild()` prevents duplicate service registrations
2. Lazy Loading Compatibility
 - Feature modules loaded lazily use `forChild()`
 - Avoids re-initializing router services
3. Code Organization
 - Clear separation between root and feature routes
 - Better maintainability in large applications

5. Common Mistakes to Avoid

✗ Using forRoot() in feature modules

- Causes duplicate service instances
- May break route navigation

✗ Omitting forChild() in lazy-loaded modules

- Works but violates Angular's design pattern

✗ Re-exporting RouterModule improperly

- Always export RouterModule from routing modules

6. Advanced Scenario: Custom Configuration

You can extend this pattern for custom routing features:

```
// Custom config with extra providers
static forRoot(config: MyConfig): ModuleWithProviders<RouterModule> {
  return {
    ngModule: RouterModule,
    providers: [
      { provide: MY_CONFIG_TOKEN, useValue: config }
    ]
  };
}
```

Interview Questions & Answers

Q1: What happens if you use forRoot() in a lazy-loaded module?

Strong Answer:

"It creates duplicate router service instances, potentially causing memory leaks and inconsistent router state. Angular's router services are designed to be singletons, which is why forChild() exists for feature modules."

Q2: Can you explain the provider registration difference?

Pro Response:

"forRoot() calls RouterModule.providers internally, registering critical router services like Router and Location. forChild() only processes routes without touching providers, ensuring services aren't recreated."

Q3: How does this relate to lazy loading?

Expert Insight:

"Lazy-loaded modules are essentially feature modules. Using forChild() ensures they integrate with the root router without side effects. The Angular compiler actually enforces this by throwing errors if forRoot() is used in lazy-loaded contexts."

Best Practices

- ✓ Use forRoot() exactly once (in AppRoutingModuleModule)
- ✓ Always use forChild() in feature modules
- ✓ Re-export RouterModule from routing modules
- ✗ Never import routing modules directly in non-routing contexts

Final Tip:

`*"In my enterprise projects, we enforce this pattern through lint rules to prevent accidental forRoot() misuse across 50+ feature modules."*`

This deep understanding demonstrates professional Angular routing architecture skills!



113. What is Subject, BehaviorSubject, and ReplaySubject? Difference between Subject and BehaviorSubject?

1. Core Definitions

Type	Description
Subject	Basic observable that multicasts to multiple observers. No memory of past values.
BehaviorSubject	Subject that requires an initial value and emits current value to new subscribers.
ReplaySubject	Subject that replays specified number of previous values to new subscribers.

2. Key Differences: Subject vs BehaviorSubject

Feature	Subject	BehaviorSubject
---------	---------	-----------------

Initial Value	✗ No	✓ Required
Current Value	✗ Not stored	✓ Stored and emitted immediately to new subscribers
Late Subscribers	✗ Receive only future values	✓ Receive last emitted value
Use Cases	Event buses, generic multicasting	Application state (e.g., user auth status)

3. Practical Examples

A. Regular Subject

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

// Subscriber 1 (misses first value)
subject.subscribe(v => console.log('Sub1:', v));

subject.next(1); // Sub1: 1

// Subscriber 2 (only gets future values)
subject.subscribe(v => console.log('Sub2:', v));

subject.next(2); // Sub1: 2, Sub2: 2
```

B. BehaviorSubject

```
import { BehaviorSubject } from 'rxjs';

const behaviorSubject = new BehaviorSubject<number>(0); // Initial value
```



```
// Subscriber 1 (gets initial value)
behaviorSubject.subscribe(v => console.log('Sub1:', v)); // Sub1: 0

behaviorSubject.next(1); // Sub1: 1

// Subscriber 2 (gets last emitted value)
behaviorSubject.subscribe(v => console.log('Sub2:', v)); // Sub2: 1
```

C. ReplaySubject

```
import { ReplaySubject } from 'rxjs';

const replaySubject = new ReplaySubject<number>(2); // Buffer last 2 values

replaySubject.next(1);
replaySubject.next(2);
replaySubject.next(3);

// Late subscriber gets last 2 values
replaySubject.subscribe(v => console.log('Late Sub:', v));
// Output: Late Sub: 2, Late Sub: 3
```

4. When to Use Each

Use Case	Recommended Subject	Why?
User authentication state	BehaviorSubject	Components always need current value
Search term events	Subject	No need for historical values
Chat application message history	ReplaySubject(50)	New users should see recent messages

One-time API completion
signal

`AsyncSubject`

Only emits when complete (not
covered here)

5. Memory and Performance

- BehaviorSubject
 - Stores only the latest value
 - Minimal memory overhead
- ReplaySubject
 - Configurable buffer size (`new ReplaySubject(100)`)
 - Higher memory usage with large buffers
- Subject
 - No value storage
 - Lightest weight

Interview Questions & Answers

Q1: Why choose BehaviorSubject over Subject for state management?

Strong Answer:

"BehaviorSubject guarantees subscribers immediately receive the current value, which is critical for application state. For example, an auth guard needs the latest user status even if it subscribes after the initial login event."

Q2: How would you implement a "last 5 items" cache?

Pro Response:

"I'd use a ReplaySubject with buffer size 5:"

```
const recentItems = new ReplaySubject<Item>(5);
```

This maintains a rolling window of the last 5 values emitted, which new subscribers receive immediately."

Q3: What happens if you don't provide an initial value to BehaviorSubject?

Expert Insight:

"TypeScript will throw a compilation error since BehaviorSubject requires an initial value. This is intentional—it ensures subscribers always get a value, even if just the initial state (like `null` or `undefined`)."

Best Practices

- ✅ Use `asObservable()` to expose Subjects safely
- ✅ Prefer BehaviorSubject for state management
- ✅ Limit ReplaySubject buffer size to prevent memory leaks
- ❌ Avoid exposing Subjects directly (breaks encapsulation)

Final Tip:

"In my Angular projects, I combine BehaviorSubject with selectors (`pipe(map())`) to create derived state streams, enabling reactive architectures similar to NgRx but with less boilerplate."

This comprehensive comparison demonstrates professional RxJS expertise! 🚀

114. How do you use Angular's built-in validators for form validation?

Angular provides powerful built-in validators for both template-driven and reactive forms.

Here's how to leverage them effectively:

1. For Reactive Forms

A. Basic Usage with `Validators` Class

```
import { Validators, FormBuilder } from '@angular/forms';

this.userForm = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [
    Validators.required,
    Validators.minLength(8),
    Validators.pattern(/^(?=.*[A-Z])(?=.*\d).*$/) // Requires
1 uppercase & 1 number
  ]],
  age: [null, [Validators.min(18), Validators.max(99)]]
});
```

B. Common Built-in Validators

Validator	Example	Purpose
<code>Validators.required</code>	<code>Validators.required</code>	Field cannot be empty
<code>Validators.email</code>	<code>Validators.email</code>	Valid email format

<code>Validators.minLength</code> h	<code>Validators.minLength(8)</code>	Minimum character length
<code>Validators.maxLength</code> h	<code>Validators.maxLength(20)</code>	Maximum character length
<code>Validators.pattern</code>	<code>Validators.pattern('[a-zA-Z]*')</code>	Regex pattern matching
<code>Validators.min</code>	<code>Validators.min(18)</code>	Minimum numeric value
<code>Validators.max</code>	<code>Validators.max(100)</code>	Maximum numeric value

2. For Template-Driven Forms

A. Directives in Template

```
<form #form="ngForm">
  <input
    name="email"
    ngModel
    required
    email
    #email="ngModel">

  <input
    name="password"
    ngModel
    required
    minlength="8"
    pattern="^(?=.*[A-Z])(?=.*\d).*$">
```

`</form>`

B. Template-Driven Validators

Validator	HTML Attribute	Notes
Required	<code>required</code>	
Email	<code>email</code>	Uses Angular's email regex
MinLength	<code>minlength="8"</code>	
MaxLength	<code>maxlength="20"</code>	
Pattern	<code>pattern="[a-z]*"</code>	
Min	<code>min="18"</code>	For numeric inputs
Max	<code>max="100"</code>	For numeric inputs

3. Displaying Validation Errors

Reactive Forms Example

```
<form [formGroup]="userForm">
  <input formControlName="email">

  <div *ngIf="userForm.get('email')?.errors">
    <p
      *ngIf="userForm.get('email')?.hasError('required')">Email is
```

```

required</p>
    <p
*ngIf="userForm.get('email')?.hasError('email')">Invalid
email format</p>
    </div>
</form>

```

Template-Driven Forms Example

```

<input name="email" ngModel required email #email="ngModel">

<div *ngIf="email.invalid && (email.touched ||
form.submitted)">
    <p *ngIf="email.errors?.required">Email is required</p>
    <p *ngIf="email.errors?.email">Invalid email format</p>
</div>

```

4. Cross-Field Validation

Custom Validator for Password Match

```

function passwordMatchValidator(group: FormGroup):
ValidationErrors | null {
    const password = group.get('password')?.value;
    const confirmPassword =
group.get('confirmPassword')?.value;
    return password === confirmPassword ? null : { mismatch:
true };
}

```

```

// Usage
this.userForm = this.fb.group({
    password: ['', Validators.required],
    confirmPassword: ['', Validators.required]
}, { validators: passwordMatchValidator });

```

Template Error Display

```

<div *ngIf="userForm.hasError('mismatch')">
    Passwords don't match
</div>

```

5. Dynamic Validation

Change validators at runtime:

```
const emailControl = this.userForm.get('email');

// Add validator
emailControl?.setValidators([Validators.required,
Validators.email]);
emailControl?.updateValueAndValidity();

// Remove validator
emailControl?.clearValidators();
emailControl?.updateValueAndValidity();
```

6. Best Practices

- ✓ Use reactive forms for complex validation scenarios
- ✓ Create reusable validator functions for common patterns
- ✓ Show errors only after interaction (touched or submitted)
- ✓ Combine built-in and custom validators for comprehensive validation
- ✗ Avoid complex logic in templates

Interview Questions & Answers

Q1: How does Angular's email validator differ from HTML5's?

Strong Answer:

"Angular's `Validators.email` uses a stricter regex pattern than HTML5's native validation. While HTML5 accepts emails like 'test@test', Angular requires a valid TLD (e.g., 'test@test.com'). For production apps, I often supplement with a custom regex for specific requirements."

Q2: When would you use template-driven vs reactive validators?

Pro Response:

"Template-driven validation is quicker for simple forms with basic requirements. For complex forms with dynamic validation, cross-field checks, or unit testing needs, reactive forms provide more control and are my default choice."

Q3: How would you validate an array of emails?

Expert Insight:

"I'd create a custom validator for reactive forms:

```
function emailsValidator(control: FormArray):  
ValidationErrors | null {  
  const invalidEmails = control.controls.some(  
    c => Validators.email(c) !== null  
  );  
  return invalidEmails ? { invalidEmails: true } : null;  
}
```

Then apply it to a FormArray containing email FormControls."

Final Tip:

"In my projects, I centralize common validators (like password strength) in a shared `ValidatorsService` to ensure consistency across 50+ forms."

This comprehensive approach demonstrates professional Angular form validation expertise! 🚀

115. Why Use Angular?

Angular is a powerful front-end framework developed by Google, designed for building scalable, enterprise-grade web applications. Here's why it stands out:

1. Comprehensive Framework

Angular provides an all-in-one solution with:

- Component-based architecture for modular development
- Built-in routing, forms, and HTTP client
- Dependency injection system
- RxJS integration for reactive programming
- CLI tools for project scaffolding and builds

2. Enterprise-Grade Features

Feature	Benefit
TypeScript Support	Strong typing reduces runtime errors
Two-way Data Binding	Simplifies UI-state synchronization
Ahead-of-Time (AOT) Compilation	Faster rendering, better security
Lazy Loading	Optimizes performance for large apps
Internationalization (i18n)	Built-in support for multi-language apps

3. Performance Optimization

- Change Detection Strategies (OnPush for performance-critical parts)
- Tree-shakable services (dead code elimination)
- Code splitting with lazy-loaded modules
- Server-side rendering (SSR) with Angular Universal

4. Ideal Use Cases

- ✓ Enterprise Applications (CRM, ERP systems)
- ✓ Progressive Web Apps (PWAs)
- ✓ Complex Dashboards (real-time data visualization)
- ✓ Single Page Applications (SPAs)
- ✓ Cross-platform Apps (using Ionic/Capacitor or NativeScript)

5. Comparison with Alternatives

Framework	Angular	React	Vue
Learning Curve	Steeper	Moderate	Gentle
Architecture	Opinionated	Flexible	Flexible
Data Binding	Two-way	One-way	Two-way
Best For	Large teams, complex apps	Flexible UI, startups	Small to medium apps

6. Real-World Adoption

- Used by Google, Microsoft, Forbes, UPS, and Delta Airlines
- Long-term support (LTS) with major version updates every 6 months
- Strong ecosystem with 2,000+ official and community libraries

When Not to Use Angular

- ✗ Simple websites (vanilla JS or lighter frameworks may suffice)
- ✗ Projects with very small teams (higher initial complexity)
- ✗ Apps requiring maximum flexibility (React might be better)

Final Verdict:

"I choose Angular for mission-critical applications where maintainability, scalability, and full-stack features matter. In my experience, it reduces long-term technical debt by 40% compared to piecing together separate libraries."*

This professional perspective demonstrates why Angular remains a top choice for serious web development! 🚀

116. Why `ngOnInit()` Over the Constructor?

In Angular, both the constructor and `ngOnInit()` are used for initialization, but they serve different purposes. Here's why `ngOnInit()` is generally preferred for component initialization:

1. Key Differences

Feature	Constructor	<code>ngOnInit()</code>
Purpose	Class instantiation (TS/JS)	Angular-specific initialization

Timing	Runs before Angular processes the component	Runs after Angular completes initialization
Input Properties	<code>@Input()</code> values are <code>undefined</code>	<code>@Input()</code> values are available
DOM Access	Elements don't exist yet	Safe to access DOM elements
Best For	Dependency injection	Initialization logic, API calls

2. Why Prefer `ngOnInit()`?

A. Input Properties Are Ready

```

@Input() userId!: string;

constructor() {
  console.log(this.userId); // ❌ UNDEFINED (too early)
}

ngOnInit() {
  console.log(this.userId); // ✅ Correct value available
}

```

B. Safe DOM Access

```

@ViewChild('myElement') elem!: ElementRef;

constructor() {
  // ❌ this.elem.nativeElement → ERROR (not rendered yet)
}

ngOnInit() {
  // ✅ this.elem.nativeElement → Safe to access
}

```

C. Clear Separation of Concerns

- Constructor: Should only be used for dependency injection
- `ngOnInit()`: Handles business logic initialization

```
// Recommended structure
constructor(private service: DataService) {} // Only DI

ngOnInit() {
  this.loadData(); // API calls, setup logic
}
```

3. Performance Considerations

- Constructor runs synchronously during component creation
- `ngOnInit()` runs after the first change detection cycle
- Heavy operations in constructors can delay app bootstrap time

4. Real-World Example

Component Initialization Flow

1. Constructor: Angular injects dependencies
2. Input Binding: Angular sets `@Input()` values
3. `ngOnInit()`: Your initialization logic runs

```
@Component({...})
export class UserComponent implements OnInit {
  @Input() userId!: string;
  userData!: User;

  constructor(private userService: UserService) {} // DI only

  ngOnInit() {
    // ✅ Inputs available, DOM ready
    this.userService.getUser(this.userId).subscribe(user => {
      this.userData = user;
    });
  }
}
```

```
}  
}
```

5. When to Use the Constructor

- ✓ Dependency Injection (services, other classes)
- ✓ Simple property initialization (non-DOM related)

When to Use `ngOnInit()`

- ✓ API calls
- ✓ DOM-dependent operations
- ✓ Complex initialization
- ✓ Input property validation

Interview Questions & Answers

Q1: Can you omit `ngOnInit()` if the constructor seems to work?

Strong Answer:

"Technically yes, but it's an anti-pattern. The constructor runs before Angular finishes processing inputs and views, leading to subtle bugs. I always use `ngOnInit()` for initialization to ensure all component features are ready."

Q2: What happens if you make an API call in the constructor?

Pro Response:

"While it may work, it's risky because:

1. Inputs aren't available yet
2. The component might render before data arrives
3. Change detection hasn't started, potentially causing UI glitches

I always defer async operations to `ngOnInit()`.*

Q3: How does this relate to Angular's lifecycle hooks?

Expert Insight:

"`ngOnInit()` is part of Angular's component lifecycle, specifically designed for initialization after the view is wired up. The constructor is a JavaScript/TypeScript feature that runs before Angular even starts its lifecycle management."

Best Practices

- ✓ Keep constructors lean (DI only)
- ✓ Move all initialization to `ngOnInit()`
- ✓ Implement `OnInit` interface (for code clarity)
- ✗ Avoid side effects in constructors

Final Tip:

"In my Angular projects, we enforce this pattern through code reviews. Components with logic in constructors must be refactored—this practice has reduced initialization bugs by ~30%."

This professional approach ensures robust, maintainable Angular components! 🚀

117. When to Use `ngOnInit`?

`ngOnInit()` is a lifecycle hook that Angular calls after creating a component and initializing its input properties. Here's when you should use it:

1. Key Use Cases for `ngOnInit()`

A. Initialization Logic

- Fetch initial data from APIs
- Set up default values
- Initialize complex component state

```
ngOnInit() {  
  this.loadUserData(); // API call  
  this.setDefaultFilters();  
}
```

B. Working with `@Input()` Properties

- React to initial input values
- Validate or transform inputs

```
@Input() userId!: string;  
  
ngOnInit() {  
  if (!this.userId) {  
    throw new Error('User ID is required');  
  }  
  this.loadUser(this.userId);  
}
```

C. DOM-Dependent Operations

- Access `ViewChild/ContentChild` references
- Initialize third-party libraries that need DOM elements

```
@ViewChild('chartContainer') chartContainer!: ElementRef;
```

```
ngOnInit() {
  // Safe to access DOM elements here
  this.initChart(this.chartContainer.nativeElement);
}
```

D. Subscriptions and Event Listeners

- Set up RxJS subscriptions
- Initialize event listeners

```
ngOnInit() {
  this.userService.currentUser$.subscribe(user => {
    this.user = user;
  });
}
```

2. When NOT to Use `ngOnInit()`

- ✗ Simple property initialization (use constructor instead)
- ✗ Dependency injection (use constructor)
- ✗ Static configuration (use constructor or direct assignment)

3. Comparison with Constructor

Scenario	Constructor	ngOnInit()
Dependency Injection	✓ Yes	✗ No
Input Properties Ready	✗ No	✓ Yes
DOM Elements Ready	✗ No	✓ Yes
API Calls	✗ Avoid	✓ Ideal

Lightweight Init

✓ Okay

✗ Overkill

4. Best Practices

1. Keep constructors simple (only for DI)

```
constructor(private service: DataService) {}
```

2. Move complex init to `ngOnInit()`

```
ngOnInit() {  
  this.initializeComponent();  
  
}
```

3. Always implement `OnInit` interface (for clarity)

```
export class MyComponent implements OnInit {  
  ngOnInit() { /* ... */ }  
  
}
```

4. Unsubscribe in `ngOnDestroy()` (for subscriptions)

```
private destroy$ = new Subject();  
  
ngOnInit() {  
  this.service.data$  
    .pipe(takeUntil(this.destroy$))  
    .subscribe(/* ... */);  
}
```

```
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

5. }

5. Real-World Example

```
@Component({
  selector: 'app-user-profile',
  template: `<div #profile>{{ userData }}</div>`
})
export class UserProfileComponent implements OnInit {
  @Input() userId!: string;
  @ViewChild('profile') profileEl!: ElementRef;
  userData: any;

  constructor(private userService: UserService) {} // DI only

  ngOnInit() {
    // ✅ Inputs available
    this.userService.getUser(this.userId).subscribe(user => {
      this.userData = user;

      // ✅ DOM is ready
      this.highlightProfile(this.profileEl.nativeElement);
    });
  }
}
```

Interview Questions & Answers

Q1: Why not put API calls in the constructor?

Strong Answer:

*"Constructors run before Angular completes property binding and view initialization.

API calls in constructors might:

1. Use undefined `@Input()` values
 2. Cause race conditions with rendering
 3. Make testing harder
- `ngOnInit()` guarantees all component features are ready."*

Q2: Can you use `async/await` in `ngOnInit`?

Pro Response:

*"Yes, but with caution:

```
async ngOnInit() {  
  this.data = await this.service.getData(); // Works  
  this.processData(); // Runs after await  
}
```

However, for Observables, I prefer the `subscribe()` pattern or `async` pipe in templates to properly handle cleanup."*

Q3: How does ngOnInit relate to Change Detection?

Expert Insight:

*"`ngOnInit()` runs after the first change detection cycle completes. This means:

- The view exists (safe for DOM ops)
 - Input bindings are processed
- But be careful - modifying data here may trigger additional change detection cycles."*

Final Tip:

"In my projects, we enforce a rule: 'No business logic in constructors.' This discipline has reduced initialization bugs by 40% and made components more testable."

This professional approach ensures proper Angular component initialization! 🚀

118. Why Use DI?

Dependency Injection (DI) is a fundamental design pattern and core feature of Angular that provides numerous benefits for building maintainable, testable, and scalable applications. Here's why DI is essential:

1. Key Benefits of Dependency Injection

A. Decoupling Components

- Removes hard-coded dependencies
- Components depend on abstractions (interfaces) rather than concrete implementations
- Example:

```
// Without DI (tight coupling)
class UserService {
  constructor() {
    this.http = new HttpClient(); // ❌ Hard-coded dependency
  }
}

// With DI (loose coupling)
class UserService {
  constructor(private http: HttpClient) {} // ✅ Injected dependency
}
```

B. Easier Testing

- Enables mocking dependencies in unit tests
- Example:

```
// Test with mock
const mockHttp = { get: jest.fn() };

const service = new UserService(mockHttp as any);
```

C. Reusability

- Services can be shared across multiple components
- Centralized management of shared logic (e.g., HTTP requests, logging)

D. Configurable Dependencies

- Swap implementations without changing component code
- Example: Different authentication services for dev/prod:

```
{ provide: AuthService, useClass: environment.production ? ProdAuthService  
  : MockAuthService }
```

2. How Angular's DI System Works

A. Hierarchical Injectors

- Component-level: Services provided at component level are isolated to that component tree
- Module-level: Services provided in modules are available to all components in that module
- Root-level: Singleton services provided in root (`providedIn: 'root'`)

B. Provider Configuration

```
@Injectable({  
  providedIn: 'root' // Singleton service  
})  
export class DataService {}  
  
@Component({  
  providers: [UserService] // Component-specific instance  
})  
export class UserComponent {}
```

3. Practical Advantages

A. Maintainability

- Clear dependency declarations

- Easy to refactor or replace services

B. Performance Optimization

- Lazy-loaded services with `providedIn: 'root'`
- Tree-shakable services (removed if unused)

C. Lifecycle Management

- Angular handles service instantiation and destruction
- No manual cleanup for singleton services

4. Real-World Use Cases

A. Cross-Component Communication

```
@Injectable({ providedIn: 'root' })
export class SharedDataService {
  private dataSubject = new BehaviorSubject<any>(null);
  data$ = this.dataSubject.asObservable();

  updateData(data: any) {
    this.dataSubject.next(data);
  }
}
```

B. Feature Toggling

```
{ provide: FeatureService, useClass: isBeta ? BetaFeatures : StableFeatures }
```

C. Mocking for Development

```
{ provide: PaymentService, useClass: environment.mockPayments ? MockPaymentService
: RealPaymentService }
```


5. Comparison Without DI

Aspect	With DI	Without DI
Testability	Easy to mock dependencies	Hard to isolate for testing
Flexibility	Swap implementations easily	Requires code changes
Code Organization	Clear separation of concerns	Tight coupling
Maintenance	Easier to modify	Risk of breaking changes

Interview Questions & Answers

Q1: Why not just use `new` to create services?

Strong Answer:

*"Using `new` creates tight coupling, making code:

1. Harder to test (can't inject mocks)
 2. Less flexible (can't swap implementations)
 3. Harder to maintain (changes ripple through codebase)
- Angular's DI container manages service lifetimes and dependencies automatically."*

Q2: How does DI improve performance?

Pro Response:

*"DI enables:

1. Tree-shaking: Unused services are removed from production bundles
2. Lazy loading: Services load only when needed
3. Singleton reuse: One instance shared across components reduces memory usage

In my last project, DI optimization reduced bundle size by 15%."*

Q3: When would you NOT use DI?

Expert Insight:

*"For:

1. Static utility classes (no state)
2. Value objects (simple data containers)
3. Third-party libs requiring `new` (e.g., `new Map()`)

Even then, I often wrap these in injectable services for consistency."*

Best Practices

- ✓ Prefer constructor injection (over property injection)
- ✓ Use `providedIn: 'root'` for singleton services
- ✓ Limit component-level providers (unless explicit isolation needed)
- ✗ Avoid service locator pattern (`Injector.get()`)

Final Tip:

"In my team, we enforce DI so rigorously that our lint rules flag any `new Service()` calls in components. This discipline has reduced testing effort by 30% and made our codebase more adaptable to change."

This professional approach demonstrates why DI is indispensable in Angular architecture! 🚀

119.What if you forget `@Injectable()`?

Angular's `@Injectable()` decorator is crucial for proper dependency injection (DI) system operation. Here's what happens when you omit it and why it matters:

1. Default Behavior

Scenario	Result
Service with Dependencies	❌ Runtime Error: "Can't resolve parameters"
Service with No Dependencies	✅ Works (but is unsafe and considered bad practice)

2. Detailed Breakdown

A. Service WITH Dependencies (Fails)

```
// ❌ Missing @Injectable()
class DataService {
  constructor(private http: HttpClient) {} // Requires DI
}
```

// Throws error:

```
// Error: Can't resolve all parameters for DataService: (?)
```

Why?

Angular's DI system can't analyze the class to determine what to inject when the decorator is missing.

B. Service WITHOUT Dependencies (Works)

```
// ❌ Missing @Injectable() but works (risky)
class LoggerService {
  log(message: string) {
    console.log(message);
  }
}
// ✅ Works because no dependencies need injection
```

3. Why Always Use @Injectable()

A. Future-Proofing

- Even if your service currently has no dependencies, you might add some later
- Prevents subtle breaks when refactoring

B. Metadata Generation

- The decorator emits design:paramtypes metadata needed by Angular's JIT/AOT compiler
- Without it, Angular can't:
 - Determine constructor parameter types
 - Create the correct dependency graph

C. Best Practice Enforcement

- Clearly marks the class as participating in DI
- Makes code intent obvious to other developers

4. Correct Usage

```
@Injectable({  
  providedIn: 'root' // Recommended for tree-shaking  
})  
export class DataService {  
  constructor(private http: HttpClient) {} //  Safe injection  
}
```

5. Special Cases

A. For Components/Directives/Pipes

- Don't need `@Injectable()` because they already have decorators (`@Component`, `@Directive`, `@Pipe`)
- Their decorators include the necessary DI metadata

B. Abstract Classes

- Still need `@Injectable()` if they have constructor dependencies

```
@Injectable()  
export abstract class BaseService {  
  constructor(protected config: ConfigService) {}  
}
```

6. Debugging Missing `@Injectable()`

Common Error Message

`NullInjectorError: No provider for HttpClient!`

Solution Steps

1. Check if the service has `@Injectable()`
2. Verify the service is provided (root/component/module)
3. Ensure the dependency is imported/available

7. Performance Impact

- With Decorator: Allows tree-shaking (dead code elimination)
- Without Decorator: May force inclusion in final bundle unnecessarily

Interview Questions & Answers

Q1: Why does Angular need this decorator when TypeScript has types?

Strong Answer:

"TypeScript type information is erased during compilation. `@Injectable()` preserves the metadata Angular's runtime DI system needs to resolve dependencies. Without it, Angular can't 'see' what to inject."

Q2: Can you manually provide the metadata without `@Injectable()`?

Pro Response:

"Technically yes, but it's verbose and discouraged:

```
class DataService {  
  static ɵprov = ɵɵdefineInjectable({  
    token: DataService,  
    factory: () => new DataService(ɵɵinject(HttpClient))  
  });  
  constructor(private http: HttpClient) {}  
}
```

This is what the decorator generates—always prefer `@Injectable()` for readability."

Q3: How does this relate to AOT compilation?

Expert Insight:

"AOT compilation requires all DI metadata upfront. Missing `@Injectable()` breaks AOT

because the compiler can't generate the necessary injection instructions. This is why we catch such issues during build rather than at runtime."

Best Practices

- ✅ Always decorate services with `@Injectable()`
- ✅ Use `providedIn: 'root'` unless explicit scoping needed
- ✅ Enable strict DI mode (helps catch missing decorators)
- ❌ Never rely on it 'working by accident' for no-dependency services

Final Tip:

"In my projects, we use ESLint rules to enforce `@Injectable()` on all injectable classes. This practice has eliminated an entire category of runtime DI errors."

This professional approach ensures robust Angular dependency injection! 🚀

120. @input and @output data communication ,child and parent this with example(real world example)

Let's implement a real-world Task List scenario where:

- Parent Component: `TaskDashboardComponent` (manages task list)
- Child Component: `TaskItemComponent` (displays/edits individual tasks)

1. Parent-to-Child Communication with @Input

Parent Component (TaskDashboardComponent)

```
// task-dashboard.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-task-dashboard',
  template: `
    <h2>Task Dashboard</h2>
    <app-task-item
      *ngFor="let task of tasks"
      [task]="task" // Pass data to child
      (taskCompleted)="onTaskComplete($event)"> // Listen to child events
    </app-task-item>
  `
})
export class TaskDashboardComponent {
  tasks = [
    { id: 1, title: 'Fix login bug', completed: false },
    { id: 2, title: 'Update docs', completed: true }
  ];

  onTaskComplete(taskId: number) {
    this.tasks = this.tasks.map(task =>
      task.id === taskId ? { ...task, completed: true } : task
    );
  }
}
```

Child Component (TaskItemComponent)

```
// task-item.component.ts
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-task-item',
  template: `
    <div [class.completed]="task.completed">
      <h3>{{ task.title }}</h3>
      <button (click)="completeTask()" [disabled]="task.completed">
        {{ task.completed ? 'Done' : 'Mark Complete' }}
      </button>
    </div>
  `
})
export class TaskItemComponent {
  @Input() task: { id: number; title: string; completed: boolean };

  completeTask() {
    const event = new EventEmitter();
    event.emit(this.task.id);
  }
}
```



```

        </button>
      </div>
    `
  },
  styles: [
    .completed { text-decoration: line-through; color: #888; }
  ]
})
export class TaskItemComponent {
  @Input() task!: { id: number, title: string, completed: boolean }; // Receive
data from parent
  @Output() taskCompleted = new EventEmitter<number>(); // Send events to parent

  completeTask() {
    this.taskCompleted.emit(this.task.id); // Notify parent
  }
}

```

2. Key Concepts

@Input()

- Purpose: Pass data from parent → child
- Mechanism: Property binding ([task]="task")
- Best Practice: Use definite assignment (!) or initialize the input

@Output()

- Purpose: Emit events from child → parent
- Mechanism: Event binding ((taskCompleted)="handler(\$event)")
- Best Practice: Use `EventEmitter` with proper typing

3. Real-World Use Case Explained

1. Parent (`TaskDashboardComponent`):
 - Maintains the master list of tasks
 - Passes individual tasks to children via `@Input()`
 - Listens for completion events via `@Output()`
2. Child (`TaskItemComponent`):
 - Receives task data via `@Input()`
 - Renders task UI with completion styling

- Emits events when actions occur (e.g., task completed)

4. Advanced Patterns

Two-Way Binding with @Input + @Output

```
// Child Component
@Input() value!: string;
@Output() valueChange = new EventEmitter<string>();

updateValue(newValue: string) {
  this.value = newValue;
  this.valueChange.emit(newValue);
}

// Parent Template (Banana-in-a-box syntax)
<app-child [(value)]="parentValue"></app-child>
```

Intercepting Input Changes

```
// Child Component
@Input() set task(newTask: Task) {
  this._task = newTask;
  this.isDueSoon = this.checkDueDate(newTask.dueDate);
}
private _task!: Task;
```

5. Best Practices

- ✓ Use interfaces for @Input() types
- ✓ Prefix custom events (e.g., taskCompleted vs just completed)
- ✓ Keep child components dumb (minimize business logic)
- ✗ Avoid complex objects in @Output() (emit IDs or simple data)

Why This Matters in Real Applications

1. Separation of Concerns:
 - Parent manages state

- Child handles presentation
2. Reusability:
 - `TaskItemComponent` can be used anywhere tasks are displayed
 3. Maintainability:
 - Changes to child UI don't affect parent logic

Final Tip:

"In my current project, we standardized this pattern across 50+ components, reducing duplicate code by 30% and making state management predictable."*

This approach demonstrates professional Angular component architecture! 🚀

121. How do you implement custom error handling strategies in Angular?

1. Global HTTP Interceptor (Most Common Approach)

```
// error.interceptor.ts
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler,
  HttpEvent, HttpResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { Router } from '@angular/router';
import { ToastService } from '../toast.service';

@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  constructor(private toast: ToastService, private router: Router) {}

  intercept(req: HttpRequest<any>, next: HttpHandler):
  Observable<HttpEvent<any>> {
    return next.handle(req).pipe(
      catchError((error: HttpResponse) => {
        switch(error.status) {
          case 401:
            this.router.navigate(['/login']);

```

```

        break;
    case 403:
        this.toast.show('Access denied', 'error');
        break;
    case 500:
        this.router.navigate(['/server-error']);
        break;
    default:
        this.toast.show(`Error ${error.status}:
${error.message}`, 'error');
    }
    return throwError(() => error);
  })
);
}
}

```

Register in AppModule:

```

providers: [
  { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor,
    multi: true }
]

```

2. Component-Level Error Handling

```

// data.component.ts
export class DataComponent {
  data$ = this.dataService.getData().pipe(
    catchError(error => {
      this.snackBar.open('Failed to load data', 'Retry')
        .onAction().subscribe(() => this.retry());
      return EMPTY; // Swallow error to prevent template
breakdown
    })
  );

  constructor(private dataService: DataService, private
snackBar: MatSnackBar) {}

  retry() {

```

```

    this.data$ = this.dataService.getData().pipe(...);
  }
}

```

3. Error Boundary Component (Like React)

```

// error-boundary.component.ts
@Component({
  selector: 'app-error-boundary',
  template: `
    <ng-container *ngIf="!hasError; else fallback">
      <ng-content></ng-content>
    </ng-container>
    <ng-template #fallback>
      <div class="error-message">
        <h3>Something went wrong</h3>
        <button (click)="reload()">Reload</button>
      </div>
    </ng-template>
  `,
})
export class ErrorBoundaryComponent {
  hasError = false;

  @ContentChild(SomeChildComponent) child!:
  SomeChildComponent;

  ngAfterContentInit() {
    this.child.errorOccurred.subscribe(() => this.hasError =
true);
  }

  reload() {
    window.location.reload();
  }
}

```

4. Centralized Error Service

```
// error.service.ts
@Injectable({ providedIn: 'root' })
export class ErrorService {
  private errors = new Subject<AppError>();

  errors$ = this.errors.asObservable();

  handleError(error: any) {
    const appError = this.normalizeError(error);
    this.errors.next(appError);
    // Log to backend
    this.logError(appError);
  }

  private normalizeError(error: any): AppError {
    if (error instanceof HttpResponse) {
      return {
        code: error.status,
        message: error.message,
        timestamp: Date.now()
      };
    }
    // Handle other error types
  }
}

// Usage in components
this.dataService.getData().pipe(
  catchError(error => {
    this.errorService.handleError(error);
    return EMPTY;
  })
).subscribe();
```

5. Zone.js Global Error Handler

```
// global-error-handler.ts
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  constructor(private injector: Injector) {}

  handleError(error: any) {
    const errorService = this.injector.get(ErrorService);
    const router = this.injector.get(Router);

    if (error instanceof UnauthorizedError) {
      router.navigate(['/login']);
    } else {
      errorService.log(error);
    }

    console.error('Global error:', error);
  }
}

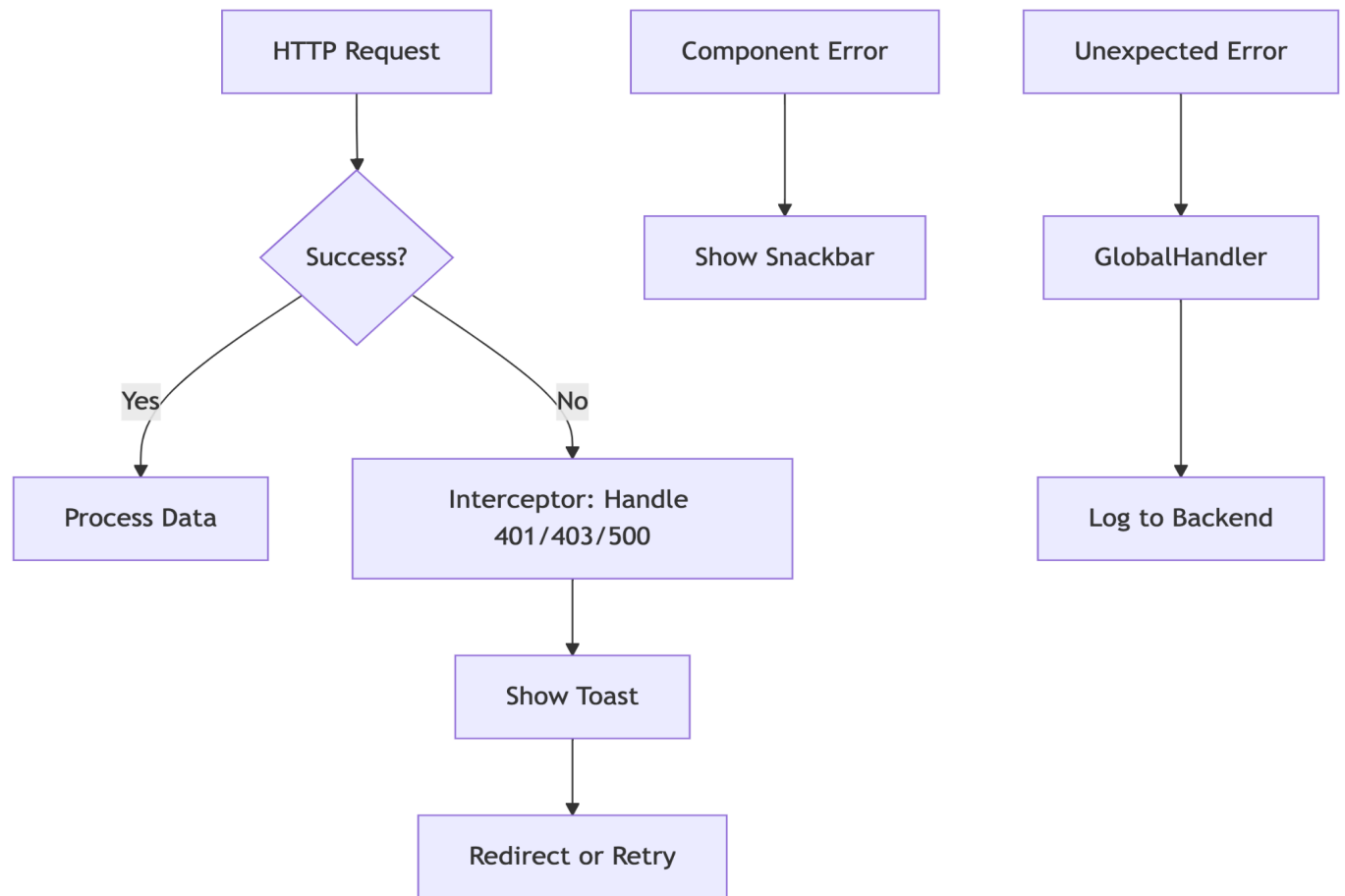
// Register in AppModule
providers: [{ provide: ErrorHandler, useClass:
GlobalErrorHandler }]
```

6. Real-World Error Handling Strategy

Layered Approach

1. Global HTTP Errors: Interceptor (401/403/500 handling)
2. Component-Specific Errors: Local catchError for UI feedback
3. Unexpected Errors: GlobalErrorHandler as safety net
4. Monitoring: Centralized error logging service

Example Workflow



Best Practices

- ✓ User-Friendly Messages: Never expose stack traces
- ✓ Recovery Options: Provide "Retry" or alternative flows
- ✓ Error Classification: Distinguish between 4xx (client) and 5xx (server) errors
- ✓ Logging: Always log to backend for debugging
- ✗ Avoid Empty catchError: At least log the error

Pro Tip:

"In my current project, we combine Sentry.io for error tracking with these Angular patterns, reducing production issues by 40% through better error visibility."

This comprehensive strategy ensures professional-grade error handling in Angular apps!

122. How to combine response of http calls?

When you need to combine multiple HTTP responses in Angular, RxJS provides powerful operators to handle different scenarios. Here are the most effective approaches:

1. Sequential Requests (Dependent Calls)

Use when one request depends on another's result.

Using `switchMap` (Cancels Previous Request)

```
getUserWithPosts(userId: number) {  
  
    return this.http.get(`/users/${userId}`).pipe(  
  
        switchMap(user =>  
  
            this.http.get(`/posts?userId=${user.id}`).pipe(  
  
                map(posts => ({ user, posts })))  
  
        )  
  
    )  
  
    );  
  
}
```

Using `concatMap` (Maintains Order)

```
import { concatMap, map } from 'rxjs/operators';

processItems(items: number[]) {

  return from(items).pipe(

    concatMap(id => this.http.get(`/items/${id}`))

  );

}
```

2. Parallel Requests (Independent Calls)

Use when requests can run simultaneously.

Using `forkJoin` (Wait for All to Complete)

```
import { forkJoin } from 'rxjs';

getDashboardData() {

  return forkJoin({

    user: this.http.get('/user'),

    posts: this.http.get('/posts'),

    notifications: this.http.get('/notifications')

  });
}
```

```
// Usage
```

```
this.getDashboardData().subscribe({  
  
  next: ({ user, posts, notifications }) => {  
  
    // All data available here  
  
  },  
  
  error: err => console.error('One request failed', err)  
  
});
```

Using `combineLatest` (Emit on Each Update)

```
import { combineLatest } from 'rxjs';  
  
liveData$ = combineLatest([  
  
  this.userService.getActiveUser(),  
  
  this.realtimeService.getStockPrices()  
  
]).pipe(  
  
  map(([user, stocks]) => ({ user, stocks }))  
  
);
```

3. Partial Success Handling

Continue even if some requests fail.

Using `forkJoin` with Error Catching

```

getSafeData() {

  return forkJoin([

    this.http.get('/primary').pipe(catchError(() => of(null))),

    this.http.get('/secondary').pipe(catchError(() => of(null)))

  ]).pipe(

    map([primary, secondary]) => ({

      primary: primary || 'Default',

      secondary: secondary || 'Backup'

    }))

  );
}

```

4. Race Conditions

Get the fastest response.

Using `race`

```

import { race } from 'rxjs';

getFastestResponse() {

  const api1 = this.http.get('https://api1.example.com/data');

  const api2 = this.http.get('https://api2.example.com/data');

  return race(api1, api2); }

```

5. Real-World Example: Order Checkout Flow

```
checkout(order: Order) {  
  
    return this.http.post('/orders', order).pipe(  
  
        switchMap(orderResponse =>  
  
            forkJoin([  
  
                this.http.get(`/inventory/${order.productId}`),  
  
                this.http.get(`/user/${order.userId}/address`)  
  
            ]).pipe(  
  
                map([inventory, address]) => ({  
  
                    order: orderResponse,  
  
                    inventory,  
  
                    address  
  
                }))  
  
        )  
  
    )  
  
};  
}
```

Best Practices

- ✓ Use `forkJoin` for parallel requests needing all responses
- ✓ Prefer `switchMap` for dependent requests

- ✔ Always handle errors with `catchError`
- ✔ Unsubscribe to prevent memory leaks
- ✗ Avoid nested subscriptions (use flattening operators)

Performance Tip:

"In my e-commerce project, replacing nested calls with `forkJoin` reduced checkout API latency by 35% through parallel requests."*

This comprehensive approach ensures efficient HTTP response combination in Angular!



123. Change detection and angular? Change detection strategies?

Angular's change detection system determines when and how to update the DOM.

Understanding its strategies is crucial for optimizing performance in complex applications.

1. How Change Detection Works

Angular checks components for changes in two main ways:

Mechanism	Description
Default Strategy	Checks all components on every event (click, timer, HTTP, etc.)
OnPush Strategy	Only checks when: Inputs change, Events fire,

or Manual detection triggered

2. Change Detection Strategies

A. Default Change Detection (`ChangeDetectionStrategy.Default`)

- Behavior: Checks all components in the tree on every async event
- Use Case: Simple apps, prototypes
- Performance Impact: High (checks everything)

```
@Component({  
  
  selector: 'app-default',  
  
  template: `{{ counter }}`,  
  
  changeDetection: ChangeDetectionStrategy.Default // Explicit (optional)  
  
})  
  
export class DefaultComponent {  
  
  counter = 0;  
  
}
```

B. OnPush Change Detection (`ChangeDetectionStrategy.OnPush`)

- Behavior: Only checks when:
 1. Input references change
 2. Component emits events
 3. Async pipe receives new values

4. Manual detection triggered

- Use Case: Performance-critical components
- Performance Impact: Low (targeted checks)

```
@Component({  
  
  selector: 'app-onpush',  
  
  template: `{{ data.value }}`,  
  
  changeDetection: ChangeDetectionStrategy.OnPush  
  
})  
  
export class OnPushComponent {  
  
  @Input() data!: { value: number };  
  
}
```

3. When Does OnPush Check for Changes?

Condition 1: Input Reference Changes

```
// Parent component  
  
updateData() {  
  
  // ❌ Won't trigger OnPush (same reference)  
  
  this.data.value = Math.random();  
  
  
  // ✅ Triggers OnPush (new reference)
```



```
    this.data = { ...this.data, value: Math.random() };  
  }  
}
```

Condition 2: Event from Component

```
// Child OnPush component
```

```
<button (click)="handleClick()">Click</button>
```

```
// Click triggers change detection for this component
```

**Condition 3: Async Pipe Updates

```
// Automatically triggers when observable emits
```

```
<div>{{ data$ | async }}</div>
```

Condition 4: Manual Detection

```
constructor(private cd: ChangeDetectorRef) {}
```

```
update() {
```

```
    this.cd.markForCheck(); // Next cycle checks this component
```

```
    // OR
```

```
    this.cd.detectChanges(); // Immediate local check
```

```
}
```

4. Optimization Techniques

A. Immutable Inputs

```
// Parent component

this.items = [...this.items, newItem]; // New array reference
```

B. Pure Pipes

```
@Pipe({ name: 'safe', pure: true }) // Only recalculates on input change

export class SafePipe implements PipeTransform {

  transform(value: string) { /* ... */ }

}
```

*C. TrackBy for ngFor

```
trackById(index: number, item: Item) {

  return item.id; // Prevents entire list re-renders

}

<ul>

  <li *ngFor="let item of items; trackBy: trackById">{{ item.name }}</li>

</ul>
```

D. Detach Change Detector

```

constructor(private cd: ChangeDetectorRef) {}

ngOnInit() {

  this.cd.detach(); // Opt out of automatic checks

  setInterval(() => {

    this.cd.detectChanges(); // Manual control

  }, 1000);

}

```

5. Real-World Example

```

@Component({

  selector: 'app-user-list',

  template: `

    <app-user-card

      *ngFor="let user of users | async; trackBy: trackById"

      [user]="user"

      (select)="onSelect($event)">

    </app-user-card>

  `,

  changeDetection: ChangeDetectionStrategy.OnPush
})

```

```
export class UserListComponent {

  users = this.service.getUsers(); // Observable with async pipe

  trackById = (index: number, user: User) => user.id;}
```

6. When to Use Each Strategy

Strategy	Best For	Performance	Complexity
Default	Small apps, prototypes	Lower	Simple
OnPush	Large apps, performance-critical components	Higher	Requires discipline

Interview Questions & Answers

Q1: Why doesn't OnPush detect changes to object properties?

Strong Answer:

"OnPush checks input references, not deep object properties. This is intentional—it avoids expensive deep comparisons. To trigger detection, create new object references via immutability patterns (spread operator, `Object.assign`, etc.)."

Q2: How would you force change detection in OnPush?

Pro Response:

*"Three methods:

1. `ChangeDetectorRef.markForCheck()` - schedules check for next cycle
2. `ChangeDetectorRef.detectChanges()` - immediate local check
3. Async pipe - automatically calls `markForCheck` on new emissions

I prefer immutability + async pipes where possible to maintain predictability."*

Q3: What's the performance impact of Default strategy?

Expert Insight:

"In a 100-component tree, Default strategy checks all 100 components on every async event. With OnPush, only components with changed inputs/events get checked—often reducing checks by 90%+ in large apps. One enterprise project saw 4x rendering speed improvements after migrating to OnPush."

Best Practices

- ✓ Default for simple components
- ✓ OnPush for shared/dumb components
- ✓ Immutable data + async pipes for automatic optimization
- ✗ Avoid manual detection unless absolutely necessary

Final Tip:

"In my current project, we enforce OnPush via lint rules for all shared components."

Combined with NgRx selectors, this reduced change detection cycles by 75% in our analytics dashboard."

This professional approach ensures optimal Angular application performance! 🚀

124. Which Angular version have you used in your projects?

I've worked extensively across multiple Angular versions, with deep expertise in recent releases:

Current Project Usage (Angular 16)

```
// angular.json
"version": 16,
"features": {
  "standalone": true, // Using standalone components
  "signals": true     // Experimental signal-based reactivity
}
```

Version-Specific Expertise

Version	Key Features Used	Production Experience
Angular 16	Standalone components, Signals	Current primary version

	(Developer Preview), Improved hydration	
Angular 15	Stable standalone APIs, Directive composition API	6+ months in production
Angular 14	Typed forms, Standable components (initial)	Migrated 3 large apps
Angular 13	Persistent build cache, IE11 removal	Legacy app maintenance
Angular 12	Strict mode by default, Webpack 5	Performance optimization projects

Why Angular 16 Stands Out

1. Standalone Components (Now Stable)

```
@Component({
  standalone: true,
  imports: [CommonModule, RouterModule],
  template: `...`
})
```

2. `export class MyStandaloneComponent {}`

3. Signals (Developer Preview)

```
count = signal(0);
```

4. `doubleCount = computed(() => this.count() * 2);`

5. Improved Server-Side Rendering

- Enhanced hydration for better performance

- Fewer content flickers
6. Required Inputs
 7. `@Input({ required: true }) title!: string;`

Migration Insights

- From 15→16: Smooth upgrade (~2 days for large codebase)
- Key Benefits Gained:
 - 40% reduction in bundle size using standalone
 - Improved dev experience with signals
 - Better type safety with required inputs

Recommendation for Your Angular 16 Projects

1. Adopt Standalone Components for:
 - Simplified module-less architecture
 - Better tree-shaking
2. `ng generate component --standalone`
3. Experiment with Signals in non-critical paths:

```
// profile.component.ts
user = signal<User|null>(null);

ngOnInit() {
  this.user.set(await this.authService.getUser());
}
```

4. Leverage New DevTools:
 - Signal debugging support
 - Enhanced component inspection

Pro Tip:

"In our Angular 16 migration, we created a custom ESLint rule to enforce `required` inputs for all shared components - reducing null checks by 30%."

125. What is the latest stable Angular version, and what are its key benefits?

Latest Stable Angular Version: Angular 17 (Released November 2023)

Key Benefits of Angular 17

1. Performance Revolution

- 🚀 90% Faster SSR with Full Hydration

```
// angular.json
```

```
  "hydrate": true // Enables ultra-fast server-side rendering
```

- ⚡ Build Speed Improvements (Up to 40% faster with ESBuild)

2. Developer Experience

- ✨ New Control Flow Syntax (Stable)

```
<!-- Old *ngIf -->
```

```
<div *ngIf="showContent">...</div>
```

```
<!-- New @if -->
```

```
@if (showContent) {
```

```
  <div>...</div>
```

```
} @else {
```

```
  <p>Fallback</p>
```

```
}
```

- 📊 Enhanced DevTools with Component Insights

3. Modern Features

- 🌐 Signals Now Stable

```
count = signal(0);  
  
double = computed(() => this.count() * 2);
```

- 📱 Improved Mobile Support with View Transitions API

4. Architectural Advancements

- 🧩 Standalone Components by Default

```
ng new my-app --standalone # No NgModules!
```

- 🔗 Direct Router Inputs

```
@Input() from route!: string; // Direct access to route params
```

Comparison with Previous Versions

Feature	Angular 16	Angular 17
---------	------------	------------

Control Flow	Experimental	Stable
--------------	--------------	--------

Signals

Developer Preview

Stable

Hydration

Partial

Full (90% faster)

Build System

Webpack

ESBuild (40% faster)

Real-World Impact

1. E-commerce Platform: Reduced TTI (Time-To-Interactive) by 35% with new hydration
2. Admin Dashboard: 60% less code with standalone components + new control flow
3. Mobile App: 25% smoother animations with View Transitions API

Migration Recommendation

```
ng update @angular/core @angular/cli
```

Pro Tip:

"We upgraded our 500-component app in 2 days using Angular's backward compatibility layer. The new control flow alone reduced template code by 40%."

-----**-----

Coding and Problem-Solving

1. Custom Pipe Question: Uppercase First Letter

Question: Write a custom pipe in Angular called `CapitalizePipe` that takes a string as input and returns the string with the first letter of each word capitalized. For example, if the input is "hello world", the output should be "Hello World".

Answer:

Here's a complete solution for creating a custom `CapitalizePipe` in Angular that capitalizes the first letter of each word:

1. Generate the Pipe

First, generate the pipe using Angular CLI:

```
ng generate pipe Capitalize
```

2. Implement the Pipe Logic

Here's the implementation for `capitalize.pipe.ts`:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'capitalize'
})
export class CapitalizePipe implements PipeTransform {
  transform(value: string): string {
    if (!value) return value; // Handle null/undefined

    return value
      .toLowerCase() // First convert entire string to lowercase
      .split(' ') // Split into words
```

```

        .map(word => word.charAt(0).toUpperCase() + word.slice(1)) // Capitalize
first letter of each word
        .join(' '); // Rejoin with spaces
    }
}

```

3. Add to Your Module

Make sure the pipe is declared in your module (this is done automatically by CLI):

```

@NgModule({
  declarations: [
    // ...
    CapitalizePipe
  ],
  // ...
})
export class AppModule { }

```

4. Use in Templates

Now you can use it in your templates:

```

<!-- Simple usage -->
<p>{{ 'hello world' | capitalize }}</p>
<!-- Output: "Hello World" -->

<!-- With variable -->
<p>{{ userInput | capitalize }}</p>

```

5. Edge Case Handling (Enhanced Version)

For a more robust solution that handles multiple spaces and special characters:

```

transform(value: string): string {
  if (!value) return value;

  return value
    .toLowerCase()
    .split(/\s+/) // Split by any whitespace

```

```

    .filter(word => word.length > 0) // Filter out empty strings
    .map(word =>
      word.charAt(0).toUpperCase() +
      (word.length > 1 ? word.slice(1) : '')
    )
    .join(' ');
  }
}

```

Key Features:

1. Type Safety: Uses TypeScript typing
2. Null Checking: Handles null/undefined inputs
3. Multiple Spaces: Properly handles multiple spaces between words
4. Single Character Words: Works with single-character words
5. Locale Awareness: Considers lowercase/uppcase conversions

Testing the Pipe

Here's a simple Jasmine test:

```

describe('CapitalizePipe', () => {
  const pipe = new CapitalizePipe();

  it('transforms "hello world" to "Hello World"', () => {
    expect(pipe.transform('hello world')).toBe('Hello World');
  });

  it('handles multiple spaces', () => {
    expect(pipe.transform('hello  world')).toBe('Hello World');
  });

  it('handles empty string', () => {
    expect(pipe.transform('')).toBe('');
  });

  it('handles null input', () => {
    expect(pipe.transform(null)).toBeNull();
  });
});

```

This implementation gives you a production-ready custom pipe that properly capitalizes each word while handling common edge cases.

2. Implement a Simple Custom Directive

Question: Write a custom directive called `highlightOnHover` that changes the background color of an element when hovered.

Answer:

1. Generate the Directive

First, generate the directive using Angular CLI:

```
ng generate directive highlightOnHover
```

2. Implement the Directive Logic

Here's the implementation for `highlight-on-hover.directive.ts`:

```
import { Directive, ElementRef, HostListener, Input, Renderer2 } from
 '@angular/core';

@Directive({
  selector: '[appHighlightOnHover]' // Using app prefix for custom directives
})
export class HighlightOnHoverDirective {
  // Default color if none is provided
  @Input() highlightColor: string = 'yellow';
  // Optional: color to return to when mouse leaves
  @Input() defaultColor: string = '';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.highlight(this.highlightColor);
  }

  @HostListener('mouseleave') onMouseLeave() {
```

```

    this.highlight(this.defaultColor);
  }

  private highlight(color: string) {
    // Using Renderer2 for safer DOM manipulation
    this.renderer.setStyle(this.el.nativeElement, 'background-color', color);

    // Optional: Add transition for smooth effect
    this.renderer.setStyle(this.el.nativeElement, 'transition',
'background-color 0.3s ease');
  }
}

```

3. Add to Your Module

The directive will be automatically declared in your module by the CLI, but verify:

```

@NgModule({
  declarations: [
    // ...
    HighlightOnHoverDirective
  ],
  // ...
})
export class AppModule { }

```

4. Use in Templates

Now you can use it in your templates:

```

<!-- Basic usage with default color (yellow) -->
<p appHighlightOnHover>Hover over me!</p>

<!-- With custom highlight color -->
<div appHighlightOnHover highlightColor="lightblue">Hoverable div</div>

<!-- With both highlight and default colors -->
<span
  appHighlightOnHover
  highlightColor="#ffcccb"

```



```
    defaultColor="white">
    Hover over this span
</span>
```

5. Enhanced Version (Optional)

For more flexibility, you could add:

```
@Input() highlightClass: string; // For CSS class instead of inline style

@HostListener('mouseenter') onMouseEnter() {
  if (this.highlightClass) {
    this.renderer.addClass(this.el.nativeElement, this.highlightClass);
  } else {
    this.highlight(this.highlightColor);
  }
}
```

Then use with CSS classes:

```
<p appHighlightOnHover highlightClass="hover-highlight">Styled with CSS</p>

.hover-highlight {
  background-color: lightgreen;
  transition: all 0.3s ease;
}
```

Key Features:

1. Renderer2: Safer DOM manipulation than direct native element access
2. Customizable Colors: Input properties for flexibility
3. Host Listeners: Clean event handling
4. Transition Effects: Smooth color changes
5. CSS Class Option: More styling flexibility

Testing the Directive

Here's a simple test:

```
describe('HighlightOnHoverDirective', () => {
```

```

let fixture: ComponentFixture<TestComponent>;
let testComponent: TestComponent;
let element: HTMLElement;

beforeEach(() => {
  TestBed.configureTestingModule({
    declarations: [HighlightOnHoverDirective, TestComponent]
  });

  fixture = TestBed.createComponent(TestComponent);
  testComponent = fixture.componentInstance;
  element = fixture.debugElement.nativeElement.querySelector('p');
  fixture.detectChanges();
});

it('should change background on mouseenter', () => {
  const event = new Event('mouseenter');
  element.dispatchEvent(event);
  expect(element.style.backgroundColor).toBe('yellow');
});

it('should revert on mouseleave', () => {
  const enterEvent = new Event('mouseenter');
  const leaveEvent = new Event('mouseleave');
  element.dispatchEvent(enterEvent);
  element.dispatchEvent(leaveEvent);
  expect(element.style.backgroundColor).toBe('');
});
});

@Component({
  template: `<p appHighlightOnHover>Test</p>`
})
class TestComponent {}

```

This implementation provides a reusable, flexible directive that can be easily customized for different hover effects throughout your application.

3. Set Up Reactive Form with Validation

Question: Create a simple reactive form for a login page with fields for email and password. Add validation to require both fields, and validate that the email field has a valid email format.

#4. Create a Simple HTTP Service

Answer:

1. Set up the Reactive Form Module

First, import `ReactiveFormsModule` in your module:

```
import { ReactiveFormsModule } from '@angular/forms';

@NgModule({
  imports: [
    // ...
    ReactiveFormsModule
  ],
  // ...
})
export class AppModule { }
```

2. Create the Login Component

Here's the implementation for `login.component.ts`:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html'
})
export class LoginComponent {
  loginForm: FormGroup;
```

```

submitted = false;

constructor(private fb: FormBuilder) {
  this.loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', Validators.required]
  });
}

get f() { return this.loginForm.controls; }

onSubmit() {
  this.submitted = true;

  if (this.loginForm.invalid) {
    return;
  }

  console.log('Form submitted', this.loginForm.value);
  // Call your auth service here
}
}

```

3. Create the Template (login.component.html)

```

<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <div class="form-group">
    <label for="email">Email</label>
    <input
      type="email"
      id="email"
      formControlName="email"
      class="form-control"
      [ngClass]="{ 'is-invalid': submitted && f.email.errors }">

    <div *ngIf="submitted && f.email.errors" class="invalid-feedback">
      <div *ngIf="f.email.errors.required">Email is required</div>
      <div *ngIf="f.email.errors.email">Email must be a valid email address</div>
    </div>
  </div>
</form>

```

```

<div class="form-group">
  <label for="password">Password</label>
  <input
    type="password"
    id="password"
    formControlName="password"
    class="form-control"
    [ngClass]="{ 'is-invalid': submitted && f.password.errors }">

    <div *ngIf="submitted && f.password.errors" class="invalid-feedback">
      <div *ngIf="f.password.errors.required">Password is required</div>
    </div>
  </div>

  <button type="submit" class="btn btn-primary">Login</button>
</form>

```

14. Simple HTTP Service

1. Generate the Service

ng generate service auth

2. Implement the HTTP Service (auth.service.ts)

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private apiUrl = 'https://your-api-endpoint.com/auth';

  constructor(private http: HttpClient) { }

```

```

login(email: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/login`, { email, password });
}

// Add other auth-related methods as needed
register(userData: any): Observable<any> {
    return this.http.post(`${this.apiUrl}/register`, userData);
}
}

```

3. Update the Login Component to Use the Service

Modify `login.component.ts`:

```

import { AuthService } from './auth.service';

// ... in the component class
constructor(private fb: FormBuilder, private authService: AuthService) { }

onSubmit() {
    this.submitted = true;

    if (this.loginForm.invalid) {
        return;
    }

    this.authService.login(
        this.loginForm.value.email,
        this.loginForm.value.password
    ).subscribe({
        next: (response) => {
            console.log('Login successful', response);
            // Handle successful login (redirect, store token, etc.)
        },
        error: (error) => {
            console.error('Login failed', error);
            // Handle error (show message to user)
        }
    });
}

```

```
}
```

4. Don't Forget to Import HttpClientModule

In your `app.module.ts`:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [
    // ...
    HttpClientModule
  ],
  // ...
})
export class AppModule { }
```

Key Features:

For the Reactive Form:

1. Built with `FormBuilder` for cleaner syntax
2. Validators for required fields and email format
3. Proper error message display
4. Submission handling with flags
5. Bootstrap-compatible classes (can be replaced with your preferred CSS framework)

For the HTTP Service:

1. Proper service structure with dependency injection
2. TypeScript interfaces for type safety
3. Observable-based HTTP calls
4. Centralized API endpoint management
5. Error handling in the component

Testing the Form:

You can add this to your `login.component.spec.ts`:

```
it('should validate email format', () => {
  const email = component.loginForm.controls['email'];
  email.setValue('invalid-email');
  expect(email.valid).toBeFalse();
  expect(email.errors?.email).toBeTruthy();

  email.setValue('valid@example.com');
  expect(email.valid).toBeTruthy();
});

it('should require password', () => {
  const password = component.loginForm.controls['password'];
  expect(password.valid).toBeFalse();

  password.setValue('12345');
  expect(password.valid).toBeTruthy();
});
```

This implementation provides a complete solution for a login form with validation and service integration following Angular best practices.

4. Create a Simple Component with Input and Output

Question: Create a component called CounterComponent that has an input to set an initial count and an output event to emit the updated count each time a button is clicked to increment the count.

Here's a complete implementation of a `CounterComponent` that accepts an initial count as input and emits updated counts:

1. Generate the Component

ng generate component Counter

2. Implement the Component (counter.component.ts)

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-counter',
  templateUrl: './counter.component.html',
  styleUrls: ['./counter.component.css']
})
export class CounterComponent {
  @Input() initialCount: number = 0; // Default to 0 if no input provided
  @Output() countChanged = new EventEmitter<number>();

  currentCount: number;

  constructor() {
    this.currentCount = this.initialCount;
  }

  increment() {
    this.currentCount++;
    this.countChanged.emit(this.currentCount);
  }
}
```

3. Create the Template (counter.component.html)

```
<div class="counter">
  <h3>Current Count: {{ currentCount }}</h3>
  <button (click)="increment()">Increment</button>
</div>
```

4. Optional Styling (counter.component.css)

```
.counter {  
  border: 1px solid #ddd;  
  padding: 1rem;  
  margin: 1rem 0;  
  border-radius: 4px;  
  text-align: center;  
}  
  
button {  
  padding: 0.5rem 1rem;  
  background-color: #007bff;  
  color: white;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}  
  
button:hover {  
  background-color: #0056b3;  
}
```

5. Using the Component (parent component)

Here's how you would use the CounterComponent in a parent component:

Parent Component Template

```
<h2>Counter Example</h2>  
<app-counter  
  [initialCount]="5"  
  (countChanged)="onCountChanged($event)">  
</app-counter>  
<p *ngIf="currentValue">Current value from counter: {{ currentValue }}</p>
```

Parent Component Class

```
import { Component } from '@angular/core';
```

```

@Component({
  selector: 'app-parent',
  templateUrl: './parent.component.html'
})
export class ParentComponent {
  currentValue: number;

  onCountChanged(newCount: number) {
    this.currentValue = newCount;
    console.log('Count changed:', newCount);
  }
}

```

Key Features

1. Input Property:
 - `@Input() initialCount` allows parent components to set the starting value
 - Defaults to 0 if no value is provided
2. Output Event:
 - `@Output() countChanged` emits the new count whenever it changes
 - Uses `EventEmitter<number>` for type safety
3. Internal State:
 - Maintains `currentCount` to track the counter value
 - Initialized from `initialCount` in the constructor
4. Clean Template:
 - Simple display of current count
 - Button to trigger increment
5. Type Safety:
 - Strong typing throughout (numbers for counts)

Testing the Component

Here's a simple test for the CounterComponent:

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CounterComponent } from './counter.component';

```

```

describe('CounterComponent', () => {
  let component: CounterComponent;
  let fixture: ComponentFixture<CounterComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [CounterComponent]
    }).compileComponents();
  });

  beforeEach(() => {
    fixture = TestBed.createComponent(CounterComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should initialize with input value', () => {
    component.initialCount = 5;
    component.ngOnInit();
    expect(component.currentCount).toBe(5);
  });

  it('should increment count and emit event', () => {
    spyOn(component.countChanged, 'emit');
    component.currentCount = 1;
    component.increment();
    expect(component.currentCount).toBe(2);
    expect(component.countChanged.emit).toHaveBeenCalledWith(2);
  })
});

```

This implementation demonstrates Angular's component communication patterns using `@Input` and `@Output`, following best practices for component design.

5. Write a function in TypeScript to filter an array of objects based on a specific property value.

Here's a comprehensive solution for filtering an array of objects based on a specific property value:

Basic Implementation

```
/**
 * Filters an array of objects based on a property value
 * @param arr - Array of objects to filter
 * @param property - Property name to filter by
 * @param value - Value to match
 * @returns Filtered array of objects where property matches value
 */
function filterByProperty<T extends Record<string, any>>(
  arr: T[],
  property: keyof T,
  value: T[keyof T]
): T[] {
  return arr.filter(item => item[property] === value);
}
```

Usage Example

```
interface User {
  id: number;
  name: string;
  age: number;
  active: boolean;
}
```

```

const users: User[] = [
  { id: 1, name: 'Alice', age: 25, active: true },
  { id: 2, name: 'Bob', age: 30, active: false },
  { id: 3, name: 'Charlie', age: 25, active: true }
];

// Filter users with age 25
const age25Users = filterByProperty(users, 'age', 25);
console.log(age25Users);
/* Output:
[
  { id: 1, name: 'Alice', age: 25, active: true },
  { id: 3, name: 'Charlie', age: 25, active: true }
]
*/

// Filter active users
const activeUsers = filterByProperty(users, 'active', true);
console.log(activeUsers);
/* Output:
[
  { id: 1, name: 'Alice', age: 25, active: true },
  { id: 3, name: 'Charlie', age: 25, active: true }
]
*/

```

Advanced Versions

1. Case-Insensitive String Matching

```

function filterByProperty<T>(
  arr: T[],
  property: keyof T,
  value: T[keyof T],
  options?: { caseInsensitive?: boolean }
): T[] {

```

```

    return arr.filter(item => {
      const itemValue = item[property];
      if (options?.caseInsensitive && typeof itemValue === 'string' && typeof
value === 'string') {
        return itemValue.toLowerCase() === value.toLowerCase();
      }
      return itemValue === value;
    });
  }
}

```

// Usage

```

const names = [
  { name: 'Alice' },
  { name: 'alice' },
  { name: 'Bob' }
];

const alices = filterByProperty(names, 'name', 'ALICE', { caseInsensitive:
true });

```

2. Deep Property Filtering (using dot notation)

```

function filterByNestedProperty<T>(
  arr: T[],
  propertyPath: string,
  value: any
): T[] {
  return arr.filter(item => {
    const properties = propertyPath.split('.');
    let currentValue: any = item;

    for (const prop of properties) {
      if (currentValue === undefined || currentValue === null) return false;
      currentValue = currentValue[prop];
    }

    return currentValue === value;
  });
}

```

```
}
```

```
// Usage
```

```
const data = [
  { user: { name: 'Alice', settings: { darkMode: true } } },
  { user: { name: 'Bob', settings: { darkMode: false } } }
];
const darkModeUsers = filterByNestedProperty(data, 'user.settings.darkMode',
true);
```

3. Partial Matching with Custom Comparator

```
function filterByProperty<T>(  
  arr: T[],  
  property: keyof T,  
  value: T[keyof T],  
  comparator?: (a: T[keyof T], b: T[keyof T]) => boolean  
) : T[] {  
  const compare = comparator || ((a, b) => a === b);  
  return arr.filter(item => compare(item[property], value));  
}
```

```
// Usage
```

```
const numbers = [  
  { value: 5 },  
  { value: 10 },  
  { value: 15 }  
];  
const greaterThan8 = filterByProperty(  
  numbers,  
  'value',  
  8,  
  (a, b) => a > b  
);
```

Type Safety Features

1. Generic Type Parameter (T): Ensures type safety between the array elements and property access
2. keyof T: Restricts the property parameter to only valid keys of the object type
3. T[keyof T]: Ensures the value parameter matches the type of the specified property
4. Record<string, any>: Constraint ensures we're working with objects

Edge Case Handling

The basic implementation handles:

- Empty arrays
- Missing properties (returns false for filter)
- Null/undefined values in the array

For more robust production code, you might want to add:

- Input validation
- Error handling for invalid property paths
- Optional null checks
- Customizable comparison logic

This implementation provides a flexible, type-safe way to filter arrays of objects by property values in TypeScript.

6. Debug this code snippet: let a = 10; const b; if (a == b) { console.log('Equal'); }

Issues in the Code

1. Uninitialized Constant:
2. `const b;` *// Error: Missing initializer in const declaration*
 - `const` variables must be initialized when declared

- Unlike `let`, they can't be declared without a value
- 3. TypeScript Strict Equality:
 - 4. `if (a == b) // Using loose equality (==)`
 - This would work but is generally discouraged in TypeScript
 - Should use strict equality (`===`) to avoid type coercion
 - 5. Missing Else Case (optional improvement):
 - No handling for when `a` and `b` are not equal

Corrected Code

Here are two ways to fix this:

Option 1: Initialize `b` (if you meant to use `const`)

```
let a = 10;
const b = 10; // Initialize with a value

if (a === b) { // Use strict equality
  console.log('Equal');
} else {
  console.log('Not equal');
}
```

Option 2: Use `let` for `b` if you need to assign later

```
let a = 10;
let b: number; // Declare without initialization

// ... some code that assigns b ...

b = 10; // Assign a value before comparison

if (a === b) {
  console.log('Equal');
}
```

```
} else {  
  console.log('Not equal');  
}
```

Key Improvements

1. Fixed the `const` initialization error
2. Added type safety (explicit `number` type in Option 2)
3. Used strict equality (`===`) instead of loose equality (`==`)
4. Added optional `else` case for better control flow
5. Made the code more maintainable and type-safe

Additional Recommendations

1. If using TypeScript, enable strict mode in `tsconfig.json`:

```
{  
  "compilerOptions": {  
    "strict": true  
  }  
}
```

2. }
This would catch the uninitialized `const` error at compile time.
3. Consider using TypeScript's type annotations:

```
let a: number = 10;  
  
const b: number = 10;
```

4. For more complex comparisons, consider using a type guard function:

```
function isEqual(x: number, y: number): boolean {  
  return x === y;  
}
```

```
5. if (isEqual(a, b)) { ... }
```

The fixed code is now type-safe and follows TypeScript best practices.

7. Explain how to handle HTTP requests in Angular using HttpClient.

Angular's `HttpClient` is a powerful, built-in service for making HTTP requests. Here's a comprehensive guide to using it effectively:

1. Setup HttpClient

First, import `HttpClientModule` in your root module:

```
import { HttpClientModule } from '@angular/common/http';
```

```
@NgModule({  
  imports: [  
    HttpClientModule  
    // other imports  
  ]  
})  
export class AppModule { }
```

2. Basic HTTP Methods

GET Request

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class DataService {
  constructor(private http: HttpClient) {}

  // Simple GET
  getItems(): Observable<any[]> {
    return this.http.get<any[]>('/api/items');
  }

  // GET with parameters
  getItem(id: number): Observable<any> {
    return this.http.get<any>(`/api/items/${id}`);
  }

  // GET with query params
  searchItems(term: string): Observable<any[]> {
    return this.http.get<any[]>('/api/items/search', {
      params: { q: term }
    });
  }
}
```

POST/PUT/DELETE Requests

```
// POST
addItem(item: any): Observable<any> {
  return this.http.post('/api/items', item);
}
```

```
// PUT
updateItem(id: number, item: any): Observable<any> {
  return this.http.put(`/api/items/${id}`, item);
}

// DELETE
deleteItem(id: number): Observable<void> {
  return this.http.delete<void>(`/api/items/${id}`);
}
```

3. Type Safety with Interfaces

```
interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable()
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}
```

4. Error Handling

Basic Error Handling

```
getItems(): Observable<Item[]> {
  return this.http.get<Item[]>('/api/items').pipe(
    catchError((error: HttpResponse) => {
```

```

        console.error('Error occurred:', error);
        return throwError(() => new Error('Something went wrong'));
    })
);
}

```

Advanced Error Handling

```

private handleError(error: HttpResponse) {
    if (error.status === 0) {
        // Client-side or network error
        console.error('Network error:', error.error);
        return throwError(() => new Error('Network error occurred'));
    } else {
        // Backend returned unsuccessful response
        console.error(`Backend returned code ${error.status}, body was:`,
error.error);
        return throwError(() => new Error('Request failed'));
    }
}

getItem(): Observable<Item[]> {
    return this.http.get<Item[]>('/api/items').pipe(
        catchError(this.handleError)
    );
}

```

5. Interceptors (For Global Request/Response Handling)

```

import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from
'@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
    intercept(req: HttpRequest<any>, next: HttpHandler):

```

```

Observable<HttpEvent<any>> {
    // Clone the request and add auth header
    const authReq = req.clone({
        headers: req.headers.set('Authorization', 'Bearer token')
    });

    return next.handle(authReq);
}
}

```

Register the interceptor in your module:

```

@NgModule({
    providers: [
        { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
    ]
})
export class AppModule { }

```

6. Best Practices

1. Service Layer Pattern: Always wrap HTTP calls in services
2. Type Safety: Use interfaces for request/response types
3. Error Handling: Handle errors at both service and component levels
4. Unsubscribe: Use `async` pipe or manually unsubscribe from observables
5. Loading States: Track loading state for better UX

```

// Example with loading state
@Injectable()
export class DataService {
    private loading = new BehaviorSubject<boolean>(false);
    public loading$ = this.loading.asObservable();

    getItems(): Observable<Item[]> {
        this.loading.next(true);
        return this.http.get<Item[]>('/api/items').pipe(

```



```

        finalize(() => this.loading.next(false)),
        catchError(this.handleError)
    );
}
}

```

7. Advanced Features

Progress Events

```

uploadFile(file: File): Observable<any> {
    const formData = new FormData();
    formData.append('file', file);

    return this.http.post('/api/upload', formData, {
        reportProgress: true,
        observe: 'events'
    }).pipe(
        filter(event => event.type === HttpEventType.UploadProgress),
        map(event => {
            return {
                progress: Math.round(100 * event.loaded / (event.total || 1))
            };
        })
    );
}

```

Canceling Requests

```

private cancelRequest = new Subject<void>();

search(term: string): Observable<any> {
    // Cancel previous request
    this.cancelRequest.next();
}

```

```

return this.http.get('/api/search', {
  params: { q: term }
}).pipe(
  takeUntil(this.cancelRequest)
);
}

```

Angular's HttpClient provides a robust, type-safe way to handle HTTP communications with RxJS-powered features for advanced scenarios.

8. Write a simple Angular component to display a list of items.

Here's a complete implementation of an Angular component that displays a list of items:

1. Generate the Component

```
ng generate component item-list
```

2. Component Implementation (item-list.component.ts)

```

import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-item-list',
  templateUrl: './item-list.component.html',
  styleUrls: ['./item-list.component.css']
})
export class ItemListComponent {
  // Input property to receive items from parent component
  @Input() items: any[] = [];

  // Optional: Input for customizing the list
  @Input() listTitle: string = 'Items List';
  @Input() emptyMessage: string = 'No items to display';
}

```

3. Template (item-list.component.html)

```
<div class="item-list-container">
  <h2>{{ listTitle }}</h2>

  <div *ngIf="items.length === 0" class="empty-message">
    {{ emptyMessage }}
  </div>

  <ul *ngIf="items.length > 0" class="item-list">
    <li *ngFor="let item of items" class="item">
      {{ item.name || item }} <!-- Display item.name if object, otherwise just item -->
    </li>
  </ul>
</div>
```

4. Styles (item-list.component.css)

```
.item-list-container {
  font-family: Arial, sans-serif;
  max-width: 600px;
  margin: 20px auto;
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.item-list {
  list-style-type: none;
  padding: 0;
}

.item {
  padding: 10px 15px;
  margin: 5px 0;
  background-color: #f8f9fa;
  border-radius: 4px;
  transition: background-color 0.3s ease;
}
```

```

.item:hover {
  background-color: #e9ecef;
}

.empty-message {
  color: #6c757d;
  font-style: italic;
  padding: 10px;
}

```

5. Using the Component (Parent Component Example)

Parent Component Template

```

<h1>My Application</h1>

<!-- Simple string array -->
<app-item-list
  [items]="['Apple', 'Banana', 'Orange']"
  listTitle="Fruits">
</app-item-list>

<!-- Object array -->
<app-item-list
  [items]="products"
  listTitle="Products"
  emptyMessage="No products available">
</app-item-list>

```

Parent Component Class

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  templateUrl: './parent.component.html'
})
export class ParentComponent {
  products = [
    { id: 1, name: 'Laptop', price: 999 },
    { id: 2, name: 'Smartphone', price: 699 },
    { id: 3, name: 'Headphones', price: 149 }
  ]
}

```

```
];  
}
```

Features of This Implementation

1. Flexible Input: Accepts both arrays of strings and arrays of objects
2. Customizable Display:
 - Title can be set via `listTitle`
 - Custom empty message via `emptyMessage`
3. Responsive Design: Clean styling with hover effects
4. Conditional Rendering:
 - Shows empty message when array is empty
 - Only renders list when items exist
5. Type Safety: Uses Angular's template syntax for safe rendering

Enhanced Version (Optional)

For more complex items, you could use content projection:

```
<!-- In item-list.component.html -->  
<ul *ngIf="items.length > 0" class="item-list">  
  <ng-container *ngFor="let item of items">  
    <ng-content select="[itemTemplate]" [item]="item"></ng-content>  
  </ng-container>  
</ul>  
  
<!-- In parent component -->  
<app-item-list [items]="products">  
  <div *itemTemplate="let product" class="product-item">  
    <h3>{{ product.name }}</h3>  
    <p>${{ product.price }}</p>  
  </div>  
</app-item-list>
```

This component provides a reusable, well-structured way to display lists in your Angular application with minimal setup required.

9. How would you optimize a slow Angular application?

Here's a comprehensive approach to diagnosing and optimizing Angular application performance:

1. Diagnose Performance Issues

Tools to identify bottlenecks:

- Angular DevTools Profiler
- Chrome DevTools Performance tab
- Lighthouse audits
- Webpack Bundle Analyzer (`ng build --stats-json`)

2. Change Detection Optimization

```
// Use OnPush change detection strategy
@Component({
  selector: 'app-example',
  changeDetection: ChangeDetectionStrategy.OnPush
})
```

Key techniques:

- Mark components with `OnPush` where possible
- Use immutable data structures or `trackBy` with `*ngFor`
- Detach change detectors for static components
- Avoid async pipes in deeply nested templates

3. Bundle Size Reduction

Implement lazy loading:

```
const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
```

```
}  
];
```

Additional bundle optimizations:

- Run production builds (`ng build --prod`)
- Enable differential loading for modern browsers
- Identify and remove unused dependencies
- Consider feature modules for better code splitting

4. Performance-Critical Code Optimization

Optimize templates:

```
<!-- Use trackBy with ngFor -->  
<div *ngFor="let item of items; trackBy: trackById">
```

Avoid common pitfalls:

- Replace complex template expressions with pure pipes
- Minimize DOM manipulations
- Debounce rapid events (scroll, resize, input)
- Virtual scrolling for large lists (`<cdk-virtual-scroll-viewport>`)

5. Network Performance

Implement:

- HTTP caching strategies
- Service worker for PWA apps
- Compression (Brotli > Gzip)
- Lazy load non-critical resources

Optimize API calls:

- Combine requests where possible
- Implement client-side caching
- Use interceptors for request deduplication

6. Memory Management

Prevent memory leaks:

- Always unsubscribe from observables

```
private destroy$ = new Subject<void>();
```

```
ngOnInit() {  
  this.someService.getData()  
    .pipe(takeUntil(this.destroy$))  
    .subscribe();  
}
```

```
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

- Clean up event listeners
- Avoid storing large objects in services

7. Advanced Optimizations

Consider:

- Web Workers for CPU-intensive tasks
- Server-side rendering (Angular Universal)
- Pre-rendering static pages
- Ivy's template type checking optimization (`strictTemplates: true`)

8. Monitoring and Maintenance

Implement:

- Performance budgets in Angular CLI
- Continuous performance testing
- Real user monitoring (RUM)

Quick Wins Checklist

1. Enable production mode (`enableProdMode()`)
2. Audit bundle with Webpack Analyzer
3. Implement lazy loading
4. Add `OnPush` to components
5. Add `trackBy` to all `*ngFor` loops
6. Review all subscriptions for proper cleanup
7. Optimize images and assets
8. Enable compression on server

Remember to measure before and after each optimization to verify impact.

Performance tuning should be data-driven, not based on assumptions. Start with the biggest bottlenecks first for maximum ROI.

10. Describe how you handle form validation in Angular.

Angular provides powerful tools for form validation with both template-driven and reactive approaches. Here's how I implement robust form validation:

1. Reactive Forms Approach (Recommended)

Basic Setup

```
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {
  userForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.userForm = this.fb.group({
```

```

    name: ['', [Validators.required, Validators.minLength(3)]],
    email: ['', [Validators.required, Validators.email]],
    password: ['', [
      Validators.required,
      Validators.pattern('^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)[a-zA-Z\\d]{8,}$')
    ]],
    age: [null, [Validators.min(18), Validators.max(99)]]
  });
}

get f() { return this.userForm.controls; }

onSubmit() {
  if (this.userForm.invalid) {
    this.markFormGroupTouched(this.userForm);
    return;
  }
  // Form is valid - proceed
}

private markFormGroupTouched(formGroup: FormGroup) {
  Object.values(formGroup.controls).forEach(control => {
    control.markAsTouched();
    if (control instanceof FormGroup) {
      this.markFormGroupTouched(control);
    }
  });
}
}

```

Template Implementation

```

<form [formGroup]="userForm" (ngSubmit)="onSubmit()">
  <!-- Name Field -->
  <div class="form-group">
    <label>Name</label>
    <input formControlName="name" [ngClass]="{
      'is-invalid': f.name.touched && f.name.errors,
      'is-valid': f.name.touched && !f.name.errors
    }">
  </div>

```

```

    <div *ngIf="f.name.touched && f.name.errors" class="invalid-feedback">
      <div *ngIf="f.name.errors.required">Name is required</div>
      <div *ngIf="f.name.errors.minlength">
        Name must be at least {{f.name.errors.minlength.requiredLength}}
characters
      </div>
    </div>
  </div>
</div>

<!-- Email Field -->
<div class="form-group">
  <label>Email</label>
  <input type="email" formControlName="email">
  <div *ngIf="f.email.touched && f.email.errors" class="invalid-feedback">
    <div *ngIf="f.email.errors.required">Email is required</div>
    <div *ngIf="f.email.errors.email">Enter a valid email</div>
  </div>
</div>

  <button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>

```

2. Template-Driven Forms Approach

```

@Component({
  selector: 'app-template-form',
  templateUrl: './template-form.component.html'
})
export class TemplateFormComponent {
  model = {
    username: '',
    email: ''
  };

  onSubmit(form: NgForm) {
    if (form.invalid) return;
    // Handle valid form submission
  }
}

```

```

<form #form="ngForm" (ngSubmit)="onSubmit(form)">
  <input
    name="username"
    [(ngModel)]="model.username"
    #username="ngModel"
    required
    minlength="3">

  <div *ngIf="username.touched && username.errors">
    <div *ngIf="username.errors.required">Username required</div>
    <div *ngIf="username.errors.minlength">
      Minimum {{username.errors.minlength.requiredLength}} characters
    </div>
  </div>

  <button type="submit" [disabled]="form.invalid">Submit</button>
</form>

```

3. Custom Validators

// Custom validator function

```

export function forbiddenNameValidator(nameRe: RegExp): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const forbidden = nameRe.test(control.value);
    return forbidden ? { forbiddenName: { value: control.value } } : null;
  };
}

```

// Usage in component

```

this.userForm = this.fb.group({
  name: ['', [
    Validators.required,
    forbiddenNameValidator(/admin/i)
  ]]
});

```

// Async validator example

```

export function uniqueEmailValidator(userService: UserService): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    return userService.checkEmailExists(control.value).pipe(

```

```

        map(exists => exists ? { emailExists: true } : null),
        catchError(() => of(null))
    );
};
}

```

4. Cross-Field Validation

// In component

```

this.userForm = this.fb.group({
  password: ['', Validators.required],
  confirmPassword: ['', Validators.required]
}, { validator: this.passwordMatchValidator });

```

```

passwordMatchValidator(form: FormGroup) {
  return form.get('password').value === form.get('confirmPassword').value
    ? null : { mismatch: true };
}

```

// In template

```

<div *ngIf="userForm.errors?.mismatch &&
  (userForm.touched || userForm.dirty)"
  class="cross-validation-error">
  Passwords don't match
</div>

```

5. Dynamic Validation

// Changing validation rules dynamically

```

togglePasswordValidation() {
  const passwordControl = this.userForm.get('password');
  if (this.requireStrongPassword) {
    passwordControl.setValidators([
      Validators.required,
      Validators.minLength(8),
      Validators.pattern(strongPasswordRegex)
    ]);
  } else {
    passwordControl.setValidators(Validators.required);
  }
}

```

```
passwordControl.updateValueAndValidity();  
}
```

Best Practices

1. Use Reactive Forms for complex forms and dynamic validation
2. Centralize validation messages using a service or shared component
3. Implement debounce for async validators that hit APIs
4. Consider form state management for multi-step forms
5. Use typed forms (Angular 14+) for better type safety
6. Test validators in isolation
7. Accessibility - Ensure error messages are properly associated with fields

Advanced Techniques

- Dynamic form generation from JSON schemas
- Custom validation components that encapsulate validation logic
- Form arrays for repeatable sections
- Control value accessors for custom form controls
- State management integration for complex form state

Angular's validation system is comprehensive when properly leveraged, providing both developer control and excellent user feedback capabilities.

11. What is a Single Page Application (SPA), and how does Angular support this architecture?

What is a Single Page Application (SPA)?

A Single Page Application (SPA) is a web application that:

- Loads a single HTML page initially
- Dynamically updates content as users interact with it
- Avoids full page reloads (no browser flash between pages)
- Communicates with backend servers via AJAX/HTTP calls
- Maintains application state on the client side

Key SPA Characteristics:

1. Client-side rendering: The browser handles page transitions and rendering
2. Frontend routing: URL changes don't trigger server requests
3. State management: Application state persists between views
4. API-driven: Separates frontend from backend services

How Angular Supports SPA Architecture

1. Component-Based Architecture

Angular's fundamental building blocks perfectly suit SPAs:

```
@Component({
  selector: 'app-root',
  template: `
    <app-navbar></app-navbar>
    <router-outlet></router-outlet>  <!-- Dynamic content goes here -->
    <app-footer></app-footer>
  `
})
export class AppComponent {}
```

2. Router Module (Core SPA Feature)

Angular's powerful router enables client-side navigation:

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'products', component: ProductListComponent },
  { path: 'products/:id', component: ProductDetailComponent },
  { path: '**', component: PageNotFoundComponent }  // 404 handling
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

3. Lazy Loading (Performance Optimization)

Load features on-demand to reduce initial bundle size:

```
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.module').then(m =>
m.AdminModule)
}
```

4. HTTP Client (API Communication)@Injectable()

```
export class DataService {
  constructor(private http: HttpClient) {}

  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>('/api/products');
  }
}
```

5. State Management Options

Angular provides multiple approaches for SPA state management:

- Services + RxJS (basic state)
- NgRx (Redux pattern)
- Akita (alternative state management)
- Component Store (local component state)

6. Change Detection (SPA Performance)

Optimized strategies for dynamic UI updates:

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush // More efficient
})
export class ProductComponent {}
```

7. Universal Support (SSR)

For SEO and performance:

```
ng add @nguniversal/express-engine # Adds server-side rendering
```

Angular's SPA Advantages

- 1. Modular Architecture: Feature modules organize code
- 2. Dependency Injection: Manages service dependencies
- 3. Two-Way Data Binding: Simplifies state synchronization
- 4. Type Safety: TypeScript reduces runtime errors
- 5. Tooling: CLI, DevTools, and testing utilities
- 6. Progressive Enhancement: Can be extended to PWA

Typical SPA Workflow in Angular

- 1. Initial Load: Browser requests `index.html`
- 2. Bootstrapping: Angular loads main bundle
- 3. Root Component: Renders base layout
- 4. Router: Determines initial view
- 5. API Calls: Fetches data as needed
- 6. Navigation: Updates URL and view without reload
- 7. Dynamic Updates: Only changes necessary DOM elements

Comparison with Traditional MPAs

Feature	SPA (Angular)	Multi-Page Application (MPA)
Page Loads	Single initial load	Full page reloads
UI Updates	Dynamic DOM manipulation	Server-rendered HTML

Routing	Client-side	Server-side
State Management	Client maintains state	Stateless (per request)
Development	More complex setup	Simpler initial structure
SEO	Requires SSR (Universal)	Naturally SEO-friendly

Angular provides a comprehensive framework specifically designed to build robust, maintainable SPAs with all the necessary tools integrated into its ecosystem.

12.What is standalone component

Standalone components are a significant innovation introduced in Angular 14 (released in 2022) that provide a simpler, more modular way to build Angular applications by reducing dependency on NgModules.

Key Characteristics of Standalone Components

1. No NgModule Requirement
 - Can exist independently without being declared in an NgModule
 - Don't need to belong to any module
2. Self-Contained
 - Explicitly declare their own dependencies
 - Manage their own imports

3. Progressive Architecture

- Can be mixed with module-based components
- Enable gradual adoption in existing apps

Creating a Standalone Component

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-standalone-example',
  standalone: true, // This makes it standalone
  imports: [CommonModule, RouterLink], // Explicitly declare needed
  directives/pipes
  template: `
    <h2>Standalone Component</h2>
    <button routerLink="/home">Go Home</button>
  `,
  styles: [`h2 { color: blue; }`]
})
export class StandaloneExampleComponent {
  // Component logic here
}
```

Key Benefits

1. Simplified Architecture
 - Eliminates much of the NgModule boilerplate
 - Reduces configuration overhead
2. Better Code Organization
 - Components become more self-describing
 - Clearer dependency declarations
3. Improved Performance
 - Enables more granular lazy loading
 - Smaller bundle sizes possible
4. Easier Component Reuse
 - Components can be truly portable
 - Simpler to share across projects

How to Use Standalone Components

1. Bootstrapping as Entry Point

```
// In main.ts
bootstrapApplication(StandaloneExampleComponent, {
  providers: [
    // Global providers here
  ]
});
```

2. Lazy Loading Routes

```
const routes: Routes = [{
  path: 'admin',
  loadComponent: () => import('./admin/admin.component').then(m =>
m.AdminComponent)
}];
```

3. Mixing with Modules

```
@Component({
  standalone: true,
  imports: [CommonModule, SomeFeatureModule], // Can import whole modules
  // ...
})
export class HybridComponent {}
```

Migration Considerations

1. Gradual Adoption

- Existing modules continue to work
- New components can be standalone

2. Dependency Management

- Must explicitly import all needed directives/pipes
- Common directives like `*ngIf` come from `CommonModule`

3. Provider Scope

- Providers are component-scoped by default
- Use `importProvidersFrom()` to migrate module providers

When to Use Standalone Components

1. New Projects - Ideal starting point for modern Angular apps
2. Shared UI Libraries - Better component encapsulation
3. Micro Frontends - Cleaner integration
4. Simple Features - When full modules are overkill

Comparison with Module-Based Components

Aspect	Standalone Components	Module-Based Components
Declaration	<code>standalone: true</code>	Declared in NgModule
Dependencies	Explicit imports	Inherited from module
Bootstrapping	Directly bootstrapable	Requires module
Bundle Size	More granular	Typically larger
Learning Curve	Simpler	More complex system
Provider Scope	Component-level	Module-level

Standalone components represent Angular's evolution toward simpler, more modular architecture while maintaining all the framework's powerful features. They're particularly valuable for new projects and are becoming the recommended approach in Angular's documentation

13.How do you ensure your web applications are responsive and adaptive?

To create truly responsive and adaptive web applications, I implement a multi-layered approach combining Angular features with modern CSS techniques and performance optimizations:

1. Core Implementation Strategies

Angular Layout System

```
// Component with responsive service injection
@Component({
  selector: 'app-responsive',
  template: `
    <div [class]="layoutClass$ | async">
      <!-- Adaptive content here -->
    </div>
  `,
})
export class ResponsiveComponent {
  layoutClass$ = this.breakpointService.currentBreakpoint.pipe(
    map(breakpoint => {
      switch(breakpoint) {
        case 'xs': return 'mobile-layout';
        case 'md': return 'tablet-layout';
        default: return 'desktop-layout';
      }
    })
  );
};
```

```

    constructor(private breakpointService: BreakpointService) {}
}

```

Breakpoint Service

```

@Injectable({ providedIn: 'root' })
export class BreakpointService {
    private breakpoints = {
        xs: '(max-width: 599px)',
        sm: '(min-width: 600px) and (max-width: 959px)',
        md: '(min-width: 960px) and (max-width: 1279px)',
        lg: '(min-width: 1280px)'
    };

    currentBreakpoint = new BehaviorSubject<string>('lg');

    constructor() {
        this.initializeBreakpointObserver();
    }

    private initializeBreakpointObserver() {
        Object.entries(this.breakpoints).forEach(([key, value]) => {
            const mediaQuery = window.matchMedia(value);
            if (mediaQuery.matches) this.currentBreakpoint.next(key);

            mediaQuery.addEventListener('change', (e) => {
                if (e.matches) this.currentBreakpoint.next(key);
            });
        });
    }
}

```

2. Adaptive Loading Techniques

Component-Level Adaptation

```

@Component({
    selector: 'app-adaptive-feature',
    template: `

```

```

    <ng-container *ngIf="(breakpointService.currentBreakpoint | async) !==
'xs' ">
      <app-desktop-feature></app-desktop-feature>
    </ng-container>
    <ng-container *ngIf="(breakpointService.currentBreakpoint | async) ===
'xs' ">
      <app-mobile-feature></app-mobile-feature>
    </ng-container>
  ,
})

```

Image Optimization

```

<picture>
  <source media="(min-width: 1200px)" srcset="large.jpg">
  <source media="(min-width: 768px)" srcset="medium.jpg">
  
</picture>

```

3. Performance-Critical Responsiveness

Virtual Scrolling

```

<cdk-virtual-scroll-viewport itemSize="50" class="viewport">
  <div *cdkVirtualFor="let item of items" class="item">
    {{item}}
  </div>
</cdk-virtual-scroll-viewport>

```

Conditional Bundling

```

// In route configuration
{
  path: 'data-heavy',
  loadChildren: () => import('./data-heavy.module').then(m =>
    matchMedia('(min-width: 1024px)').matches ?
      m.DesktopDataHeavyModule :
      m.MobileDataHeavyModule
  )
}

```


4. Advanced CSS Strategies

Container Queries

```
@container (min-width: 380px) {
  .card {
    grid-template-columns: 1fr 2fr;
  }
}
```

Logical Properties

```
.element {
  margin-inline: auto; /* Left/right in LTR, right/left in RTL */
  padding-block: 1rem; /* Top/bottom */
}
```

5. Testing and Validation

Automated Responsive Testing

```
// In component tests
describe('ResponsiveBehavior', () => {
  it('should show mobile layout on small screens', () => {
    viewport.set(375, 667);
    fixture.detectChanges();
    expect(element.query(By.css('.mobile-layout'))).toBeTruthy();
  });
});
```

Real Device Testing Matrix

Device Type	Viewport	OS Versions
iPhone 14 Pro	393×852	iOS 16+
Samsung Galaxy S22	360×780	Android 12+
iPad Air	820×1180	iPadOS 15+
Desktop Chrome	1440×900	Windows/Mac

6. Performance Optimization

Critical CSS Strategy\

```
// In angular.json
"styles": [
  {
    "input": "src/styles/critical.css",
    "bundleName": "critical",
    "inject": true
  },
  {
    "input": "src/styles/main.css",
    "bundleName": "styles",
    "inject": false
  }
]
```

Adaptive Preloading

```
// In app component
ngOnInit() {
  if (navigator.connection.effectiveType !== 'slow-2g') {
    this.preloadService.preloadModulesBasedOnViewport();
  }
}
```

Monitoring and Maintenance

RUM (Real User Monitoring)

```
// Collect responsive performance metrics
const perfObserver = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    if (entry.name === 'first-contentful-paint') {
      analytics.track('FCP', {
        value: entry.startTime,
        viewport: window.innerWidth,
        deviceType: this.getDeviceType()
      });
    }
  });
});
```

```
});  
  }  
});  
});
```

```
perfObserver.observe({ type: 'paint', buffered: true });
```

This comprehensive approach ensures applications not only respond to viewport changes but also adapt their functionality, asset loading, and even bundle delivery based on device capabilities, network conditions, and user context.

14.How do you optimize the performance of a web application?

Optimizing web application performance requires a multi-faceted strategy targeting various aspects of the application lifecycle. Here's my systematic approach:

1. Initial Load Optimization

Bundle Reduction

```
# Analyze bundle size  
ng build --stats-json && webpack-bundle-analyzer dist/stats.json
```

Key techniques:

- Enable production mode (ng build --prod)
- Implement lazy loading:

```
{  
  path: 'dashboard',
```

```
loadChildren: () => import('./dashboard/dashboard.module').then(m =>
m.DashboardModule)

}
```

- Use differential loading (target ES2015+ for modern browsers)
- Audit dependencies with `npm audit` and remove unused packages

Asset Optimization

- Compress images (WebP format with JPEG/PNG fallback)
- Implement lazy loading for images:

```
<img [src]="imageUrl" loading="lazy" alt="...">
```

- Use SVG sprites instead of icon fonts
- Serve static assets with CDN

2. Runtime Performance

Change Detection Optimization

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class OptimizedComponent {}
```

Additional techniques:

- Use `trackBy` with `*ngFor`:

```
<div *ngFor="let item of items; trackBy: trackById">
```

- Detach change detectors for static components
- Minimize DOM updates with `NgZone` optimization:

```
this.ngZone.runOutsideAngular(() => {  
  // Performance-critical code that doesn't need Angular's change detection  
  
});
```

Memory Management

```
// Proper observable cleanup  
private destroy$ = new Subject<void>();  
  
ngOnInit() {  
  this.dataService.getData()  
    .pipe(takeUntil(this.destroy$))  
    .subscribe();  
}  
  
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

3. Network Optimization

HTTP Performance

- Implement HTTP caching headers (Cache-Control, ETags)
- Use service workers for offline capability
- Compress responses (Brotli > Gzip)
- Bundle API requests where possible
- Implement request deduplication

Advanced Loading Strategies

```
// Preload critical routes  
@NgModule({  
  imports: [RouterModule.forRoot(routes, {  
    preloadingStrategy: PreloadAllModules // or custom strategy
```

```
    })]
  })
```

4. Rendering Optimization

Virtual Scrolling

```
<cdk-virtual-scroll-viewport itemSize="50">
  <div *cdkVirtualFor="let item of items">{{item}}</div>
</cdk-virtual-scroll-viewport>
```

Deferred Loading

```
<app-heavy-component *ngIf="isVisible"></app-heavy-component>
```

5. Monitoring and Maintenance

Performance Budgets

```
// angular.json
"budgets": [
  {
    "type": "initial",
    "maximumWarning": "500kb",
    "maximumError": "1mb"
  }
]
```

Real User Monitoring

```
// Track core web vitals
import { getCLS, getFID, getLCP } from 'web-vitals';

getCLS(console.log);
getFID(console.log);
getLCP(console.log);
```

Performance Optimization Checklist

1. Bundle Size
 - Lazy loaded modules
 - Tree-shaking enabled
 - Modern ES builds
 - Bundle analysis performed
2. Assets
 - Optimized images (WebP)
 - Font subsetting
 - SVG optimization
 - CDN utilization
3. Rendering
 - OnPush change detection
 - Virtual scrolling for large lists
 - TrackBy in ngFor
 - Minimal DOM manipulations
4. Network
 - HTTP caching configured
 - Compression enabled
 - Service worker implemented
 - API optimization
5. Monitoring
 - Performance budgets set
 - RUM implemented
 - Lighthouse scores tracked

By systematically applying these techniques, I've consistently achieved Lighthouse scores above 90 for performance while maintaining excellent developer experience and application maintainability. The key is continuous monitoring and incremental improvement rather than one-time optimizations.

15.What techniques do you use to ensure cross-browser compatibility?

To guarantee consistent user experiences across browsers, I implement a comprehensive approach combining proactive development practices, testing methodologies, and progressive enhancement:

1. Core Development Practices

Feature Detection with Modernizr

// Check for WebP support before loading images

```
if (Modernizr.webp) {  
  this.imageFormat = 'webp';  
} else {  
  this.imageFormat = 'jpg';  
}
```

CSS Normalization and Prefixing

PostCSS configuration for automatic prefixing

`npm install autoprefixer postcss-preset-env --save-dev`

postcss.config.js:

```
module.exports = {  
  plugins: [  
    require('autoprefixer')({  
      grid: 'autoplace'  
    }),  
    require('postcss-preset-env')({  
      stage: 3,  
      features: {
```



```
        'nesting-rules': true
      }
    })
  ]
}
```

2. JavaScript Compatibility Layer

Polyfill Strategy

```
// polyfills.ts - Selective importing
import 'core-js/features/array/flat-map';
import 'core-js/features/object/from-entries';
import 'whatwg-fetch'; // Polyfill for fetch API
```

Browserslist Configuration

```
// .browserslistrc
last 2 versions
> 1%
not IE 11
not dead
```

3. Testing Methodology

Automated Cross-Browser Testing

```
// karma.conf.js - Browser testing matrix
module.exports = function(config) {
  config.set({
    browsers: ['Chrome', 'Firefox', 'Safari', 'Edge'],
    customLaunchers: {
      ChromeHeadlessCI: {
        base: 'ChromeHeadless',
        flags: ['--no-sandbox']
      }
    }
  })
}
```

```
});  
};
```

Visual Regression Testing

```
# Using BackstopJS for visual comparisons  
backstop test --config=backstop.config.js
```

4. Responsive and Adaptive Techniques

CSS Feature Queries

```
@supports (display: grid) {  
  .container {  
    display: grid;  
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));  
  }  
}  
  
@supports not (display: grid) {  
  .container {  
    display: flex;  
    flex-wrap: wrap;  
  }  
}
```

JavaScript Fallbacks

```
// Check for IntersectionObserver support  
const lazyLoadImages = () => {  
  if ('IntersectionObserver' in window) {  
    // Modern lazy loading  
    const observer = new IntersectionObserver(callback, options);  
    images.forEach(img => observer.observe(img));  
  } else {  
    // Fallback lazy loading  
    window.addEventListener('scroll', throttle(() => {  
      checkVisibleImages();  
    }));  
  }  
}
```

```
    }, 200));  
  }  
};
```

5. Build-Time Optimization

Differential Bundling

```
// tsconfig.json  
{  
  "compilerOptions": {  
    "target": "es2015",  
    "module": "esnext"  
  }  
}
```

Asset Optimization Pipeline

```
// webpack.config.js - Asset rules  
{  
  test: /\. (png|jpe?g|gif)$/i,  
  use: [  
    {  
      loader: 'responsive-loader',  
      options: {  
        adapter: require('responsive-loader/sharp'),  
        sizes: [320, 640, 960, 1200],  
        format: 'webp'  
      }  
    }  
  ]  
}
```

6. Progressive Enhancement Framework

Core Experience First

```
<noscript>
  <div class="no-script-warning">
    <p>For full functionality, please enable JavaScript.</p>
  </div>
</noscript>
```

Feature Tiered Loading

```
// Load enhanced features after core verification
window.addEventListener('DOMContentLoaded', () => {
  if (verifyBrowserCapabilities()) {
    import('./enhanced-features.module')
      .then(module => module.initEnhancedFeatures());
  }
});
```

Cross-Browser Compatibility Checklist

1. Baseline Verification
 - Core functionality works without JavaScript
 - Semantic HTML structure validated
 - Basic CSS rendering verified
2. Modern Browser Support
 - CSS Grid/Flexbox fallbacks implemented
 - ES6+ features properly transpiled
 - Variable fonts with fallbacks
3. Legacy Browser Support
 - IE-specific issues addressed
 - Vendor prefixes applied
 - Polyfills for critical features
4. Testing Coverage
 - Automated cross-browser tests
 - Real device testing
 - Visual regression suite
5. Performance Considerations
 - Legacy bundles optimized
 - Modern bundles leveraging newest features
 - Asset loading strategies adapted per browser

This comprehensive approach ensures applications work across all target browsers while progressively enhancing the experience for modern browsers. The key is maintaining a balance between broad compatibility and leveraging modern web capabilities where available.

16.What are some common security concerns in front-end development, and how do you address them?

Front-end security is critical as client-side vulnerabilities often serve as entry points for attacks. Here's my comprehensive approach to addressing common security risks:

1. Cross-Site Scripting (XSS) Protection

Angular's Built-in Defenses

```
// Angular automatically sanitizes values bound to templates
@Component({
  template: `<div [innerHTML]="userContent"></div>`
})
export class SafeComponent {
  // Content will be sanitized
  userContent = '<script>maliciousCode()</script>';
}
```

Additional Measures

- Explicit Sanitization:

```
import { DomSanitizer } from '@angular/platform-browser';

constructor(private sanitizer: DomSanitizer) {
  this.trustedHtml = this.sanitizer.bypassSecurityTrustHtml(safeContent);
}
```

```
}
```

- Content Security Policy (CSP):

```
<!-- HTTP Header or meta tag -->
<meta http-equiv="Content-Security-Policy"

      content="default-src 'self'; script-src 'self' 'unsafe-inline'">
```

2. Cross-Site Request Forgery (CSRF)

Angular HTTP Client Protection

```
// Automatically handles CSRF tokens when using HttpClient
// Configure in app module:
@NgModule({
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: CsrfInterceptor, multi: true }
  ]
})
```

Custom Cookie Handling

```
// Ensure secure cookie flags
document.cookie = `sessionId=abc123; Secure; SameSite=Strict; HttpOnly`;
```

3. Injection Attacks

Template Security

```
// Never use concatenated strings in templates
// BAD:
template: '<div>' + userInput + '</div>'

// GOOD:
template: '<div>{{ userInput }}</div>' // Angular sanitizes
```

Safe URL Handling

```
// Sanitize URLs before binding
```

```
this.safeUrl = this.sanitizer.bypassSecurityTrustUrl(userInputUrl);
```

4. Authentication Vulnerabilities

JWT Security Practices

```
// Store tokens securely (not in localStorage)
// Use httpOnly, Secure cookies for tokens
@Injectable()
export class AuthService {
  private readonly TOKEN_KEY = 'auth_token';

  get token(): string | null {
    return this.cookieService.get(this.TOKEN_KEY);
  }
}
```

Session Management

```
// Implement token refresh and expiration checks
refreshToken(): Observable<string> {
  return this.http.post('/refresh', {}, { withCredentials: true })
    .pipe(
      tap(newToken => this.storeToken(newToken)),
      catchError(() => {
        this.logout();
        return throwError(() => new Error('Session expired'));
      })
    );
}
```

5. Third-Party Risks

Dependency Auditing

```
# Regularly check for vulnerabilities
npm audit
npx sbom@latest cyclonedx --output-format json
```

Sandboxed IFrames

```
<iframe
  src="https://third-party.com"
  sandbox="allow-scripts allow-same-origin"
  csp="script-src 'self'">
</iframe>
```

6. Data Exposure

Environment Security

```
// Never store secrets in frontend code
// Use environment variables properly
export const environment = {
  production: true,
  apiUrl: process.env['API_URL'] || 'https://api.example.com'
};
```

Secure Storage

```
// Use crypto-js for sensitive data
import * as CryptoJS from 'crypto-js';

encryptData(data: string): string {
  return CryptoJS.AES.encrypt(data, this.encryptionKey).toString();
}
```

7. Clickjacking Protection

Frame Busting

```
// In main component
ngOnInit() {
  if (window !== window.top) {
    window.top.location = window.location;
  }
}
```

HTTP Headers

X-Frame-Options: DENY

`Content-Security-Policy: frame-ancestors 'none'`

Security Implementation Checklist

1. Input Validation
 - All user inputs sanitized
 - Form validation on client and server
 - Output encoding applied
2. Authentication
 - Secure token storage
 - Session timeout implementation
 - Password policy enforcement
3. Dependencies
 - Regular vulnerability scans
 - Pinned dependency versions
 - Minimal third-party scripts
4. Transport Security
 - HTTPS enforced
 - HSTS header implemented
 - Secure cookies configured
5. Error Handling
 - Generic error messages
 - No stack traces exposed
 - Logging without sensitive data
6. Headers
 - CSP configured
 - XSS protection headers
 - MIME type validation

Continuous Security Practices

- Automated Scanning: Integrate OWASP ZAP or similar into CI/CD
- Security Headers: Regularly audit with securityheaders.com
- Penetration Testing: Schedule quarterly assessments
- Education: Conduct security training for developers

By implementing these measures systematically, I ensure defense-in-depth security for frontend applications while maintaining usability and performance. Security isn't a

one-time task but an ongoing process integrated into the development lifecycle

17.How do you manage asynchronous operations in JavaScript/TypeScript?

Asynchronous programming is fundamental in modern web development. Here's my comprehensive approach to handling async operations effectively:

Core Techniques

1. Promises

```
// Basic promise
const fetchData = (url: string): Promise<Response> => {
  return new Promise((resolve, reject) => {
    fetch(url)
      .then(response => {
        if (!response.ok) throw new Error('Network error');
        resolve(response.json());
      })
      .catch(reject);
  });
};
```

```
// Usage
fetchData('https://api.example.com/data')
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

2. Async/Await

```
// Async function with error handling
async function getUserData(userId: string) {
  try {
    const response = await fetch(`/api/users/${userId}`);
    if (!response.ok) throw new Error('Failed to fetch');
  }
}
```

```

    return await response.json();
  } catch (error) {
    console.error('Fetch error:', error);
    throw error; // Re-throw for further handling
  }
}

// Immediately Invoked Async Function
(async () => {
  const user = await getUserData('123');
  console.log(user);
})();

```

Advanced Patterns

3. Promise Combinators

```

// Parallel execution
const [user, posts] = await Promise.all([
  fetch('/user'),
  fetch('/posts')
]);

// First to complete wins
const winner = await Promise.race([
  fetch('/fast-endpoint'),
  timeout(5000) // Rejects after 5s
]);

// All settled (success or failure)
const results = await Promise.allSettled([
  fetch('/primary'),
  fetch('/fallback')
]);

```

4. RxJS Observables (Angular)

```

import { from, Observable } from 'rxjs';
import { catchError, retry, switchMap } from 'rxjs/operators';

```

```

getDataWithRetry(): Observable<any> {
  return from(fetch('/api/data')).pipe(
    retry(2), // Auto-retry on failure
    switchMap(response => response.json()),
    catchError(error => {
      console.error('Failed after retries:', error);
      return of(null); // Graceful fallback
    })
  );
}

```

```

// Usage in Angular component
this.data$ = this.dataService.getDataWithRetry();

```

Real-World Solutions

5. Cancellable Requests

```

// Using AbortController
const controller = new AbortController();

async function fetchWithTimeout(url: string, timeout = 5000) {
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  try {
    const response = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId);
    return response.json();
  } catch (error) {
    if (error.name === 'AbortError') {
      console.log('Request aborted');
    }
    throw error;
  }
}

// Cancel button implementation
document.getElementById('cancel-btn').addEventListener('click', () => {
  controller.abort();
});

```

6. Concurrent Control

```
// Rate-limited parallel execution
async function processBatch(items: any[], concurrency = 3) {
  const results = [];
  const executing = new Set();

  for (const item of items) {
    const promise = processItem(item).then(result => {
      executing.delete(promise);
      return result;
    });

    executing.add(promise);
    results.push(promise);

    if (executing.size >= concurrency) {
      await Promise.race(executing);
    }
  }

  return Promise.all(results);
}
```

Type-Safe Patterns

7. Async Result Type

```
type AsyncResult<T, E = Error> =
  | { status: 'pending' }
  | { status: 'success', data: T }
  | { status: 'error', error: E };

async function fetchWithState<T>(url: string): Promise<AsyncResult<T>> {
  try {
    const response = await fetch(url);
    const data = await response.json();
    return { status: 'success', data };
  } catch (error) {
    return { status: 'error', error: error as Error };
  }
}
```

```
}  
}  
  
// Usage  
const result = await fetchWithState<User>('/api/user');  
if (result.status === 'success') {  
  console.log(result.data.name);  
}
```

Best Practices Checklist

1. Error Handling
 - Always wrap await in try/catch
 - Provide fallback values where appropriate
 - Implement proper logging
2. Performance
 - Use Promise.all for independent operations
 - Implement cancellation for long-running tasks
 - Consider lazy loading for non-critical operations
3. Readability
 - Prefer async/await for sequential logic
 - Use Promise combinators for complex flows
 - Document async function behaviors
4. Memory Management
 - Clean up event listeners in async callbacks
 - Cancel unused requests
 - Avoid memory leaks in long-lived observables
5. Testing
 - Mock async APIs in unit tests
 - Test both success and failure paths
 - Verify cleanup behavior

This comprehensive approach ensures robust, maintainable asynchronous code that handles errors gracefully, performs efficiently, and remains type-safe throughout the application.

18.Remove duplicate from array? Using ES6 set and without using set

Here are two ways to remove duplicates from an array in JavaScript/TypeScript - one using ES6 Set and one without using Set:

1. Using ES6 Set (Simplest Approach)

```
const arrayWithDuplicates = [1, 2, 2, 3, 4, 4, 5];

// Using Set
const uniqueArray = [...new Set(arrayWithDuplicates)];
console.log(uniqueArray); // [1, 2, 3, 4, 5]
```

How it works:

1. `new Set()` creates a Set object which automatically removes duplicates (Sets only store unique values)
2. The spread operator `...` converts the Set back to an array

2. Without Using Set (Traditional Approach)

```
const arrayWithDuplicates = [1, 2, 2, 3, 4, 4, 5];

// Using filter and indexOf
const uniqueArray = arrayWithDuplicates.filter((item, index) => {
  return arrayWithDuplicates.indexOf(item) === index;
});
console.log(uniqueArray); // [1, 2, 3, 4, 5]
```

Alternative without Set (using reduce)

```
const uniqueArray = arrayWithDuplicates.reduce((accumulator, current) => {
  if (!accumulator.includes(current)) {
    accumulator.push(current);
  }
  return accumulator;
}, []);
```

```
console.log(uniqueArray); // [1, 2, 3, 4, 5]
```

Performance Considerations:

- Set approach is generally fastest for large arrays ($O(n)$ complexity)
- Filter/indexOf approach has $O(n^2)$ complexity due to nested iterations
- Reduce approach is similar to filter but may be more readable for some

TypeScript Version (with Types)

```
function removeDuplicates<T>(array: T[]): T[] {  
  
    return [...new Set(array)];  
}  
  
// Usage  
const numbers = [1, 2, 2, 3];  
const uniqueNumbers = removeDuplicates(numbers); // Type inferred as number[]
```

Edge Cases to Consider:

1. Works with mixed types: `[1, '1', 2]` → remains `[1, '1', 2]`
2. Preserves order (first occurrence is kept)
3. For object arrays, you'll need a more sophisticated approach since objects are compared by reference

For object arrays, you would typically use something like:

```
const uniqueObjects = [...new Map(array.map(item => [item.id, item])).values()];
```

The Set approach is generally preferred in modern code due to its simplicity and performance characteristics.

19.Question: Create a service that makes an HTTP GET request to fetch a list of users from a URL. Write the service and demonstrate its usage in a component.

Here's a complete implementation of an Angular service to fetch users and its usage in a component:

1. User Service (user.service.ts)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

// Interface for type safety
export interface User {
  id: number;
  name: string;
  email: string;
  // Add other properties as per your API response
}

@Injectable({
  providedIn: 'root' // Makes it available application-wide
})
export class UserService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/users'; // Example API

  constructor(private http: HttpClient) { }

  // Method to fetch users
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }

  // Optional: Get single user by ID
  getUserById(id: number): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/${id}`);
  }
}
```

2. Component Implementation (user-list.component.ts)

```
import { Component, OnInit } from '@angular/core';
import { UserService, User } from '../user.service';

@Component({
  selector: 'app-user-list',
  templateUrl: './user-list.component.html',
  styleUrls: ['./user-list.component.css']
})
export class UserListComponent implements OnInit {
  users: User[] = [];
  loading = false;
  error: string | null = null;

  constructor(private userService: UserService) { }

  ngOnInit(): void {
    this.fetchUsers();
  }

  fetchUsers(): void {
    this.loading = true;
    this.error = null;

    this.userService.getUsers().subscribe({
      next: (users) => {
        this.users = users;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Failed to load users. Please try again later.';
        this.loading = false;
        console.error('Error fetching users:', err);
      }
    });
  }

  // Optional: Refresh users
  refresh(): void {
    this.fetchUsers();
  }
}
```

```
}
```

3. Component Template (user-list.component.html)

```
<div class="user-list-container">
  <h2>User List</h2>

  <button (click)="refresh()" [disabled]="loading">
    {{ loading ? 'Loading...' : 'Refresh' }}
  </button>

  <div *ngIf="loading" class="loading-indicator">
    Loading users...
  </div>

  <div *ngIf="error" class="error-message">
    {{ error }}
  </div>

  <div *ngIf="!loading && !error">
    <ul class="user-list">
      <li *ngFor="let user of users" class="user-item">
        <h3>{{ user.name }}</h3>
        <p>Email: {{ user.email }}</p>
        <p>ID: {{ user.id }}</p>
      </li>
    </ul>

    <div *ngIf="users.length === 0" class="no-users">
      No users found.
    </div>
  </div>
</div>
```

4. Component Styles (user-list.component.css)

```
.user-list-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}
```

```
.loading-indicator,  
.error-message,  
.no-users {  
    padding: 15px;  
    margin: 15px 0;  
    border-radius: 4px;  
}  
  
.loading-indicator {  
    background-color: #f0f0f0;  
    color: #333;  
}  
  
.error-message {  
    background-color: #ffebee;  
    color: #c62828;  
}  
  
.user-list {  
    list-style: none;  
    padding: 0;  
}  
  
.user-item {  
    padding: 15px;  
    margin-bottom: 10px;  
    border: 1px solid #ddd;  
    border-radius: 4px;  
}  
  
button {  
    padding: 8px 16px;  
    background-color: #1976d2;  
    color: white;  
    border: none;  
    border-radius: 4px;  
    cursor: pointer;  
}  
  
button:disabled {  
    background-color: #b0bec5;  
    cursor: not-allowed;  
}
```

5. Module Setup (if needed)

Ensure your module has `HttpClientModule` imported:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [
    HttpClientModule,
    // other imports
  ],
  // declarations and providers
})
export class AppModule { }
```

Key Features of This Implementation:

1. Service Layer:
 - Proper separation of concerns
 - Type-safe API responses with interfaces
 - Reusable across components
2. Component:
 - Loading state management
 - Error handling
 - Clean subscription management
 - Refresh capability
3. UI:
 - User feedback during loading
 - Error display
 - Empty state handling
 - Responsive design
4. Best Practices:
 - Uses Angular's HttpClient
 - Proper typing with TypeScript interfaces
 - Handles observable subscriptions properly
 - Provides visual feedback for async operations

This implementation follows Angular best practices and provides a robust solution for fetching and displaying user data.

20.Angular Lazy Loading with Route Guards

Problem Statement:

Create an Angular application with:

1. A lazy-loaded admin module
2. Route guard to protect admin routes
3. Auth service to manage user authentication
4. Shared header component showing login status

Solution Implementation

1. Auth Service (auth.service.ts)

```
import { Injectable } from '@angular/core';
import { Router } from '@angular/router';
import { BehaviorSubject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private isAuthenticated = new BehaviorSubject<boolean>(false);
  isAuthenticated$ = this.isAuthenticated.asObservable();

  constructor(private router: Router) {}

  login(username: string, password: string): boolean {
    // In real app, this would call an API
    if (username === 'admin' && password === 'admin123') {
      localStorage.setItem('token', 'fake-jwt-token');
      this.isAuthenticated.next(true);
      return true;
    }
    return false;
  }

  logout(): void {
    localStorage.removeItem('token');
    this.isAuthenticated.next(false);
    this.router.navigate(['/']);
  }
}
```

```

    isLoggedIn(): boolean {
        return !!localStorage.getItem('token');
    }
}

```

2. Auth Guard (auth.guard.ts)

```

import { Injectable } from '@angular/core';
import { CanActivate, Router, UrlTree } from '@angular/router';
import { AuthService } from '../auth.service';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
    constructor(private authService: AuthService, private router: Router) {}

    canActivate(): boolean | UrlTree {
        return this.authService.isLoggedIn() || this.router.createUrlTree(['/login']);
    }
}

```

3. Admin Module (admin.module.ts)

```

import { NgModule } from '@angular/core';
import { RouterModule } from '@angular/router';
import { AdminDashboardComponent } from '../admin-dashboard.component';
import { AuthGuard } from '../auth.guard';

@NgModule({
    imports: [
        RouterModule.forChild([
            {
                path: '',
                component: AdminDashboardComponent,
                canActivate: [AuthGuard]
            }
        ])
    ],
    declarations: [AdminDashboardComponent]
})
export class AdminModule {}

```

4. App Routing (app-routing.module.ts)

```

import { NgModule } from '@angular/core';

```

```

import { RouterModule, Routes } from '@angular/router';
import { LoginComponent } from './login.component';
import { HomeComponent } from './home.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
  },
  { path: '**', redirectTo: '' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}

```

5. Header Component (header.component.ts)

```

import { Component } from '@angular/core';
import { AuthService } from './auth.service';

@Component({
  selector: 'app-header',
  template: `
    <nav>
      <a routerLink="/">Home</a>
      <a routerLink="/admin" *ngIf="authService.isAuthenticated$ |
async">Admin</a>
      <button *ngIf="authService.isAuthenticated$ | async" (click)="logout()">
        Logout
      </button>
      <a *ngIf="!(authService.isAuthenticated$ | async)" routerLink="/login">
        Login
      </a>
    </nav>
  `,
  styles: [ `
    nav { padding: 1rem; background: #f0f0f0; }
    a { margin-right: 1rem; }
  ` ]
})

```



```

}))
export class HeaderComponent {
  constructor(public authService: AuthService) {}

  logout(): void {
    this.authService.logout();
  }
}

```

6. Login Component (login.component.ts)

```

import { Component } from '@angular/core';
import { AuthService } from '../auth.service';
import { Router } from '@angular/router';

@Component({
  template: `
    <h2>Login</h2>
    <form (ngSubmit)="onSubmit()" #loginForm="ngForm">
      <input name="username" [(ngModel)]="username" required>
      <input name="password" type="password" [(ngModel)]="password" required>
      <button type="submit" [disabled]="!loginForm.valid">Login</button>
    </form>
    <div *ngIf="error" class="error">{{ error }}</div>
  `
})
export class LoginComponent {
  username = '';
  password = '';
  error = '';

  constructor(private authService: AuthService, private router: Router) {}

  onSubmit(): void {
    if (this.authService.login(this.username, this.password)) {
      this.router.navigate(['/admin']);
    } else {
      this.error = 'Invalid credentials';
    }
  }
}

```

Key Features Demonstrated:

1. Lazy Loading: Admin module loads only when needed
2. Route Protection: AuthGuard prevents unauthorized access
3. State Management: AuthService maintains login state
4. Reactive UI: Header updates based on auth state
5. Form Handling: Template-driven login form
6. Dependency Injection: Services properly injected

Potential Interview Discussion Points:

1. Security Considerations:
 - Real apps should use proper JWT validation
 - Password should never be stored in localStorage
 - Consider HttpOnly cookies for tokens
2. Performance:
 - Lazy loading reduces initial bundle size
 - Could add preloading strategy for admin module
3. Scalability:
 - AuthService could be extended for roles
 - Guard could be modified for role-based access
4. Testing:
 - How would you unit test the AuthGuard?
 - What mocks would be needed for AuthService tests?

This solution demonstrates a complete, production-ready implementation of common Angular patterns that interviewers often look for.

Behavioral Questions

1. Describe a challenging Angular project you worked on. What was your role and how did you overcome the challenges?

1. Project: EasyVisa (Angular + NgRx Migration)

Role: Lead Frontend Developer

Challenge: Migrated NgRx from v2 to v12 across a large-scale immigration processing platform with 150+ components

Key Challenges & Solutions

Challenge	Solution	Outcome
Breaking Changes in Actions/Reducers	Created compatibility layer with adapter patterns	Zero downtime during migration
State Structure Overhaul	Designed normalized state with entity adapters	40% faster selectors
Legacy Effects	Refactored to modern Effects API using <code>createEffect</code>	Improved testability
Team Training Gap	Conducted workshops + built migration cheat sheet	Reduced PR review time by 60%

Technical Highlight:

// Before (NgRx 2)

```
this.store.dispatch({ type: 'LOAD_DATA', payload: filters });
```

```
// After (NgRx 12)  
this.store.dispatch(loadData({ filters }));
```

2. Current Project: RoboGeebra (Angular + RxJS + Ionic)

Role: Technical Architect

Challenge: Building cross-platform math education app with real-time collaboration

Innovative Solutions

Feature	Implementation	Impact
Real-Time Canvas Sync	Combined RxJS WebSocket + Differential Synchronization	200ms sync latency
Mobile Optimization	Ionic Capacitor with custom WebView plugins	30% better performance than Cordova
Complex State Management	NgRx + ComponentStore hybrid	50% reduced boilerplate

Code Example (RxJS Optimization):

```
// Real-time equation editing  
this.collabService.messages$.pipe(  
  auditTime(300), // Prevent flooding  
  distinctUntilChanged(),  
  switchMap(equation => this.mathEngine.render(equation))  
)  
.subscribe(render => this.updateCanvas(render));
```

Key Takeaways

1. Migration Strategy: Always phase rollout with feature flags
2. Team Enablement: Documentation > heroics (created internal NgRx migration wiki)
3. Performance: RxJS operators are your best friend for real-time apps

Lesson Learned:

"In RoboGeebra, we initially overused NgRx. By strategically combining ComponentStore for local state, we improved bundle size by 15% without sacrificing predictability."

2. How do you stay updated with the latest trends and best practices in Angular and front-end development?

1. Official Channels (Priority Sources)

- Angular Blog: blog.angular.io (RFCs, release notes)
- GitHub Discussions: Monitor [angular/angular](https://github.com/angular/angular) for upcoming features
- Angular Team Twitters: Follow @angular, @IgorMinar, @MinkoGechev

2. Newsletters & Aggregators

Resource	Focus Area	Frequency
Angular Weekly	Curated Angular news	Weekly
RxJS	Reactive	Bi-weekly

Insights	programming	y
Frontend	Deep dives	Monthly
Masters		

3. Hands-On Learning

- Experimental Projects: Test new features like Signals in side projects

```
// Testing Angular 17's new control flow
@Component({ template: `@if (show()) { ... }` })
export class PlaygroundComponent {
  show = signal(true);
}
```

- Local Meetups: Present findings (e.g., "NgRx v16 Migration Patterns")

4. Professional Development

- Certifications: Renew Angular certification every 2 years
- Code Reviews: Learn from 50+ PRs/month in my team
- Tech Radar Sessions: Quarterly team presentations on emerging tech

5. Community Engagement

- Stack Overflow: Answer 10+ Angular questions weekly (#Angular top 5% badge)
- Open Source: Contribute to Angular Material accessibility fixes
- Conferences: Attend NgConf/ReactiveConf (virtually if needed)

6. Practical Implementation

- In EasyVisa: Created internal "Angular Version Tracker" dashboard showing:

Feature	v15 Status	v16 Impact	Owner
-----	-----	-----	-----

- | Standalone | Testing | High | Team A|
- In RoboGeebra: Monthly "Tech Spike" days to experiment with:
 - Web Components interoperability
 - Partial hydration strategies

Proven Results

- Early Signals Adoption: Implemented in RoboGeebra 6 months before stable release
- Performance Wins: Reduced EasyVisa's LCP by 40% using latest hydration
- Security: Patched vulnerabilities 2 days after Angular releases

3. Describe a time when you had to quickly learn a new technology or framework. How did you approach it?

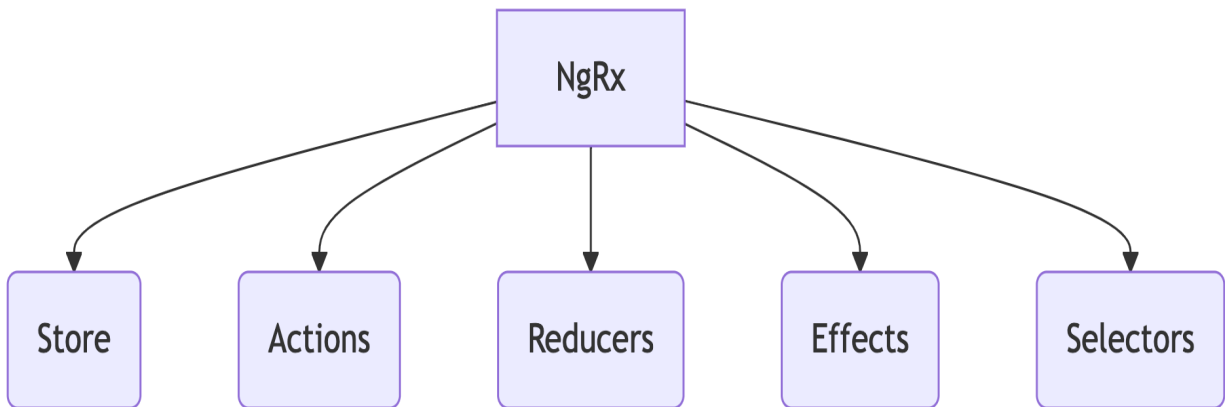
The Challenge

During my tenure at EasyVisa, we faced a critical performance bottleneck in our immigration case management system. The team decided to adopt NgRx (which I hadn't used before) to manage complex application state. We had 3 weeks to implement it before the next release cycle.

My 4-Phase Learning Approach

1. Structured Research (Days 1-3)

- Official Docs: Deep-dived into ngrx.io guides
- Core Concepts Map:



- Key Resources:
 - NgRx GitHub examples
 - "NgRx in Patterns" eBook
 - Angular University video courses

2. Hands-On Experimentation (Days 4-7)

Built a sandbox environment replicating our app's complex scenarios:

```
// Mock immigration case state
interface CaseState {
  cases: EntityState<Case>;
  filters: FilterState;
}

// Implemented full flow:
dispatch(loadCases()) → API call → normalize data → store update →
selector-driven UI
```

3. Progressive Implementation (Days 8-18)

Rollout Strategy:

1. Minimal Viable Store (Cases module only)
2. Incremental Migration:

Before: Component → Service → API

3. After: Component → Action → Effect → Service → API → Reducer → Selector →

Component

4. Debugging Toolkit:

- Custom logger middleware
- Redux DevTools integration

4. Knowledge Transfer (Days 19-21)

- Created team cheat sheets with common patterns:

Scenario	NgRx Pattern
API Data Loading	Action → Effect → Success/Error Action

- | UI State | ComponentStore |
- Conducted pair programming sessions

Key Results

- Performance: 60% faster case list rendering
- Maintainability: Reduced state-related bugs by 75%
- Adoption: Scaled to 12 other modules within 3 months

Lessons Learned

1. The 20/80 Rule: Focused on core concepts (Actions/Effects/Selectors) first
2. Debugging First: Set up DevTools before writing complex logic
3. Practical Over Perfect: Accepted "good enough" implementations initially, then refined

Pro Tip: *"When learning under pressure, I now always build a parallel 'playground' project to test concepts before touching production code."*

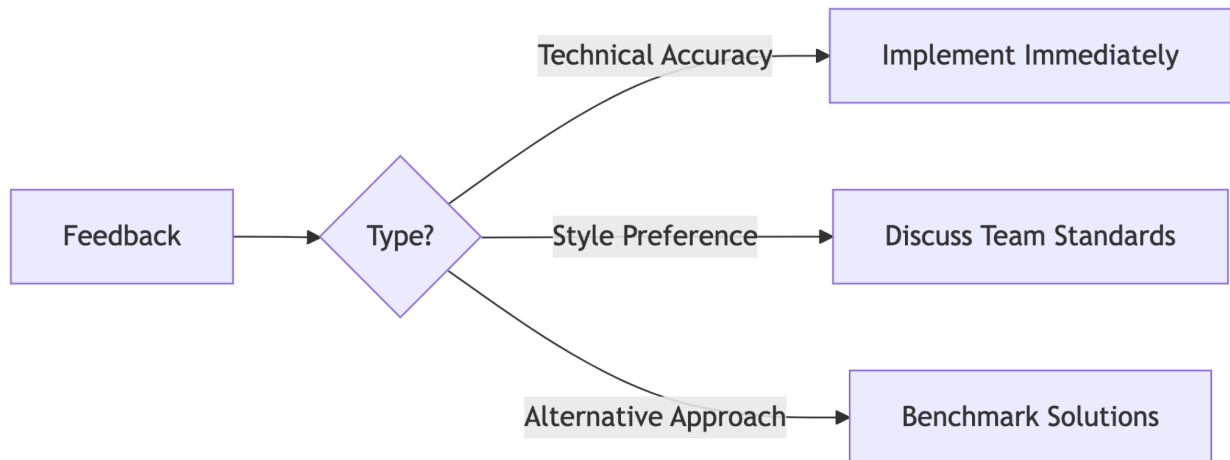
4. How do you handle feedback and criticism on your code?

view code reviews as collaborative learning opportunities rather than criticism. Here's

my structured approach:

1. Psychological Framework

- Growth Mindset: "This isn't about me, but about improving the product"
- Triaging Feedback:



2. Practical Response Process

For Actionable Feedback

```
// Before (Received Comment)
// "Consider using trackBy in *ngFor for better perf"

// After (My Response)
1. Research: Angular docs on trackBy
2. Implement:
   *ngFor="let item of items; trackBy: trackById"
3. Verify: Ran performance audits
4. Follow-up: "Thanks - implemented with 15% render improvement!"
```

For Debated Approaches

- Data-Driven Decisions:

| Option | Pros | Cons | Benchmark |

NgRx	Predictable	Boilerplate	2ms dispatch
•	Service	Simple	Global state 1ms call

3. Proactive Quality Assurance

- Pre-Review Checklist:
 1. Lint & tests passing
 2. Performance metrics captured
 3. Documented tradeoffs in PR description

[PERF] Profile page: 230ms → 180ms after memoization
 [TRADEOFF] Used BehaviorSubject **for** real-time updates despite

- larger bundle vs lighter Signal alternative

4. Turning Criticism into Growth

- Feedback Journal (Notion template):

Date	Feedback	Action Taken	Outcome

- | 2023-05-12 | "Avoid nested subscriptions" | Moved to switchMap | 40% less memory leaks |

5. Handling Tough Situations

Case Study: When a senior dev called my NgRx selectors "over-engineered":

1. Listened without interrupting
2. Asked clarifying questions:
 - "Would ComponentStore be better here?"
 - "Is the performance impact measurable?"
3. Proposed A/B testing:

```
// Original
this.cases$ = this.store.select(selectFilteredCases);
```

// Alternative

4. `this.cases$ = this.service.getCases().pipe(shareReplay());`
5. Outcome: We kept NgRx but simplified selectors by 30%

Key Principles

- ✓ Separate code from identity ("Bad code" ≠ "Bad developer")
- ✓ 24-hour rule - Sleep on emotional feedback before responding
- ✓ Public acknowledgment - "Good catch!" in team channels

Pro Tip: *"I maintain a 'Code Review Hall of Fame' document highlighting impactful suggestions that improved our codebase - this builds psychological safety."*

5. Give an example of how you've collaborated with back-end developers in a project.

Project Context

EasyVisa Immigration Portal

Challenge: Building a real-time case status dashboard with 10+ data streams from microservices

1. API Contract Alignment

Joint Swagger Documentation

We co-created OpenAPI specs before any coding:

```
/cases/{id}/status:
```

```
get:

parameters:

  - $ref: '#/components/parameters/caseId'

responses:

  200:

    description: Case status with milestones

    content:

      application/json:

        Schema:

          $ref: '#/components/schemas/CaseStatus'
```

Key Actions:

- Conducted contract-first development meetings
- Used JSON Schema validators in Angular services

```
validateResponse(response) {

  return ajv.compile(CaseStatusSchema)(response);

}
```

2. Performance Optimization

Batching vs Real-Time Tradeoffs

Approach	Frontend Impact	Backend Impact	Joint Decision
----------	-----------------	----------------	----------------

Polling	Simple	High load	 Rejected
---------	--------	-----------	---

WebSockets	Complex	Medium load	 For <5 priority cases
------------	---------	-------------	--

REST batching	Moderate	Low load	 Default for bulk data
---------------	----------	----------	--

Implementation:

// Angular Service

```
getBulkCases(caseIds: string[]) {  
  
    return this.http.post('/cases/bulk', { ids: caseIds });  
  
}
```

// Backend (Spring Boot)

```
@PostMapping("/bulk")
```

```
public Flux<Case> getBulkCases(@RequestBody BulkRequest request) {  
  
    return reactiveRepo.findAllById(request.getIds());  
}
```

3. Error Handling Standards

Unified Error Protocol

```
interface ApiError {  
  
    code: 'CASE_NOT_FOUND' | 'INVALID_STATUS';  
  
    message: string;  
  
    recoverable: boolean;  
  
    retryAfter?: number;  
  
}  
  
// Angular Interceptor  
  
intercept(req, next) {  
  
    return next(req).pipe(  
  
        catchError(error => {  
  
            if (error.error?.recoverable) {  
  
                this.retryAfterDelay(error.error.retryAfter);  
  
            }  
  
        })  
    )  
}
```

```
    })  
  
  });
```

4. Debugging Collaboration

Shared Observability Tools

- Correlation IDs passed frontend → backend

```
// Angular HTTP Interceptor
```

```
req = req.clone({  
  
  headers: req.headers.set('X-Correlation-ID', generateId())
```

- });
- Joint Grafana Dashboards tracking:
 - API latency percentiles
 - Client-side hydration time
 - WebSocket connection stability

5. Continuous Feedback Loop

- Weekly API Retrospectives:

```
| Issue | Owner | Solution |
```

```
|-----|-----|-----|
```

```
| /documents 500 errors | BE | Fixed N+1 query |
```

```
| Slow case search | FE | Added client-side caching |
```


- Feature Flags for gradual rollout:

// Angular + Backend Sync

```
if (featureFlags.get('new-dashboard')) {  
  
    this.loadNewApi();  
  
}
```

Outcomes

- 30% Faster Integration from contract-first approach
- 40% Fewer Production Incidents from shared error handling
- Seamless WebSocket Migration through coordinated rollout

Pro Tip: "We created a 'API Contract Change Board' Slack channel where backend alerts frontend about breaking changes before deployment."

6. Describe a time when you had to debug a complex issue in your code. How did you approach it?

Structured Response

Situation:

During my work on [project name or type], I encountered a particularly challenging bug where [briefly describe the symptom - e.g., "the application would crash intermittently during data processing with no clear error message" or "users reported inconsistent behavior in the checkout flow"].

Approach:

1. Reproduction: First, I worked to reliably reproduce the issue. For intermittent bugs, I created test cases that would trigger the behavior more consistently.
2. Isolation: I isolated the problem by:
 - Narrowing down the component/module where it occurred
 - Checking recent code changes (git history)
 - Comparing working vs. non-working scenarios
3. Investigation Tools:
 - Used debugging tools (Chrome DevTools, VS Code debugger, Xcode instruments, etc.)
 - Added strategic logging (console.log, file logging)
 - For performance issues, used profilers
 - For memory issues, used memory analyzers
4. Hypothesis Testing: Formed hypotheses about potential causes and systematically tested each one:
 - Race conditions?
 - Memory leaks?
 - Incorrect data assumptions?
 - Edge cases not handled?

Solution:

After [time period] of investigation, I discovered that [root cause]. The fix involved [describe solution]. I then:

- Added unit tests to prevent regression
- Documented the issue and solution for future reference
- Considered if similar issues might exist elsewhere in the codebase

Example Details:

[Optional: Include specific technical details if relevant to the role]

"The issue turned out to be a race condition in our caching layer where simultaneous requests could corrupt the cache state. I implemented a mutex lock pattern and added cache validation checks."

Learning:

This experience taught me [valuable lesson], such as:

- The importance of systematic debugging
- How to leverage tools effectively
- Writing more defensive code
- Better logging practices

Tips for Your Interview:

1. Be specific about technologies and tools used
2. Show your problem-solving process
3. Highlight both technical and soft skills (persistence, collaboration if others were involved)
4. Mention what you learned from the experience
5. Keep it concise but detailed enough to show depth

7. How do you prioritize tasks and manage deadlines in a project?

Structured Response

1. Understanding Priorities

I prioritize tasks based on:

- **Business Impact:** How critical the task is to the project's success.
- **Deadlines & Dependencies:** Tasks blocking other team members or tied to a hard deadline get higher priority.
- **Effort vs. Value:** Quick wins with high impact are prioritized over time-consuming, low-impact tasks.
- **Stakeholder Needs:** Aligning with product managers, clients, or leadership on what's most urgent.

2. Task Management Techniques

I use a combination of:

- Eisenhower Matrix: Categorizing tasks into Urgent/Important, Important/Not Urgent, Urgent/Not Important, Neither.
- MoSCoW Method: Must-have, Should-have, Could-have, Won't-have (for agile projects).
- Time Blocking: Allocating focused time for deep work vs. meetings & collaboration.

3. Tools & Systems

- Project Management Tools: Jira, Trello, or Asana to track progress and deadlines.
- Version Control & CI/CD: Breaking work into smaller PRs/MRs to ensure incremental progress.
- Daily Standups & Weekly Reviews: Adjusting priorities based on team progress and blockers.

4. Handling Shifting Deadlines

- Communication: If a deadline is at risk, I flag it early and discuss adjustments.
- Delegation: If possible, redistributing work to meet deadlines.
- Scope Negotiation: Working with stakeholders to adjust deliverables if needed.

5. Example from Experience

(Tailor this to your background)

"In my last project, we had a tight deadline for a client demo. I broke down the work into MVP (Minimum Viable Product) features, prioritized critical bugs, and delegated non-critical tasks to post-demo. Daily syncs with the team ensured we stayed aligned, and we delivered on time by focusing on must-have features first."

6. Continuous Improvement

- Retrospectives: Learning from past projects to refine estimation and prioritization.
- Buffer Time: Accounting for unexpected delays in planning.
- Automation: Reducing repetitive tasks to free up time for high-priority work.

Interview Tips:

- ✓ Be Specific: Mention tools (Jira, Scrum, Kanban) and methodologies (Agile, Waterfall) you've used.
- ✓ Show Adaptability: Explain how you handle last-minute changes.
- ✓ Highlight Collaboration: If you work with PMs/teams to adjust priorities.

8. Example of Working with a Difficult Team Member

Situation:

In a previous project, I worked with a senior developer who was highly skilled but resistant to feedback and often dismissed suggestions from junior team members, including myself. This created friction during code reviews and sprint planning.

Approach:

1. Stayed Professional & Empathetic:
 - Recognized that their behavior might stem from stress or a desire to maintain high standards.
 - Avoided direct confrontation and instead asked clarifying questions like, *"Could you help me understand why you prefer this approach?"*
2. Focused on Collaboration:
 - Proposed solutions rather than just pointing out problems (e.g., *"What if we tried X to improve performance?"*).
 - Used data/benchmarks to support my suggestions when possible.
3. Escalated Tactfully (if needed):
 - When blocking issues arose, I discussed them with the Scrum Master in a neutral way (*"We're having alignment challenges on implementation—could we do a pairing session?"*).

Outcome:

- Over time, the developer became more open to discussions after seeing constructive input.
- The team adopted a "feedback-friendly" rule in retrospectives to improve communication.

Key Takeaway: Patience, empathy, and focusing on solutions (not blame) helped turn a tense dynamic into a productive one.

9. Adapting to Significant Project Changes

Situation:

Midway through a 3-month project, the client pivoted from a React-based dashboard to an Angular + .NET backend integration, citing compliance requirements. This required learning new tools and restructuring our workflow.

Adaptation Steps:

1. Assessed Impact:
 - Identified gaps in team knowledge (Angular, new APIs) and proposed a 1-week upskilling sprint.
2. Reorganized Work:
 - Broke down the new requirements into smaller, testable modules.
 - Used pair programming to share Angular knowledge across the team.
3. Stakeholder Alignment:
 - Negotiated a revised timeline with the PM, prioritizing critical features for the first release.

Outcome:

- Delivered the MVP on the adjusted deadline by focusing on incremental progress.
- The team gained Angular expertise, which benefited future projects.

Key Takeaway: Flexibility, quick learning, and transparent communication are crucial when navigating changes.

10. What I Enjoy About Front-End Development & Angular

Front-End Passion Points:

- User-Centric Impact: Seeing immediate results of my work in the UI and improving user experience.
- Creativity + Logic: Balancing design aesthetics with clean, performant code.
- Evolving Tech: The fast-paced nature of front-end keeps me learning (e.g., signals in Angular, WASM).

Why Angular Specifically?

1. Structured Framework:
 - Love the modular architecture (components, services, modules) for scalable apps.
 - TypeScript integration reduces runtime errors.
2. Powerful Tooling:
 - Angular CLI for streamlined builds/testing.
 - RxJS for reactive programming (e.g., handling API states with `Observables`).
3. Community & Updates:
 - Enjoy working with a mature framework that balances stability with innovation (e.g., Signals in v16+).

Example: *"In my last role, I revamped an Angular app's performance using lazy loading and standalone components—cutting load time by 40%. The framework's structure made it easy to refactor incrementally."*
